

A scalable, open-source data platform

# Architecting an Apache Iceberg Lakehouse

Alex Merced

MEAP



MANNING





**MEAP Edition  
Manning Early Access Program**

**Architecting an Apache Iceberg Lakehouse**  
**A scalable, open-source data platform**  
**Version 3**

**Copyright 2025 Manning Publications**

For more information on this and other Manning titles go to [manning.com](https://manning.com).

© Manning Publications Co. To comment go to liveBook

Licensed to UNKNOWN UNKNOWN <weiss.itamar@gmail.com>

# welcome

---

Thank you for purchasing the MEAP edition of *Architecting an Apache Iceberg Lakehouse*. This book is designed for data architects, engineers, and technology leaders who are preparing to design or evolve their data architecture using Apache Iceberg. To get the most out of the material, you should already be familiar with modern data platform concepts, such as data lakes, cloud storage, and query engines, and have experience working with big data tools and distributed systems.

Apache Iceberg has quickly become a foundational component in modern data lakehouses. Yet, for many practitioners, the challenge lies not in understanding what Iceberg is, but in how to build a whole architecture around it; one that is scalable, governed, and adaptable to evolving business needs. This book aims to bridge that gap.

Rather than focusing solely on API usage or vendor-specific implementations, *Architecting an Apache Iceberg Lakehouse* provides a framework for making critical architectural decisions at each layer of the lakehouse stack, from storage and ingestion to cataloging, query, and governance. Along the way, you'll explore trade-offs, real-world patterns, and practical exercises to help ground your knowledge in experience.

My goal is to help you navigate the growing Iceberg ecosystem with clarity and confidence. Whether you're transitioning from legacy Hadoop platforms, experimenting with open table formats, or designing your first lakehouse from the ground up, this book will support you in aligning technical choices with strategic outcomes.

As you read, I encourage you to share feedback in the [liveBook Discussion forum](#). Your insights help shape a better book for everyone in the community.

—Alex Merced

# *brief contents*

---

## **PART 1: THE VALUE OF THE APACHE ICEBERG LAKEHOUSE**

- 1 The world of the Apache Iceberg Lakehouse*
- 2 Hands-on with Apache Iceberg*

## **PART 2: DESIGNING YOUR ICEBERG ARCHITECTURE**

- 3 Preparing for your move to Apache Iceberg*
- 4 Selecting the storage layer*
- 5 Architecting the ingestion layer*
- 6 Implementing the catalog layer*
- 7 Designing the federation layer*
- 8 Understanding the consumption layer*

## **PART 3: OPERATING YOUR APACHE ICEBERG LAKEHOUSE**

- 9 Maintaining an Iceberg lakehouse*
- 10 Operationalizing Apache Iceberg*

## *APPENDIXES*

- Appendix A. The metadata tables*
- Appendix B. Python for Apache Iceberg*
- Appendix C. The Apache Iceberg specification*



# ***1 The world of the Apache Iceberg Lakehouse***

## **This chapter covers**

- What a data lakehouse is and how it differs from traditional data architectures
- How Apache Iceberg shapes the lakehouse paradigm
- When and why you should implement an Apache Iceberg lakehouse

The evolution of data architecture has been shaped by a constant struggle to balance performance, cost, and flexibility while ensuring data remains accessible and governed. Over the years, businesses have cycled through various approaches—data warehouses (analytics-optimized databases), data lakes (analytics on files stored on distributed storage), and hybrid solutions—each attempting to solve the challenges of scaling analytics, reducing complexity, and controlling costs.

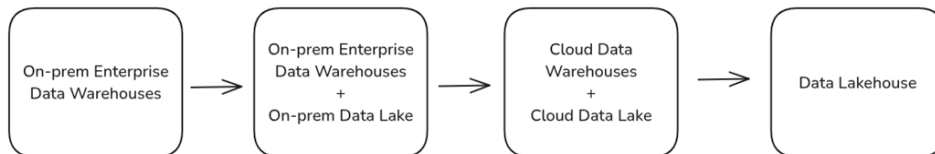
Enter Apache Iceberg, an open table format that allows groups of data files on distributed storage systems to be treated as traditional database tables. With Apache Iceberg, you can now have your analytics datasets efficiently accessible by multiple tools, improving collaboration between business units and reducing the costs of excessive Extract Transform Load (ETL) and data replication by working from a canonical single copy of your data on your data lake instead of multiple copies in multiple analytical systems. The Apache Iceberg Lakehouse is a modular, scalable, and cost-effective architecture that integrates the best aspects of data lakes and warehouses while preserving openness and flexibility. Apache Iceberg is revolutionizing the data industry. But why? What makes Iceberg a disruptive force in modern data platforms? Why are companies like Netflix, Apple, Dremio, AWS, Snowflake, Databricks, and many others using Iceberg, developing and building tooling around it? Apache Iceberg provides a community-led standard format for storing analytical datasets, which can be flexibly leveraged by the entire ecosystem of analytics tools while providing the ACID (Atomicity, Consistency, Isolation, Durability) guarantees and performance of closed data warehouse systems.

This book will focus on understanding and effectively making the choices needed to architect an Iceberg lakehouse to meet your use cases. Throughout this book, we will explore what a data lakehouse and Apache Iceberg are, enable you to get hands-on with a data lakehouse on your local environment, and go through the journey understanding your data platform needs and examining the ecosystem of each component to understand the choices you have to build your ideal data platform. In this chapter, we'll explore the context in which Apache Iceberg exists, looking at the challenges enterprises have faced with traditional architectures and the emergence of the data lakehouse pattern as a response. We'll then dive into how Apache Iceberg pushes this pattern forward, offering a scalable, high-performance, and open solution for modern data architectures.

## 1.1 What is a data lakehouse

Data doesn't exist in a vacuum—it's always in motion, powering applications, analytics, and AI-driven decision-making. Over the years, organizations have built infrastructure layers to handle the ever-growing complexity of data management. The data lakehouse architecture responds to decades of evolving needs, balancing cost, scalability, and accessibility while embracing openness and interoperability.

Though the data lakehouse has become the dominant paradigm, to fully appreciate its influence, it's essential to understand the architectural challenges that preceded it. By revisiting the evolution from OLTP (Online Transaction Processing) databases to data warehouses, data lakes, and beyond, we can better grasp why the lakehouse emerged and how it resolves long-standing trade-offs in data management. Figure 1.1 gives a visual overview of this evolution.



**Figure 1.1** The evolution of data platforms from on-prem warehouses to data lakehouses.

### 1.1.1 The rise of data warehouses

In the early days, most data lived inside OLTP databases—traditional relational systems designed to support real-time application interactions. These databases, like Oracle, DB2, PostgreSQL, MySQL, NoSQL stores, and SQL Server, were optimized for transactional operations, efficiently inserting, updating, and retrieving individual records. However, when organizations began wanting to analyze their data, generating roll-up reports, analytical dashboards, aggregations, and complex joins, their transactional databases struggled to process growing numbers of records stored in row-based structures, not optimal for analytics.

The reasons were clear: OLTP systems were designed for fast, transactional queries, not for scanning and aggregating millions, or even billions, of rows. While indexing and partitioning could help optimize some queries, analytical workloads still significantly strain operational databases, often slowing down transaction processing. These systems weren't built for massive-scale storage and retrieval, making retaining historical data for long-term analysis difficult.

Enter the enterprise data warehouse (EDW), a dedicated analytics system. Solutions like Teradata, IBM Netezza, and Oracle Exadata allowed companies to extract data from their OLTP systems, transform it into optimized schemas, and load it into centralized warehouses for analytical workloads known as OLAP (Online Analytical Process) systems. But there was a catch: cost and rigidity.

### 1.1.2 The move to cloud data warehouses

Traditional enterprise data warehouses were expensive in terms of storage and compute. Scaling them required massive on-premises (on-prem) hardware investments. Managing ETL (Extract, Transform, Load) pipelines was time-consuming.

With the rise of cloud computing, vendors like Snowflake, Google BigQuery, and Amazon Redshift introduced cloud data warehouses, offering elasticity and pay-as-you-go pricing. These platforms deliver several key benefits:

- *Compute and storage were decoupled*, allowing businesses to scale workloads independently.
- *Self-service analytics became easier*, enabling simplified access to analytics for all users..
- *Operational overhead was reduced*, as cloud providers handled infrastructure management.

However, cloud data warehouses still had fundamental limitations, especially as big data became the norm. While they were more cost-effective than traditional enterprise data warehouses, they still came with premium pricing for storage and compute, making cost scalability a concern. Additionally, data movement inefficiencies became apparent—businesses often had to copy data from source systems to the warehouse for analysis, introducing latency and complexity. On top of that, vendor lock-in remained a challenge, as proprietary storage formats made it difficult to move data between cloud providers or analytical engines. To address these challenges, businesses began to rethink how they approached large-scale data storage and analytics.

### 1.1.3 The data lake and the Hadoop era

While data warehouses provided efficient analytics on structured data, companies still needed a place to cost-effectively store massive, unstructured datasets—clickstream logs, IoT sensor data, social media feeds, and more. The solution? The data lake. A paradigm of running analytics directly on files containing data in a distributed storage system.

Early data lakes, built on Hadoop and its HDFS (Hadoop Distributed File System), revolutionized data storage by allowing businesses to store raw data in a central repository without upfront structuring and using lower-cost off-the-shelf hardware. This flexibility was a game-changer, significantly reducing storage costs compared to traditional enterprise data warehouses. Unlike rigid relational systems, Hadoop-based lakes could store all types of structured, semi-structured, and unstructured data, offering a single repository for diverse datasets. Additionally, powerful processing frameworks like Hadoop MapReduce and Apache Spark enabled large-scale computations directly on raw data, making extracting insights from vast amounts of information possible at scale.

However, significant challenges emerged as organizations leaned more heavily on Hadoop-based data lakes. Querying data stored in HDFS was painfully slow and inefficient, even with tools like Hive, which attempted to provide a SQL-like interface. Ensuring data consistency across teams became increasingly difficult, often leading to stale or conflicting datasets. Moreover, schema evolution was cumbersome—unlike relational databases, Hadoop lacked built-in schema enforcement, making data governance and management a complex and time-consuming task. As a result, many businesses struggled with "data swamps," which are massive repositories of unstructured and poorly managed data that were difficult to navigate, analyze, and extract value from.

At the same time, the cost of cloud data warehouses continued to soar as businesses needed to repeatedly copy subsets of their data into cloud data warehouse platforms for different use cases. A new approach was needed that combined the performance, ease of use, and structure of a data warehouse with the flexibility and cost-efficiency of a data lake.

#### **1.1.4 Apache Iceberg: The key to the data lakehouse**

This is where Apache Iceberg enters the story. Originally developed at Netflix, Iceberg was designed to address the challenges of managing large-scale datasets in data lakes. Specifically, Netflix engineers were frustrated with Apache Hive tables, which suffered from:

- *Slow metadata operations*—Hive struggled to handle tables with millions of partitions while tracking those partitions in its metastore.
- *Lack of schema evolution*—Schema updates would require expensive table rewrites.
- *Lack of ACID (Atomicity, Consistency, Isolation, Durability) guarantees*—data consistency across writes was challenging to enforce unless using file formats like ORC.
- *Inefficient query performance*—scanning large datasets often meant reading entire directories instead of just relevant files.

Apache Iceberg introduces a modern table format specification for analytics at a big data scale, bringing warehouse-like capabilities to the data lake. Unlike traditional data lake architectures, which struggled with consistency and performance, Iceberg provided a metadata layer to ensure consistent data definition and performant scans. It provides enterprises with extensive table metadata, which can help eliminate the need for expensive and time-consuming full-table scans when they aren't necessary. With built-in support for ACID transactions, Iceberg ensures data consistency and allows multiple users to read and write concurrently without sacrificing accuracy. Additionally, it optimized query performance through pruning based on partition and column statistics in the metadata, significantly improving efficiency without requiring manual tuning.

Iceberg marks a turning point in how organizations approach data lakes, transforming them from raw storage repositories into high-performance analytical platforms. With Apache Iceberg, businesses could use the data lake as a data warehouse, without sacrificing performance, consistency, or governance.

### **1.1.5 The data lakehouse: the best of both worlds**

The data lakehouse, in concept, emerged as the next evolution in data architecture. This system combines data lakes' cost efficiency with warehouses' performance and ease of use using open table formats like Apache Iceberg.

The lakehouse approach directly solves three significant problems:

1. **Data consistency across business units**
  - Instead of each team ingesting the same data into different proprietary data warehouses, which results in vendor lock-in and data portability challenges, a lakehouse allows them to query a single source of truth—the data files stored in their data lake.
2. **High-performance analytics on data stored in a data lake**
  - With Iceberg's optimizations, teams can run fast analytical workloads on raw data, reducing the need for expensive ETL pipelines and data movement and eliminating the need for extensive manual data management and tuning.
3. **Cost savings through reduced duplication**
  - Traditional architectures require multiple copies of the same data—one in a data lake, another in a warehouse, plus additional marts and extracts. The lakehouse minimizes redundant storage and lowers compute costs.

Apache Iceberg enables organizations to break free from legacy constraints and build scalable, cost-efficient, AI-ready architectures. In the next section, we'll look at Apache Iceberg itself—what it is, how it works, and why it's such a game-changer for the modern data lakehouse.

## APACHE ICEBERG AND THE ALTERNATIVES

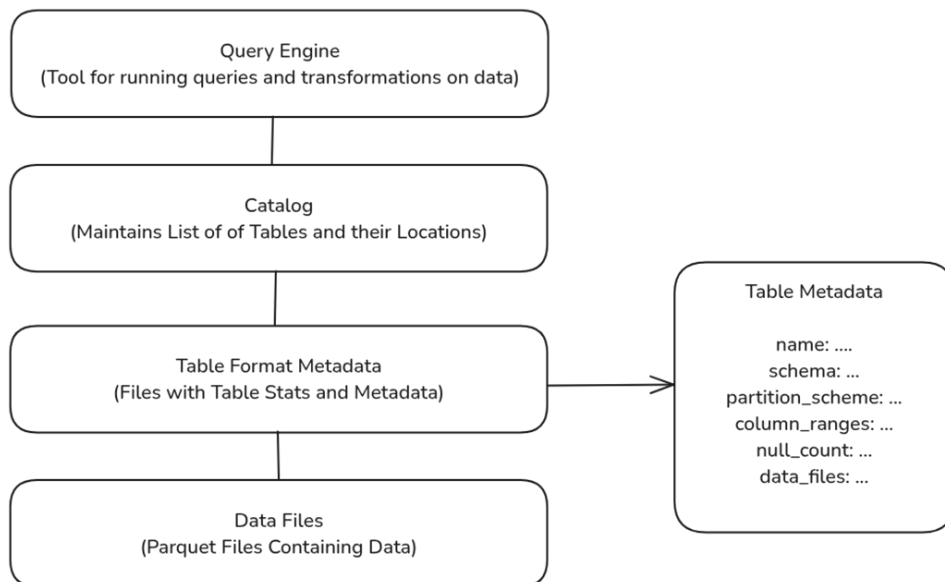
When comparing Apache Iceberg to alternative table formats like Apache Hudi, Delta Lake, and Apache Paimon, the differences often come down to flexibility, ecosystem support, and ease of management. While all these table formats have evolved toward feature parity, offering capabilities like ACID transactions, time travel, and schema evolution, Iceberg still stands out in key areas. One of its most notable advantages is its approach to partitioning. Unlike Delta Lake and Hudi, which rely on traditional static partitioning, Iceberg supports partition evolution and hidden partitioning, making it significantly easier to work with changing data structures over time. Partition evolution allows updates to partitioning strategies without rewriting existing data. In contrast, hidden partitioning eliminates the need for users to manually define and manage partition columns, simplifying queries and reducing operational complexity. These features make Iceberg particularly well-suited for dynamic and large-scale analytical workloads.

Beyond its technical capabilities, Apache Iceberg's most significant advantage is its rapidly expanding ecosystem of tools that go beyond just reading and writing the format. While Delta Lake is now an open-source Linux Foundation project, it is primarily steered and influenced by Databricks. Hudi is optimized for streaming use cases and has a smaller integration ecosystem than Iceberg and Delta. Iceberg, on the other hand, enjoys broad support across multiple platforms, from data warehouses (Snowflake, BigQuery) to query engines (Trino, Dremio, Presto) and processing frameworks (Spark, Flink), and is a transparently community-run Apache Foundation project. The Apache Foundation has stricter rules regarding diverse governance. More importantly, the Iceberg ecosystem includes advanced lakehouse management features, such as automated table optimization, lakehouse cleanup, access controls, and catalog integrations, giving users greater control over performance and governance. This broad compatibility ensures that organizations adopting Iceberg have the most flexible path to the lakehouse, allowing them to scale and optimize their architecture without being locked into a single vendor or toolset.

## 1.2 What is Apache Iceberg?

Modern data platforms demand flexibility, performance, scale, and consistency, but traditional approaches have forced businesses to compromise. Either they store data in data warehouse tables with strong ACID guarantees but high costs, or opt for raw files in a data lake with lower costs but weaker consistency and governance. Apache Iceberg changes this equation.

Iceberg is an open, community-driven, vendor-agnostic standard table format—an abstraction layer that sits on top of raw files (typically Apache Parquet) to make them behave like fully managed tables, similar to those in a data warehouse. Figure 1.2 illustrates the role of a table format in a data lakehouse, which acts as an abstraction or bundling of data that makes up a dataset, plus the metadata that provides information to search that data efficiently, like a library index. But unlike proprietary warehouse formats, Iceberg is open and vendor-agnostic, allowing teams to access the same data with the tools they prefer, whether it's Dremio, Spark, Flink, Trino, Snowflake, Qlik, or even graph processing engines like Puppygraph.



**Figure 1.2** The role of the table format in data lakehouses.

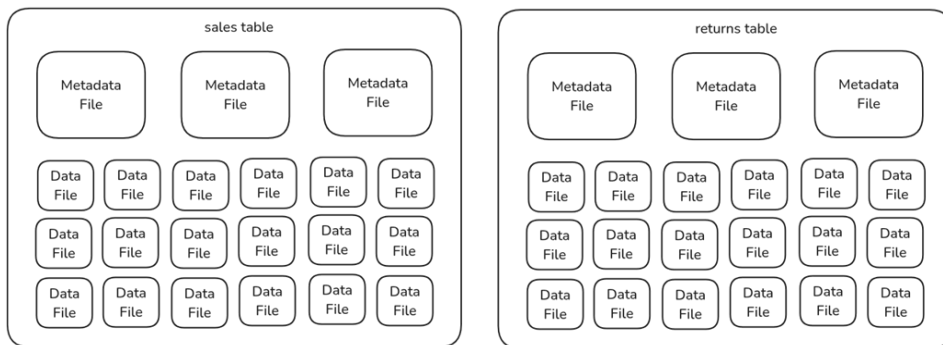
Let's break down the problem Apache Iceberg solves, its architecture, and the unique features that set it apart from other table formats.

### 1.2.1 The need for a table format

At its core, a data lake is simply a collection of files, often stored in the Apache Parquet format within an object store such as AWS S3, Azure Blob Storage, or Google Cloud Storage. While Parquet is well-suited for analytical workloads, relying solely on raw Parquet files introduces several challenges. One of the most significant limitations is that query engines do not automatically recognize multi-file Parquet datasets as singular structured tables with ACID guarantees and Schema evolution, without using catalogs like Apache Hive and similar solutions that track dataset directories and partitions to enable recognition as singular tables. These solutions still have limited ACID guarantees and evolution capabilities. Without a dedicated table format, there is no built-in mechanism to track schema changes, partitions, or snapshots, leading to inefficient metadata operations and slower queries.

Managing data consistency also becomes a challenge in a Parquet-only setup. Conflicts can arise when multiple users or tools write to the same dataset, resulting in duplicate records or inconsistent data views across different queries. Additionally, performance optimization is severely limited, as query engines must often scan entire directories of files to retrieve relevant data, even when only a small subset is needed. These inefficiencies explain why modern table formats, such as Apache Iceberg, Delta Lake, and Apache Hudi, have become essential for turning raw data lakes into high-performance, well-governed data platforms that enable ACID guarantees on table updates and deletes.

A table format like Apache Iceberg solves these issues by wrapping raw files in a metadata layer. This allows query engines to interact with them as if they were traditional database tables while maintaining the cost benefits of a data lake. As you can see in figure 1.3, a lakehouse table is made up of data files with the dataset and metadata files that allow query engines to scan that data more efficiently.



**Figure 1.3 The anatomy of a lakehouse table, metadata files, and data files.**

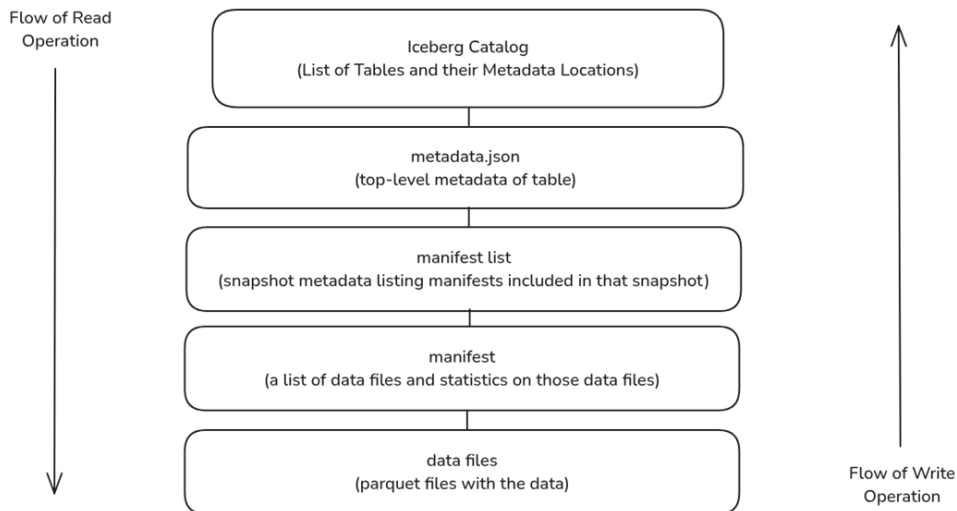
Iceberg achieves this with a multi-layer metadata structure that enables fast reads, efficient writes, and complete transactional consistency across multiple engines.



## 1.2.2 How Apache Iceberg manages metadata

One of Apache Iceberg's most powerful innovations is how it structures metadata, which is crucial in making lakehouses fast and efficient. But what exactly is metadata, and why does it matter so much in a data lakehouse? Imagine a large kitchen filled with drawers and cabinets. If you need a spoon, you could search through every drawer until you find one, but that would take time, especially if the kitchen is massive. Now, imagine a clipboard hanging on the wall with a detailed guide listing where every item is stored. Instead of rummaging through every drawer, you can simply glance at the clipboard and go directly to the right place. That clipboard is metadata—a structured index that helps locate and retrieve information efficiently.

In traditional data lakes, query engines often have to "search the entire kitchen," scanning through thousands (or even millions) of Parquet files to find relevant data, making queries slow and inefficient (parquet datasets may be partitioned for performance improvement, but as we'll see later Iceberg also improves on partitioning capabilities). Iceberg changes this by introducing a structured, multi-layered metadata system that acts like a well-organized clipboard, guiding queries to the exact files they need. Instead of relying on a single monolithic metadata file, where every change could slow down operations, Iceberg organizes metadata into three distinct layers (metadata.json, manifest list, and manifests), each serving a specific purpose (illustrated in figure 1.4 below). A catalog that tracks tables helps discover the three layers of metadata, which enables an engine to determine which data files to scan. This layered approach drastically improves query planning, enables better schema evolution, and optimizes the scalability of large datasets, making Iceberg a fundamental building block of the modern lakehouse.

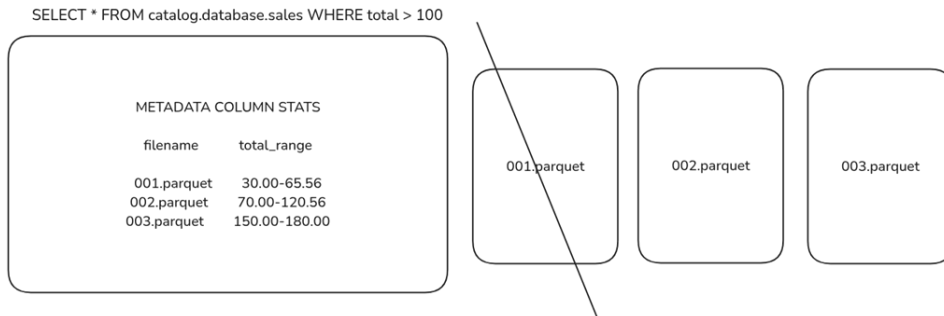


**Figure 1.4** The structure and flow of an Apache Iceberg table read and write operation.

An Apache Iceberg table's metadata files are primarily composed of three types of files containing data that enable engines to scan and understand the table more efficiently.

1. **Metadata.json (Table-Level Metadata)**
  - Tracks snapshots (versions of the table over time).
  - Stores schema definitions and partitioning strategies.
  - Allows query engines to understand the table's structure.
  - Enables features like schema evolution, time travel, and data consistency.
2. **Manifest Lists (Snapshot-Level Metadata)**
  - Represents a single snapshot of the table.
  - Lists groups of files (manifests) and summary statistics to help prune unnecessary file scans.
3. **Manifests (File-Level Metadata)**
  - Lists individual Parquet files within a dataset.
  - Stores file locations and statistics to help query engines filter out irrelevant data before scanning.

By organizing metadata in this way, Iceberg allows query engines to skip unnecessary files when executing queries, improving performance without needing complex indexing systems. For example, suppose a query only requires data from January 2024. Iceberg can use Manifest List stats to eliminate unnecessary partitions in that case. Then, Manifest files filter out individual Parquet files that don't match the query conditions before scanning any data. This intelligent pruning significantly reduces the amount of data scanned, improving performance and lowering compute costs. Figure 1.5 illustrates the process of metadata pruning.



**Figure 1.5 Engines use metadata statistics to eliminate data files from being scanned for faster queries.**

### 1.2.3 Key features of Apache Iceberg

Apache Iceberg delivers all the core capabilities expected from a modern table format while introducing groundbreaking innovations that set it apart from alternatives like Delta Lake and Apache Hudi. At its foundation, Iceberg offers schema evolution, allowing users to modify table schemas seamlessly without breaking existing queries. Unlike traditional systems that require expensive full-table rewrites, Iceberg enables effortless column additions, renaming, and deletions, making it easier to adapt data models as business needs evolve. Iceberg also ensures ACID guarantees, meaning every transaction, whether a small update or a large-scale ingestion, remains atomic, consistent, isolated, and durable. This level of reliability is critical in multi-writer environments where concurrent modifications could otherwise lead to inconsistencies.

Beyond these fundamental features, Iceberg introduces innovations that redefine partitioning and data access. With time travel, users can query historical versions of a dataset, enabling auditing, debugging, or even rolling back unintended changes. Where other formats require full rewrites to alter partitioning strategies, Iceberg's partition evolution allows modifications without costly data reorganization, making data more flexible over time.

Additionally, hidden partitioning automates partition column tracking within metadata, eliminating the need for users to specify partition filters in queries manually. This simplifies query logic and boosts efficiency, ensuring engines can optimize execution without requiring deep knowledge of the underlying data layout. These innovations make Iceberg a flexible and future-proof table format, providing organizations with a robust foundation for scalable, high-performance data lakehouses.

### 1.2.4 Apache Iceberg as an open-source standard

Apache Iceberg is not controlled by a single vendor. Unlike proprietary formats tied to specific platforms, it is an Apache Software Foundation project with contributors from Netflix, Apple, Dremio, Snowflake, AWS, and many more.

This broad industry support has led to deep integration across tools and engines, including:

- *Dremio*—Federated SQL & Native query acceleration and Reflections for fast BI performance on Iceberg tables.
- *AWS*—Major cloud vendor with a growing number of Apache Iceberg integrations.
- *Puppygraph*—Running graph queries on Iceberg datasets.
- *Apache Spark & Flink*—Large-scale ETL and streaming workloads.
- *Kafka Connect*—Streaming data ingestion into Iceberg tables.
- *Qlik Talend Cloud*—Full-featured data integration and management solution using Iceberg

Because Iceberg is open, organizations aren't locked into a single vendor or technology stack. Teams can build a flexible, interoperable lakehouse, ensuring they can adopt new tools and workflows without re-architecting their entire data platform.

Iceberg enables organizations to unlock the full potential of their data lakes, treating them as scalable, cost-efficient alternatives to traditional data warehouses. Let's explore the benefits of Apache Iceberg and why it's becoming the de facto standard for modern data platforms.

## 1.3 The benefits of Apache Iceberg

Apache Iceberg isn't just another table format—it fundamentally changes how organizations manage, query, and optimize data in a data lake. Traditional data lakes, while offering cheap and scalable storage, often suffer from slow query performance, data inconsistency, and complex governance challenges, all of which would require extensive manual data management to overcome. Iceberg addresses these issues by introducing a structured, high-performance approach that brings warehouse-like capabilities to the data lake. With features like schema evolution, ACID transactions, and advanced partitioning, Iceberg enables organizations to efficiently store, manage, and analyze massive datasets without the typical trade-offs of legacy architectures.

However, adopting Iceberg isn't without its considerations. While it significantly improves performance and flexibility, it requires careful integration with query engines and catalog systems to unlock its full potential. Organizations must also rethink traditional data lake practices as Iceberg shifts from working with raw files to treating data as managed tables. Below, we'll explore some of its most potent advantages and how they solve long-standing data management problems.

### 1.3.1 ACID transactions

Multiple jobs writing to the same dataset without proper transactional guarantees can lead to conflicting updates, duplicate records, and unreliable query results. This lack of coordination often forces organizations to implement complex workarounds, such as writing temporary files, using external locking mechanisms, or scheduling jobs to minimize conflicts. However, these solutions add operational overhead and still don't eliminate the risk of inconsistencies.

Apache Iceberg solves this problem by providing full ACID guarantees, ensuring that every change to a dataset is reliable and predictable. ACID stands for atomicity, consistency, isolation, and durability. With atomicity, updates are applied as a single, all-or-nothing operation, preventing partial writes that could leave data in an inconsistent state. Consistency ensures that all writes follow strict integrity rules, eliminating conflicts across different tools and processing frameworks. Isolation allows query engines to always see a stable dataset version, preventing read operations from being disrupted by ongoing writes. Finally, durability ensures that once a change is committed, it is permanent and recoverable, preventing accidental data loss. With these safeguards, Iceberg enables multiple teams and tools to read and write to the same dataset without fear of corruption or inconsistency, making data lakes more reliable and easier to manage.

### 1.3.2 Table evolution

Modifying schemas and partitioning in traditional data lakes has always been cumbersome, often requiring entire datasets to be reloaded or rewritten to accommodate a simple structural change. This rigidity forces organizations to make upfront design decisions that may not scale well as business needs evolve. Over time, these limitations can lead to inefficiencies, unnecessary data duplication, and complex workarounds that slow down analytics and data engineering workflows.

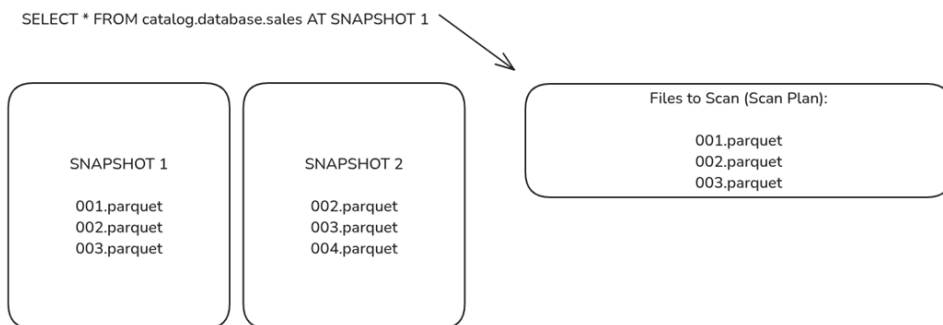
Apache Iceberg eliminates these challenges by introducing seamless schema evolution, allowing datasets to adapt over time without costly rewrites. With Iceberg, users can add, rename, or delete columns without breaking existing queries, ensuring backward compatibility across different applications. Unlike Hive, where partition schemes are fixed at table creation, Iceberg's partition evolution feature enables dynamic modifications to partitioning strategies without data migration.

Additionally, Iceberg tracks schema history, providing visibility into how a table's structure has changed over time—an essential feature for auditing and long-term data management. This flexibility makes Iceberg ideal for long-lived datasets that must evolve alongside business requirements, removing the friction traditionally associated with schema and partition management.

### 1.3.3 Time travel & snapshot-based queries

In traditional data lakes, once data is overwritten, it's gone. There's no easy way to recover previous versions for historical analysis or undo mistakes. This lack of version control makes disaster recovery and historical analysis far more challenging, often forcing organizations to create duplicate datasets or rely on manual backups. Without a structured way to track changes, even a small error in data ingestion can result in irreversible data loss or corrupted records.

Apache Iceberg changes this by enabling Time Travel. This powerful feature allows users to query past table versions precisely as they existed at a specific time. If incorrect or incomplete data is ingested, teams can roll back to a previous snapshot, restoring the dataset without complex recovery processes. Additionally, Iceberg enables versioned analytics, allowing users to analyze historical trends without duplicating data or managing multiple copies of the same dataset. These capabilities make Iceberg an invaluable tool for organizations that require strong data governance, compliance, and debugging capabilities, ensuring that data remains flexible and resilient to human or system errors. Figure 1.6 illustrates how time travel works by using a different data file listing depending on which snapshot is being scanned.



**Figure 1.6** Engines can scan older snapshots, which will provide a different list of files to scan, enabling scanning older versions of the data.

### 1.3.4 Hidden partitioning for reduced accidental full-table scans

Partitioning is crucial in optimizing query performance, but traditional Hive-style tables require users to manually specify partition columns in their queries. If a partition filter is missing in a query's WHERE clause, query engines must scan large amounts of unnecessary data, leading to slow performance and inefficient resource usage. This manual approach adds complexity for users and increases the risk of suboptimal queries that fail to take advantage of partition pruning.

Apache Iceberg eliminates this burden with Hidden Partitioning, a feature that automatically applies partition filters behind the scenes. Query engines optimize scans seamlessly, even if users don't explicitly specify partition keys in their queries. Analysts and data engineers no longer worry about manually including specific partition filters. Iceberg ensures that queries remain efficient without extra effort. By reducing the likelihood of costly full-table scans and making queries more straightforward to write, Hidden Partitioning significantly improves performance and usability, making large-scale data analysis faster and more accessible.

### 1.3.5 Cost efficiency & optimized query performance

Organizations often maintain multiple copies of the same data across warehouses, lakes, and data marts. This duplication drives up storage costs, increases ETL complexity, and leads to excessive compute expenses when processing large datasets. A common scenario is extracting raw data into a lake, transforming it via an ETL pipeline, and loading it into a warehouse for analytics—each step consuming additional storage and compute resources. These inefficiencies quickly add up, forcing companies to spend millions to keep their data infrastructure running.

Apache Iceberg dramatically reduces these costs by eliminating the need for unnecessary data duplication and optimizing query performance. Instead of copying data from a lake into a warehouse for analytics, teams can query Iceberg tables directly from the data lake, avoiding redundant storage expenses. Iceberg also optimizes data scans, allowing query engines to intelligently prune unnecessary files before execution, reducing the amount of data read and cutting down compute costs.

Iceberg simplifies ETL workflows by enabling streaming and batch ingestion into a single table, eliminating the need for complex data movement between different environments. By moving less data, minimizing copies, and optimizing queries, organizations can achieve high-performance analytics at a fraction of the cost, making Iceberg a game-changer for data infrastructure efficiency. One example of these cost savings is Insider, which reduced its S3 costs by 90% with Apache Iceberg (<https://medium.com/insiderengineering/apache-iceberg-reduced-our-amazon-s3-cost-by-90-997cde5ce931>).

## 1.4 The components of an Apache Iceberg lakehouse

A lakehouse is more than just a place to store data. It's an architecture that integrates flexibility, performance, and governance into a unified system. At the heart of an Apache Iceberg Lakehouse is its modular design, which allows organizations to select the best tools for each component while maintaining interoperability and openness.

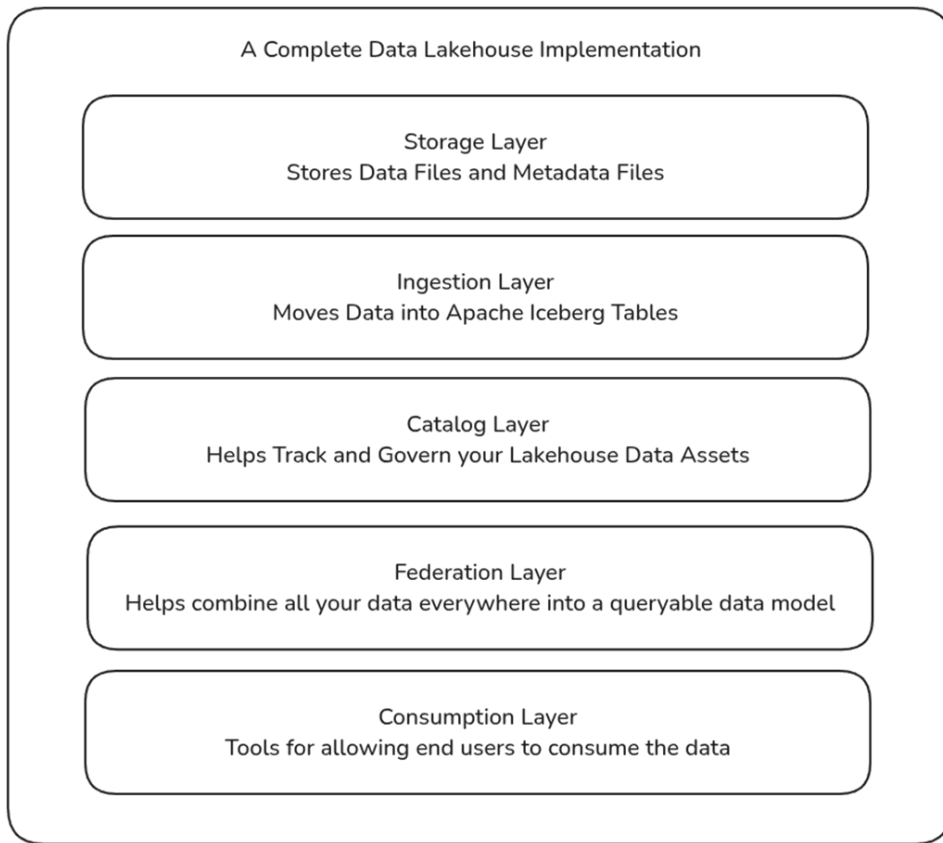
Unlike traditional monolithic data warehouses, which often lock organizations into a single vendor's ecosystem, a modular lakehouse architecture enables businesses to:

- Scale each component independently based on evolving needs.
- *Reduce vendor lock-in* by leveraging open standards like Apache Iceberg.
- *Optimize performance and cost* by choosing the right tools for ingestion, federation, and consumption.

By structuring the lakehouse into five key components, organizations can mix and match the best tools for their specific needs without vendor lock-in.

- *Storage Layer*—Stores data cost-effectively in object storage.
- *Ingestion Layer*—Streams or batches data into Iceberg tables.
- *Catalog Layer*—Tracks and governs all Iceberg metadata.
- *Federation Layer*—Models and accelerates data for analytics.
- *Consumption Layer*—Enables BI, AI, and real-time applications.

Let's break down the five key components of an Apache Iceberg Lakehouse, providing a high-level overview of how they work together, as shown in figure 1.7. In part two of this book, we'll explore each component in greater depth.



**Figure 1.7 The components of a complete data lakehouse implementation**

### 1.4.1 The storage layer: The foundation of your lakehouse

The storage layer is where your raw data and metadata live. This layer is typically a distributed object store or filesystem optimized for scalability, durability, and cost efficiency.

#### Where data is stored:

- *Cloud*—Amazon S3, Azure Blob Storage, Google Cloud Storage
- *On-premises*—HDFS, MinIO, Ceph
- *Hybrid*—A combination of both for greater flexibility

#### What is stored:

- *Data files*—Typically Apache Parquet (can be ORC or AVRO), optimized for analytics.
- *Metadata files*—Iceberg's table metadata, manifests, and snapshots.



## WHY IT'S IMPORTANT

One of the lakehouse model's most significant advantages is separating storage from compute. Unlike data warehouses, which bundle storage and compute together, Iceberg allows organizations to scale compute workloads independently, reducing storage costs while improving performance and allowing different teams to use other tools they prefer.

### 1.4.2 The ingestion layer: Feeding data into Iceberg tables

Once data is collected across various systems, it must be processed and stored as Iceberg tables in the storage layer. The ingestion layer comprises tools and frameworks that help transform, clean, and load data into Iceberg tables.

#### Batch ingestion:

- *Apache Spark*—Large-scale ETL workloads into Iceberg.
- *Fivetran*—managed ingestion service for ETL workloads

#### Streaming ingestion:

- *Apache Flink*—Real-time data processing into Iceberg.
- *Kafka Connect*—Streaming ingestion from Kafka topics into Iceberg tables.
- *Estuary*—Managed platform for real-time and batch ingestion
- *Qlik Talend Cloud*—Supports batch and streaming workloads

These are just a few options; we will visit many more in chapter 5 on ingestion.

## WHY IT'S IMPORTANT

A flexible ingestion layer ensures batch and streaming data can be written to Iceberg tables efficiently. This modularity allows organizations to optimize data freshness, reducing latency for analytics and AI use cases.

### 1.4.3 The catalog layer: The entry point to your lakehouse

A Lakehouse Catalog is what makes Iceberg queryable and manageable at scale. It acts as the interface between your storage layer and query engines, tracking all tables and the location of their metadata.

#### Open Catalog Options:

- *Apache Hive Metastore*—Legacy support, but not ideal for Iceberg.
- *AWS Glue Data Catalog*—Cloud-native option for managing Iceberg tables.
- *Project Nessie*—A Git-like versioned catalog that enables time travel and data as code.
- *Apache Polaris*—An open-source Lakehouse Catalog with robust Role-Based Access Controls (RBAC) features
- *Apache Gravitino*—Geo-Distributed Lakehouse Catalog

- *Lakekeeper*—A lakehouse catalog with data contract features
- *Iceberg REST Catalog*—An open standard for Iceberg-compatible catalogs.

## WHY IT'S IMPORTANT

The catalog layer is the single source of truth for all Iceberg tables, ensuring consistency and governance across multiple query engines. It enables:

- **Atomic table updates** without disrupting users.
- **Cross-tool interoperability** allows Dremio, Spark, Flink, Trino, and Snowflake to see the same table.
- **Portable access controls across your tools** are possible thanks to Lakehouse catalogs.

A well-chosen catalog ensures that teams across the organization can securely collaborate on the same Iceberg datasets, eliminating data silos.

### 1.4.4 The federation layer: Modeling & accelerating data

The federation layer is where raw data is modeled, enriched, and optimized for analytics, AI, and reporting. This is where teams define business logic, unify data across sources, and optimize query performance.

#### Key capabilities:

- *Semantic Layer*—Defines consistent metrics across the organization.
- *Federated Query Engines*—Enables querying across Iceberg and external databases.
- *Query Acceleration*—Speeds up analytics without duplicating data.

#### Example tools:

- *Dremio*—Unifies Iceberg, databases, and warehouses with a built-in semantic layer and its Reflections feature for acceleration.
- *Trino*—Provides open-source SQL-based data unification across multiple sources. It may require an external semantic layer and acceleration tools.
- *dbt*—Enables transformations and modeling directly on Iceberg.

## WHY IT'S IMPORTANT

The federation layer ensures that **Iceberg tables don't exist in isolation**. Organizations often have data spread across:

- **Relational databases** (PostgreSQL, MySQL, SQL Server)
- **Cloud data warehouses** (Snowflake, Redshift, BigQuery)
- **Other lakehouse formats** (Delta Lake, Apache Hudi)

A strong federation layer allows teams to create unified business views, ensuring that Iceberg fits seamlessly into an enterprise-wide data strategy.

### 1.4.5 The consumption layer: Delivering value to the business

Data is only as valuable as its impact. The consumption layer is where data is used for analytics, AI, and operational workflows.

#### Use Cases:

- *Business Intelligence (BI)*—Power dashboards & reporting.
- *AI & Machine Learning (ML)*—Train models and fuel AI applications and AI Agents.
- *Embedded Analytics*—Deliver insights directly into business apps.
- *Data APIs & Apps*—Build customer-facing applications on Iceberg data.

#### Example tools:

- *BI & Dashboards*—Tableau, Power BI, Looker, Preset (Superset).
- *AI & ML*—Databricks, Hugging Face, Jupyter, LangChain.
- *Data APIs*—GraphQL, REST APIs for application integration.

## WHY IT'S IMPORTANT

The modular nature of lakehouses and the open-standard nature of Iceberg ensure that data is available across all consumption tools without unnecessary ETL duplication. This eliminates:

- The need to copy data into multiple warehouses for different workloads.
- The risk of data inconsistencies across departments.
- Expensive cloud warehouse query costs for operational analytics.

With an Iceberg lakehouse, data products can be defined once and consumed everywhere, ensuring a single, optimized source of truth for the entire business.

In the next chapter, we'll get hands-on so you can see the data lakehouse in action before exploring the five components for architecting your Apache Iceberg Lakehouse in part two of this book.

## 1.5 Summary

- Data lakehouse architecture combines the scalability and cost-efficiency of data lakes with the performance, ease of use, and structure of data warehouses, solving key challenges in governance, query performance, and cost management.
- Apache Iceberg is a modern table format that enables high-performance analytics, schema evolution, ACID transactions, and metadata scalability. It transforms data lakes into structured, mutable, governed storage platforms.
- Iceberg eliminates significant pain points of OLTP databases, enterprise data warehouses, and Hadoop-based data lakes, including high costs, rigid schemas, slow queries, and inconsistent data governance.
- With features like time travel, partition evolution, and hidden partitioning, Iceberg reduces storage costs, simplifies ETL, and optimizes compute resources, making data analytics more efficient.
- Iceberg integrates with query engines (Trino, Dremio, Snowflake), processing frameworks (Spark, Flink), and open lakehouse catalogs (Nessie, Polaris, Gravitino), enabling modular, vendor-agnostic architectures.
- The Apache Iceberg Lakehouse has five key components: storage, ingestion, catalog, federation, and consumption.

## 2 *Hands-on with Apache Iceberg*

### **This chapter covers**

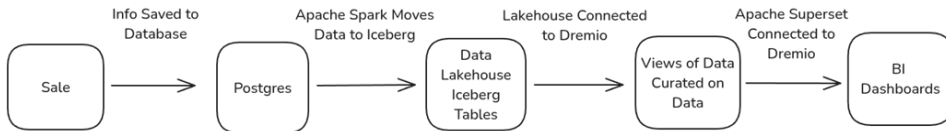
- Setting up an Apache Iceberg environment
- Creating Iceberg tables in Spark
- Reading Iceberg tables in Dremio
- Building a business intelligence (BI) dashboard

We've explored the theory behind the Apache Iceberg lakehouse—its architecture, components, and benefits. Now, it's time to bring it to life with a hands-on exercise: set up a fully functional Iceberg lakehouse on your laptop.

This chapter will walk you through an end-to-end setup, from installing your environment to creating Iceberg tables, querying them with Dremio, and visualizing data in a Business Intelligence (BI) dashboard. By the end of this chapter, you'll have a working Iceberg lakehouse that you can experiment with, giving you the confidence and foundation needed to design a production-scale implementation.

In this scenario, we will ingest sales transactions from a fashion retail chain, capturing not just product and category but also location, customer demographics, and the associated marketing campaign. Initially housed in a PostgreSQL database, this operational data is extracted and transformed into Iceberg tables using Apache Spark. These Iceberg tables are then curated in Dremio, where virtual views are created to reflect sales trends by region, customer segment, and campaign performance. These views power interactive dashboards in Apache Superset, providing real-time insights for marketing, sales, and merchandising teams to optimize product offerings and campaign strategies.

We can see an illustration of the flow of this exercise below in figure 2.1:



**Figure 2.1** The flow of the hands-on exercise in chapter 2.

The code for this chapter can be found at <https://github.com/AlexMercedCoder/architecting-an-apache-iceberg-lakehouse>.

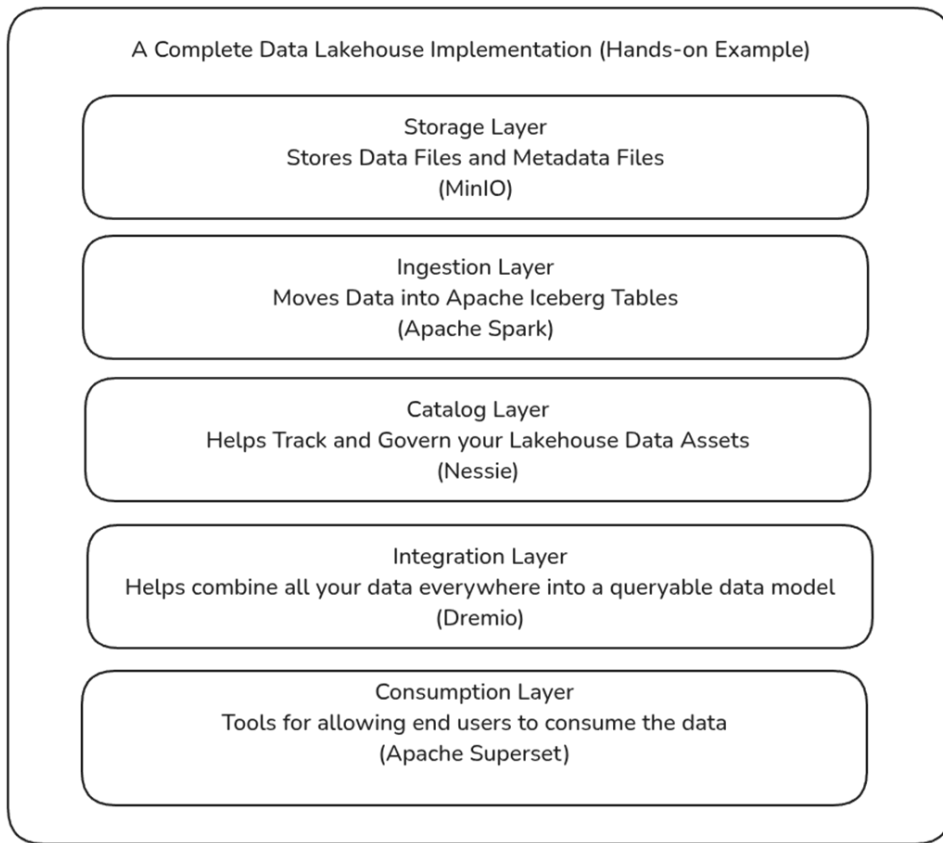
Let's dive in and get our hands dirty with Apache Iceberg!

## 2.1 Setting up an Apache Iceberg environment

To illustrate working with Iceberg tables, we will set up an end-to-end local lakehouse environment. We'll be using Docker Compose to spin up a collection of essential services representing a possible analytics stack, including:

1. **Operational Database:** PostgreSQL (simulating a transactional system)
2. **Data Lake Storage Layer:** MinIO (an S3-compatible object store)
3. **Ingestion & Processing Layer:** Apache Spark
4. **Catalog Layer:** Nessie (for tracking Iceberg tables)
5. **Query Engine:** Dremio (for running fast SQL queries)
6. **BI & Visualization:** Apache Superset (for dashboards)

Each of these components can play a critical role in an Apache Iceberg lakehouse, and by setting them up locally, we can observe how they interact in real-world scenarios. We will examine other options for many of these roles as we discuss each layer of architecting an Iceberg lakehouse. Figure 2.2 illustrates the components of the Iceberg lakehouse environment we discussed in chapter 1.



**Figure 2.2 The components of an Apache Iceberg Data Lakehouse.**

### 2.1.1 Prerequisites: Install Docker

Before we begin, you'll need Docker Desktop installed on your machine. If you don't have it, download and install it from [docker.com](https://docker.com) for free. Docker allows us to create virtual environments to run software called containers, enabling us to run software like Nessie, Apache Spark, and Dremio without installing them on our laptops.

Once installed, ensure that Docker is running by executing the following command in your terminal:

```
docker --version
```

```

~ on ☁️ (us-east-1)
😊 docker --version
Docker version 20.10.14, build a224086
~ on ☁️ (us-east-1) took 2s

```

**Figure 2.3 Output to expect when checking if Docker is installed.**

You should see output similar to figure 2.3, confirming that Docker is installed.

### 2.1.2 Creating the Docker compose file

Now, let's define our environment. In an empty directory, create a file named `docker-compose.yml`, then add the following configuration in the code snippet (for the full code snippet, refer to the GitHub repo linked below). In this Docker Compose configuration, we are configuring the different services to deploy our data lakehouse to our host machine, including our catalog (Nessie), storage layer (MinIO), integration/federation tool (Dremio), ingestion tool (Spark), and consumption tool (Superset). We are configuring which Docker images to use and which network ports to map to each container to have a complete lakehouse environment running on our local machine.

<https://github.com/AlexMercedCoder/architecting-an-apache-iceberg-lakehouse/blob/main/ch2/docker-compose.yml>

In this file, we'll find the following code:

```

version: "3"  #A
Services:  #B
  Nessie:  #C
    image: projectnessie/nessie:latest
    container_name: nessie
    networks:
      lakehouse-net:
    ports:
      - 19120:19120

```

#A The version of the docker-compose file being used

#B Our list of services making up our environment

#C Our first service, which is our nessie catalog\



This Docker Compose configuration defines a service for running a Project Nessie catalog server using its in-memory storage backend. The service is named `nessie` and is built from the latest Project Nessie image. It is assigned to a custom network called `lakehouse-net`, allowing it to communicate with other services in the same network, such as compute engines Apache Spark and Dremio. The container exposes port 19120 on both the host and the container, enabling external access to Nessie's REST API. Since this configuration uses in-memory storage, it is suitable for development or testing environments where persistence across restarts is not required.

```
Minio: #A
  image: minio/minio:latest
  container_name: minio
  environment:
    - MINIO_ROOT_USER=admin
    - MINIO_ROOT_PASSWORD=password
    - MINIO_REGION=us-east-1
  networks:
    lakehouse-net:
  ports:
    - 9001:9001
    - 9000:9000
  command: ["server", "/data", "--console-address", ":9001"]
```

#### **#A Our data lake storage service using the latest Minio image**

This Docker Compose service definition sets up a local MinIO server, a high-performance object storage system compatible with the Amazon S3 API. The service is named `Minio` and uses the latest MinIO image. It specifies environment variables to configure the root user credentials and the region (`us-east-1`). The container runs within the `lakehouse-net` network, allowing it to communicate with other components of the lakehouse architecture. It exposes two ports: 9000 for S3-compatible API access and 9001 for accessing the MinIO web-based console. The command instructs MinIO to use `/data` as its storage directory and binds the admin console to port 9001, making it accessible through a browser. This configuration is useful for testing or prototyping object storage layers in an Iceberg-compatible environment.

```

Dremio: #A
  image: dremio/dremio-oss:latest
  container_name: dremio
  networks:
    lakehouse-net:
  ports:
    - 9047:9047
    - 31010:31010
    - 32010:32010

```

**#A Our analytics engine service, Dremio, which will handle running our analytical queries**

This Docker Compose service defines a container for running Dremio, an open-source, high-performance SQL query engine designed for lakehouse architectures. The service uses the latest open-source Dremio image and is named `dremio`. It is attached to the `lakehouse-net` network, enabling it to interact with other components like MinIO for storage or Nessie for catalog management. The container exposes three ports: 9047 for accessing the Dremio UI and REST API, 31010 for JDBC client connections, and 32010 for internal communication within Dremio's distributed engine. This setup allows Dremio to serve as a central interactive query platform, capable of federating across multiple data sources and providing a unified SQL interface over Iceberg tables.

```

Spark: #A
  image: alexmerced/spark35nb:latest
  container_name: spark
  networks:
    lakehouse-net:
  ports:
    - 8080:8080 # Master Web UI
    - 7077:7077 # Master Port
    - 8888:8888 # Jupyter Notebook
  environment:
    - AWS_ACCESS_KEY_ID=admin
    - AWS_SECRET_ACCESS_KEY=password
    - AWS_DEFAULT_REGION=us-east-1
    - AWS_REGION=us-east-1

```

**#A Our ingestion engine service with Apache Spark, which we will use to write data to Iceberg tables**

This Docker Compose service launches an Apache Spark container configured for development and data lakehouse experimentation. It uses the `alexmerced/spark35nb:latest` image, which includes Spark 3.5 along with Jupyter Notebook for interactive development. The container, named `spark`, is connected to the `lakehouse-net` network, allowing it to interact with other services like MinIO and Nessie. It exposes three ports: 8080 for the Spark master web UI, 7077 for Spark's master port (used by workers and drivers), and 8888 for accessing Jupyter Notebook. Environment variables are set to provide Spark with access credentials for AWS S3-compatible storage, enabling seamless reading and writing to MinIO using the S3 API. This setup allows users to run Spark jobs or explore Iceberg tables interactively through Jupyter, making it ideal for local testing and prototyping.

```
Postgres: #A
  image: postgres:latest
  container_name: postgres
  environment:
    POSTGRES_DB: mydb
    POSTGRES_USER: myuser
    POSTGRES_PASSWORD: mypassword
  networks:
    lakehouse-net:
  ports:
    - "5435:5432"
```

**#A Our database service representing our operational data source**

This Docker Compose service configures a PostgreSQL container to act as the operational data source in the lakehouse architecture example. The service uses the latest official PostgreSQL image and sets up a database named `mydb` with user credentials specified via environment variables. It runs on the shared `lakehouse-net` network, allowing other services like Spark or ingestion tools to connect for data extraction. The container exposes port 5432 internally, mapped to 5435 on the host to avoid conflicts with other local PostgreSQL instances. This setup simulates a transactional data system from which structured data can be periodically or continuously ingested into the lakehouse for downstream analytics.

```
Superset: #A
  image: alexmerced/dremio-superset
  container_name: superset
  networks:
    lakehouse-net:
  ports:
    - 8088:8088
```

```
Networks: #B
  lakehouse-net:
```

#A Our BI tool service using Apache Superset for Creating Dashboards from our data

#B We declare our shared network so all our services can communicate.

This Docker Compose service sets up Apache Superset, a modern open-source business intelligence (BI) platform, pre-configured to work with Dremio for visual analytics in a lakehouse environment. It uses the alexmerced/dremio-superset image, which simplifies integration with Dremio as a backend SQL engine. The container is named superset and is connected to the lakehouse-net network, enabling seamless connectivity with services like Dremio and the Iceberg catalog. The service exposes port 8088, providing access to Superset's web-based interface for building dashboards and exploring datasets. This configuration allows users to interactively query Iceberg tables through Dremio and visualize results in Superset, completing the end-to-end analytics workflow.

### 2.1.3 Running the environment

Now that we have our Docker Compose file, let's spin up our lakehouse environment.

From the terminal, navigate to the directory where you saved docker-compose.yml and run:

```
docker-compose up -d
```

This will:

- Start the **Nessie catalog**, which will track our Iceberg tables.
- Launch **MinIO**, our object store for Iceberg metadata and data files.
- Deploy **Dremio**, our SQL-based query engine.
- Set up **Apache Spark**, which we'll use to move data into Iceberg tables.
- Create a **PostgreSQL database** simulating an operational system.
- Initialize **Apache Superset**, where we'll build BI dashboards.

## VERIFYING THAT EVERYTHING IS RUNNING

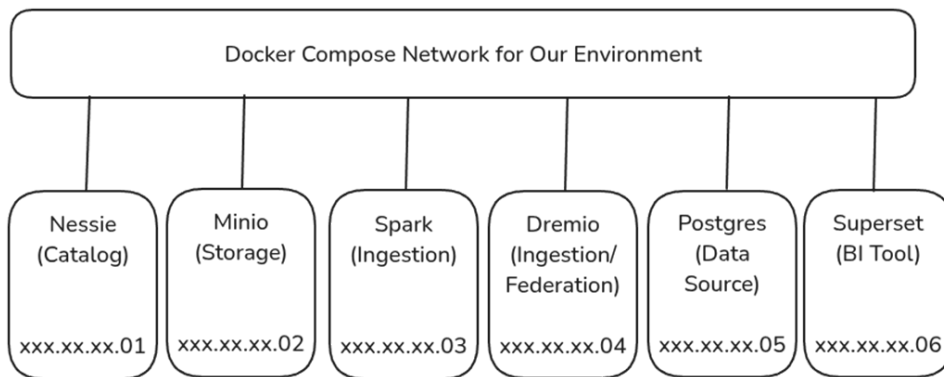
Once Docker Compose finishes, check the running containers with:

```
docker ps
```

You should see something like this:

| CONTAINER ID | IMAGE                       | STATUS       | PORTS                     |
|--------------|-----------------------------|--------------|---------------------------|
| xxxxxxxxxx   | projectnessie/nessie:latest | Up 5 minutes | 19120/tcp                 |
| xxxxxxxxxx   | minio/minio:latest          | Up 5 minutes | 9000-9001/tcp             |
| xxxxxxxxxx   | dremio/dremio-oss:latest    | Up 5 minutes | 9047/tcp, 31010-32010/tcp |
| xxxxxxxxxx   | alexmerced/spark35notebook  | Up 5 minutes | 7077/tcp, 8888/tcp        |
| xxxxxxxxxx   | postgres:latest             | Up 5 minutes | 5435->5432/tcp            |
| xxxxxxxxxx   | alexmerced/dremio-superset  | Up 5 minutes | 8088/tcp                  |

If all containers are running, your Iceberg lakehouse is ready to go! Refer to figure 2.4 to see the resulting network of containers that Docker should create.



**Figure 2.4** The network of containers for this exercise, each with its own IP Address.

The flow of how these containers will work together is that when we want to ingest new data, we will write Python scripts that will create a job for Spark to complete (in this case, taking data from Postgres and writing it to our Iceberg Lakehouse). These jobs will configure Nessie as the catalog and Minio as the storage layer. This lets Spark know that any data and metadata files should be written to the configured location in Minio. Then, the catalog should be updated to include the new metadata for the table being created or updated. Dremio will be similarly configured with Nessie and Minio to enable viewing of our datasets, allowing us to curate logical views on those tables to clean and prepare the data for our specific use cases. We can connect a BI tool like Apache Superset with Dremio as a source for our BI Dashboard use case and run useful visual dashboards directly from our data.

### 2.1.4 Accessing the services

Now, let's verify that each service is working:

- **MinIO (Object Storage):** Open `http://localhost:9001` and log in with `admin/password`.
- **Dremio (Query Engine):** Open `http://localhost:9047` and create an admin account.
- **Spark (Data Processing):** Open `http://localhost:8888` to access Jupyter Notebook.
- **PostgreSQL (Operational DB):** Connect via a Postgres client using `localhost:5435`, user `myuser`, and password `mypassword`.
- **Superset (BI Dashboards):** Open `http://localhost:8088` and log in with `admin/admin`. We will initialize Superset later on.

Now that our lakehouse environment is running, it's time to use it!

## 2.2 Creating Iceberg tables in Spark

Now that we have our Apache Iceberg Lakehouse environment, it's time to begin working with accurate data. In this journey stage, we'll simulate the data that typically lives in operational systems like a transactional PostgreSQL database. For this exercise, the data represents sales transactions in a retail environment, including product details, sales amounts, customer demographics, and store locations. This dataset provides a rich foundation for downstream analytics, allowing us to explore trends in customer behavior, campaign effectiveness, and regional performance.

The first step is to extract this data from PostgreSQL. This could involve connecting to production systems that support business applications such as point-of-sale systems, CRMs, or e-commerce platforms in real-world scenarios. Here, we're simulating that environment locally using a PostgreSQL instance populated with sample sales data. It's essential to consider how frequently this data changes and whether you're performing batch loads, micro-batches, or streaming ingestion—factors that influence your architecture decisions later on. This will be covered in our later chapter on ingestion.

Next, we use Apache Spark to transform and load this data into Apache Iceberg tables. Apache Iceberg offers powerful features such as schema evolution, time travel, and partitioning, essential for managing large-scale, analytic-ready datasets in a data lakehouse.

Once the data is loaded, we verify that it is correctly stored in our object storage backend, MinIO, and that the Nessie Iceberg catalog tracks each operation. Nessie acts as our catalog tracking and governing our Iceberg tables, so any tool that wants to work with our tables can easily discover and access them, enabling easy tool interoperability. This mirrors the workflow data engineers follow in modern lakehouse architectures: ingesting data from transactional sources into an open, governed, and scalable storage format, ready for analytics, reporting, or even AI/ML training pipelines.

## 2.2.1 Populating the PostgreSQL database

Our PostgreSQL container is already running from our docker-compose up command. Now, let's populate it with sample data representing a sales dataset. The data we are populating inside PostgreSQL represents the data our operational systems may generate. We want to move this data into analytical systems optimized for analytical workloads like an Iceberg lakehouse.

You can also find these SQL statements in the book's GitHub repo at:

<https://github.com/AlexMercedCoder/architecting-an-apache-iceberg-lakehouse/blob/main/ch2/postgres.sql>

Run the following command in your terminal to enter the Postgres shell:

```
docker exec -it postgres psql -U myuser mydb
```

Once inside, create a sales\_data table with mock data with the following SQL:

```
CREATE TABLE fashion_sales (
  id SERIAL PRIMARY KEY,
  product_name VARCHAR(255),
  category VARCHAR(50),
  sales_amount DECIMAL(10, 2),
  sales_date DATE,
  store_location VARCHAR(100),
  customer_age_group VARCHAR(50),
  campaign_name VARCHAR(100)
); #A

INSERT INTO fashion_sales (product_name, category, sales_amount, sales_date,
store_location, customer_age_group, campaign_name)
VALUES
  ('Slim Fit Jeans', 'Denim', 89.99, '2024-03-01', 'New York', '18-24', 'Spring
Launch'),
  ('Leather Jacket', 'Outerwear', 249.99, '2024-03-01', 'Los Angeles', '25-34',
'Spring Launch'),
  ('Graphic T-Shirt', 'Tops', 39.99, '2024-03-02', 'Chicago', '18-24', 'March
Madness'),
  ('Summer Dress', 'Dresses', 129.99, '2024-03-03', 'New York', '35-44', 'March
Madness'),
  ('Casual Sneakers', 'Footwear', 99.99, '2024-03-03', 'Los Angeles', '25-34',
'Spring Launch'); #B
```

**#A** Create our fashion\_sales table that will hold our data

**#B** Add a few records of data to the fashion\_sales table

Exit the Postgres shell with the following command:

```
\q
```

Our PostgreSQL database has a dataset ready to be ingested into Iceberg tables.

### 2.2.2 Starting the Apache Spark environment

Now, we'll use Apache Spark, a popular open-source data processing tool, to extract data from PostgreSQL and write it into Iceberg tables. Before working with Spark, we need to create a storage location for our Iceberg tables. MinIO will serve as our object storage backend for this purpose. Open your browser and navigate to <http://localhost:9001> to access the MinIO console. Log in using the credentials admin for the username and password for the password. Once logged in, create a new bucket named warehouse—this is where Spark will write the Iceberg table data.

Spark already runs in our Docker environment and includes Jupyter Notebook for interactive data processing.

- Open your browser and go to <http://localhost:8888>
- Create a new Python notebook to run our PySpark script (Python-based Spark Code).

You will need the IP address of the Minio container; run the following commands to get the IP address in a terminal:

```
docker network ls
```

This will get you the list of your networks; find the correct network name, which should be a mix of the directory name your docker-compose file is in and the network name at the bottom of the docker compose file. Once you have the network name, run the following command:

```
docker network inspect <network_name>
```

Look for the IPv4 address for the minio container and note it for our next step. This address is the unique address of this container on the docker network. Usually, it is sufficient to use the container name, for example, <http://minio:9001>, to address the container. Still, sometimes the Docker DNS (domain name service) may not resolve these names correctly, which tends to be the case for minio in this exercise. To fix this issue, we use the IP address to directly point our PySpark script to where the Minio container runs on the network so that they can communicate.



```

"Containers": {
  "05c1c8a2d57315e83bd51ab8f645cea493db62b1092ed3d4fcc5326c6d54bf33": {
    "Name": "minio",
    "EndpointID": "b81a31c524afed31304cbb027e3302952e70615a01f47d9815879a272ace6ca2",
    "MacAddress": "02:42:ac:12:00:02",
    "IPv4Address": "172.18.0.2/16",
    "IPv6Address": ""
  },

```

**Figure 2.5 Inspecting the Docker Network to Find the IP address of different containers**

## 2.2.3 Configuring Apache Spark for Iceberg

In the Jupyter Notebook, paste and run the following Python code, arguably the most popular language for writing Spark code versus writing in Spark's original Scala/Java API (make sure to put the minio IP address where noted).

In this Python code, we load the Spark libraries and set up variables with the network addresses of our catalog and storage layer, along with the specific address where we want to store the data to be written to our storage, expressed as an S3 address. We then create configurations of our Spark session that allow Spark to be aware of our catalog and storage settings, and then we start the session, allowing us to send directions to Spark on what it should do for our job.

To view the complete code, go to: <https://github.com/AlexMercedCoder/architecting-an-apache-iceberg-lakehouse/blob/main/ch2/example.py>

```

import pyspark from pyspark.sql import SparkSession #A

## DEFINE VARIABLES #B
CATALOG_URI = "http://nessie:19120/api/v1"
WAREHOUSE = "s3://warehouse/"
STORAGE_URI = "http://<minio_ip>:9000"

#A Import the pyspark library to send Spark directions
#B Define variables we will use in our Iceberg catalog configuration

```

This Python code snippet initializes configuration variables for use in a PySpark-based data pipeline that interacts with an Apache Iceberg lakehouse setup.

- `import pyspark` and `from pyspark.sql import SparkSession` brings in PySpark libraries required to create and manage Spark sessions, which are the entry point for Spark DataFrame operations.
- `CATALOG_URI` defines the REST endpoint for the Nessie catalog service, which is responsible for tracking metadata and versions of Iceberg tables. The URI points to the Nessie container on the local Docker network.
- `WAREHOUSE` specifies the root location in object storage where Iceberg tables will physically reside. It uses the S3-compatible path syntax and references a bucket named `warehouse`, which is assumed to be hosted on the local MinIO service.

- `STORAGE_URI` defines the base address for accessing the MinIO object store. This allows Spark to connect to MinIO using the S3 API for reading and writing table data. We will need the Minio Service IP address, which we retrieved earlier.

Together, these variables configure Spark to use Nessie for table versioning and MinIO as the Iceberg-compatible storage layer.

```
conf = (
    pyspark.SparkConf()
        .setAppName('Iceberg Ingestion')
        .set('spark.jars.packages',
            'org.postgresql:postgresql:42.7.3,'
            'org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.5.0,'
            'org.projectnessie.nessie-integrations:nessie-spark-extensions-3.5_2.12:0.77.1,'
            'software.amazon.awssdk:bundle:2.24.8,'
            'software.amazon.awssdk:url-connection-client:2.24.8')
        .set('spark.sql.extensions',
            'org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions,'
            'org.projectnessie.spark.extensions.NessieSparkSessionExtensions')
        .set('spark.sql.catalog.nessie', 'org.apache.iceberg.spark.SparkCatalog')
        .set('spark.sql.catalog.nessie.uri', CATALOG_URI)
        .set('spark.sql.catalog.nessie.ref', 'main')
        .set('spark.sql.catalog.nessie.authentication.type', 'NONE')
        .set('spark.sql.catalog.nessie.catalog-impl',
            'org.apache.iceberg.nessie.NessieCatalog')
        .set('spark.sql.catalog.nessie.s3.endpoint', STORAGE_URI)
        .set('spark.sql.catalog.nessie.warehouse', WAREHOUSE)
        .set('spark.sql.catalog.nessie.io-impl',
            'org.apache.iceberg.aws.s3.S3FileIO')
    ) #A
```

**#A create spark configuration setting up our Apache Iceberg Catalog**

This block of code sets up a customized Spark configuration for ingesting data into Apache Iceberg tables, using Project Nessie as the versioned catalog and MinIO as the S3-compatible storage backend. It begins by creating a SparkConf object and setting the application name to "Iceberg Ingestion." The `spark.jars.packages` property loads a set of required libraries: the PostgreSQL JDBC driver for reading from a Postgres source, Iceberg's Spark runtime for table operations, Nessie's Spark extensions for catalog integration, and AWS SDK components for interacting with S3-compatible storage like MinIO.

Next, the code registers the necessary Spark session extensions to enable Iceberg and Nessie functionality. It defines a logical catalog named `nessie` and specifies that it should use the Iceberg `SparkCatalog` class with Nessie-specific settings. These include the REST URI for the Nessie server, a reference branch (`main`), and a disabled authentication mode for simplicity. The catalog is configured to use the Nessie-specific implementation for Iceberg and access storage through the AWS S3-compatible `S3FileIO` interface, pointing to the defined MinIO endpoint and warehouse directory. Together, these settings enable a Spark job to write data into Iceberg tables managed by Nessie and store it in MinIO, with full support for features like time travel, branching, and atomic commits.

```
spark = SparkSession.builder.config(conf=conf).getOrCreate()  
print("Spark Running")
```

This code initializes a Spark session using the previously defined custom configuration `conf`, which includes all the necessary settings to interact with Apache Iceberg, Project Nessie, and MinIO. The `SparkSession.builder.config(conf=conf).getOrCreate()` call creates a new Spark session if one doesn't already exist, or retrieves the existing session if it does. This session is the main entry point for interacting with Spark's APIs, enabling operations like reading from and writing to Iceberg tables. The `print("Spark Running")` line confirms that the Spark session has been successfully started, signaling that the environment is ready for data processing tasks.

## 2.2.4 Loading data from PostgreSQL into Iceberg

Now, let's connect to PostgreSQL, read the `sales_data` table, and write it into an Iceberg table in our data lake. The data lake (MinIO in our example) is our storage repository for our data files, typically using a distributed storage solution like HDFS or Object Storage.

Paste the following code into a new cell in Jupyter Notebook and run it:

```
# Define the JDBC connection properties
jdbc_url = "jdbc:postgresql://postgres:5432/mydb" #A
properties = {
    "user": "myuser",
    "password": "mypassword",
    "driver": "org.postgresql.Driver"
} #B

# Read the sales_data table from Postgres into a Spark DataFrame
sales_df = spark.read.jdbc(url=jdbc_url, table="fashion_sales",
properties=properties) #C

# Show the first few rows of the dataset
sales_df.show() #D
```

**#A** Save connection string to postgres database in a variable

**#B** Connection properties for our database connection

**#C** Use connection to pull dataset into a dataframe

**#D** Print dataset to make sure it loaded correctly

You should see an output displaying the PostgreSQL sales data, as in figure 2.6.

```
[2]: # Define the JDBC connection properties
jdbc_url = "jdbc:postgresql://postgres:5432/mydb"
properties = {
    "user": "myuser",
    "password": "mypassword",
    "driver": "org.postgresql.Driver"
}

# Read the sales_data table from Postgres into a Spark DataFrame
sales_df = spark.read.jdbc(url=jdbc_url, table="fashion_sales", properties=properties)

# Show the first few rows of the dataset
sales_df.show()
```

| id | product_name    | category  | sales_amount | sales_date | store_location | customer_age_group | campaign_name |
|----|-----------------|-----------|--------------|------------|----------------|--------------------|---------------|
| 1  | Slim Fit Jeans  | Denim     | 89.99        | 2024-03-01 | New York       | 18-24              | Spring Launch |
| 2  | Leather Jacket  | Outerwear | 249.99       | 2024-03-01 | Los Angeles    | 25-34              | Spring Launch |
| 3  | Graphic T-Shirt | Tops      | 39.99        | 2024-03-02 | Chicago        | 18-24              | March Madness |
| 4  | Summer Dress    | Dresses   | 129.99       | 2024-03-03 | New York       | 35-44              | March Madness |
| 5  | Casual Sneakers | Footwear  | 99.99        | 2024-03-03 | Los Angeles    | 25-34              | Spring Launch |

**Figure 2.6 Loading the Data from the Source System into a Dataframe**

Now, let's write this data into an Apache Iceberg table tracked by Nessie. The commands below will create a "namespace," which, like a subfolder on your computer, is a unit for organizing datasets. Then we will take the table for Postgres, which we saved to a Spark dataframe, and write that to an Iceberg table, write data files for our Minio storage, and update Nessie. Hence, Nessie knows the table exists and where the file location of its metadata is:

```
#Create a namespace
spark.sql("CREATE NAMESPACE nessie.sales;") #A

# Write the DataFrame to an Iceberg table in the Nessie catalog
sales_df.writeTo("nessie.sales.fashion_sales").createOrReplace() #B

# Verify that the data was written to Iceberg by reading the table
spark.read.table("nessie.sales.fashion_sales").show() #C
```

#A Create a namespace called "sales" to organize datasets under  
 #B Write the postgres data to a new table in that namespace called "fashion\_sales"  
 #C Print the new Iceberg table to ensure it was created correctly

If successful, you should see the same sales data stored in Iceberg, as shown in figure 2.7!

```
#Create a namespace
spark.sql("CREATE NAMESPACE IF NOT EXISTS nessie.sales;")

# Write the DataFrame to an Iceberg table in the Nessie catalog
sales_df.writeTo("nessie.sales.fashion_sales").createOrReplace()

# Verify that the data was written to Iceberg by reading the table
spark.read.table("nessie.sales.fashion_sales").show()
```

| id | product_name    | category  | sales_amount | sales_date | store_location | customer_age_group | campaign_name |
|----|-----------------|-----------|--------------|------------|----------------|--------------------|---------------|
| 1  | Slim Fit Jeans  | Denim     | 89.99        | 2024-03-01 | New York       | 18-24              | Spring Launch |
| 2  | Leather Jacket  | Outerwear | 249.99       | 2024-03-01 | Los Angeles    | 25-34              | Spring Launch |
| 3  | Graphic T-Shirt | Tops      | 39.99        | 2024-03-02 | Chicago        | 18-24              | March Madness |
| 4  | Summer Dress    | Dresses   | 129.99       | 2024-03-03 | New York       | 35-44              | March Madness |
| 5  | Casual Sneakers | Footwear  | 99.99        | 2024-03-03 | Los Angeles    | 25-34              | Spring Launch |

**Figure 2.7 Writing Data to an Iceberg table and Reading it**

## 2.2.5 Verifying data storage in MinIO

Since we use MinIO as our object storage, let's check if our data is stored correctly.

- Open <http://localhost:9001>
- Log in with admin/password
- Click on the warehouse bucket
- Navigate to sales/sales\_data/

You should see the Parquet files containing our data and metadata files in JSON and AVRO. The Parquet files represent our data, while the JSON files are the iceberg’s “Metadata” files at the top of its metadata structure, which we covered in chapter 1. The AVRO files are the “Manifest” and “Manifest List” files within that structure. The data has been successfully ingested into Iceberg tables using Spark and stored as Parquet in our MinIO object store.



**Figure 2.8 Seeing the Iceberg Table Data in the Minio Data Lake**

Now that our Iceberg table is created, we can query it with Dremio, our powerful SQL engine for fast analytics on the data lake.

## 2.3 Reading Iceberg tables in Dremio

Now that we have successfully ingested data into Apache Iceberg using Spark, it's time to query our Iceberg tables using Dremio—a lakehouse query engine designed to deliver analytics on the data lake. Throughout this book, we will discuss other engines that integrate with Apache Iceberg, such as Trino and DuckDB, as we explore the Iceberg ecosystem in selecting tools for each layer of a lakehouse.

In this section, we'll walk through the process of connecting Dremio to the Nessie catalog, enabling us to access and explore our Apache Iceberg tables. With this connection in place, we'll begin querying the sales dataset we previously ingested, using standard SQL within Dremio's interface. This demonstrates how easy it is to work with Iceberg tables as if they were in a traditional data warehouse, without the need to move or replicate the data for modeling your warehouse.

As we run queries, we'll also explore how Apache Iceberg enhances performance through features such as metadata pruning. Iceberg maintains rich metadata about your tables, including partition information and file-level statistics, which enables Dremio to skip over unnecessary data during query execution. This results in faster query performance, even as your dataset scales. By the end of this section, you'll see how Iceberg enables fast, efficient analytics directly on your data lake, making it possible to deliver warehouse-like query performance without the complexity and cost of data duplication.

### 2.3.1 Starting Dremio

Dremio is already running as part of our Docker Compose setup. To access it:

1. Open your browser and go to `http://localhost:9047`
2. Log in with the admin account you created earlier

This will take you to the Dremio UI. Dremio allows us to connect to different data sources, run SQL against datasets in those data sources, and run queries across these sources, known as query federation. One type of source Dremio can connect to is lakehouse catalogs like Nessie, allowing Dremio to quickly discover and work with your Iceberg tables tracked by your catalog.

### 2.3.2 Connecting Dremio to the Nessie Catalog

In this case, Dremio needs to be connected to an Iceberg Catalog to query Iceberg tables, such as Nessie, which serves as our Iceberg catalog. We are running the Nessie container without configuring authorization, simplifying this exercise. In production, whichever catalog you use, you'll want to have any governance features used to control catalog users' access. We will explore the world of Iceberg catalogs and the different options available in the catalogs chapter of this book. Follow these steps:

1. Click "**Add Source.**"
2. Select "**Nessie**" from the available source types
3. Enter the following settings:

#### GENERAL SETTINGS

- **Source Name:** nessie
- **Nessie Endpoint URL:** `http://nessie:19120/api/v2`
- **Auth Type:** None

## New Nessie Source

General

Storage

Advanced Options

Reflection Refresh

 Nessie Source

Name

nessie

Nessie endpoint URL

http://nessie:19120/api/v2

Nessie authentication type

☒ None ☐ Bearer

No authentication is enforced on Nessie server.

Figure 2.9 Configuring Nessie General Settings

### STORAGE SETTINGS

- **AWS Root Path:** warehouse
- **AWS Access Key:** admin
- **AWS Secret Key:** password
- **Uncheck "Encrypt Connection"** (since we aren't using SSL)



**New Nessie Source**

---

General

Storage

Advanced Options
Reflection Refresh

Storage provider

☒ AWS
☐ Azure
☐ Google (Preview)

**S3 Storage**

AWS root path ⓘ

s3:// warehouse

**S3 Authentication**

Authentication method

☒ AWS Access Key
☐ EC2 Metadata
☐ AWS Profile
☐ No Authentication

AWS access key

admin

AWS access secret

.....|

**Figure 2.10 Configuring Nessie Storage Settings**

## CONNECTION PROPERTIES

- `fs.s3a.path.style.access = true`
- `fs.s3a.endpoint = minio:9000`
- `dremio.s3.compat = true`

These connection properties allow Dremio to connect to other “S3-compliant” storage layers as if they were AWS S3. Minio is an S3-compliant storage system. These settings enable us to connect to Minio, allowing Dremio to locate the files.

New Nessie Source

×

General

admin

Storage

AWS access secret

.....

Advanced Options

IAM role to assume

Reflection Refresh

Other

Connection properties

Name

Value

fs.s3a.path.style.access

true

×

Name

Value

fs.s3a.endpoint

minio:9000

×

Name

Value

dremio.s3.compat

true

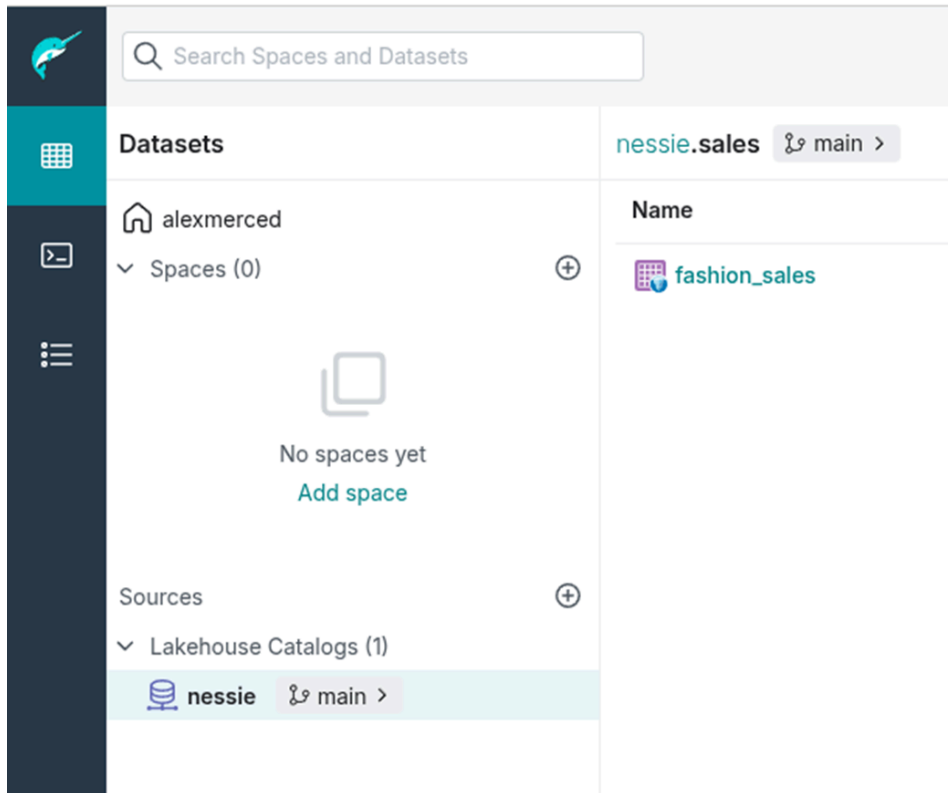
×

⊕ Add property

☐ Encrypt connection

**Figure 2.11 Configuring Nessie Connection Properties**

Click **"Save"** to add the Nessie source. Once added, Dremio will automatically detect the Iceberg tables registered in Nessie.



**Figure 2.12 Seeing Our Iceberg Tables from the Dremio Platform**

### 2.3.3 Querying Iceberg tables in Dremio

Now that Dremio is connected to Nessie, let's run some SQL queries on our Iceberg sales dataset. In the Dremio SQL Editor, run:

```
SELECT * FROM nessie.sales.fashion_sales;
```

You should see the same sales data we originally ingested from PostgreSQL, which is now stored as an Iceberg table in MinIO. One of Iceberg's most significant advantages is its metadata-driven pruning, which enables fast queries by skipping unnecessary data scans. Instead of scanning all the data in every file, the metadata of a table provides statistics on each file that allow engines like Dremio to skip reading files with no relevant data. For example, if only scanning for records whose `sales_amount` is 35, then the engine can skip a file whose metadata states that it has a range for this column of 38-40, because it would have no records being sought.

## CHECKING TABLE METADATA

To view one of the Iceberg metadata tables, run:

```
SELECT * FROM TABLE( table_snapshot( 'nessie.sales.fashion_sales' ) )
```

This data will show a list of snapshots, which are the historical versions of the table; their snapshot IDs can be used for time travel queries. Time travel queries allow us to query historical versions of the table. The information, like the `snapshot_id`, can be used for Iceberg features like Time Travel.

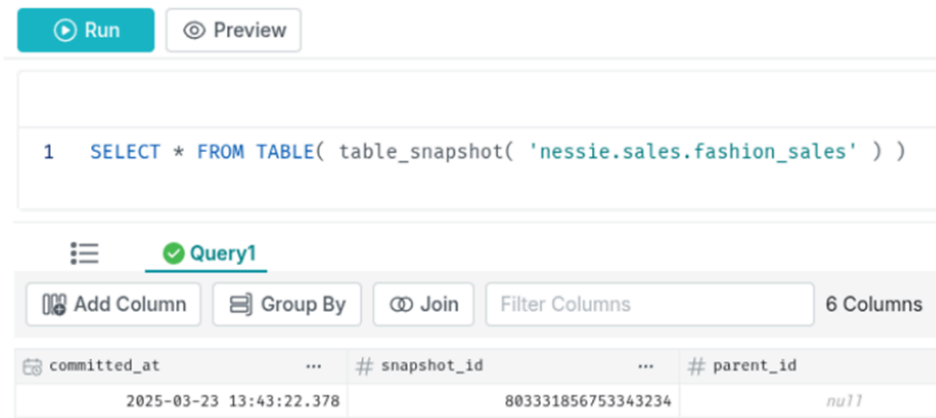


Figure 2.13 Querying a Table's Snapshot History in Dremio

## USING TIME TRAVEL WITH ICEBERG SNAPSHOTS

Iceberg supports time travel, allowing you to query historical versions of a dataset.

To time travel using a snapshot ID::

```
SELECT * FROM nessie.sales.sales_data AT SNAPSHOT <snapshot_id>;
```

To time travel based on a particular timestamp:

```
SELECT * FROM nessie.sales.sales_data AT TIMESTAMP <timestamp>;
```

Time travel queries can be used to debug data by examining previous versions of the table to help isolate errors. They can also be used for auditing and compliance by making historical data available for examination.

## 2.4 Creating a BI dashboard from your Iceberg tables

Now that we've successfully ingested data into Apache Iceberg and queried it using Dremio, it's time to take things a step further and build a BI dashboard to visualize insights.

In this section, we'll connect Apache Superset to Dremio, create a dataset from our Iceberg table, and design interactive charts and dashboards. By the end, you'll have a live BI dashboard powered by Dremio + Apache Iceberg, demonstrating the power of open lakehouse analytics.

### 2.4.1 Starting Apache Superset

Apache Superset is an open-source BI tool that allows users to create interactive dashboards and visualizations with SQL-based data sources. If Superset isn't already running, start it with:

```
docker-compose up -d superset
```

Then, make sure to initialize Superset, which will set up Superset's default users, metadata table, and roles with the following command:

```
docker exec -it superset superset init
```

Once it completes initializing, open your browser and go to <http://localhost:8088>. Login using the following credentials.

- Username: admin
- Password: admin

This will take you to the Superset home page, where we can connect to Dremio.

### 2.4.2 Connecting Superset to Dremio

We need to configure Dremio as a data source in Superset.

- Click **Settings Database Connections**
- Click + **Add Database**
- Select "**Other**" as the database type
- In the **Dremio URI**, enter::

```
dremio+flight://USERNAME:PASSWORD@dremio:32010/?UseEncryption=false
```

(Replace USERNAME and PASSWORD with your Dremio credentials.)

- Click Test Connection to ensure it's working.
- Click Save.

Dremio is now successfully connected to Superset! This URI uses Dremio's Arrow Flight protocol (more on communication protocols in chapters 7 and 8) for fast, efficient data access. Encryption is disabled for this local setup, but in production, enable `UseEncryption=true` for secure connections. Replace `USERNAME` and `PASSWORD` with your Dremio admin credentials.

## Connect a database

×

STEP 2 OF 2

### Enter Primary Credentials

Need help? Learn how to connect your database [here](#).

BASIC

ADVANCED

DISPLAY NAME \*

Dremio

Pick a name to help you identify this database.

SQLALCHEMY URI \*

dremio+flight://xxxxxxx:xxxxxx@dremio:32010/?UseEncryption=fals

Refer to the [SQLAlchemy docs](#) for more information on how to structure your URI.

TEST CONNECTION

Connect this database using the dynamic form instead ⓘ

ⓘ Additional fields may be required

Select databases require additional fields to be completed in the Advanced tab to successfully connect the database. Learn what requirements your databases has [here](#).

BACK

CONNECT

Figure 2.14 Connecting to Dremio from Superset

### 2.4.3 Creating a dataset from Iceberg tables

Now that we're connected, let's create a dataset from our Iceberg sales\_data table.

- Click the + button in the upper right Create Dataset
- Select Dremio as the database
- Navigate to nessie.sales.fashion\_sales
- Click Create Dataset


[Dashboards](#)
[Charts](#)
[Datasets](#)
[SQL ▾](#)

---

## fashion\_sales

DATABASE

dremio Dremio ▾

SCHEMA

nessie.sales ▾ ↺

TABLE

 fashion\_sales ▾ ↺

 **fashion\_sales**

**Table columns**

| Column Name ▴▾     |
|--------------------|
| id                 |
| product_name       |
| category           |
| sales_amount       |
| sales_date         |
| store_location     |
| customer_age_group |
| campaign_name      |

**Figure 2.15 Creating a New Dataset from Our Iceberg Table in Superset**

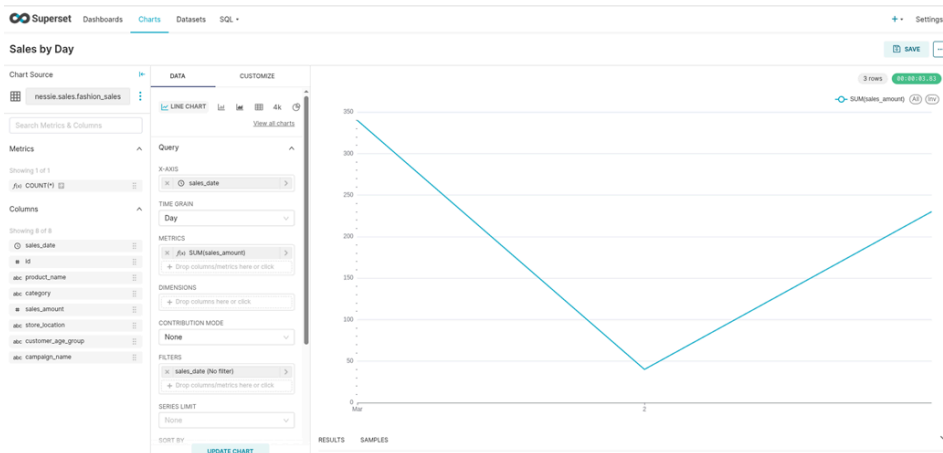
This makes our Iceberg table available for creating charts and dashboards.



## 2.4.4 Building charts and dashboards

Now, let's visualize our Iceberg data with some interactive charts. Apache Superset has a straightforward workflow for adding datasets from your sources, creating charts based on those datasets, and compiling those charts into a single dashboard that can then be shared with anyone who needs to view the data visually.

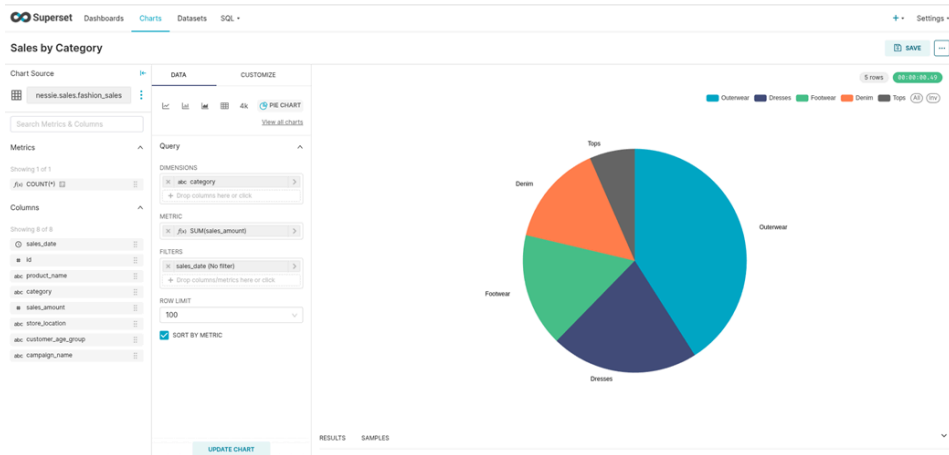
- Click the + button Create Chart
- Select the sales\_data dataset
- Choose a Line Chart
- Set:
  - X-Axis: sales\_date
  - Metric: SUM(sales\_amount)
- Click Run Query to preview the chart
- Click Save, name it Sales by Day, and add it to a new dashboard



**Figure 2.16 A line chart made from our Iceberg Data**

Let's create one more chart.

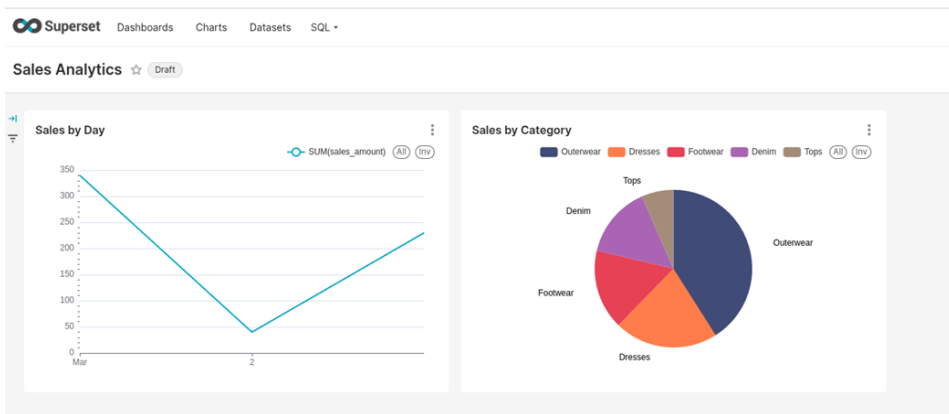
- Click + Create Chart
- Select sales\_data as the dataset
- Choose a Pie Chart
- Set:
  - Dimension: category
  - Metric: SUM(sales\_amount)
- Click Run Query to generate the chart
- Click Save, name it Sales by Category, and add it to the dashboard



**Figure 2.17 A Pie Graph made from our Iceberg Data in Superset**

Now that we've built multiple charts, combine them into a single dashboard.

- Click Dashboards + Create Dashboard
- Name it Sales Analytics
- Click + Add Chart, then select:
  - Sales by Day
  - Sales by Category
- Resize and arrange your charts
- Click Save & Publish



**Figure 2.18 A Dashboard built from our Iceberg Data in Superset**

You now have a fully functional BI dashboard powered by Apache Iceberg on your data lake! You may notice that working with tools like Apache Superset is not any different than working with any table in a database and data warehouse, and that is the point. You now have tables that can serve your existing workflows in an open format, usable across the entire enterprise without changing how you consume the data. Since it's Iceberg, any of the tools we used in this exercise can be swapped out for others, illustrating the value of Apache Iceberg's modular ecosystem.

To turn off the environment when you are done, run the following command:

```
docker compose down -v
```

The -V flag in this command instructs Docker to clean out the storage volumes it creates for each container, freeing up the storage that may be used when running these containers.

In the next chapter, we'll take a step back to evaluate your current data infrastructure and begin planning how to adopt Iceberg in a real-world environment, from defining architectural requirements to choosing the right components for a successful implementation.

## 2.5 Summary

- Apache Iceberg turns a traditional data lake into a high-performance analytics platform by providing rich metadata management, schema evolution, and ACID guarantees.
- A local Apache Iceberg lakehouse environment can be set up using Docker Compose to deploy services such as MinIO, Apache Spark, Nessie, Dremio, Superset, and PostgreSQL.
- MinIO acts as the object storage layer for Iceberg table data files and metadata files, simulating cloud-based storage locally.
- Project Nessie serves as a catalog, tracking Iceberg tables so that Apache Spark and Dremio can discover them.
- Apache Spark extracts sales data from a PostgreSQL operational database, transforms it, and loads it into Iceberg tables tracked by Nessie.
- Dremio connects to the Nessie catalog to query Iceberg tables directly from object storage to power our analytics workloads.
- Apache Superset is connected to Dremio to create interactive BI dashboards that visualize insights from the Iceberg lakehouse environment.

## ***3 Preparing for your move to Apache Iceberg***

### **This chapter covers**

- Performing an infrastructure audit
- Engaging stakeholders to surface technical and organizational needs
- Documenting current tooling, storage systems, and governance practices
- Converting audit findings into prioritized, actionable requirements
- Considerations for storage, ingestion, catalog, federation, and data consumption

Implementing Apache Iceberg is not simply a technical upgrade. It's a shift in how your organization handles data, from ingestion and storage to governance and analytics. While Chapter 2 gave you hands-on exposure to Iceberg's capabilities, moving to production requires more than experimentation. It demands a structured understanding of your current environment.

Every organization's data landscape is unique. You may be using different file formats, legacy ETL tools, or navigating region-specific compliance constraints, and attempting to integrate Iceberg into this environment without first conducting an audit. This approach risks introducing inefficiencies, unexpected costs, or governance gaps. A thoughtful audit helps you avoid these pitfalls by turning a broad set of options into a focused strategy.

But this chapter won't tell you which catalog to pick or which query engine is "best." That's by design, although we will explore the options for the lakehouse components in depth in future chapters. The lakehouse ecosystem is too diverse for a one-size-fits-all recommendation. Instead, our goal is to help you clarify your requirements. These will serve as a filter when evaluating tools, many of which are marketed to highlight strengths that may not necessarily align with your goals.

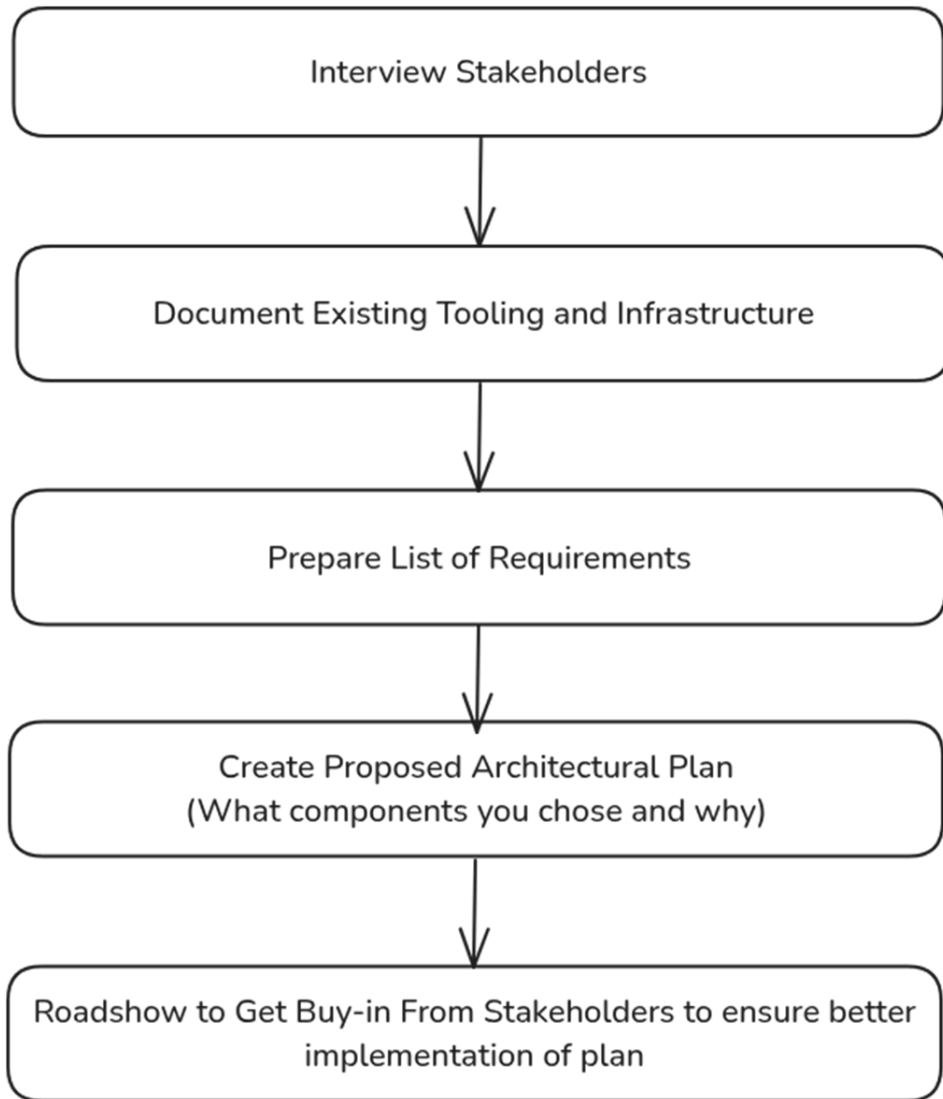
A successful audit begins by identifying the right stakeholders. Each team interacts with your data platform in a unique way, bringing essential perspectives. Data engineers can highlight brittle pipelines and technical debt. Analysts can speak to accessibility and freshness. Compliance teams understand regulatory requirements. Business leaders define the strategic outcomes the platform must support. By interviewing these groups early, you not only gather critical input, you also build a network of champions who can help drive adoption and remove roadblocks later.

This chapter will walk you through what to ask, whom to ask, and why each answer matters. To ground these ideas, we'll follow Hamerliva Bank Incorporated, a fictional financial firm navigating its own Iceberg transition. Their audit will serve as a practical guide to uncovering your own platform's constraints, strengths, and opportunities. By the end of this chapter, you'll have the foundation for a tailored Iceberg implementation—one rooted in reality, aligned with business needs, and ready to scale.

### **3.1 Conducting your data platform audit**

Before building a modern lakehouse, you need to know what you're working with—and what your organization actually needs. A successful Apache Iceberg implementation starts not with tools or infrastructure, but with a plan. This section introduces a structured audit process that surfaces technical constraints, organizational priorities, and opportunities for simplification. The audit is your first step toward building a list of actionable requirements that will shape your architectural decisions.

Figure 3.1 outlines the architectural journey from audit to implementation. Each phase builds on the last: stakeholder discovery leads to requirement gathering, which in turn drives your technical blueprint. Skipping the audit risks building a solution optimized for someone else's problem.

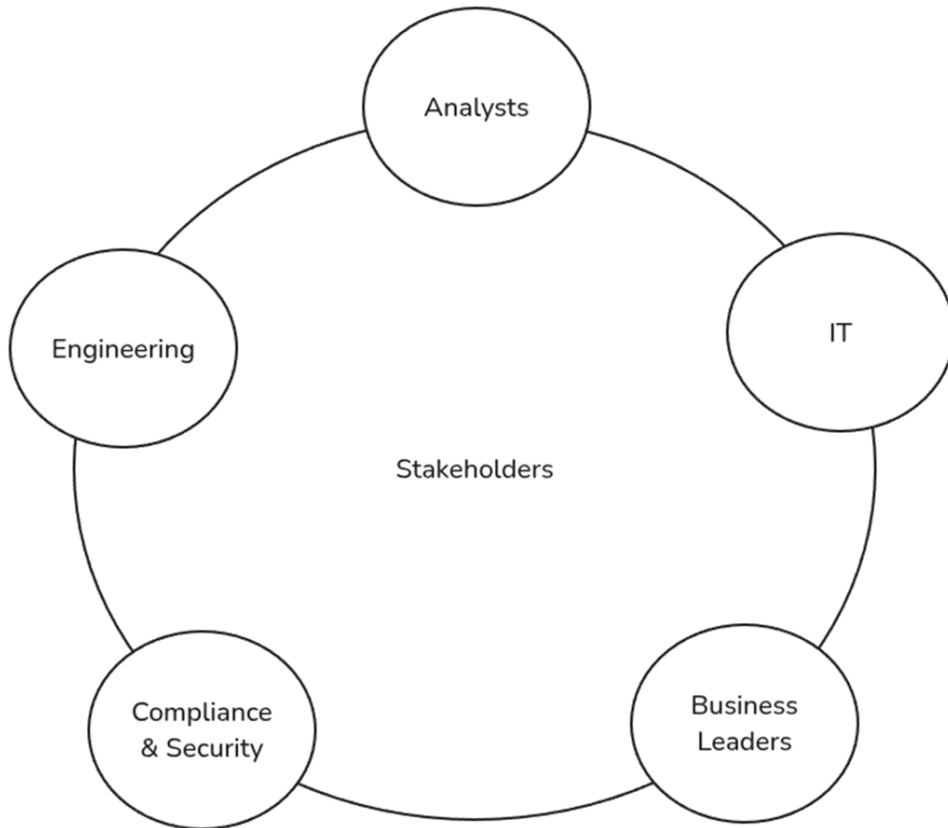


**Figure 3.1** The journey from gathering information, establishing requirements, and implementing a solution.

This work isn't about creating an exhaustive inventory of your entire data estate. It's about establishing firm convictions on what your platform must deliver to support your business goals. With that clarity, you'll be able to evaluate vendors, components, and deployment models with confidence, rather than being swayed by marketing claims that don't align with your priorities.

### 3.1.1 Who are the stakeholders?

To design a data platform that delivers measurable value, you must first understand who your platform serves. A lakehouse is not built for a single user or team—it is a shared foundation for data across departments, functions, and roles. Each group illustrated in figure 3.2 interacts with the platform differently, and each brings a unique set of expectations, constraints, and concerns. Identifying and engaging the right stakeholders at the start of your audit is critical. It ensures your implementation addresses not only technical feasibility but also organizational fit and long-term usability.



**Figure 3.2** Interviewing all your stakeholders will help you create the most holistic requirements possible.

One of the most central perspectives will come from the data engineering team. These engineers are responsible for designing, building, and maintaining the ingestion pipelines and transformation jobs that move data from raw sources into analytics-ready formats. Often, they are also tasked with managing the underlying storage infrastructure, whether on-premises or in the cloud. Their input is essential in uncovering the real costs of existing technical debt—systems that are brittle, overly complex, or ill-suited for growth. Engineers can also offer insight into which tools are working well, which ones impose limitations, and where the architecture may be introducing operational bottlenecks that impact the entire organization.

Data analysts and data scientists offer a different, but equally important, perspective. They are the consumers of the data platform—those who depend on clean, timely, and trustworthy data to drive decision-making. Analysts often reveal where the data value chain breaks down: slow refresh cycles, opaque data definitions, or inaccessible tables that require constant engineering intervention. These pain points can quietly erode confidence in the platform, leading to the development of shadow systems or duplicated effort. Engaging these users directly helps you understand how the current state of the platform affects business insight, speed to value, and trust in data.

Your audit should also bring in compliance and security professionals. In many industries, these stakeholders are responsible for ensuring that your data architecture complies with legal and regulatory standards. Whether it's data residency rules under GDPR, auditability requirements from financial regulators, or internal policies around encryption and access control, their input defines many of the non-negotiables in your architecture. These requirements must be accounted for from the start, not bolted on later. Without their buy-in, even technically sound solutions may stall in implementation due to unaddressed risk.

Business leaders and product owners help articulate the strategic objectives of your data platform. They are less concerned with which engine you use and more focused on what that engine enables: faster time to insight, lower cost of analytics, support for new customer-facing features, or increased operational agility. Their vision defines the outcomes your platform must support and will help you distinguish between features that are essential versus those that are merely nice to have. They also play a key role in securing funding, prioritizing resources, and championing adoption across teams.

Finally, the audit must account for the role of IT and infrastructure teams. These are the stewards of the hardware, networking, and core services that underpin your platform. Their responsibilities often include provisioning and securing cloud or on-prem resources, managing service-level agreements, and maintaining uptime. Their feedback helps you understand operational constraints, such as storage availability, network throughput, or support for hybrid deployment models. They are also the ones who will raise concerns about complexity, maintainability, and total cost of ownership if not consulted early.

Bringing these voices into the conversation from the outset isn't just about collecting input. It is about building alignment and shared ownership. By surfacing stakeholder priorities early, you reduce the risk of later surprises and increase the likelihood of smooth implementation. You also begin the process of building internal advocates—people who will help explain, support, and promote the transition to Apache Iceberg when the time comes to deploy. In the long run, this alignment is what transforms a technically successful project into one that actually gets adopted and delivers lasting value.





Discussions with analysts and data scientists should focus on data accessibility and trust. You should understand how long it takes these users to get the data they need to build reports, models, or dashboards. Delays in data access often reveal deeper architectural inefficiencies or gaps in automation. You should also investigate the level of trust analysts have in the data itself. If datasets are inconsistent, poorly documented, or prone to silent errors, users will often resort to copying data into personal silos, creating risk and fragmentation. Their feedback helps define expectations around data quality, freshness, and usability.

The conversations with compliance and security teams must address regulatory fit and governance capabilities. You need to explore whether the current architecture supports or complicates compliance efforts. Certain storage models or metadata systems may make auditability easy or extremely burdensome. You must ask what types of access controls, lineage tracking, and encryption standards are non-negotiable, and where the current platform either supports or falls short of these expectations. Understanding these constraints early allows you to bake compliance into the platform design rather than treating it as an afterthought.

When interviewing business leaders, the goal is to translate technical capabilities into strategic outcomes. Business sponsors are often less concerned with how the architecture is built and more focused on what it will allow the company to achieve. You must ask which analytics use cases are currently blocked by technical limitations. It is equally important to understand what a successful platform would enable: faster customer insights, reduced analytics costs, or support for new lines of business. Their answers define the value targets your platform must hit to justify investment.

Finally, you must engage IT and infrastructure teams to understand the operational boundaries within which your new platform must exist. Questions should explore whether there are current storage, compute, or networking limitations that any new architecture must respect. It is also important to assess the appetite for new operating models. Some organizations are open to fully cloud-native deployments; others prefer a hybrid approach or have regulatory requirements that necessitate maintaining on-premises infrastructure. These conversations help you anticipate constraints related to deployment (such as regional regulatory concerns and egress fees), cost control, and ongoing maintenance.

Through these discussions, you are not merely compiling a checklist. You are translating abstract goals into concrete, technical realities. You are also revealing interdependencies: decisions made about the ingestion engine will affect metadata management; catalog choices will influence governance options; storage decisions will impact query performance. Understanding these linkages early ensures that your Iceberg lakehouse will be architected deliberately, not reactively, setting the stage for a platform that is not only technically sound but strategically aligned with your business priorities.

### **3.1.3 Conducting a technological audit**

While stakeholder interviews reveal how your platform is experienced, a technological audit uncovers how it works. This phase complements your discovery conversations by examining the hard facts of your data environment: which systems are in use, how they interact, and where friction or misalignment may occur. Without this grounding in the current state, you risk architecting an Iceberg lakehouse that either duplicates past mistakes or overlooks practical constraints.

The goal of the audit is not simply to compile an inventory of tools and platforms. It is to map your current architecture to your organizational needs and identify which components can evolve naturally into an Iceberg-compatible ecosystem, and which will require rethinking. This assessment provides the architectural reality check that ensures future design decisions are informed, feasible, and context-aware.

Begin by documenting your storage architecture. This includes identifying all systems currently in use, such as Hadoop Distributed File System (HDFS), Amazon S3, on-premises network-attached storage (NAS), or cloud-native warehouses like Snowflake or BigQuery. It's essential to determine where data resides, what regulatory or geographic boundaries apply, and how each system is used in practice. For example, a cloud warehouse might handle ad hoc analytics, while HDFS supports archival or regulatory workloads. Each of these has different performance, cost, and compliance implications that will influence how Iceberg is introduced.

Next, evaluate your file formats. Many organizations have accumulated data in a patchwork of formats—such as CSV, JSON, Avro, and Parquet—often driven by the tools or vendor systems in place at the time. Understanding which formats are most prevalent and which are used in analytics pipelines versus those used for raw ingestion helps determine the scope of transformation needed. Iceberg is optimized for columnar formats like Parquet. If large portions of your data still live in row-based or semi-structured formats, you may need to plan conversion pipelines or define staging layers during ingestion.

Your ETL and ELT frameworks are another area of focus. Most modern stacks include a mix of batch and streaming jobs. Spark and Flink may already be in place, but other tools like NiFi or Informatica often persist in regulated or long-standing systems. By reviewing your current orchestration and transformation tools, you can identify which workloads are ready to move to Iceberg today and which may require rewrites, connector upgrades, or a phased transition strategy.

In addition to processing, metadata management and cataloging must be audited. Systems such as the Hive Metastore, AWS Glue, or custom catalog databases may currently handle schema definitions, ownership metadata, and access controls. These systems play a central role in how your data platform enforces governance and evolves schemas—core areas where Iceberg offers powerful capabilities. However, not all metadata systems are equally compatible with Iceberg, especially when it comes to advanced features like multi-table transactions (only available with REST Catalog compliant Iceberg catalogs). You'll need to assess whether your current approach can be extended or if a switch to a REST-based catalog, such as Apache Polaris or Project Nessie, will be required.

Finally, analyze the query engines and business intelligence tools that interact with your data. These systems—whether it's Dremio, Trino, Presto, or traditional BI tools like Tableau and Power BI—define how users consume data and what performance expectations exist. Understanding which engines are already in place and how they connect to your existing tables will inform your Iceberg deployment strategy and guide decisions around interoperability and query optimization.

To summarize, your technical audit should capture:

- *System landscape*—Storage layers, processing engines, catalogs, and query tools currently in use.
- *Operational context*—How these systems are configured, where they fall short, and what dependencies or constraints must be accounted for.

This phase is where architecture meets reality. By completing the audit, you gain a grounded view of your technical foundation—what can stay, what must change, and where Iceberg can create immediate value. Combined with the insights from stakeholder interviews, you are now positioned to define requirements that are not only visionary but executable.

## 3.2 Hamerliva Bank's audit in action

With the principles of a comprehensive audit now established, it is helpful to see how they play out in practice. In this section, we will follow Hamerliva Bank Incorporated, our fictional financial services firm, as they apply the audit framework to their environment. Hamerliva Bank's journey illustrates one possible path through the audit process—not a universal prescription, but an example of how a strategic and thorough approach can set the stage for a successful Iceberg adoption.

Hamerliva Bank faces a complex landscape. As a multinational firm operating under strict regulatory regimes in both Europe and North America, they must balance compliance, performance, and operational efficiency. Their platform supports everything from real-time trading analytics to regulatory reporting, and their challenges mirror those of many real-world organizations: siloed systems, costly pipelines, and data access barriers. Through their audit, Hamerliva Bank moves from a scattered status quo to a coherent, prioritized understanding of what their future lakehouse must deliver.

### 3.2.1 Hamerliva Bank interviews their stakeholders

The first step in Hamerliva Bank's audit process is to identify and interview stakeholders across the organization. Rather than assuming technology alone would dictate the architecture, they began by grounding their understanding in the needs, frustrations, and goals of the people who interact with their data.

Data engineers surfaced deep concerns around the fragility of existing pipelines. Many critical ingestion flows relied on legacy ETL tools that required manual intervention for schema changes, resulting in slow and error-prone updates. Engineers also noted that maintaining separate on-premises and cloud pipelines for different regions created redundant effort and increased operational risk.

Data analysts and scientists shared frustrations about the difficulty of accessing timely and trustworthy data. Analysts explained that preparing datasets for use often required informal, undocumented transformations, leading to inconsistencies between different reports. Data scientists pointed to the need for more real-time access to streaming data to improve the responsiveness of their machine learning models.

Compliance and security teams emphasized the critical need for comprehensive audit trails, strict adherence to data residency regulations such as GDPR for EU-based data, and encryption of data both at rest and in transit. Beyond these foundational requirements, they emphasized the importance of robust data access logging to support regulatory mandates, such as SEC Rule 17a-4, which requires the maintenance of immutable and auditable records of financial transactions. Systems lacking built-in lineage tracking or requiring extensive custom tooling to fulfill these obligations were flagged as operational and compliance risks.

Business leadership provided a strategic lens, articulating a need for faster insights, more agile experimentation, and a reduction in the overall cost of analytics. They highlighted opportunities to launch new client-facing data products if internal data accessibility and trust could be improved.

The IT and infrastructure teams clarified operational constraints. They underscored the challenges of scaling their on-premises environments and the need to support a hybrid deployment model that bridges regulated on-premises data with cloud-native analytics workloads.

By gathering these diverse viewpoints as summarized in table 3.1, Hamerliva Bank built a detailed map of both technical challenges and business opportunities. Importantly, they also began cultivating buy-in from across the organization, turning stakeholders into active participants in the upcoming platform transformation.

**Table 3.1 Stakeholder interview findings**

| Stakeholder Group             | Key Observations                                                                  | Challenges Highlighted                                                        |
|-------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Data Engineers                | Fragile, manually intensive pipelines                                             | Complex schema evolution, redundant regional pipelines, operational overhead  |
| Data Analysts and Scientists  | Slow and inconsistent access to trusted data                                      | Need for improved data freshness, clarity, and reduced manual preprocessing   |
| Compliance and Security Teams | Strong regulatory compliance needs around data residency, encryption, and lineage | Lack of built-in auditability and dynamic access controls                     |
| Business Leadership           | Demand for faster insights and lower analytics costs                              | Desire for new data-driven products is blocked by slow platform agility       |
| IT and Infrastructure         | Scalability and operational complexity challenges                                 | Hybrid cloud/on-prem needs; difficulties maintaining legacy on-prem resources |

### 3.2.2 Hamerliva Bank audits its technology

Armed with stakeholder input, Hamerliva Bank moved into the technical audit phase. Here, they sought not just to inventory their tools, but to understand how each piece of their environment would support—or hinder—their move to an Iceberg-based architecture.

Their storage landscape revealed significant fragmentation. On-premises HDFS clusters housed historical trading data, while on-prem NAS systems held vendor and compliance records. Several business units had independently adopted cloud data warehouses, such as Snowflake, resulting in duplicated datasets and inconsistent governance. A small but growing footprint in AWS S3 hinted at emerging cloud-native practices, particularly in newer analytics initiatives.

The file format audit revealed a significant reliance on CSV and JSON, particularly for data sourced from external vendors. While some newer datasets were stored in Parquet, large portions of high-value data required format conversion to fully leverage Iceberg's performance optimizations.

In terms of processing frameworks, Hamerliva Bank's stack was a blend of the old and the new. Apache Spark powered many of their batch ETL pipelines, especially in the European division. Meanwhile, other tools, such as Informatica and NiFi, were still central to regulatory data processing in the U.S. Real-time ingestion was handled through Kafka pipelines, but the data often arrived in custom formats that did not integrate smoothly with downstream analytics.

Metadata and governance systems were equally fragmented. Hive Metastore managed schema metadata for on-prem HDFS environments, while ad hoc relational databases stored lineage and audit information separately. Schema changes were manual and often risky, leading to occasional downtime and slow innovation.

Finally, their query and BI tools revealed a diverse but uncoordinated ecosystem. Dremio and Presto supported exploratory analysis on raw data lakes and dashboards on that data. At the same time, Tableau, Power BI, and Snowflake were the preferred platforms for executive dashboards and financial reporting. However, data duplication and inconsistent semantics across tools frequently led to confusion and inefficiency.

This technical audit, as summarized in table 3.2, confirmed many of the pain points stakeholders had raised. It also illuminated the depth of the transformation needed: success would not come from incremental improvement, but from rethinking how storage, processing, governance, and analytics were organized and integrated.

**Table 3.2 Technical audit findings**

| Domain                  | Current State                                     | Pain Points Identified                                                      |
|-------------------------|---------------------------------------------------|-----------------------------------------------------------------------------|
| Storage                 | HDFS, NAS, S3, Snowflake, BigQuery                | Fragmentation, duplicated data, and inconsistent governance                 |
| File Formats            | CSV, JSON dominant; Parquet used in newer systems | Performance bottlenecks, widespread need for format standardization         |
| ETL/ELT Frameworks      | Spark, Kafka, NiFi, Informatica                   | Legacy tool reliance, batch/stream silos, inconsistent transformation logic |
| Metadata and Governance | Hive Metastore, relational metadata stores        | Manual schema changes, limited lineage tracking, governance inconsistencies |
| Query and BI Tools      | Dremio, Presto, Tableau, Power BI, Snowflake      | Data duplication across tools, inconsistent semantics, and governance gaps  |

### 3.2.3 Hamerliva Bank summarizes its audit findings

With both stakeholder input and a comprehensive technical audit completed, Hamerliva Bank compiled its findings into a structured report. The goal was not to prescribe solutions or lock in architectural decisions, but to clarify the current state of their data platform and identify the core challenges that needed to be addressed in any future architecture. The report served as a centralized reference for aligning internal teams and preparing for the next phase: defining requirements.

The report was organized into four major domains—storage, processing, governance, and data consumption—each reflecting a different layer of the data platform. For each domain, the team identified observed pain points, operational inefficiencies, and questions that needed to be addressed before moving forward.

In the storage layer, they documented the fragmented nature of their infrastructure. Data spanned HDFS, on-prem NAS, and multiple cloud warehouses. Duplication of datasets across these environments had led to inconsistencies in governance, increased costs, and additional operational overhead. There were open questions about data residency, integration complexity, and long-term scalability.

In the data processing layer, the audit revealed a mix of modern and legacy tooling. While Spark and Kafka supported parts of their workflows, older ETL systems remained embedded in critical pipelines. The coexistence of batch and streaming flows across teams highlighted the need to unify ingestion strategies, while also raising concerns about compatibility and operational maturity.

Governance and metadata management surfaced as one of the most fragmented areas. Schema definitions were managed inconsistently across Hive Metastore, internal databases, and ad hoc documentation. Manual processes dominated schema evolution, and tracking lineage or enforcing data access policies often required custom scripts or external coordination. These issues introduced friction during audits and raised concerns about compliance and the trustworthiness of data.

Finally, the query and consumption layer was marked by tool sprawl and semantic inconsistency. Multiple engines and business intelligence (BI) platforms were in use, each with its own set of assumptions and permissions. Business users often had conflicting views of the same data, leading to duplicated effort and prolonged validation cycles. While these tools were practical in isolation, their integration with upstream systems varied widely, limiting end-to-end visibility.

**Table 3.3 Findings of pre-architectural interviews and audit**

| Domain           | Key Findings                                                               | Problems to Address                                                                  |
|------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Storage          | Fragmented infrastructure across HDFS, NAS, and cloud warehouses           | Data duplication, inconsistent governance, and unclear long-term storage strategy    |
| Processing       | Mix of Spark, Kafka, NiFi, and Informatica across batch and streaming jobs | Tool incompatibility, redundant pipelines, and manual schema coordination            |
| Governance       | Metadata scattered across Hive, relational DBs, and manual processes       | Manual schema evolution, audit complexity, and inconsistent access controls          |
| Data Consumption | Use of Dremio, Presto, Snowflake, Tableau, and Power BI                    | Semantic drift across tools, multiple data copies, and inconsistent user experiences |

### 3.3 From audit to requirements: Laying the foundation for design

With your audit complete, the next step is translating those findings into a structured set of requirements that will guide the design of your Iceberg lakehouse. This isn't the stage for selecting tools or locking in vendors. Instead, it's about defining what success looks like—both technically and organizationally—so you can later evaluate options with clarity and confidence.

At the center of this translation process are use cases. Use cases serve as the bridge between business goals and architectural choices. While your audit surfaced pain points, constraints, and existing systems, your requirements must reflect how the platform will be used to create value. For example, enabling sub-second dashboard refreshes for investment analysts is a use case. Supporting nightly regulatory audit extracts is another. These use cases define the operational scenarios your architecture must support and shape the technical requirements needed to meet those expectations.

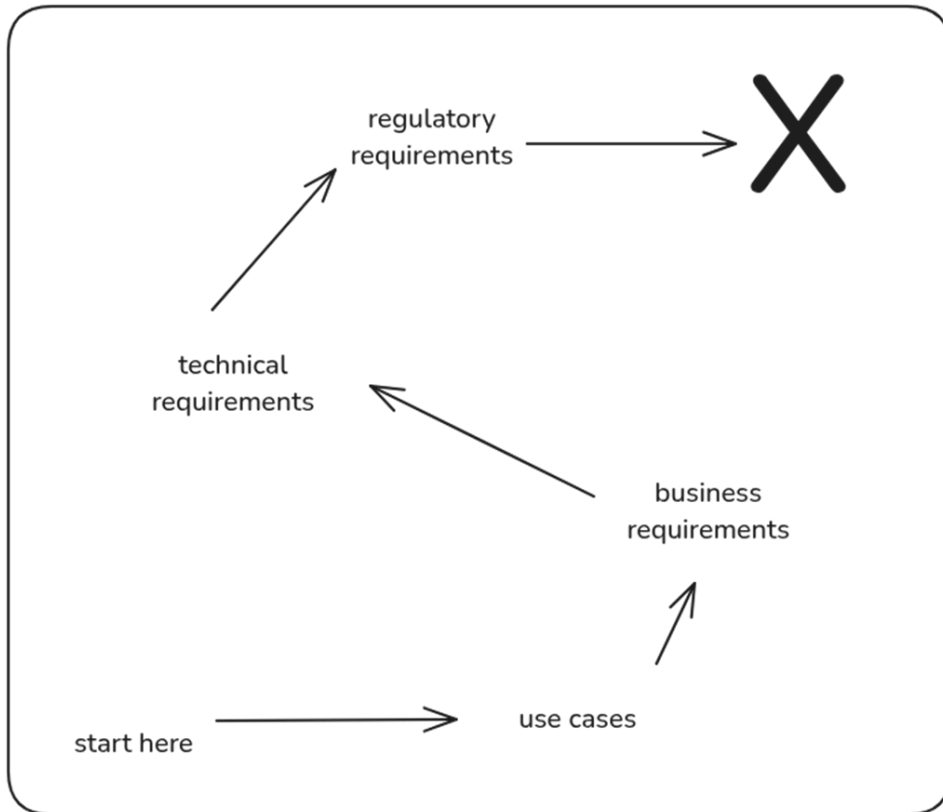
Once these use cases are understood, they can be mapped to the five foundational components of the Iceberg lakehouse: storage, ingestion, catalog, federation, and consumption. For each of these, you'll define specific outcomes that your data platform must deliver. These outcomes become your requirements—decision filters that ensure you assess architectures and tools based on how well they address your real-world needs, rather than relying on feature lists or vendor claims.

A requirement might state: “Data residency must be enforced at the region level” (a storage outcome) or “Streaming ingestion must support incremental upserts with no downtime” (an ingestion need tied to a low-latency use case). These requirements should be written in plain language, organized by component, and prioritized according to business impact and implementation complexity.

A well-crafted set of requirements will not only clarify design trade-offs but also provide a defensible rationale for the architectural decisions that follow. They connect stakeholder priorities to concrete technical goals and anchor your Iceberg deployment in the actual value it must deliver.



In the sections that follow, we'll explore how to define these requirements across each lakehouse component, guided by the use cases surfaced in your audit. This requirements framework sets the stage for building your architectural blueprint, as illustrated in figure 3.4. In later chapters, we'll examine each of these components in depth, exploring the tools, trade-offs, and design patterns that can help bring your lakehouse to life.



**Figure 3.4** Your audit acts as the map navigating your architectural decision.

### 3.3.1 Defining storage requirements

Storage is the physical foundation of your lakehouse. It determines not only where your data resides but also how it is governed, accessed, and scaled over time. Your storage requirements must be grounded in four key areas: compliance obligations, performance expectations, cost constraints, and operational realities. These factors will shape whether you lean toward cloud, on-premises, or a hybrid deployment, and your requirements should be crafted to support that decision with clarity, not bias.

Start by revisiting your audit results. What storage systems are currently in place? How is data distributed across environments? Are some systems tightly coupled to compliance-sensitive workloads, such as trading data governed by regional residency rules? Are others driving up costs due to redundancy or lack of tiering?

From these observations, begin to articulate requirements in outcome-focused terms. These requirements should be specific and testable, but not prescriptive about technologies or vendors. For example, if your audit uncovered regulatory needs, one requirement might be: “All PII data collected in the EU must remain physically stored within EU data centers.” If cost emerged as a concern, you might write: “Cold data not accessed within 90 days must be automatically migrated to low-cost archival storage.” These requirements express needs, not solutions, leaving flexibility for implementation while ensuring alignment with your constraints.

It’s also important to consider the broader trade-offs of different deployment models:

*Cloud-based storage* offers scalability, elasticity, and simplified maintenance. Object storage services, such as Amazon S3, Google Cloud Storage, and Azure Blob Storage, provide high durability and integrate well with Iceberg’s architecture. For organizations seeking to streamline infrastructure management or expand global access, cloud-native storage often aligns with their strategic goals. However, cloud introduces new challenges: data egress fees, latency for specific workloads, and more complex cost forecasting. If cloud storage is part of your architecture, your requirements should reflect operational goals such as “Storage must support tiering and archival for cold data,” “All data must be encrypted at rest and in transit,” or “Access must be auditable for compliance and incident response.”

*On-premises storage* remains essential for many industries, especially where data sovereignty, physical control, or regulatory restrictions are non-negotiable. In these environments, systems like HDFS or enterprise-grade object stores (e.g., MinIO, NetApp) may still serve as the foundation. The requirements for these deployments might include “System must support WORM (Write Once, Read Many) policies for regulatory compliance” or “Storage must be accessible from isolated network zones without public internet access.” While on-prem storage offers control, it also demands internal expertise, upfront capital investment, and a plan for long-term capacity management.

*Hybrid storage models* are becoming increasingly common as organizations strive to balance control with flexibility. These architectures allow specific datasets—often those with latency, cost, or compliance sensitivities—to remain on-premises, while offloading analytics or archival workloads to the cloud. Requirements in this scenario should address the interaction between environments. For example: “Storage must expose a consistent object store interface across cloud and on-prem systems,” or “Platform must support metadata and catalog integration across heterogeneous storage backends.”

Once you’ve written your storage requirements, prioritize them. Not every goal will be equally urgent. Some may be foundational to compliance or performance, such as encryption and access latency. Others may reflect long-term ambitions, such as automatic tiering and cross-region replication. Categorize your requirements into “critical,” “nice to have,” and “future-facing.” This exercise will help you navigate vendor discussions and architectural decisions with confidence. It will also provide stakeholders with a clear framework for evaluating trade-offs, ensuring that the final storage solution balances flexibility, compliance, cost, and performance in a manner that aligns with your organization’s needs.

Ultimately, the strength of your storage requirements will shape how resilient, efficient, and scalable your Iceberg lakehouse becomes. Whether you choose cloud, on-prem, or hybrid, your architecture should be a response to clearly stated needs, not assumptions or default choices.

### 3.3.2 Defining ingestion requirements

Ingestion is the front door to your lakehouse, making it one of the most strategic components to define early. Before selecting tools or defining latency expectations, start by identifying the types of source systems you'll need to integrate. Your ingestion requirements should account for the diversity and complexity of these sources, ranging from relational databases and APIs to SaaS platforms and event streams. Adding or updating sources is often the most difficult and time-consuming part of ingestion, requiring a deep understanding of source system behavior, data structures, and constraints on inserts, updates, and deletes.

Once sources are identified, the next step is understanding ingestion patterns: which data arrives via batch, which pipelines operate in real time, and what refresh frequencies different teams rely on. Just as critically, examine where current workflows break down—pipelines that fail under schema drift, hand-coded update processes, or jobs that bottleneck delivery. These insights shape not only tool selection but the broader design of a platform that's robust, scalable, and responsive to business needs.

One of the most critical dimensions to assess and express is data freshness. While it is tempting to treat streaming ingestion as the gold standard, not all datasets benefit from real-time pipelines, and not all teams are equipped to operate and maintain them. Streaming architectures introduce operational complexity: they require checkpointing, error recovery, backpressure handling, and often a separate set of tools from batch systems. Adopting streaming where it is not truly necessary may result in wasted engineering effort and a higher operational burden.

Instead, your requirements should reflect the actual decision-making cadence of your business. For example, if internal dashboards are updated once per hour and users are satisfied with that latency, batch ingestion may be the simplest and most cost-effective choice. On the other hand, if certain use cases—such as fraud detection, algorithmic trading, or scoring an ML model—require fresh data within seconds, you should define those latency targets precisely. A requirement like “Streaming data must be queryable within five seconds of event arrival” sets a clear engineering objective that can be validated in practice.

In some cases, hybrid ingestion models are appropriate. Apache Iceberg supports both batch and streaming ingestion patterns, allowing your platform to evolve from batch to near-real-time ingestion as needed, without requiring a re-architecting of your entire stack. A thoughtful requirement might state, “Ingestion must support concurrent batch and streaming updates to the same table with transactional consistency,” allowing teams to increase freshness while preserving operational stability gradually.

Beyond freshness, your ingestion requirements should define how your platform handles schema evolution, late-arriving data, and data corrections. These are common challenges in evolving pipelines, particularly when external sources deliver data or originate from operational systems that frequently update. Requirements like “Schema changes must be propagated without stopping pipelines across downstream systems” or “Ingestion must support upserts and deduplicated inserts from streaming sources” help ensure that your platform remains agile and reliable over time.

Operational concerns must also be captured. Ask: Who owns and maintains the ingestion jobs? What tools are used for orchestration? How is monitoring and alerting managed? In environments with multiple teams and pipelines, requirements should cover maintainability and integration. For instance, “Ingestion tools must integrate with an orchestration tool and support job lineage metadata,” or “Ingestion failures must be traceable and alertable via existing observability systems.”

Finally, pay attention to data integrity guarantees. Depending on your use case, you may need ingestion pipelines to enforce ordering, handle idempotency, or guarantee exactly-once delivery semantics. If these aspects are not defined up front, they often become sources of bugs, delays, complexity, costs, or silent data corruption.

Once documented, your ingestion requirements should fall into categories that strike a balance between technical rigor and business value. Some may be essential for regulatory or operational correctness, while others may serve as optimization targets over time. Prioritizing your requirements will help you evaluate ingestion frameworks, whether Spark, Flink, Kafka Connect, or vendor solutions, based primarily on their ability to support the source connectors you need, not just on general features or performance claims.

A well-defined ingestion strategy isn’t about choosing the most modern or powerful tool—it’s about ensuring that data flows into your lakehouse in a way that is reliable, maintainable, and aligned with how your organization makes decisions. With clear ingestion requirements, you create a blueprint for data movement that supports growth without introducing unnecessary complexity.

### 3.3.3 Defining catalog requirements

The catalog is the central nervous system of your Iceberg lakehouse. Every read or write operation—whether it comes from Spark, Flink, Trino, or a BI dashboard—relies on the catalog to discover, access, and modify Iceberg tables. It is the interface that makes your lakehouse not just a collection of files, but a governed, queryable, and evolving data platform. Because of this, selecting the right catalog and defining its requirements is one of the most strategic decisions you’ll make during your Iceberg implementation. At its core, the catalog manages metadata in the form of tracking table metadata references similar to what is illustrated in table 3.4.

**Table 3.4 Iceberg catalogs track metadata by tracking the location of metadata files**

| Table               | Metadata Location                                                                   |
|---------------------|-------------------------------------------------------------------------------------|
| sales_data_2024     | s3://company-data/warehouse/sales_data_2024/metadata/00004-abcdefg.metadata.json    |
| customer_profiles   | s3://company-data/warehouse/customer_profiles/metadata/00012-hijklm.metadata.json   |
| inventory_snapshots | s3://company-data/warehouse/inventory_snapshots/metadata/00003-nopqrs.metadata.json |
| web_traffic_logs    | s3://company-data/warehouse/web_traffic_logs/metadata/00008-tuvwxy.metadata.json    |

One of the most critical decisions when defining catalog requirements is determining whether your catalog must support the Iceberg REST Catalog specification. This standard, adopted by projects such as Apache Polaris, Project Nessie, Lakekeeper, Apache Gravitino, and supported by AWS Glue Data Catalog, ensures that your catalog can interoperate with the broadest range of compute engines and data tools in the Iceberg ecosystem. REST catalogs expose a standardized API that allows query engines and orchestration tools to read and write metadata in a consistent, engine-agnostic way.

In contrast, catalogs that do not support the REST spec, such as the traditional Hive Metastore or specific vendor-specific integrations, may have more limited compatibility. These catalogs can still function with Iceberg in some environments, but their support across modern engines may require custom adapters or compromise advanced functionality such as multi-table transactions. If your audit revealed a diverse ecosystem of tools or the desire to decouple storage from compute, Iceberg REST-compatible catalogs are likely a better fit.

Another key dimension of your catalog requirements is governance. The catalog often acts as the central control point for defining table-level access policies across engines. Support for role-based access control (RBAC) and attribute-based access control (ABAC), whether built-in or integrated with identity providers, can dramatically simplify and unify security management. Equally important is data discoverability: the ability for users to locate relevant datasets through metadata search, tagging, and lineage visibility. A well-governed catalog should not only enforce permissions but also enable analysts, engineers, and data scientists to find and understand the datasets available to them. Requirements might include: “Catalog must support RBAC integration with enterprise identity systems,” or “Catalog must expose robust permissions and searchable metadata to facilitate dataset discoverability.”

## CATALOGS AND FINE-GRAINED ACCESS CONTROLS

It's essential to note, however, that fine-grained access control (e.g., row-level or column-level security) is typically enforced by the query engine, rather than the catalog. While the catalog can help define and register policies or annotations, it's up to the engine (like Trino, Dremio, or Snowflake) to apply those rules at query time. Therefore, when assessing governance needs, it is essential to consider the interaction between the catalog and query engines in your architecture. A catalog might offer strong policy modeling, but without engine enforcement, it may fall short of your security goals.

Your catalog requirements should also address metadata resilience and traceability. Features such as time travel, rollback, and commit history are central to Iceberg's value proposition, and your catalog will often help maintain your table by coupling optimization and cleanup features. Requirements in this area may include: "The catalog must autonomously expire snapshots older than 30 days," or "Table access should be logged as auditable events."

Ultimately, you must decide whether your organization prefers to manage the catalog itself or utilize a managed catalog service. Open-source projects, such as Polaris, Nessie, Gravitino, Lakekeeper, or even the legacy Hive Metastore, can be deployed in-house, giving your teams complete control over updates, integration, and scaling. This model appeals to organizations with strong platform engineering teams and a desire for infrastructure independence. In contrast, managed services like AWS Glue Data Catalog, Dremio Catalog, or Snowflake's Open Catalog offer simplicity, scalability, and operational support out of the box—often at the cost of deeper customization or portability.

To summarize, your catalog requirements should clarify:

- *Interoperability:* Which query engines must be supported natively?
- *Governance:* What level of access control (RBAC/ABAC) must the catalog enable?
- *Deployment model:* Will the catalog be self-hosted or managed?

The catalog is not an afterthought. It is the heartbeat of your Iceberg lakehouse. With a strong set of catalog requirements, you ensure that your metadata layer is not only reliable and flexible but also a foundation for interoperability, governance, and long-term scalability.

### 3.3.4 Defining federation requirements

Even in mature data environments, it's rare for all data to live in one place. Whether due to regulatory constraints, organizational silos, or the pace of modernization, many organizations continue to operate with distributed data sources—some in cloud warehouses, others in on-premises relational databases, and some in legacy Hadoop environments. Your Iceberg lakehouse does not need to consolidate all data physically. But it must provide seamless access to that data logically. This is where federation requirements come into play.

Federation refers to the ability of your query engine to access and join data from multiple systems in real time, without requiring upfront movement or duplication. Federation refers to how well your platform connects to various systems, including operational databases, cloud APIs, and third-party SaaS platforms. Together, these capabilities enable you to present a unified data experience while avoiding the complexity, delay, and cost associated with centralizing every dataset.

Your audit should have revealed whether key workloads are dependent on data that spans environments. For example, suppose analysts routinely pull client profile data from an operational database and join it with analytics data stored in Iceberg. In that case, this is a strong signal that federated access is a requirement. Similarly, suppose teams are duplicating data across systems solely to enable access via a preferred business intelligence (BI) tool. In that case, federation can reduce complexity and cost by allowing direct cross-system queries.

Begin by articulating the requirements that define connectivity and coverage. For instance: “Query engine must support federated access across Iceberg, JDBC-compatible databases, and cloud object stores.” At a minimum, your platform should be able to interact with the systems uncovered in your audit, whether they are SQL-based data warehouses, NoSQL systems, or file-based object stores.

But federation is more than connectivity. To be effective, your platform must support semantic consistency. This means offering a semantic layer—a place where datasets are modeled, governed, and described in ways that business users understand. Some query engines provide this capability natively; others integrate with external semantic layers, such as dbt’s Semantic Layer, or tools like AtScale or Cube. Your requirement here might state: “Query engine must expose a unified semantic model across federated sources,” or “Semantic layer integration must support consistent metric definitions across Iceberg and Snowflake datasets.”

Performance is another critical factor. Federated queries are only helpful if they are performant enough to be interactive and scalable. This is where support for Apache Arrow becomes significant. Apache Arrow has emerged as the standard for high-performance in-memory columnar data interchange. Engines that use Arrow for data transport or execution, such as through Arrow Flight or ADBC, can deliver significantly faster performance and lower latency. When defining your requirements, consider specifying: “Federated query engine must support Arrow Flight for cross-system data transport,” or “Platform must support vectorized execution using Apache Arrow.”

You should also assess how the engine handles query acceleration, especially when accessing remote or heterogeneous sources. Does the engine support result caching, materialized views, or dynamic pruning? Can it push down filters, joins, or aggregations to the source systems? More advanced platforms offer cross-source acceleration, allowing optimizations to be applied even when querying across Iceberg and external systems within the same query plan. A requirement might state: “Query acceleration must apply across federated sources without requiring manual materialization.”

Your federation strategy should also include interfaces and application programming interfaces (APIs). A modern query engine should expose a set of standard data access interfaces to ensure compatibility with business intelligence (BI) tools, notebooks, and applications. This includes JDBC/ODBC drivers, REST APIs for query execution and platform management, as well as newer protocols such as Apache Arrow Flight and ADBC for high-performance access. You might write: “Platform must support JDBC/ODBC, REST, and Arrow Flight protocols for all federated datasets.”

Finally, consider how automation and operational management are handled. Federation introduces architectural complexity, and your platform must help manage this complexity through intelligent resource scaling, automatic metadata synchronization, and policy enforcement. Requirements in this space might include: “Federated queries must be automatically optimized and routed based on data location and source performance,” or “Platform must support autoscaling and monitoring of federated query workloads.”

Together, these requirements ensure that your Iceberg lakehouse can reach beyond itself, interacting with the broader data ecosystem without compromising performance, governance, or user experience. Federation is not just about convenience—it’s about building a unified analytical platform that reflects the reality of how your organization stores and uses data. With clear federation and integration requirements, you empower your teams to access and analyze data where it resides, while laying the foundation for future consolidation and simplification.

### 3.3.5 Defining consumption requirements

The final category encompasses how data is accessed and utilized—whether through business intelligence (BI) tools, interactive queries, or machine learning workflows. These requirements align closely with user experience and query performance.

Your audit should have revealed which tools users depend on and how they interact with data today. Are dashboards slow to refresh? Do analysts rely on manually exported datasets for modeling? Is access governed consistently across teams?

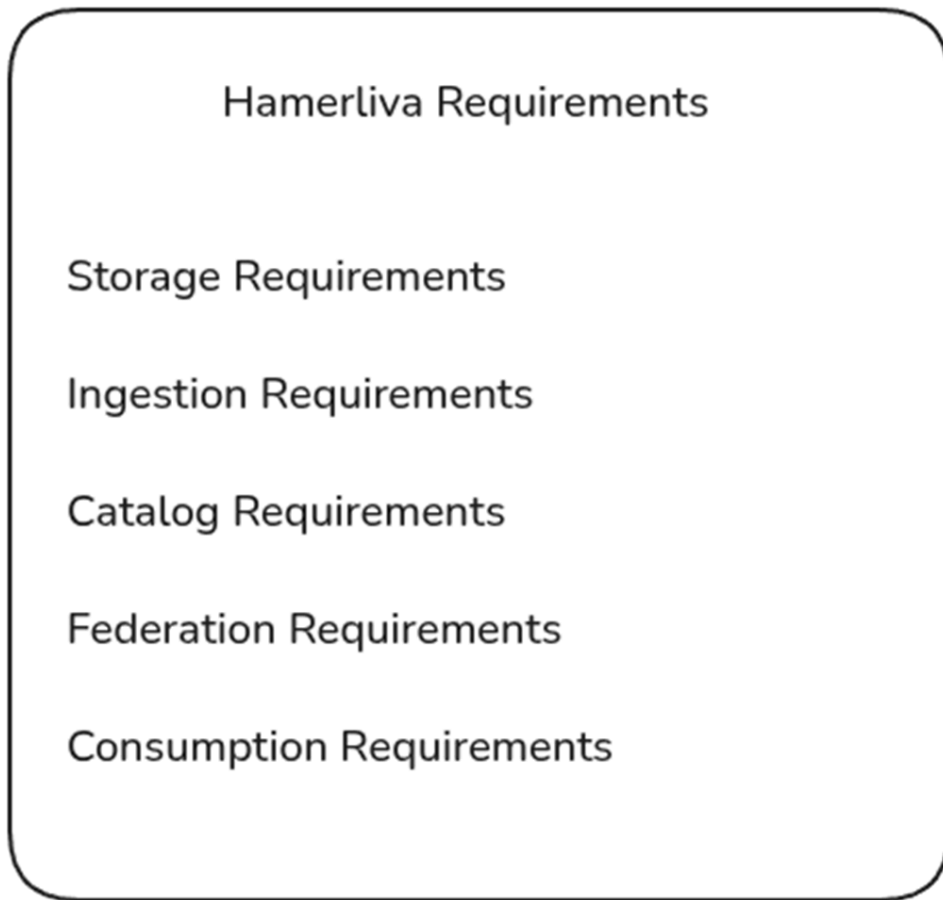
Use this information to define consumption requirements, such as: “Users must be able to securely access data from local notebooks outside the corporate network,” or “The lakehouse must support secure cross-region querying, including access to EU-stored datasets from US-based analysts.”

Also consider interoperability. If your users span Dremio, Trino, and Snowflake, your platform must ensure semantic consistency across engines. Your requirements should reflect this, stating what kind of tooling and interfaces users need, not just which tools they currently use.

### 3.3.6 Hamerliva Bank establishes its requirements

Having completed a thorough audit of their technical environment and stakeholder priorities, Hamerliva Bank Incorporated moved to the next critical step: converting insights into a formal list of architectural requirements. These requirements do not prescribe specific technologies or tools, but instead establish the criteria by which future design decisions would be evaluated across the five key Iceberg lakehouse components: storage, ingestion, catalog, federation, and consumption.





**Figure 3.5 Hamerliva Bank builds out its architectural requirements.**

## **STORAGE REQUIREMENTS**

Hamerliva Bank's audit revealed significant fragmentation in their storage infrastructure. Data lived across on-prem HDFS clusters, NAS systems, and multiple cloud warehouses. Regulatory compliance requirements, particularly the GDPR in the EU and SEC rules in the U.S., necessitated that specific datasets be physically isolated by geography. At the same time, storage duplication across regions and systems was driving up costs and complexity.

To address these needs, Hamerliva Bank defined requirements that emphasized compliance, consistency, and compatibility:

- "All regulated data collected in the EU must remain in regionally bound, on-premises storage."
- "Storage systems must expose S3-compatible APIs to ensure integration with all query engines and ingestion frameworks."

- “Cold data older than 60 days must be eligible for archival tiering, while remaining queryable through Iceberg metadata.”

These requirements provided a clear lens for comparing on-premises object store vendors and assessing the feasibility of hybrid cloud deployments without compromising their regulatory posture.

## INGESTION REQUIREMENTS

Hamerliva Bank supported both batch and streaming data pipelines; however, their audit revealed operational silos and inconsistent tooling across regions. Real-time ingestion was mission-critical for algorithmic trading teams, whereas compliance and financial reporting workloads were largely batch-driven and less latency-sensitive.

To balance these needs, they crafted a dual-mode ingestion strategy with targeted requirements:

- “Streaming ingestion must support upserts and late-arriving records using merge-on-read semantics.”
- “Batch jobs must run nightly and be orchestrated through Apache Spark integrated with existing Airflow DAGs.”
- “All ingestion pipelines must support schema evolution and emit lineage metadata for catalog registration.”

These requirements highlighted the need for flexible ingestion tools that could evolve in response to workload demands, while preserving data integrity and traceability.

## CATALOG REQUIREMENTS

The metadata layer in Hamerliva Bank’s current environment was fragmented across Hive Metastore, spreadsheets, and internal relational databases. Manual schema changes, limited versioning, and inconsistent access control mechanisms were frequent pain points.

To unify metadata and improve manageability, Hamerliva Bank defined the following:

- “The catalog must support the Iceberg REST Catalog specification to ensure compatibility across all query engines.”
- “Versioned schema evolution must be enabled, with rollback support and user attribution for all changes.”
- “Catalog must support RBAC for dataset-level permissions and integrate with corporate identity providers (e.g., Active Directory).”

These statements enabled Hamerliva Bank to shortlist catalogs, such as Polaris, Nessie, or AWS Glue, based on functionality and governance compatibility, rather than name recognition or legacy use.

## FEDERATION REQUIREMENTS

Although Hamerliva Bank was beginning to consolidate its workloads, many datasets were still scattered across multiple systems. Teams often had to duplicate data or manually transform exports to bridge environments, which introduced latency, inconsistencies, and extra cost.

To address this, their federation requirements focused on interoperability and unified access:

- “Query engines must support federated access across Iceberg tables, S3 buckets, and on-prem HDFS without data duplication.”
- “The platform must provide a common semantic layer to define reusable business logic across federated queries.”
- “Federated queries must enforce row- and column-level access controls consistently across all sources.”

This allowed Hamerliva Bank to evaluate engines like Dremio or Trino not just on their ability to query Iceberg, but on their ability to do so alongside legacy and operational data stores, while maintaining governance fidelity.

## CONSUMPTION REQUIREMENTS

The most significant issues uncovered in the audit pertained to data consumption. Dashboards suffered from latency spikes, and semantic inconsistencies emerged across tools, leading analysts to frequently create shadow pipelines to work around these limitations. Consumption requirements focused on creating a consistent, performant, and governed experience for all data consumers.

Key requirements included:

- “Interactive dashboards must return query results under five seconds for Iceberg tables partitioned by time.”
- “Query engines must enforce row-level access policies defined in the catalog and aligned with enterprise roles.”
- “Consumption interfaces must include JDBC, ODBC, and REST APIs, with support for Apache Arrow Flight for high-performance use cases.”
- “Data science users must be able to query Iceberg tables via Python using PyArrow and pandas-compatible connectors.”

These requirements positioned Hamerliva Bank to evaluate not just whether a tool could “query Iceberg,” but whether it could serve the diverse needs of analysts, business users, and data scientists under consistent semantics and security rules.

## FROM REQUIREMENTS TO DESIGN

Together, these requirements formed the foundation for Hamerliva Bank's architectural design process. While they did not resolve every open question, they provided clear, testable criteria for evaluating tools, making trade-offs, and ensuring that each decision aligned with business objectives and stakeholder expectations. More importantly, these requirements established a shared language and direction, enabling engineering, governance, and analytics teams to move forward with confidence, clarity, and cohesion. We will explore each component of the iceberg lakehouse and how different tools attempt to meet these and other requirements in other chapters.

### 3.4 Architectural plan and road show

With a complete set of requirements in hand, the next step in the Iceberg implementation journey is architectural planning. This is the phase where decisions must be made—choosing technologies, defining federation strategies, and outlining how the components of your lakehouse will work together to meet the defined goals.

But an architectural plan isn't just a technical diagram. It is a narrative that explains how your platform will evolve, what trade-offs were made, and how those decisions align with stakeholder priorities. To succeed, this plan must be communicated clearly and validated by the very teams who will use, operate, or be impacted by the new architecture. In other words, implementation begins not with infrastructure changes, but with alignment.

The architectural plan should not only cover each of the five Iceberg lakehouse components—storage, ingestion, catalog, federation, and consumption—but also illustrate how these components will interact in practice. The objective is to define a blueprint that reflects both the audit findings and the practical realities of your organization's scale, compliance posture, and team capabilities.

Once the plan is complete, it's critical to socialize it broadly through a stakeholder "road show." This involves presenting the proposed architecture to each stakeholder group, walking through how their pain points and goals are addressed, and gathering feedback to refine the design before implementation begins. This step builds trust, uncovers blind spots, and cultivates champions within each department, paving the way for smoother adoption and sustained cooperation. Let's look at how Hamerliva Bank Incorporated executed this phase of the process.

#### 3.4.1 Hamerliva Bank creates its architectural plan

With a detailed set of requirements established, Hamerliva Bank Incorporated entered the architectural planning phase. This was not a solitary exercise performed by the data team in isolation; it was a coordinated effort involving a cross-functional working group composed of platform engineers, data architects, compliance officers, and analytics leaders. Each participant brought their domain expertise to the table, ensuring the resulting plan reflected technical feasibility, regulatory compliance, and real-world usage patterns.

The group's mandate was to design a production-grade Iceberg lakehouse architecture that met the company's immediate needs while allowing room for future evolution. Their guiding principle was to build a system that was not only modern but also maintainable—one that would streamline data operations, respect governance constraints, and enhance analytical responsiveness across the organization.

Rather than rushing into tool selection, they began by revisiting each of the five core Iceberg lakehouse components, mapping the documented requirements to candidate architectural patterns and technologies. Each decision was made in the context of the trade-offs between cost, control, performance, and integration complexity.

### *Storage*

Hamerliva Bank chose a hybrid storage model as the backbone of their lakehouse. AWS S3 was designated as the primary object store for analytics workloads, particularly for data that benefited from elasticity, scalability, and proximity to cloud-native compute. To satisfy data residency and regulatory requirements—especially under GDPR and SEC Rule 17a-4—they deployed MinIO clusters on-premises within both their European and U.S. data centers. These MinIO instances provided S3-compatible object storage with the added benefit of local control, air-gapped environments, and encrypted storage at rest.

### *Ingestion*

For data movement and preparation, Hamerliva Bank adopted a dual-mode ingestion strategy grounded in their existing infrastructure and source diversity. Apache Spark was retained for batch workflows, orchestrated with Apache Airflow to process structured reporting data, compliance logs, and historical trade records. To support real-time ingestion, Apache Flink and Kafka Connect were selected for handling high-frequency sources like market tick data and order events, writing directly to Iceberg tables with merge-on-read semantics for efficient updates.

Recognizing the complexity of integrating APIs, SaaS platforms, and operational databases, Hamerliva Bank also incorporated a connector-focused tool to simplify onboarding new sources. Solutions like Fivetran, Airbyte, or Qlik were evaluated to automate ingestion from commonly used applications, ensuring a smoother experience for both data engineers and domain teams. This layered approach enabled Hamerliva Bank to modernize incrementally while maintaining operational continuity.

### *Catalog*

The group selected Apache Polaris as their central metadata and catalog service. Polaris supported the Iceberg REST Catalog specification, which was a critical requirement for enabling engine interoperability and API-based table management. It also provided native RBAC support and the ability to integrate with Hamerliva Bank's existing identity provider. Because Polaris can serve both on-premises and cloud environments, it provides a single pane of visibility and control across all datasets, regardless of physical location. Its time-travel capabilities and atomic commit log further reinforced the platform's reliability and traceability.

### *Federation*

For federated query and cross-system federation, Hamerliva Bank standardized on Dremio as its primary query engine. Dremio's support for Iceberg-native acceleration, semantic layer modeling, and Apache Arrow-based execution made it well-suited for both high-performance analytics and user-friendly exploration. Dremio's ability to federate queries across S3, HDFS, and relational systems allowed Hamerliva Bank to minimize data duplication and serve analytical use cases that spanned multiple data domains. It also offered REST APIs and Arrow Flight endpoints, meeting federation requirements for both application developers and data scientists.

### *Consumption*

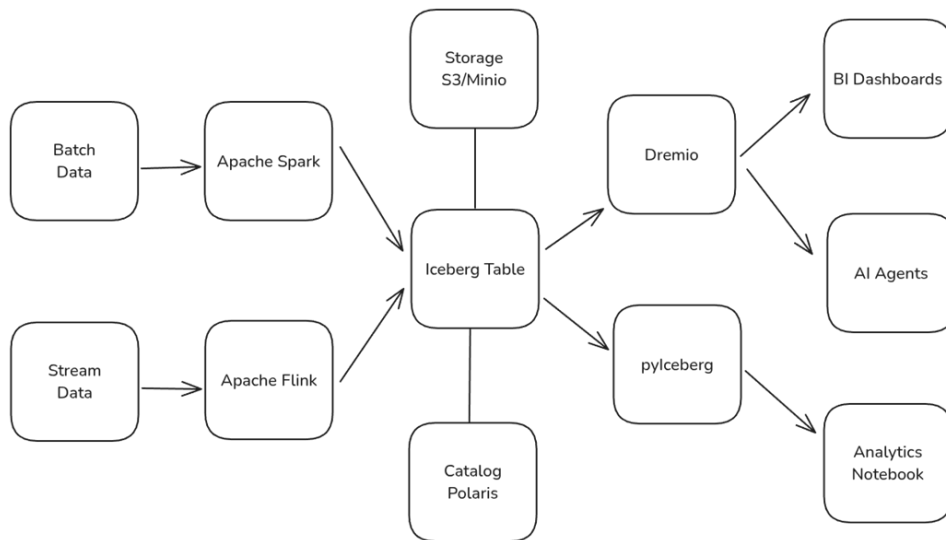
To serve business users, Tableau was integrated with Dremio via standard ODBC connectors, enabling governed self-service BI with low-latency response times for tasks like ad hoc sales reporting and executive dashboards. For data scientists, Hamerliva BankXYZ provisioned access to Iceberg tables using PyArrow and Arrow Flight, allowing notebook users to directly explore curated datasets for advanced analytics and machine learning model training without relying on custom ETL extracts. Typical workflows included customer segmentation models and fraud detection pipelines that required timely access to transaction-level data. Across all consumption paths, semantic consistency was maintained through Dremio's native semantic layer, while access controls defined in Polaris were enforced by Dremio's query engine, ensuring data consumption remained aligned with governance policies.

Once these component-level decisions were finalized, the working group assembled a comprehensive architecture guide. This document went beyond technical specifications. It included:

- End-to-end data flow diagrams and system topologies
- Storage tiering strategies across S3 and MinIO
- A map of catalog-registered datasets, indicating regulatory classifications
- Lineage mappings to show how ingestion pipelines connect to analytical outputs
- A staged migration plan prioritizing datasets based on frequency of access, sensitivity, and transformation complexity
- Fallback procedures, including options for handling catalog or engine outages
- A change management plan for deprecating legacy ETL jobs and onboarding users to new tools

Importantly, Hamerliva Bank did not present this plan as a finalized directive. It was explicitly framed as a proposal, subject to review and revision. This framing signaled to stakeholders that their feedback remained essential and that the implementation would be shaped through ongoing collaboration. The architecture guide was written in clear language, with annotations and visual aids tailored for both technical and non-technical audiences, preparing it for the stakeholder road show to follow.

This plan, summarized in figure 3.6, served as the reference architecture for aligning technical components with user-facing outcomes. Pylceberg played a critical role in enabling direct programmatic access to Iceberg tables from notebooks and machine learning pipelines, allowing data scientists to explore features, engineer datasets, and train models without intermediate ETL steps. The inclusion of AI agents represented emerging use cases such as automated anomaly detection and real-time recommendation engines, which relied on fast, governed access to curated datasets. Presenting this design to stakeholders allowed teams to validate assumptions, secure alignment, and refine the implementation plan. The time invested in cross-functional planning ensured the lakehouse would be both well-architected and well-supported across the organization.



**Figure 3.6** Hamerliva Bank architectural proposal shows how all the components relate to one another

### 3.4.2 Hamerliva Bank conducts a road show

With a thoughtfully prepared architectural plan in hand, Hamerliva Bank Incorporated turned its focus to one of the most essential yet often overlooked phases of platform modernization: the road show. This phase was about more than simply presenting a technical design—it was about securing alignment, building trust, and ensuring every stakeholder group had a voice before implementation began. The success of the project would depend as much on organizational engagement as on architectural soundness.

The road show was structured as a series of targeted, role-specific briefings. Each session was tailored to the audience's interests and responsibilities, reinforcing the message that this was a collaborative effort rather than a top-down directive. Sessions were scheduled with data engineers, business analysts, data scientists, compliance officers, IT operations staff, and executive stakeholders. The goal was to foster transparency, surface unanticipated concerns, and generate early champions for the new platform.

Each briefing began with a restatement of the context: the goals uncovered during the audit, the constraints identified during requirements gathering, and the broader business motivation for transitioning to Apache Iceberg. This framing helped each audience understand how the architecture proposal was not simply a technical overhaul, but a strategic response to shared pain points and ambitions.

The briefers then walked through the relevant sections of the architectural plan. For example:

- In sessions with compliance and risk management, the team opened by revisiting GDPR and SEC Rule 17a-4 obligations. They then demonstrated how the hybrid storage architecture—featuring MinIO clusters for regional data residency—and Polaris’s RBAC support would maintain and in many cases improve regulatory alignment.
- In meetings with data engineering teams, the focus shifted to ingestion strategy. Detailed diagrams illustrated how existing Spark pipelines would be preserved, how Flink would be introduced for real-time use cases, and how tools such as NiFi and Informatica would be phased out through a staged migration plan. Engineers were tasked with validating assumptions about pipeline dependencies and job handoffs.
- For business analysts and data scientists, the sessions focused on enhancing data discoverability, freshness, and accessibility. The team demonstrated how Dremio’s semantic layer would standardize metric definitions across tools, how dashboards would benefit from improved performance through Iceberg’s hidden partitioning, and how data scientists would gain access to Arrow Flight for programmatic workflows.

Throughout each session, the architecture guide served as a shared anchor, not a static document, but a living reference. Attendees were encouraged to annotate, critique, and explore the guide during discussions. Facilitators explicitly requested feedback on whether the proposed changes aligned with actual workflows and where operational frictions might still exist.

The discussions led to several meaningful refinements. For instance:

- The analytics team expressed a need for early access to curated datasets in S3 to accelerate the development of prototypes. This led to a reprioritization of migration phases, with more heavily used dashboards front-loaded in the rollout schedule.
- Compliance officers requested tighter controls over metadata auditing. As a result, the architecture team added log export and monitoring capabilities to Polaris and planned for audit trail integrations with internal Security Information and Event Management (SIEM) systems.
- IT operations flagged concerns about managing hybrid storage costs. The architecture guide was updated to include tiered storage strategies and operational SLAs for capacity alerts and automated lifecycle policies.



These sessions not only improved the plan, but they also cultivated buy-in. Stakeholders saw their input taken seriously, and as a result, they began to internalize ownership of the plan's success. In several teams, participants volunteered to act as transition liaisons, colleagues who would stay engaged with the implementation process, help onboard others, and troubleshoot cross-functional concerns during the rollout.

By the end of the road show, Hamerliva Bank had accomplished more than just a multi-team presentation. They had achieved cross-functional alignment. Each department understood the rationale for the change, the scope of what was to come, and their role in executing the transformation. More importantly, they recognized how the proposed lakehouse would directly improve their work, whether by reducing manual steps, increasing data trust, or accelerating insight delivery.

The road show turned a proposed architecture into an organizational commitment. It transformed potential resistance into active support. And it ensured that when implementation began, it did so not with hesitation, but with shared momentum and a clear, company-wide sense of purpose.

In the chapters that follow, we'll examine each of the five Iceberg lakehouse components in detail. We'll explore the trade-offs, design patterns, and tool options that can help you make the right choices for your organization, starting with the foundation: storage.

### 3.5 Summary

- Transitioning to an Apache Iceberg-based lakehouse involves making informed, strategic decisions that are grounded in your organization's real-world needs.
- Conduct a structured audit of your current data architecture, translate those insights into actionable requirements, and build a stakeholder-aligned architectural plan.
- Engage stakeholders across roles and functions to surface technical and organizational needs.
- Document and categorize your current tooling, storage systems, and governance practices.
- Before moving forward with implementation, it's crucial to gain consensus and clarity across all departments.
- The roadshow process helps create shared understanding and feedback loops which can transform a proposed architecture into a cross-functional initiative, reducing friction and fostering long-term support.

## 4 *Selecting the storage layer*

### **This chapter covers**

- Defining storage performance, security, and integrity requirements
- Comparing block and object storage architectures
- Understanding Parquet and the S3 API as foundational standards
- Exploring storage solutions including HDFS, MinIO, and Pure Storage

The storage layer is the foundation of any Apache Iceberg lakehouse. While tools for ingestion, cataloging, and querying often receive attention for their immediate impact on user experience, it is the storage layer that ultimately determines the reliability, scalability, and cost-efficiency of the platform. Poor choices here can lead to performance bottlenecks, security gaps, or unsustainable operational complexity. Sound decisions, on the other hand, enable long-term flexibility, reduced costs, and future-proof integrations.

Building on the requirements surfaced during your audit, this chapter guides you through the key dimensions that should shape your storage strategy. We begin by revisiting the most critical requirement categories: performance, security, integrity and cost. With these in mind, we then examine the two main architectural paradigms for lakehouse storage, block storage and object storage, and explain how they differ in structure, access patterns, and suitability for Iceberg workloads.

Next, we explore two technical standards that underpin the majority of modern storage solutions: the Parquet file format and the S3 API. Parquet's columnar structure makes it ideal for Iceberg's analytics-oriented workloads, while the S3 API has become the lingua franca of object storage, offering broad compatibility and deployment flexibility.

Finally, we compare a variety of storage solutions, some designed for on-premises infrastructure, others built for the cloud. These include legacy systems like Hadoop HDFS, modern object stores such as MinIO and Ceph, and enterprise-focused platforms like NetApp StorageGRID and Pure Storage. For each, we'll highlight history, design philosophy, and the types of environments where they shine.

By the end of this chapter, you'll be equipped to assess which storage technologies best align with your lakehouse requirements, balancing technical constraints with business priorities to make a durable, strategic choice.

## 4.1 Storage requirements

Before selecting a storage solution for your Apache Iceberg lakehouse, you must first clarify what your storage layer needs to deliver. Requirements vary significantly across organizations, and choosing the wrong fit can introduce inefficiencies that undermine the broader architecture. That's why storage decisions should not begin with technology preferences or vendor comparisons, they should begin with clearly articulated goals. Keep in mind, any storage system can possibly work with Apache Iceberg so it becomes a matter of choosing the one that meets your needs.

The audit process introduced in chapter 3 provides the foundation for this discussion. By engaging stakeholders across engineering, analytics, security, and operations, you should now have a detailed understanding of how data is used, where pain points exist, and what constraints must be respected. This insight translates directly into requirements that your storage layer must fulfill, requirements that span performance, security, data integrity, cost, and operational complexity.

In this section, we will focus on the most common requirement categories that influence storage decisions in a lakehouse context:

- *Performance*, which addresses how quickly files can be retrieved and scanned by query engines.
- *Security*, which ensures sensitive data is protected through access controls and encryption.
- *Integrity*, which speaks to resilience and recoverability in the face of hardware failures or human error.
- *Operational and cost considerations*, which capture the day-to-day realities of managing storage, including scalability, cloud costs, and ease of maintenance.

By defining these dimensions in concrete terms, you will be better prepared to evaluate the diverse storage solutions we'll examine later in this chapter, not based on their marketing claims, but on their ability to meet the specific needs of your organization.

### 4.1.1 File retrieval performance requirements

At the core of every lakehouse workload is the act of accessing files, both reading and writing. Whether serving a dashboard query, ingesting new datasets, training a machine learning model, or conducting exploratory analysis, efficient file access directly affects user experience, system throughput, and infrastructure cost. In Apache Iceberg, performance depends not only on the query engine and metadata optimization but also on how files are retrieved from and written to storage.

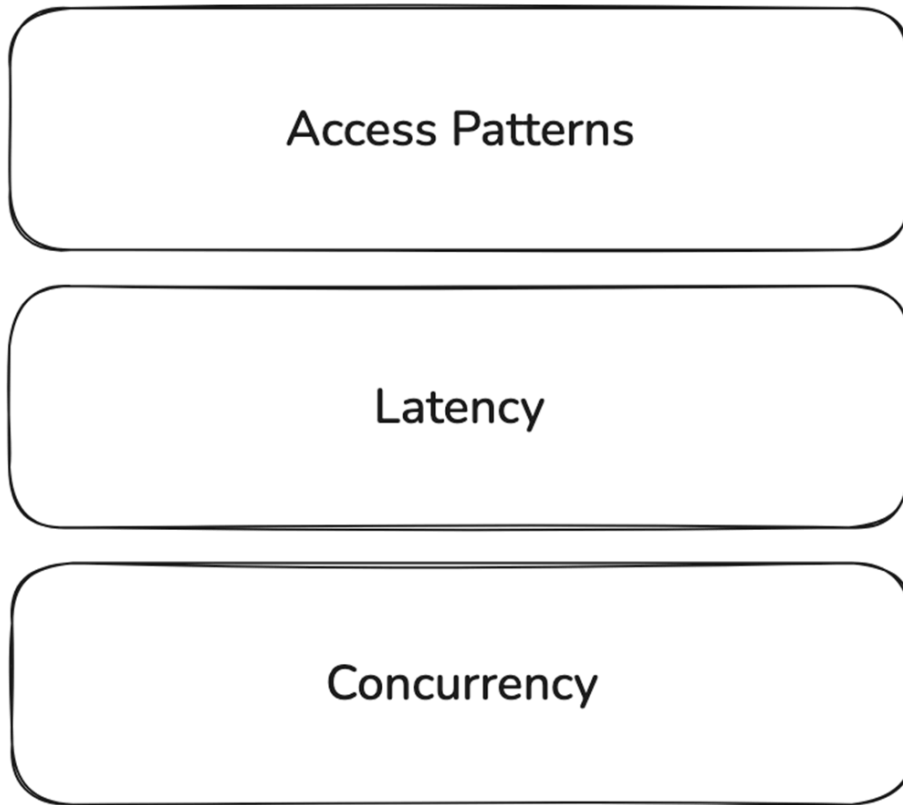
File retrieval and write latency are shaped by several factors: the physical layout of data on disk, the characteristics of the storage medium, network throughput, and caching strategies. For instance, object stores often prioritize scalability and durability over raw speed, resulting in higher latency than block storage systems that are directly attached to compute nodes. This tradeoff may be acceptable for batch ingestion jobs but becomes problematic for interactive queries or workloads involving frequent access to small files.

Concurrency also plays a critical role. Storage systems must support multiple compute engines, users, and pipelines operating simultaneously. As Iceberg adoption grows, concurrent file operations during peak hours or reporting cycles can overwhelm systems that lack efficient request handling. Solutions that throttle throughput or introduce read contention under load may become performance bottlenecks.

Another key factor is access patterns, how data is read or written in typical workloads. For example, many analytical queries exhibit large, sequential reads across columnar files, while ingestion jobs may involve frequent appends and compactions. A storage system optimized for sequential reads may struggle with concurrent random writes. Understanding these patterns is crucial when selecting and configuring your storage layer.

The structure and distribution of data files further impact retrieval efficiency. Iceberg typically manages large numbers of Parquet files, leveraging features like predicate pushdown and file pruning. These optimizations only yield benefits if the underlying storage serves files predictably and with low variance. Inconsistent or slow retrieval erodes the value of Iceberg's metadata-driven query planning.

To evaluate real-world performance, organizations should go beyond advertised throughput. Benchmarking access patterns, measuring tail latency, and stress-testing concurrency are essential steps in making a storage decision aligned with actual workload demands. Figure 4.1 highlights the key factors to assess when analyzing storage performance.



**Figure 4.1** The factors to consider when understanding a storage layers performance.

#### **4.1.2 Security requirements**

Security requirements are a critical dimension in selecting the right storage layer for your Iceberg lakehouse. While the architecture may support sophisticated access controls and governance features at the catalog or query engine layers, those mechanisms must ultimately rely on the integrity of the storage system beneath them. A weak or misconfigured storage layer can expose sensitive data to risk, even when higher-level controls are in place.

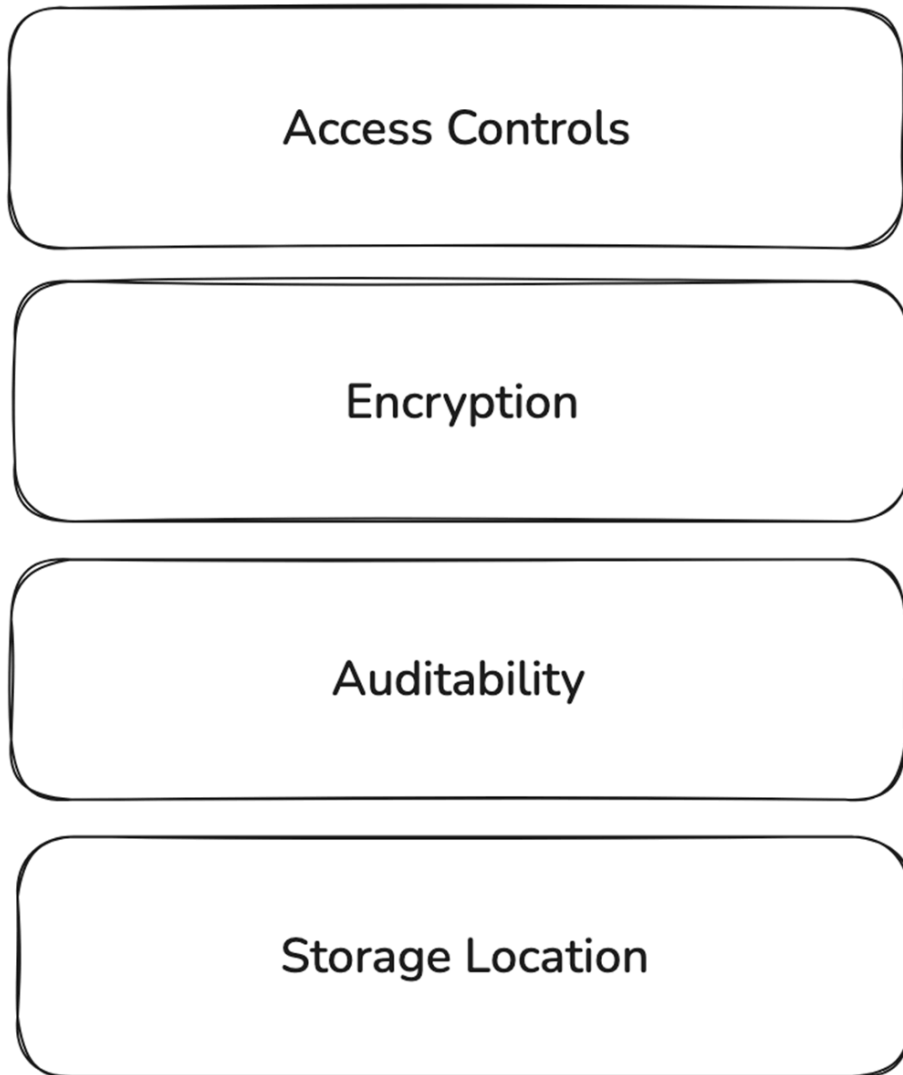
At a minimum, the storage solution must support fine-grained access control. This includes the ability to restrict read and write permissions by user, role, or service identity. For object storage systems, this typically means supporting bucket-level and prefix-level permissions, enforced through identity and access management policies. For block storage, access is usually governed through filesystem permissions or host-level isolation.

Encryption is another essential component of secure storage. Many storage vendors offer server-side encryption options, allowing data to be automatically encrypted at rest. Some support bring-your-own-key (BYOK) or key management systems (KMS) integration, providing greater control over encryption lifecycle and compliance alignment. In regulated industries, this capability is often not optional. Organizations subject to data residency laws, financial compliance standards, or healthcare regulations must ensure that their data is encrypted and auditable at every stage of storage and retrieval.

Auditability itself is increasingly a requirement. Storage systems should support logging of access attempts, modifications, and deletions, with sufficient granularity to detect anomalous behavior or respond to incidents. These logs are especially valuable in forensic investigations and must be retained according to organizational policy or legal requirements.

Finally, storage location matters. Public cloud services may store data in multiple geographic regions by default, which can introduce compliance challenges. Conversely, on-premises solutions offer greater control but may lack the geographic redundancy required for disaster recovery. Ensuring that your storage system can be configured to meet jurisdictional requirements is vital, especially for global enterprises.

Security cannot be an afterthought. When evaluating storage options, it is essential to determine how well they integrate with your organization's identity systems, key management infrastructure, and compliance processes. The storage layer is a foundational trust boundary in the lakehouse architecture, and its design must reflect that responsibility focusing on the areas illustrated in figure 4.2.



**Figure 4.2** The factors to consider when analyzing a storage layers security.

#### **4.1.3 Integrity requirements**

Data integrity in the storage layer refers to the assurance that data remains accurate, complete, and recoverable across its lifecycle. For lakehouse architectures built on Apache Iceberg, where large volumes of critical analytical data are managed in distributed filesystems, ensuring integrity is both a technical necessity and a business imperative.

At its most basic, integrity begins with durability. The storage system must be able to retain data reliably without corruption or loss, even in the face of hardware failure, network interruption, or unexpected shutdowns. Object storage platforms typically offer built-in redundancy by replicating data across multiple nodes or geographic zones. Block storage systems, especially in on-premises environments, may require external tools or configurations to achieve similar fault tolerance. Understanding how each system handles replication, consistency, and eventual recovery is key when assessing integrity guarantees.

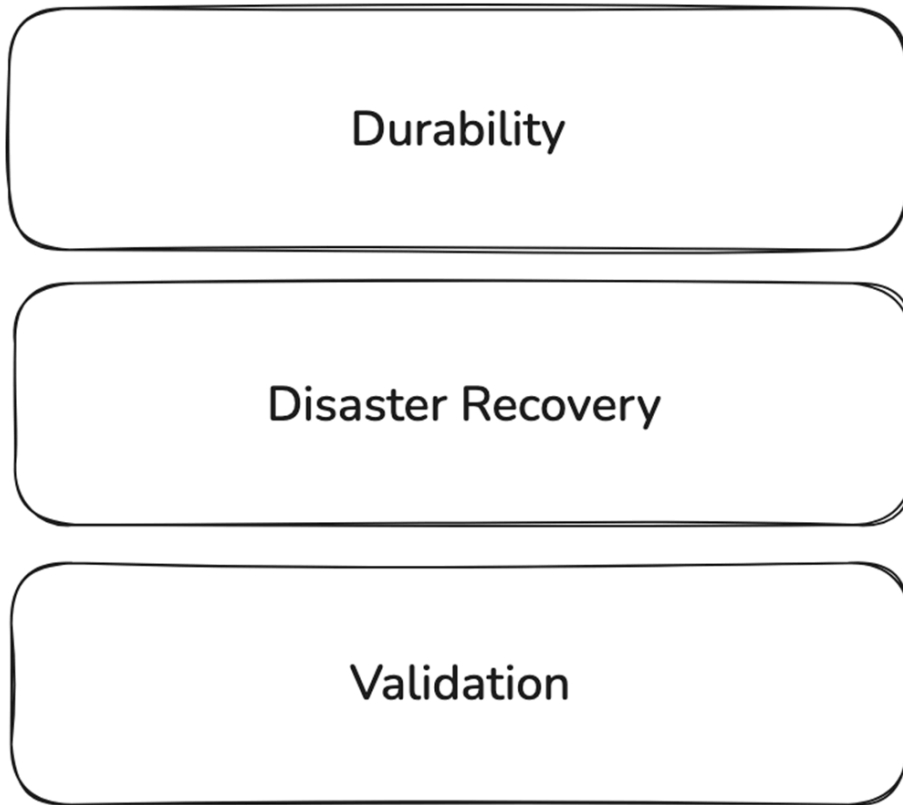
Backup and disaster recovery are also central to integrity planning. In distributed environments, file deletions, schema changes, or software bugs can affect many files at once. Effective storage systems must support point-in-time snapshots or versioning features, allowing organizations to roll back to a known good state. This is particularly important in Iceberg lakehouses, where transactional metadata updates must remain synchronized with data files to avoid inconsistencies. Misalignment between table metadata and underlying files can result in query failures or data anomalies.

Checksumming and validation mechanisms further enhance integrity. Systems that implement automatic verification of data blocks or objects during write and read operations help detect corruption early, reducing the risk of silent errors that may propagate through downstream pipelines.

High availability complements data integrity by ensuring that storage remains accessible even when individual components fail. In a lakehouse environment, where multiple services and users depend on continuous access to data, downtime can halt critical processes ranging from ingestion to analytics. A highly available storage system minimizes single points of failure through redundancy at both the hardware and software levels. This may include active-active replication across availability zones, failover mechanisms for network paths, and load-balanced access endpoints. Object stores, in particular, often support geographic distribution, allowing read and write operations to continue uninterrupted during localized outages. Incorporating high availability into your storage strategy ensures that data remains not only intact but accessible at all times, supporting uninterrupted operation and platform resilience.

In a lakehouse architecture, data is constantly in motion. Without integrity guarantees at the storage level, the risk of silent failures, data loss, or inconsistent reads grows significantly. A mature storage layer must not only preserve data but do so in a way that is observable, recoverable, and trustworthy under pressure factoring the factors illustrated in figure 4.3.





**Figure 4.3** The integrity factors to consider when assessing a storage system.

#### **4.1.4 Cost and operational overhead requirements**

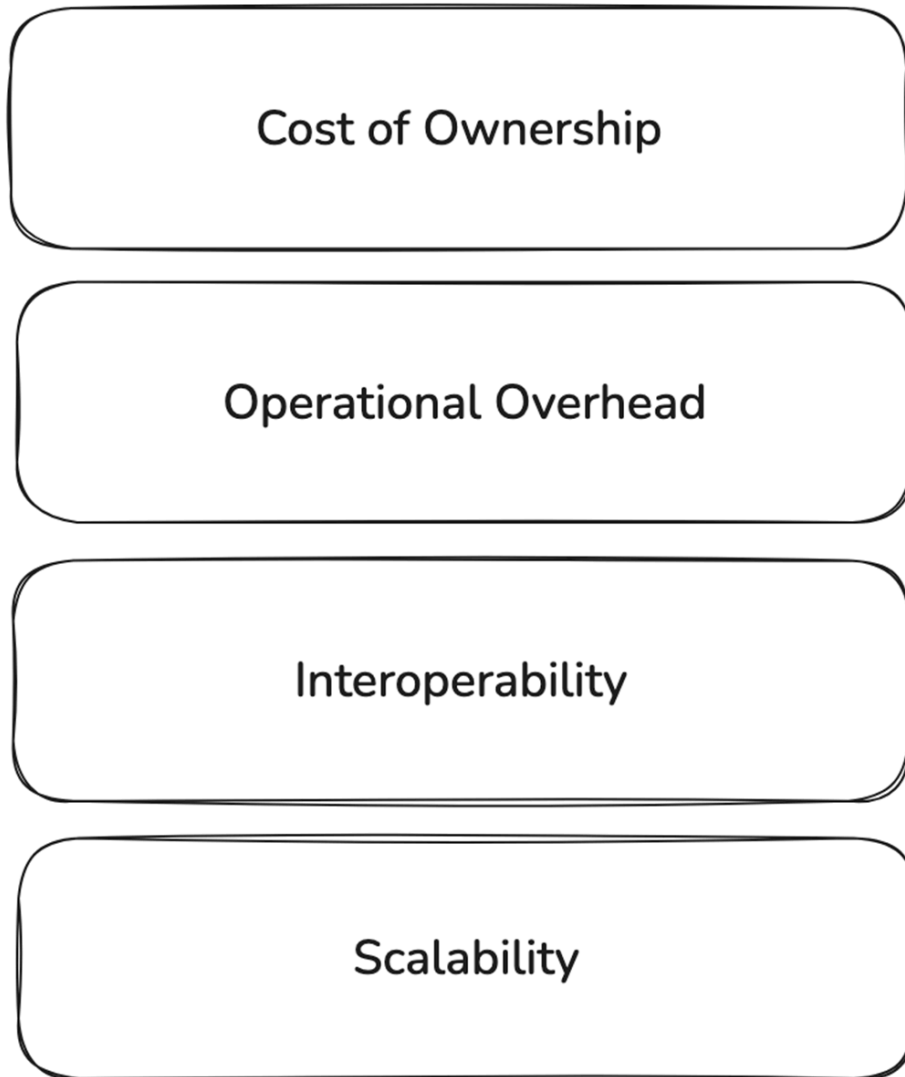
No storage decision is complete without a full accounting of its cost and operational burden. While performance, security, and integrity are all critical dimensions, they must be weighed against the financial and human resources needed to sustain the storage layer over time.

The first consideration is the total cost of ownership. This includes not just storage capacity, but also data egress charges, replication costs, access frequency pricing, and long-term archival strategies. Cloud-based object storage solutions often appear inexpensive at first glance but can accumulate substantial costs through frequent access or inter-region data movement. On-premises block storage may avoid some of these variables but typically incur higher capital expenditure, ongoing maintenance, and infrastructure management costs.

Operational overhead refers to the effort required to manage, scale, monitor, and troubleshoot the storage environment. Systems that demand constant tuning, manual scaling, or deep vendor-specific expertise can limit agility and slow down platform evolution. Conversely, solutions with strong automation, observability tooling, and self-service capabilities can empower platform teams to maintain performance and compliance without disproportionate staffing.

Interoperability also affects operational cost. Storage systems that support standard APIs and protocols reduce friction across ingestion tools, processing engines, and query services. Those that rely on proprietary interfaces may require custom integrations, introduce lock-in, or constrain flexibility as architectural needs evolve.

Finally, scalability must be considered from both a cost and labor perspective. Can the system handle rapid growth in data volume or user demand without triggering major reconfigurations? Are scaling events predictable and manageable, or do they involve complex provisioning and downtime? These questions are especially important in environments with seasonal workloads, real-time analytics, or high rates of schema evolution. Make sure to weigh the cost considerations illustrated in figure 4.4.



**Figure 4.4 Factors when considering the cost of a storage system.**

Choosing a storage layer is a long-term commitment. Its cost profile and operational demands will influence not only your budget but also your team's ability to focus on innovation rather than firefighting. By clarifying these requirements early, you can prioritize solutions that align with your financial model and operational culture, not just your technical checklist.

## 4.2 Block vs object

Once your storage requirements are defined, the next step is to evaluate the fundamental architectural models available to implement them. In the world of data lakehouses, nearly all storage options fall into one of two categories: block storage or object storage, both of which can be used with Apache Iceberg Lakehouses. Each model represents a distinct philosophy in how data is organized, accessed, and managed, and each brings a unique set of trade-offs that affect performance, scalability, cost, and operational complexity.

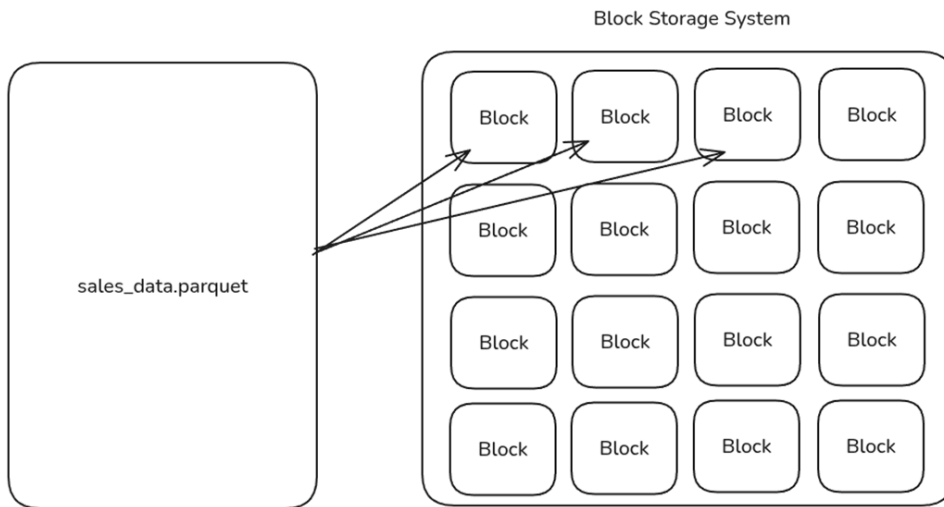
Block storage operates at a lower level of abstraction, breaking data into fixed-size chunks that are managed by the operating system. It offers high throughput and low latency, making it a natural fit for high-performance computing and traditional data processing frameworks like Hadoop. Block storage is typically co-located with compute resources, meaning storage and compute scale together. While this can yield performance benefits, it also drives up costs and complicates failure recovery, as scaling storage independently or recovering from node-level failures often requires manual intervention. Additionally, block storage demands external systems to manage file structures, permissions, and availability, adding to the overall complexity and operational overhead.

Object storage, by contrast, abstracts files into discrete objects, each stored with associated metadata and a unique identifier. This model emphasizes scalability, durability, and accessibility, often operating over HTTP-based APIs such as the S3 protocol. One of its most significant advantages is that it enables the separation of compute and storage, allowing each to scale independently. This decoupling reduces infrastructure costs and enhances flexibility across diverse workloads. As a result, object storage has become the dominant approach in modern data lakehouses due to its compatibility with cloud-native services, lower operational burden, and tight integration with analytical file formats like Parquet.

In this section, we will explore how each model works, where they excel, and what limitations they introduce in the context of an Iceberg lakehouse. Understanding these core differences will clarify why certain storage solutions align better with specific workload patterns, deployment models, or regulatory needs.

### 4.2.1 Block storage

Block storage is one of the oldest and most widely used approaches to data persistence in enterprise computing. At its core, block storage divides data into evenly sized chunks known as blocks as illustrated in figure 4.5, which are individually addressed and managed by the underlying operating system. These blocks are then assembled by the file system to present a coherent view of the data to applications.



**Figure 4.5** Illustrating how a single file is stored across multiple blocks in a block storage system.

This architecture is particularly effective when paired with distributed file systems like Hadoop Distributed File System (HDFS). In HDFS, block storage serves as the foundational layer, enabling the system to stripe data across multiple machines for redundancy and parallel access. This design was instrumental in the early era of big data, offering high-throughput access to large files across commodity hardware. Hadoop's reliance on block storage gave organizations fine-grained control over replication factors, fault tolerance, and storage locality, which in turn allowed compute jobs to operate with predictable performance. In the earliest days of Apache Iceberg, it was designed to overcome the eventual limitations of Hadoop and Hive, which is why its file storage cataloging mechanism is canonically called the "Hadoop Catalog" as it was designed initially with Hadoop in mind despite working with any storage system.

While less common in new lakehouse deployments, block storage remains relevant in legacy Hadoop environments and high-performance private cloud installations. Many on-premises data platforms continue to rely on self-managed disks and distributed file systems like HDFS, where block storage is the foundational layer. These setups can support Apache Iceberg, enabling organizations to modernize their table formats without overhauling their entire infrastructure. However, managing block storage comes with trade-offs. It lacks native integration with cloud-native services, requires separate systems for replication and access control, and demands significant operational effort to ensure reliability and scalability. For organizations maintaining legacy systems, layering Iceberg onto existing block storage can offer a transitional path—but it also means missing out on the elasticity, automation, and cost-efficiency that cloud-based object storage typically provides.

## REPLICATION FACTORS AND FAULT TOLERANCE IN STORAGE SYSTEMS

In storage systems, the replication factor refers to the number of times a piece of data is duplicated and stored across different locations, such as servers, disks, or data centers. For example, a replication factor of three means that there are three copies of each data block stored in the system. This concept is critical in distributed storage architectures like HDFS (Hadoop Distributed File System), object stores, and cloud-native file systems to ensure data availability and resilience.

Fault tolerance is the system's ability to continue operating correctly even when some components fail. By having multiple replicas of data spread across different physical machines or locations, a system can withstand disk failures, server crashes, or even data center outages without losing data or access. If one replica becomes unavailable due to hardware or network issues, the system can automatically serve the data from one of the remaining copies, often without noticeable downtime or impact to users.

Replication and fault tolerance are essential for maintaining reliability, high availability, and data durability in modern storage systems. Without them, a single point of failure—like a corrupted disk or a crashed server—could result in permanent data loss or service interruption. While higher replication factors provide better protection, they also consume more storage and resources, so system designers must balance cost, performance, and risk tolerance when setting replication policies.

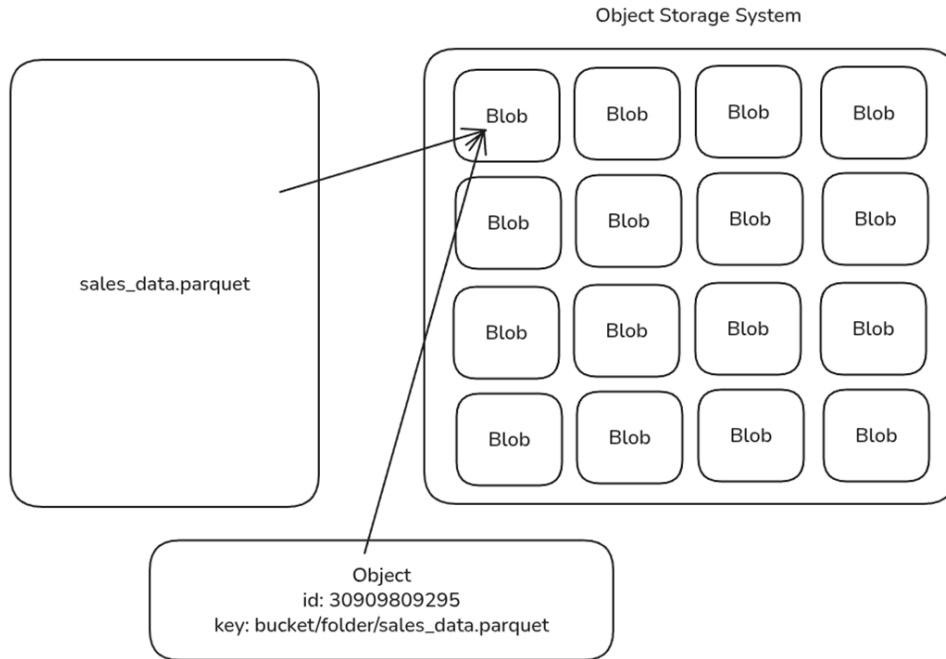
Block storage systems also support direct mounting by compute nodes. This allows engines such as Apache Spark, when deployed near the storage layer, to perform low-latency reads and writes with minimal overhead. In environments that are latency-sensitive or compute-intensive, this proximity can deliver substantial performance gains. These benefits are especially pronounced in workloads that demand high and consistent input/output operations per second (IOPS), a measure of how many read or write operations a storage system can handle in a given second. Consistent IOPS is critical for applications that require steady, predictable performance, such as large-scale ETL jobs or real-time analytics.

However, block storage comes with trade-offs. It is inherently infrastructure-heavy. Setting up and maintaining block-based storage systems often requires configuring volume managers, maintaining redundant arrays of independent disks (RAID), and deploying monitoring and alerting systems to detect failures. Additionally, because block storage lacks native support for metadata-rich access and HTTP-based protocols, it is less suited for the loosely coupled, cloud-native architectures that are now common in data platforms.

Scalability can also become a challenge. Expanding a block storage system may involve provisioning new disks, redistributing data, and coordinating changes across a tightly coupled network of nodes. Compared to object storage, which can elastically scale with far less manual intervention, block storage may impose more constraints as data volumes grow or as workloads evolve.

## 4.2.2 Object storage

Object storage has become the dominant model for modern lakehouse architectures, particularly in cloud-native environments. Unlike block storage, which operates at the infrastructure level, object storage abstracts data into self-contained units called objects. Each object consists of the data itself, associated metadata, and a unique identifier as illustrated in figure 4.6, allowing it to be accessed directly via standard APIs, most commonly the S3 protocol.



**Figure 4.6** A file is stored as a blob (binary large object) with an Object with metadata and id used to access it, user requests the file by key and the system then uses the metadata to locate and retrieve the file.

### AN OBJECT'S METADATA AND ID

From the user's perspective, object storage systems present a flat namespace where each object is accessed using a unique key or full path, similar to a URL. Unlike traditional file systems that use nested directories, object stores organize data in a non-hierarchical structure. This design simplifies scalability and eliminates limitations tied to directory depth or file system hierarchies, making it well-suited for large-scale environments with millions or billions of files.

Behind the scenes, each object is stored along with metadata and managed through a unique identifier. While the exact mechanisms vary by vendor, users typically interact with objects through their assigned keys, which can be structured to mimic directory paths (e.g., `logs/2024/01/file.parquet`). Object metadata includes system-level details like timestamps, size, and access permissions, as well as optional user-defined tags. These metadata fields enable advanced features like search, lifecycle rules, and access control without modifying the object's contents.

Because data is automatically distributed across multiple storage nodes, users benefit from high availability and seamless access regardless of the object's physical location. Features like versioning, replication, and policy-based management are supported out of the box, abstracted away from the user's experience. This makes object storage a natural fit for data lakehouses, where performance, scalability, and metadata-rich access are essential for managing large volumes of analytical data.

Modern object stores are built on distributed architectures that automatically partition data (auto-sharding) and provide load balancing, fault tolerance, and high availability. These systems scale horizontally and replicate data across multiple servers or geographic regions to meet stringent durability guarantees, often cited as eleven nines (99.999999999%) of durability. Beyond core storage, object stores include a range of enterprise-grade features. Lifecycle policies allow data to be automatically moved between storage tiers or deleted after a defined period. Built-in auditing and logging record every access or change to stored data, supporting security monitoring and compliance. Server-side encryption ensures that data is encrypted at rest without requiring changes to client applications. For access control, object stores use identity and access management (IAM), a framework that enables administrators to define who can perform specific actions (like reading or writing data) on particular resources. IAM policies typically allow permissions to be granted based on user roles, groups, or service identities, providing fine-grained control over data access. Together, these features minimize operational complexity while strengthening the platform's security and regulatory posture.

Object storage also aligns with the financial and operational needs of lakehouse deployments. Cloud services like Amazon S3, Google Cloud Storage, and Azure Blob Storage use a usage-based pricing model, charging per GB stored and per request made. There is no need for the end user to manage physical disks or overprovision storage in advance. Even on-premises solutions such as MinIO and Ceph adopt similar models while offering more control over deployment and data residency.

Apache Iceberg natively supports object storage, taking advantage of its scalability and API-driven access model. However, latency characteristics differ from block storage. While throughput remains high, access times can be more variable—particularly for workloads involving frequent access to many small files or object listings. For instance, S3 typically exhibits a 100–200 millisecond latency per request under load, which may be noticeable in low-latency scenarios. Developers designing ingestion pipelines or interactive queries must account for this, especially when workloads demand sub-second responsiveness.

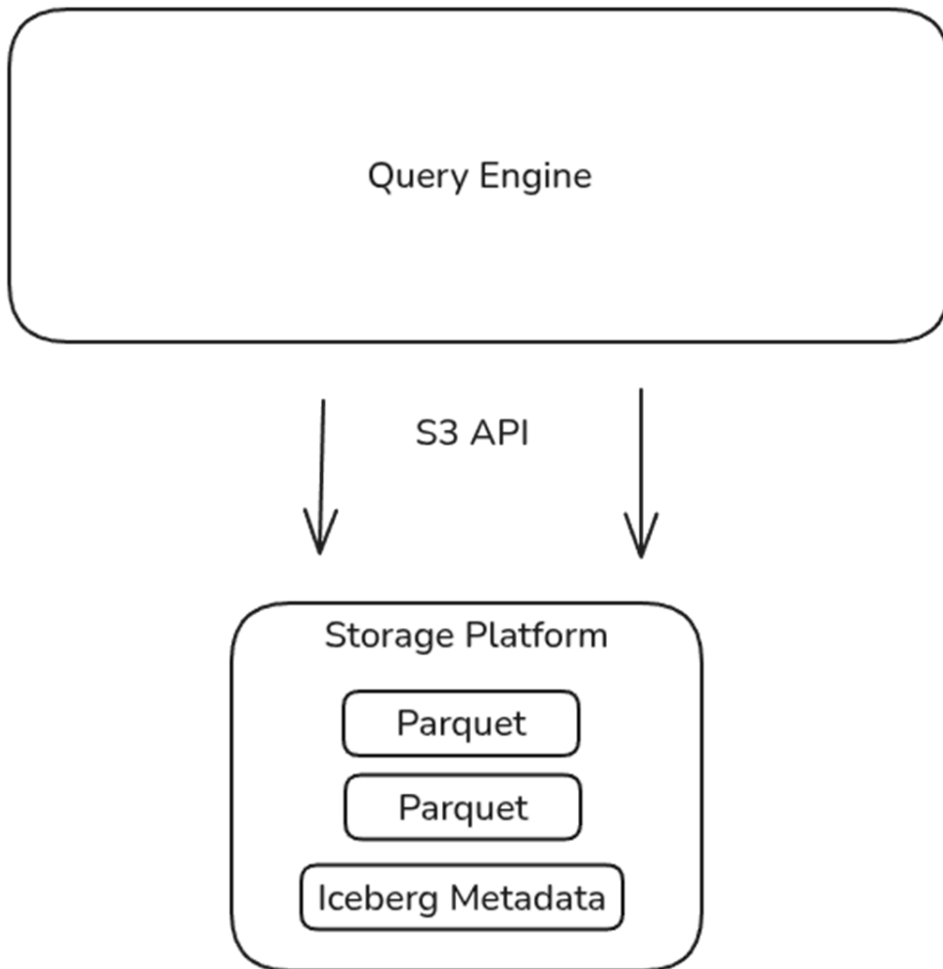


Another consideration is eventual consistency. Some object stores may delay visibility of new or updated objects in directory listings or across regions. While Apache Iceberg's commit protocol and metadata handling mitigate many of these challenges, understanding these consistency behaviors is critical for designing reliable, fault-tolerant pipelines.

Despite these nuances, object storage is the default choice for most Apache Iceberg lakehouse deployments. Its elasticity, managed service model, and broad compatibility make it ideal for handling evolving data architectures—from batch analytics to real-time machine learning pipelines—at enterprise scale.

### **4.3 The standards in the storage layer**

While the choice between block and object storage defines how data is physically stored and accessed, it is the standards layered on top of these systems that determine interoperability, performance, and long-term sustainability. In the world of Apache Iceberg lakehouses, two standards dominate the storage layer: the Parquet file format and the S3 API. These are two primitives you'll find as part of your architecture likely regardless how you design it.



**Figure 4.7** The engine doing an Iceberg transaction will use the S3 API to talk to an object storage platform where it will store data as a parquet file along with the Iceberg metadata.

These standards are not incidental. They represent years of ecosystem convergence around practical needs, efficient analytical storage and broad access compatibility. Their widespread adoption has made it easier for tools, platforms, and teams to collaborate across organizational and technological boundaries. For Iceberg, which aims to unify data access across engines and clouds, these standards are not just convenient, they are foundational.

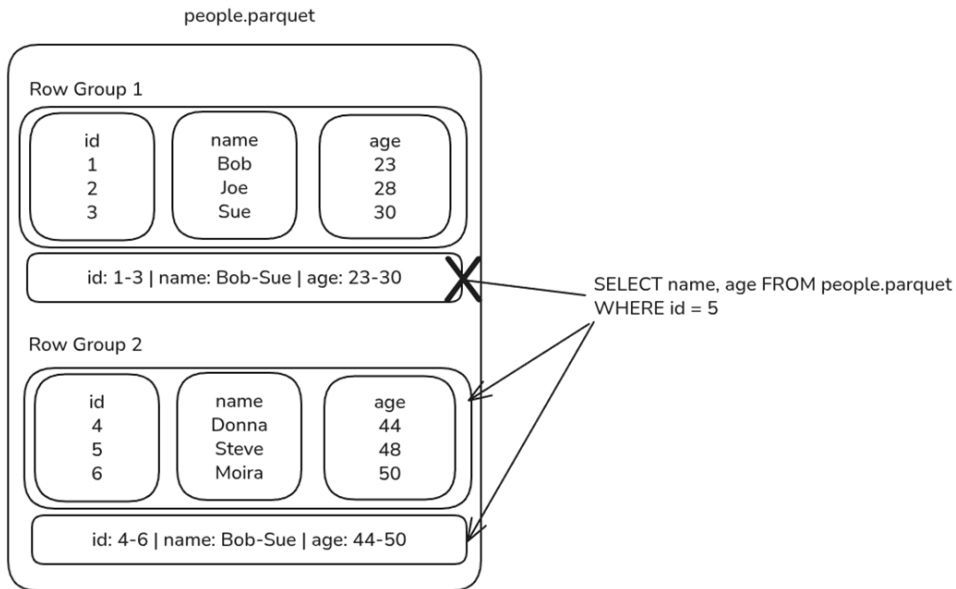
In this section, we will take a closer look at these two technologies. We'll explore Apache Parquet as the prevailing file format used in Iceberg tables and examine how its design supports fast, scalable analytics. We'll also discuss the S3 API, originally developed by Amazon but now embraced by nearly every major object storage solution. Understanding these standards will help you appreciate why certain storage systems integrate more smoothly with Iceberg, and what to look for when evaluating their support.

### 4.3.1 Apache Parquet

In Iceberg lakehouses, Parquet is more than just a file format, it is a pillar of the performance, flexibility, and openness that define the architecture. Its synergy with Iceberg's metadata model and query planning capabilities makes it the default choice for nearly all production deployments.

Apache Parquet is the most widely adopted file format for columnar data storage in modern analytics systems. Originally developed by Twitter and Cloudera and now part of the Apache Software Foundation, Parquet was designed to address the limitations of row-oriented formats such as CSV and JSON in analytical workloads. Its columnar design makes it ideally suited for Iceberg tables, where performance, efficiency, and schema evolution are central concerns.

At its core, Parquet stores data by columns rather than rows in distinct row groups illustrated in figure 4.7. This design enables a range of optimizations. When a query engine such as Apache Spark, Dremio, or Trino scans a Parquet file, it can read only the columns relevant to the query, skipping over unnecessary data. This selective access significantly reduces I/O, especially for wide tables with many fields. Parquet also supports advanced compression techniques such as dictionary and run-length encoding, which compress data within each column block far more effectively than generic file-level compression.



**Figure 4.8** Based on the metadata in a parquet row group footers, we can determine whether a particular row group needs to be scanned or can be skipped for faster reading.

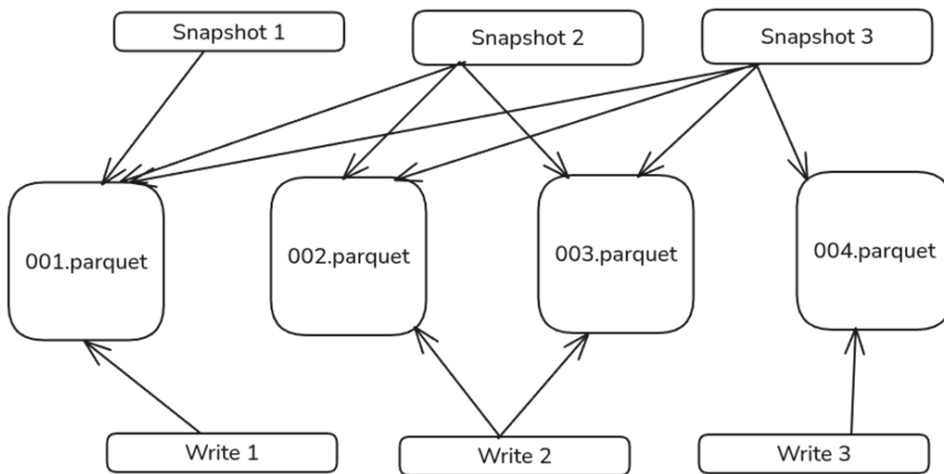
## ROW GROUPS AND PREDICATE PUSHDOWN

Rather than storing data row by row like traditional formats (e.g., CSV), Parquet organizes data column by column, enabling better compression and faster read performance for analytical workloads. Parquet files are divided into row groups, which are the fundamental units of storage and I/O. Each row group contains a subset of the dataset's rows, and within each row group, data is stored in column chunks, making it efficient to scan only the columns relevant to a query.

Row groups play a critical role in optimizing performance. Because each row group contains its own column-wise metadata—such as minimum and maximum values for each column—the system can make intelligent decisions about which row groups to scan when processing a query. Larger row groups reduce metadata overhead and improve scan throughput, while smaller row groups can offer more precise data skipping. Striking the right balance when defining row group size is essential for tuning performance, particularly in distributed processing environments like Spark, Dremio, or Trino.

This is where predicate pushdown becomes powerful. Predicate pushdown allows query engines to push filters—such as `WHERE id = 2`—down to the storage level before data is read into memory. Using the statistics stored in the row group's metadata, the engine can skip entire row groups if it determines that none of the rows match the filter condition. This significantly reduces the amount of data read and processed, improving performance and lowering resource usage. Together, row groups and predicate pushdown are central to Parquet's ability to enable fast, scalable analytics.

For Iceberg, which is built to manage large collections of immutable data files, Parquet offers several strategic advantages. Each write to an Iceberg table typically produces one or more new Parquet files. These files are then tracked by Iceberg's metadata structures, allowing query engines to prune files based on statistics, partition values, and file-level metadata. Parquet's structure supports this model elegantly, with each file containing embedded metadata about column ranges, null counts, and data sizes. This makes it possible for Iceberg to skip irrelevant files during query planning, reducing scan time and compute usage.



**Figure 4.9** Each write to the table introduces new files, while the metadata of each snapshot tracks the files in that table at that time.

Parquet is also schema-aware. It stores not only the data but also the schema, including field names, types, and optionality. This makes it resilient to schema evolution, a critical feature for dynamic datasets. When Iceberg performs schema changes such as column renaming, reordering, or type widening, the compatibility between Parquet files can be validated and enforced. Parquet's support for nested data structures using a compact encoding model also enables rich data modeling without sacrificing performance.

Performance tuning in Parquet involves controlling the size and layout of files. Optimal file sizes typically fall between 128 MB and 1 GB, depending on workload characteristics and query patterns. Small files can degrade performance by increasing metadata overhead and opening too many I/O streams, while very large files may reduce the effectiveness of pruning. Understanding this balance is key to getting the most out of Parquet in an Iceberg lakehouse.

Because of its ubiquity, Parquet is supported across nearly all data processing and analytics tools. This broad compatibility simplifies integration and futureproofs your storage layer. Whether you are using cloud-native compute engines or running on-premises Hadoop clusters, Parquet offers a consistent and efficient foundation for analytical data.

In Apache Iceberg, Parquet's structure is not just a file format—it's a foundational part of how data is organized and accessed efficiently. Iceberg builds on Parquet's columnar design by maintaining a metadata layer that includes manifests: structured files that act as routing indexes to Parquet data files. These manifests store file-level statistics such as minimum and maximum values for each column, null counts, and row counts. By scanning this metadata first, query engines can avoid reading files that don't match filter conditions, significantly reducing I/O. Furthermore, because Parquet files themselves contain column-level statistics, Iceberg can combine these with its own metadata to perform precise pruning at both the file and column level. This metadata-driven access model allows Iceberg to scale queries to petabyte-scale datasets while keeping response times low. In this way, Parquet's internal features—when leveraged by Iceberg—become the engine for intelligent, efficient data access.

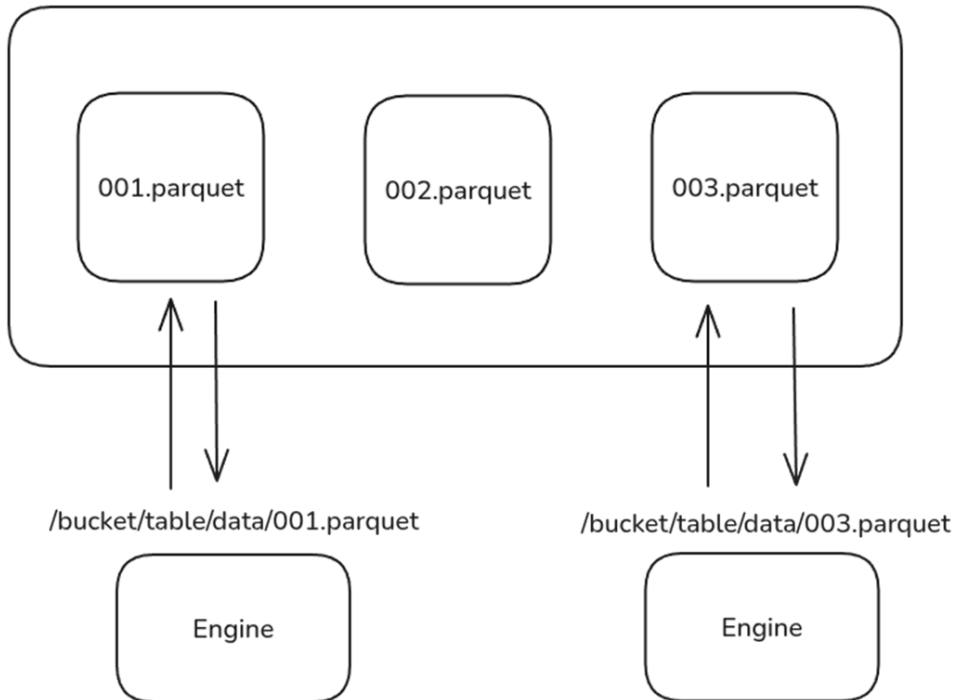
### 4.3.2 The S3 API

The S3 API, short for the Simple Storage Service API, originated as the interface for Amazon S3 and has since become the dominant protocol for interacting with object storage systems. What began as a proprietary access method for Amazon's cloud storage quickly evolved into a de facto standard across the industry. Today, the S3 API underpins nearly every major object store, from public cloud services to on-premises solutions, enabling consistent, HTTP-based interaction with large-scale storage systems.

The widespread adoption of the S3 API has had a transformative effect on data architecture. By standardizing the way systems create, retrieve, and manage objects, it has abstracted storage access from vendor-specific infrastructure. This has opened the door for broad interoperability between storage systems, data platforms, and processing engines. Apache Iceberg, for instance, can operate seamlessly on any S3 API-compatible object store, whether hosted by Amazon, MinIO, Ceph, or another vendor.

From a design perspective, the S3 API is built around flat object namespaces rather than traditional hierarchical filesystems. Objects are accessed via unique keys within a bucket, and each object is treated as an immutable binary blob. While this flat structure may seem simplistic compared to a file system, it enables scalability and parallelism at cloud scale. Because each object is independently addressable via HTTP, clients can retrieve data from multiple objects concurrently without concern for locking or directory structure.

For Iceberg lakehouses, this model is ideal. Iceberg tables are composed of large numbers of immutable files, both data files and metadata files, each of which can be independently read, written, and versioned. The S3 API allows Iceberg-compatible tools to interact with these files using predictable, resilient patterns. For example, when committing a new snapshot, Iceberg writes metadata files and then performs an atomic commit operation by updating a pointer file. This workflow maps cleanly to the write-once-read-many semantics of the S3 API.



**Figure 4.10** Different engines working with the same Iceberg table can individually request files concurrently from the object storage system.

Another advantage of the S3 API is its ecosystem. Many operational tools, such as backup systems, monitoring dashboards, and access control frameworks, are designed to work natively with S3 endpoints. This reduces integration overhead and allows platform teams to apply common policies and tooling across storage deployments. The S3 API also supports features such as versioning, lifecycle policies, and pre-signed URLs, which can be leveraged to enforce governance, manage cost, or support temporary data sharing.

However, it is important to note that while the S3 API is widely supported, not all implementations behave identically. Differences in eventual consistency, performance characteristics, and API completeness can affect how well a given object store supports Iceberg's operational model. For this reason, it is essential to evaluate not only whether a storage solution claims S3 compatibility, but also how mature and performant that implementation is under production workloads.

In modern Iceberg lakehouses, the S3 API serves as the connective tissue between the storage layer and the rest of the architecture. Its ubiquity enables flexibility, its simplicity supports scale, and its alignment with Iceberg's design principles makes it a cornerstone of any serious deployment.

## 4.4 Storage solutions

With a solid understanding of your storage requirements, the core architectural models, and the underlying standards, you're now ready to explore the concrete storage technologies available for implementing an Apache Iceberg lakehouse. This section surveys a range of storage solutions, each with distinct design philosophies, deployment models, and operational trade-offs. Rather than prescribing a single "best" choice, the goal here is to provide you with the historical context, technical capabilities, and practical fit of each option so you can map them to the requirements surfaced during your platform audit.

### HOW STORAGE ARCHITECTURE SHAPES DURABILITY, AVAILABILITY, AND LATENCY

When evaluating storage solutions for an Iceberg lakehouse, it's critical to understand how key characteristics—durability, high availability (HA), and latency, are implemented within different architectural models. These features are not just vendor capabilities; they emerge from how storage is physically and logically organized.

#### Co-located Storage (e.g., block storage with HDFS)

In this model, storage is tightly coupled with compute resources. Data resides on disks directly attached to the same machines that run processing engines. Durability and availability depend on software-managed replication (e.g., HDFS typically uses 3x replication across nodes). Failures require intervention from cluster orchestration tools, and recovery can be slow if data needs to be re-replicated manually or from cold storage. Latency is typically low, as reads and writes happen close to the processor, but scaling capacity means provisioning more compute as well—a costly and rigid tradeoff.

#### Separated Storage (e.g., object storage like S3 or MinIO)



In this model, storage is decoupled from compute and accessed via APIs over the network. Durability is achieved through automatic, vendor-managed replication across regions or availability zones. For instance, AWS S3 replicates data across multiple facilities, with no user configuration required. High availability is built into the platform through distributed architecture and global failover support. Latency is higher and more variable than block storage, due to network hops and eventual consistency mechanisms, but the model allows independent scaling of compute and storage, reduced operational overhead, and built-in resiliency.

Understanding these distinctions helps align storage choices with workload needs. Low-latency, tightly controlled environments may still prefer co-located storage, while scalable, cost-efficient, and cloud-native architectures are increasingly built on separated object stores.

We'll begin with Hadoop Distributed File System (HDFS), which continues to be widely used in legacy big data environments and is often the starting point for organizations modernizing their data platforms. From there, we'll turn to the major public cloud providers: Amazon S3, Google Cloud Storage, and Azure Blob Storage. These services represent the mainstream of cloud-native object storage and form the backbone of many Iceberg deployments today.

Next, we'll explore a variety of on-premises and hybrid solutions that offer S3-compatible object storage, including MinIO, Ceph, NetApp StorageGRID, Pure Storage, Dell ECS, and Wasabi. These options are particularly relevant for organizations with regulatory, residency, or performance requirements that rule out a fully public cloud approach. Each vendor brings different strengths, from cost predictability to Kubernetes-native integration, and understanding these distinctions is key to making an informed, sustainable choice.

For each storage solution, we'll look at its origins, how it fits into the Iceberg ecosystem, and which requirement profiles it best supports. Whether you're extending an existing infrastructure or starting fresh in the cloud, the following sections will help you choose the right foundation for your Iceberg lakehouse.

#### **4.4.1 Vendor Comparison Summary**

With so many storage options available, selecting the right solution for your Apache Iceberg lakehouse ultimately comes down to aligning technical capabilities with your specific performance, governance, and operational requirements. The vendors discussed in this section represent a spectrum, from highly performant enterprise appliances to simple, cost-effective cloud offerings. Each brings its own trade-offs around scalability, ecosystem integration, and ease of management.

To support a more structured evaluation, table 4.1 summarizes key characteristics across the ten vendors we've reviewed. While this comparison does not capture every nuance, it offers a useful high-level view of how these platforms differ in areas that commonly influence Iceberg storage decisions.

**Table 4.1 Summary of Storage Vendors**

| Vendor               | Deployment Model      | S3 API Support | Security & Compliance    | Cost Model                   |
|----------------------|-----------------------|----------------|--------------------------|------------------------------|
| HDFS                 | On-prem               | No             | Varies, typically basic  | CapEx-heavy                  |
| Amazon S3            | Cloud (AWS)           | Native         | Enterprise-grade         | Pay-per-use (incl. egress)   |
| Google Cloud Storage | Cloud (GCP)           | Native         | Enterprise-grade         | Tiered, usage-based          |
| Azure Blob/ADLS      | Cloud (Azure)         | Native         | Enterprise-grade         | Tiered, usage-based          |
| MinIO                | On-prem, Hybrid       | Full           | Strong with custom setup | Free/open-source             |
| Ceph                 | On-prem               | Full (via RGW) | Highly configurable      | Open-source, infra-dependent |
| NetApp StorageGRID   | On-prem, Hybrid       | Full           | Strong, WORM, lifecycle  | Enterprise licensing         |
| Pure Storage         | On-prem Appliance     | Full           | Robust, integrated       | Premium appliance            |
| Dell ECS             | On-prem, Hybrid       | Full           | Enterprise-grade         | Enterprise licensing         |
| Wasabi               | Cloud (Vendor-hosted) | Full           | Basic, improving         | Flat-rate, no egress fees    |

This table is not a substitute for hands-on evaluation, but it can guide initial conversations and help stakeholders narrow the field. For many organizations, the right solution may involve multiple storage backends working together, such as pairing on-prem object storage for sensitive data with cloud-native stores for scalable analytics. The modular nature of Iceberg allows for this kind of hybrid flexibility, provided each backend adheres to the principles of immutability, consistency, and S3-compatible access.

As you proceed, use the requirements outlined earlier in this chapter to benchmark options against your operational realities and strategic goals. Storage may be the lowest layer in your lakehouse stack, but the choices you make here ripple upward, affecting performance, governance, and ultimately, your ability to deliver trusted, scalable insight from your data.

#### 4.4.2 Hadoop (HDFS)

For organizations deeply invested in Hadoop, HDFS remains a stable and performant option. It allows for continued use of existing clusters while adopting Iceberg's benefits, such as ACID guarantees, schema evolution, and time travel, on top of a familiar storage foundation. However, for new deployments or those planning to scale beyond the bounds of a single data center, HDFS increasingly serves as a transitional layer rather than a long-term strategic choice.

Hadoop Distributed File System (HDFS) has played a foundational role in the evolution of big data platforms. Designed as part of the original Apache Hadoop project, HDFS was built to provide scalable, fault-tolerant storage on clusters of commodity hardware. It divides large files into fixed-size blocks, replicates them across nodes for resilience, and exposes a distributed file system interface to clients and processing frameworks.

For many organizations, HDFS was the first viable alternative to traditional data warehouses for storing and analyzing massive volumes of structured and semi-structured data. Paired with processing engines like MapReduce and later Apache Spark, HDFS enabled parallel data access at scale and laid the groundwork for the early big data ecosystem. Even today, it remains a staple in on-premises environments where data locality, low-latency access, and tight integration with existing Hadoop tooling are prioritized.

In an Apache Iceberg lakehouse, HDFS continues to offer value under specific conditions. Because Iceberg is format-agnostic at the storage layer, it can manage Parquet files stored in HDFS just as easily as in object storage. Organizations with mature Hadoop deployments may choose to introduce Iceberg gradually, using it to modernize metadata and table management without immediately replacing the underlying storage infrastructure. This allows for an incremental transition toward a more flexible and modular architecture.

That said, there are limitations to consider. HDFS was not designed for the elastic scaling, global accessibility, or API-driven workflows that characterize modern lakehouses. It requires significant operational overhead, including disk provisioning, replication configuration, cluster balancing, and fault recovery. It also lacks the native support for HTTP-based access and object-level metadata that many newer tools and platforms expect.

Moreover, HDFS has limited support for multi-tenancy and fine-grained access control compared to cloud-based object stores or newer S3-compatible solutions. These constraints can create friction in environments where multiple teams share infrastructure or where regulatory compliance demands robust isolation and auditing capabilities.

#### 4.4.3 Amazon S3

For organizations already committed to the AWS ecosystem, Amazon S3 provides a reliable, performant, and secure foundation for Iceberg lakehouses. Its global infrastructure, mature feature set, and first-class support across the data ecosystem make it the default option for many cloud-native deployments. However, it's essential to pair this power with thoughtful cost and architecture management to ensure long-term efficiency and sustainability.

Amazon Simple Storage Service (S3) is the original implementation of what is now the industry-standard S3 API and remains the most widely used object storage solution in the cloud. Since its launch in 2006, S3 has evolved into a highly durable, massively scalable storage platform that supports a wide range of analytics, machine learning, and operational workloads. For Iceberg lakehouses deployed on AWS, Amazon S3 is the natural choice for object storage.

S3's architecture abstracts storage into buckets and objects, enabling highly concurrent access via simple HTTP operations. Its global availability, native integration with AWS identity and access management (IAM), and mature ecosystem of tooling make it particularly attractive for enterprise-scale data platforms. Features such as versioning, object locking, lifecycle rules, and cross-region replication provide fine-grained control over data retention, security, and disaster recovery.

In the context of Apache Iceberg, Amazon S3 offers seamless compatibility. Iceberg's metadata and data files can be stored directly in S3 buckets, and most compute engines, including Apache Spark, Dremio, Trino, and Amazon Athena, offer native support for querying Iceberg tables stored in S3. The combination of Iceberg's transactional metadata and S3's object immutability model allows for safe concurrent reads and writes, supporting scalable workloads across multiple teams and services.

Performance on S3, while generally strong, does require some architectural consideration. Because S3 is designed for eventual consistency in some operations (such as list and overwrite), care must be taken when designing commit workflows and cleanup jobs. Iceberg's design helps mitigate many of these concerns by using atomic pointer updates in catalogs and file-level tracking in metadata files, but environments with very high commit frequency or real-time ingestion should still monitor latency and consistency behavior carefully.

Another important consideration is cost. S3 pricing is based on a combination of storage volume, request frequency, data transfer, and optional features such as intelligent tiering or Glacier archival. For data platforms with frequent small object access or large-scale inter-region queries, these costs can become significant. However, AWS offers tools and configuration options to optimize usage patterns and control expenses, including tiered storage classes and intelligent lifecycle policies.

Security and governance are also well-supported. Integration with AWS IAM enables access control, and native encryption options, both at rest and in transit, support compliance with enterprise and regulatory requirements. S3 also integrates with services like AWS CloudTrail and AWS Config for auditing and policy enforcement, making it easier to meet internal and external standards for data protection.

### **AMAZON S3 TABLE BUCKETS**

Amazon S3 Table Buckets are a specialized type of S3 bucket designed for analytical tables as first-class objects, optimized for analytics. Unlike general-purpose buckets, table buckets integrate natively with Apache Iceberg, allowing you to define and manage Iceberg tables directly within S3.

These buckets support automatic maintenance to optimize storage costs and performance, and they integrate with AWS Glue Data Catalog and other AWS analytics services. You can also access them through open-source query engines using the Amazon S3 Tables Catalog and its REST API.

Each table bucket is private by default and governed through IAM-based policies. This ensures secure, fine-grained access control across data consumers and roles. While table buckets currently have regional limits (10 per region by default), they provide a scalable and managed option for Iceberg users looking to align tightly with AWS-native infrastructure.

#### 4.4.4 Google Cloud Storage

Google Cloud Storage is particularly compelling for organizations already leveraging Google's analytics ecosystem, including BigQuery, Dataflow, and Vertex AI. Its strong consistency model, competitive performance, and cost-effective tiering make it an excellent choice for Iceberg lakehouses that prioritize elasticity and integration across the data lifecycle.

Google Cloud Storage (GCS) is Google's fully managed, scalable object storage service and a foundational component of its data analytics ecosystem. Built on the same infrastructure that powers products like Gmail and YouTube, GCS is designed for high availability, global accessibility, and deep integration with Google's suite of data services. For organizations building Iceberg lakehouses on Google Cloud, GCS offers a reliable and flexible storage backend.

Like Amazon S3, GCS organizes data into buckets and objects and is accessed via a RESTful API. It supports strong read-after-write consistency, which can be beneficial for Iceberg's transactional model, especially during concurrent metadata commits or snapshot reads. GCS's consistency guarantees reduce the risk of delayed file visibility, simplifying architecture decisions around ingestion and cleanup.

GCS offers multiple storage classes optimized for different access patterns, including Standard, Nearline, Coldline, and Archive. These classes allow organizations to balance cost and performance by segmenting data according to usage frequency and retention policies. Iceberg tables, for example, can store frequently accessed metadata and recent partitions in Standard storage, while older snapshots and historical data can be transitioned automatically to colder, more cost-effective tiers.

In terms of performance, GCS is well-suited for Iceberg workloads. Its low-latency object retrieval and strong internal network backbone support high-throughput analytics jobs across large datasets. When used with engines like BigQuery, Spark on Dataproc, or open-source query engines like Trino, GCS provides consistent performance for reading and writing Iceberg tables. Google's data transfer acceleration and parallel object access further enhance throughput for batch and interactive queries.

Security and compliance are also strengths of GCS. The platform supports server-side encryption by default, with options for customer-managed encryption keys (CMEK) and hardware security module (HSM) integration. Access control is enforced through IAM policies, which can be configured at the bucket, object, or project level. GCS also integrates with Google Cloud's audit logging and policy management tools, providing visibility and control over storage usage.

#### 4.4.5 Azure Blob Storage and ADLS

Microsoft Azure offers two closely related object storage services that can serve as the foundation for Iceberg lakehouses: Azure Blob Storage and Azure Data Lake Storage (ADLS). Both are built on the same core platform but are differentiated by their access semantics and feature sets. Together, they provide a flexible and scalable foundation for storing data in the Azure ecosystem, supporting a wide range of analytics and governance requirements.

**Azure Blob Storage** is the fundamental object storage service in Azure. It provides a flat namespace for storing blobs (binary large objects), which can be accessed over HTTPS using the REST-based Blob API or the S3-compatible API in certain configurations. Blob Storage is designed for high durability, geo-redundancy, and broad compatibility, making it a viable backend for Iceberg tables in Azure-centric deployments.

Blob Storage supports several performance tiers, Hot, Cool, and Archive, that align with access frequency and cost goals. This tiered structure allows organizations to optimize storage cost over time by automatically transitioning older Iceberg data to cooler tiers without affecting accessibility. Integration with Azure services like Event Grid and Azure Functions enables event-driven workflows for ingestion, processing, and lifecycle management.

**Azure Data Lake Storage Gen2 (ADLS)** builds directly on Blob Storage but introduces a hierarchical namespace and file system semantics. This includes directory structures, atomic rename operations, and fine-grained access control using Azure Active Directory (AAD) and POSIX-style ACLs. These enhancements are especially useful in analytics scenarios where workloads expect a file system-like interface and where security policies require granular permission management.

#### AZURE ACTIVE DIRECTORY AND ACLS

Azure Active Directory (Azure AD) is Microsoft's cloud-based identity and access management service. It helps organizations manage users, groups, and access to applications and resources across cloud and on-premises environments. Azure AD is integral to managing authentication and authorization in Microsoft Azure, allowing enterprises to enforce secure sign-ins, enable single sign-on (SSO), and implement role-based access control (RBAC). It supports integration with thousands of SaaS applications and can federate identities across domains, making it a central piece in modern identity management strategies.

On the other hand, POSIX-style Access Control Lists (ACLs) are a file system-level permission model traditionally used in Unix and Linux environments. POSIX ACLs provide fine-grained control over who can read, write, or execute files and directories. Unlike basic Unix permissions (owner/group/other), ACLs allow administrators to specify permissions for individual users or groups beyond the file owner or group. This makes them highly flexible for multi-user environments where access requirements can't be easily expressed with standard permission bits alone.

In the context of cloud storage—such as Azure Data Lake Storage Gen2—both Azure AD and POSIX-style ACLs can work together to manage access. Azure AD handles authentication (verifying who the user is), while POSIX ACLs manage authorization (what the user can do with the data). For example, a user may be authenticated via Azure AD and then granted read-only or write access to specific directories or files based on the ACLs set within the storage account. This layered security model allows for robust, enterprise-grade governance over data access.

For Apache Iceberg, both Blob Storage and ADLS are compatible storage backends. Most engines that support Iceberg on Azure, such as Spark on Azure Synapse, Dremio, and Trino, offer configuration options for either service. ADLS is often preferred when tighter access controls and file system operations are needed, while Blob Storage may be chosen for its simplicity and broader ecosystem support.

Performance in both services is competitive, particularly for batch workloads and large file scans. ADLS's hierarchical namespace introduces some additional latency for metadata-intensive operations, but this is typically offset by improved compatibility with Hadoop-style processing engines. Blob Storage's flat namespace, on the other hand, performs better in object-intensive operations where directory traversal is less relevant.

Security is a strong point across both services. Azure provides built-in encryption at rest, with support for customer-managed keys and integration with Azure Key Vault. Access is tightly integrated with Azure IAM and AAD, allowing consistent policy enforcement across storage and compute services. Diagnostic logging and activity tracking can be enabled for auditing and compliance monitoring, making these services well-suited for regulated environments.

When architecting an Iceberg lakehouse on Azure, the choice between Blob Storage and ADLS often depends on workload patterns and governance needs. For simple, large-scale object access, Blob Storage is often sufficient and easier to integrate. For environments that demand fine-grained control, compatibility with Hadoop file systems, or enterprise-scale security policies, ADLS provides a more specialized and capable foundation.

## MICROSOFT ONELAKE AND THE FABRIC DATA PLATFORM

Microsoft OneLake is the unified data lake storage layer underpinning Microsoft Fabric, a SaaS-based analytics platform that integrates data engineering, data science, real-time analytics, and business intelligence. OneLake provides a single, logical storage system for the entire Fabric ecosystem, similar in concept to a "data mesh" on top of Azure Data Lake Storage (ADLS) Gen2.

OneLake supports native integration with open table formats like Apache Iceberg and Delta Lake, enabling interoperability with external engines and third-party tools. It also provides shortcuts to external storage (such as linking to existing ADLS Gen2 or AWS S3 buckets) without physically moving the data, supporting hybrid and multi-cloud strategies.

### 4.4.6 MinIO

While it lacks the global infrastructure and built-in services of a public cloud provider, MinIO's flexibility, standards compliance, and performance make it a strong candidate for production Iceberg deployments that demand control and speed. As Iceberg adoption grows in hybrid and edge environments, MinIO continues to be one of the most versatile object storage options available.

MinIO is an open-source, high-performance object storage solution that is fully compatible with the S3 API. Designed to bring cloud-like scalability and simplicity to on-premises and hybrid environments, MinIO has become a popular choice for organizations that want to run Iceberg lakehouses outside of public cloud providers without sacrificing modern architecture principles.

At its core, MinIO offers a minimalist, container-friendly design that is easy to deploy, scale, and manage. It supports deployment across physical servers, virtual machines, Kubernetes clusters, and even edge environments. Its lightweight footprint and strong performance characteristics make it ideal for scenarios where control, cost predictability, and performance tuning are top priorities.

MinIO's S3 compatibility is more than superficial. It is frequently updated to maintain close alignment with Amazon's API specification, enabling seamless integration with the broader Iceberg ecosystem. Processing engines such as Apache Spark, Dremio, and Trino can interact with MinIO in the same way they do with AWS S3, allowing developers to transition workflows without reengineering ingestion or query pipelines.

Performance is one of MinIO's key differentiators. It is optimized for small object operations and high-throughput data workloads, offering impressive benchmarks for both read and write operations. For Iceberg deployments, this translates into fast metadata commits, efficient data file retrieval, and reduced latency for concurrent access across multiple workloads. MinIO also includes features like erasure coding for durability, inline compression, and object locking to support compliance and fault tolerance.



## ERASURE ENCODING, INLINE COMPRESSION AND OBJECT LOCKING

Erasure encoding (or erasure coding) is a data protection technique used in modern storage systems to provide fault tolerance with greater storage efficiency than traditional replication. Instead of storing multiple full copies of data, erasure encoding breaks data into fragments and generates additional parity fragments using mathematical algorithms. These fragments are then distributed across different storage nodes. If a portion of the data becomes unavailable due to hardware failure or corruption, the original data can be reconstructed from the remaining fragments and parity. This method is widely used in distributed object storage systems like Azure Blob Storage, AWS S3 with S3 Glacier, and MinIO to achieve durability and availability while using less space than full replication.

Inline compression refers to compressing data on the fly as it is being written to storage. This reduces the storage footprint without requiring a separate post-processing step. Inline compression is especially valuable in columnar formats like Parquet or ORC, and is also built into many object storage systems. It minimizes I/O and improves query performance by reducing the amount of data read from disk during processing. The key advantage is that it's transparent to the user or application—data is automatically compressed as it's stored and decompressed when retrieved, providing seamless performance and cost benefits.

Object locking is a feature in object storage systems that prevents objects from being modified or deleted for a specified period, even by users with full access privileges. This is crucial for compliance and regulatory use cases, such as financial record retention or legal holds, where data must remain immutable. Object locking can be implemented in two modes: governance mode, which allows authorized users to override the lock, and compliance mode, which does not permit any changes until the retention period expires. This feature is supported in systems like AWS S3 and MinIO, where it helps meet requirements like SEC Rule 17a-4 or HIPAA. It plays a critical role in building tamper-proof, audit-ready data architectures.

Security is well-supported in MinIO. It includes built-in support for encryption at rest and in transit, fine-grained access control through IAM-like policies, and integration with external identity providers such as LDAP or OpenID Connect. For organizations with strict governance or data residency requirements, these features provide a flexible and auditable security model without relying on external cloud services.

From an operational standpoint, MinIO is designed for simplicity. Its management API and user interface provide visibility into object usage, system health, and replication status. Administrators can configure bucket policies, monitor performance, and scale clusters with minimal overhead. For Kubernetes users, MinIO offers native operators to simplify lifecycle management and auto-scaling in containerized environments.

MinIO is particularly attractive for organizations that want to build Iceberg lakehouses in environments where cloud options are not viable, whether due to regulatory restrictions, cost concerns, or the need for localized data processing. It also serves well as a testing and development backend for teams that want a cloud-like experience on a laptop or in a CI/CD pipeline.

#### 4.4.7 Ceph

For Iceberg lakehouses, Ceph provides a compelling path to S3-compatible object storage in environments that prioritize openness, on-prem control, and architectural unification. It is a powerful option for mature infrastructure teams that want to avoid cloud dependency while retaining Iceberg's full capabilities.

Ceph is an open-source, unified storage platform that provides object, block, and file storage under a single distributed system. Originally developed by the open-source community and now maintained under the Ceph Foundation with support from Red Hat and others, Ceph is designed for flexibility, high availability, and scalability. In Iceberg lakehouse architectures, Ceph is most commonly deployed as an S3-compatible object store, offering a robust alternative to cloud-based storage in private data centers or hybrid environments.

At the heart of Ceph is its RADOS (Reliable Autonomic Distributed Object Store) layer, which handles data distribution, replication, and consistency across a cluster of nodes. On top of RADOS, Ceph provides several interfaces, including RBD (block storage), CephFS (POSIX-compliant file system), and RGW (RADOS Gateway), which exposes an S3-compatible API. It is this RGW component that enables Ceph to function as a backend for Apache Iceberg tables in environments where object storage is preferred but public cloud usage is constrained.

One of Ceph's strengths is its fault tolerance. It uses distributed replication or erasure coding to ensure data durability across nodes and supports self-healing and rebalancing to recover from hardware failures with minimal manual intervention. This makes it a strong fit for organizations operating large-scale, multi-node storage clusters that need to ensure uptime and data integrity.

Ceph's performance profile can vary depending on deployment configuration, underlying hardware, and network bandwidth. When tuned properly, it supports high-throughput and concurrent access patterns typical of Iceberg workloads. However, it does require careful planning around cluster topology, replication factors, and IOPS provisioning to achieve optimal results. Ceph's operational complexity is generally higher than that of MinIO or commercial solutions, making it best suited for teams with deep systems expertise.

In terms of Iceberg compatibility, Ceph's S3 API support allows it to integrate with a wide range of engines and tools. However, implementations may occasionally lag behind the latest S3 features or exhibit subtle differences in consistency behavior, which can affect certain edge cases in high-concurrency environments. It is important to validate these behaviors when deploying Iceberg in production on top of Ceph.

Security in Ceph is highly configurable. It supports access control via role-based permissions, TLS encryption for data in transit, and integration with external authentication systems such as LDAP or Keystone (in OpenStack environments). While powerful, these features often require custom configuration, and organizations must invest in proper monitoring and policy enforcement to maintain a secure posture.

Ceph is particularly well-suited for organizations operating private clouds, research institutions, or service providers looking to offer multi-tenant object storage. Its flexibility to support different types of storage workloads under one system can also simplify infrastructure management. However, its adoption comes with an expectation of operational discipline and resource planning.

#### 4.4.8 NetApp StorageGRID

StorageGRID is a strong choice for organizations seeking a stable, policy-driven, and compliant storage platform for Iceberg lakehouses. It offers the consistency and compatibility needed for Iceberg metadata management, while also supporting enterprise priorities around cost control, data governance, and infrastructure integration.

NetApp StorageGRID is an enterprise-grade, software-defined object storage solution designed for large-scale, hybrid cloud deployments. With strong support for the S3 API and an architecture focused on long-term data retention, policy-driven lifecycle management, and regulatory compliance, StorageGRID is well-suited for organizations operating Iceberg lakehouses in complex, multi-environment infrastructures.

Originally built for archiving and unstructured data storage, StorageGRID has evolved into a versatile platform capable of serving as the primary object storage layer for analytics and data lake workloads. It is available as a software appliance for deployment on existing hardware, as an integrated hardware solution from NetApp, or as part of a hybrid cloud strategy integrating on-premises systems with public cloud storage.

For Apache Iceberg, StorageGRID's S3-compatible interface enables seamless integration with engines such as Apache Spark, Trino, and Dremio. Iceberg tables can be stored and managed on StorageGRID with the same API semantics used in cloud-native environments, making it a practical option for organizations that require on-premises deployment due to data residency, performance control, or regulatory mandates.

A distinctive feature of StorageGRID is its support for intelligent data lifecycle policies. Administrators can define rules for object placement, retention, deletion, and tiering based on metadata, access patterns, or organizational policies. This capability aligns well with Iceberg's snapshot-based model, where different versions of the same dataset may have different storage and retention requirements. StorageGRID can automate archival and cost-optimization strategies without disrupting data availability or consistency.

Performance is tuned for large object workloads, with strong support for concurrent access and high-throughput ingestion. While not as latency-optimized as systems designed for transactional use cases, StorageGRID performs well in batch and analytical workflows typical of Iceberg usage. It also supports erasure coding, geo-dispersed replication, and service-level objectives that help ensure durability and availability across distributed environments.

From a governance and security perspective, StorageGRID includes robust features such as WORM (write once, read many) compliance, access logging, and S3 Object Lock. These capabilities make it particularly attractive for sectors like finance, healthcare, and government, where auditability and immutability are legal or regulatory requirements. Integration with Active Directory, TLS encryption, and bucket-level IAM controls further strengthen its enterprise readiness.

### WRITE ONCE, READ MANY (WORM)

WORM (Write Once, Read Many) is a data storage model designed to ensure that once data is written, it cannot be altered or deleted. This immutability makes WORM storage ideal for preserving records that must remain unchanged for regulatory, legal, or audit purposes. Common examples include financial records, medical data, logs, and compliance-related documents. By guaranteeing data integrity over time, WORM helps organizations meet strict data retention requirements and protect critical information from tampering or accidental loss.

WORM can be implemented at both the hardware and software levels. Traditional WORM solutions were built on specialized hardware like optical discs or tape drives. Today, cloud object storage platforms—such as AWS S3 Object Lock, Azure Immutable Blob Storage, and Google Cloud Storage Retention Policies—offer software-defined WORM capabilities. These platforms allow users to set retention periods during which objects are locked, ensuring data cannot be modified or deleted, even by administrators. Some systems also support legal hold features that preserve data until a specific condition is cleared, regardless of the initial retention period.

The WORM model plays a crucial role in compliance and governance frameworks. Industries such as finance, healthcare, and government are often subject to strict regulations like SEC Rule 17a-4, HIPAA, or GDPR, which require long-term, tamper-proof data retention. By using WORM storage, organizations can demonstrate adherence to these standards, protect against data-related liabilities, and support secure archival and audit processes. As data volumes grow and regulatory scrutiny increases, WORM capabilities have become a fundamental requirement in enterprise storage strategies.

Operationally, StorageGRID is designed for centralized management at scale. Its web-based interface and automation features enable IT teams to monitor system health, configure multi-site deployments, and enforce policies across vast datasets. Support for Kubernetes CSI drivers and integration with NetApp's cloud-native tools makes it increasingly viable in containerized and hybrid environments.

#### 4.4.9 Pure Storage

For Iceberg lakehouses, Pure Storage offers a high-end storage layer that can keep pace with the growing demands of data-hungry applications. Its enterprise orientation, predictable performance, and streamlined operations make it an excellent fit for organizations looking to modernize their data platform with minimal risk and maximum efficiency.

Pure Storage is a leading provider of high-performance storage hardware and software solutions, best known for its all-flash storage arrays. With the introduction of Pure Storage FlashBlade and the Pure Storage S3-compatible object interface, the company has entered the object storage domain with a strong emphasis on speed, simplicity, and enterprise integration. For Iceberg lakehouses that demand consistent high throughput, predictable latency, and easy operational management, Pure Storage offers a compelling option.

At the center of Pure Storage's object storage offering is FlashBlade, a scale-out file and object storage platform optimized for analytics, backup, and AI/ML workloads. It provides native S3 API support, allowing it to serve as a backend for Apache Iceberg table storage with minimal configuration. FlashBlade is engineered for performance at scale, enabling low-latency access to large volumes of data with consistent throughput across thousands of parallel clients.

This performance profile makes Pure Storage particularly attractive for high-concurrency environments, where Iceberg's separation of metadata and data can result in large numbers of small object operations during query planning and execution. FlashBlade's ability to serve these operations quickly and reliably helps reduce query times and enhances responsiveness for interactive workloads.

In addition to speed, Pure Storage emphasizes operational simplicity. Its storage systems are delivered as turnkey appliances with automated provisioning, intelligent performance tuning, and seamless upgrades. The Pure1 management platform provides cloud-based monitoring, predictive analytics, and proactive support features that reduce administrative overhead and improve system reliability. For platform teams managing an Iceberg lakehouse, this translates to more time spent building data products and less time tuning infrastructure.

Security and compliance features are also robust. FlashBlade supports encryption at rest, secure multi-tenancy, and detailed access controls. Audit logs and integration with identity providers such as LDAP or Active Directory support enterprise governance requirements. These capabilities make Pure Storage suitable for regulated industries that must maintain strict data controls without sacrificing performance.

## **LDAP AND EXTERNAL ENTITY PROVIDERS**

LDAP (Lightweight Directory Access Protocol) is a protocol used to access and manage directory information services, such as user identities, roles, and permissions, in a networked environment. LDAP is commonly implemented through directory services like Microsoft Active Directory, OpenLDAP, or 389 Directory Server. These systems store identity data in a hierarchical structure, allowing applications to authenticate users, look up user attributes, and manage access control. LDAP has long been a cornerstone of on-premises identity management, supporting centralized authentication and simplifying user provisioning across internal systems.

External Identity Providers (IdPs) extend the concept of identity management to the cloud and federated systems. These providers—such as Azure Active Directory, Okta, Auth0, and Google Identity—offer authentication and authorization services that can integrate across cloud applications, third-party services, and enterprise systems. Many support modern standards like SAML, OAuth2, and OpenID Connect, enabling Single Sign-On (SSO), multi-factor authentication, and conditional access. External IdPs decouple authentication from individual applications, providing a centralized way to enforce security policies, monitor identity activity, and reduce password sprawl.

Together, LDAP and external identity providers are essential to securing access in hybrid and multi-cloud environments. Organizations often integrate LDAP with external IdPs to bridge on-premises and cloud identity systems, allowing seamless access regardless of where applications are hosted. This hybrid model supports both legacy applications and modern SaaS platforms, providing flexibility, scalability, and stronger governance. As identity becomes the new perimeter in enterprise security, using robust directory protocols and cloud-native identity providers is critical for protecting data, enforcing compliance, and enabling secure collaboration across users and devices.

While Pure Storage is often viewed as a premium solution, it delivers value in scenarios where performance, simplicity, and support are top priorities. Organizations running latency-sensitive analytics, training large-scale ML models, or consolidating infrastructure under a single high-throughput platform may find the investment well justified.

#### 4.4.10 Dell ECS

For organizations seeking a mature, scalable, and secure object storage system to support an Iceberg lakehouse, Dell ECS delivers enterprise reliability with the flexibility of S3 compatibility. It is especially valuable in regulated, security-conscious environments where hybrid or on-prem storage is a requirement rather than a preference.

Dell EMC Elastic Cloud Storage (ECS) is Dell's enterprise-grade object storage platform designed for scalability, resiliency, and deep integration within hybrid and on-premises infrastructures. Built with a strong focus on durability, regulatory compliance, and multi-tenancy, ECS offers S3 API compatibility, making it a viable storage backend for Iceberg lakehouses in enterprise environments where cloud options may be limited or hybrid architectures are preferred.

ECS is engineered for large-scale object storage deployments across multiple sites, with built-in support for geo-replication, erasure coding, and software-defined storage. These capabilities make it well suited for disaster recovery, high availability, and long-term data preservation. Its design also supports horizontal scaling, enabling organizations to add capacity and performance incrementally as data needs grow.

For Apache Iceberg, Dell ECS provides the necessary object storage semantics to store and retrieve table data and metadata using the S3 API. It integrates with a variety of query engines and data processing tools, including Apache Spark and Presto, and can serve as a drop-in replacement for cloud object stores in fully on-prem or hybrid environments. This flexibility is particularly valuable for organizations that need to maintain data sovereignty or operate within constrained regulatory environments.

Performance-wise, ECS is tuned for high-throughput workloads and concurrent object access, which aligns well with Iceberg's pattern of frequent metadata reads and data file scans. However, as with many enterprise storage systems, achieving peak performance often depends on careful configuration of networking, storage pools, and replication policies. ECS deployments benefit from thoughtful infrastructure planning and proactive monitoring to ensure consistent responsiveness.

ECS distinguishes itself through its support for multi-tenancy and governance. It enables organizations to isolate storage environments by tenant, apply granular access controls, and enforce retention and compliance policies. These features make it a strong candidate for environments where storage must be securely partitioned between business units or external clients.

Security and compliance are central to ECS's value proposition. It offers robust access controls, encryption at rest and in transit, WORM capabilities, and audit logging. Integration with Active Directory and other enterprise identity providers allows centralized policy enforcement, while its ability to retain objects for legally mandated durations supports use cases in healthcare, finance, and government.

ECS is managed through a centralized administrative interface, which provides visibility into system health, capacity usage, and policy enforcement across clusters. Dell also offers tight integration between ECS and its broader infrastructure ecosystem, allowing customers to build out end-to-end data platforms using Dell networking, compute, and storage technologies.

#### **4.4.11 Wasabi**

Wasabi provides a lightweight, reliable, and cost-predictable object storage option for Iceberg lakehouses. Its full S3 API compatibility and no-surprises pricing model make it ideal for organizations looking to control cloud costs while maintaining compatibility with modern data platforms.

Wasabi is a cloud object storage provider known for its simplicity and aggressively competitive pricing model. Marketed as "hot cloud storage," Wasabi offers S3-compatible object storage without the complexities and variable costs associated with traditional cloud services. For organizations building cost-conscious Iceberg lakehouses or managing large volumes of frequently accessed data, Wasabi presents a compelling alternative.

Wasabi's storage model is designed to be straightforward: a flat per-terabyte monthly fee with no charges for data egress or API requests. This eliminates much of the unpredictability in budgeting and simplifies operational planning. For Iceberg workloads that involve frequent table scans, metadata reads, and periodic snapshot rewrites, this pricing structure can result in significant cost savings compared to tiered cloud services that meter each access or transfer.

From a technical standpoint, Wasabi supports the S3 API, making it compatible with the wide ecosystem of Iceberg-integrated tools and engines. Apache Spark, Trino, Dremio, and other compute engines can interact with Wasabi storage as they would with Amazon S3 or any other S3-compatible system, requiring only minor configuration adjustments.

Performance on Wasabi is designed for high availability and consistent throughput, particularly for workloads with high read and write intensity. While it may not match the ultra-low-latency characteristics of some premium-tier cloud storage services or all-flash on-premises systems, Wasabi performs reliably for most analytical scenarios, especially those with batch-oriented or periodic access patterns.

Wasabi's simplicity extends to its operational model. There are no storage tiers to manage, no lifecycle rules to configure for cost optimization, and no egress budgeting required. This makes it especially appealing for smaller data teams, startups, educational institutions, or organizations that want to avoid cloud lock-in or unexpected billing fluctuations. Wasabi also offers integrations with backup, archiving, and media management tools, extending its utility across non-analytic workloads.

In terms of security, Wasabi includes features such as immutable buckets for ransomware protection, encryption at rest, and support for MFA. While it does not offer the same depth of access control granularity as enterprise-focused platforms like AWS or Azure, it meets the core security needs of most analytics use cases and integrates with identity providers through S3-compatible authentication schemes.

The primary consideration with Wasabi is its focus. It is purpose-built for storage and does not offer the broader cloud services or data ecosystem found in hyperscale providers. As such, it is best positioned as a storage backend in a modular architecture, where compute and catalog services are hosted separately. This makes it well-suited for Iceberg lakehouses built on open, decoupled architectures where flexibility, transparency, and cost control are priorities.

## 4.5 Selecting based on requirements

Now that we've explored the core storage requirements and reviewed a broad range of storage solutions, it's time to bring these ideas together through a set of illustrative examples. This section presents a series of hypothetical scenarios mapped to the requirement categories introduced in section 4.1: performance, security, integrity, and operational cost. Each scenario highlights how varying priorities might influence your storage decisions.

These examples are not prescriptive recommendations. Rather, they are meant to demonstrate how you might think through the trade-offs between different storage options when applying Iceberg to diverse analytical needs. In practice, most organizations support multiple use cases on a single storage platform. You are unlikely to select different storage systems per use case due to the operational and financial overhead this would introduce. Instead, you will likely converge on a solution that meets your broadest set of needs within the boundaries of your existing cloud provider, regulatory obligations, and infrastructure maturity.



For example, if your architecture is anchored in AWS, Amazon S3 will naturally shape your storage design. If you're running Spark or Hive on-premises, HDFS may remain central, potentially mirrored by MinIO for cloud compatibility. These broader deployment considerations; cloud strategy, compliance boundaries, and existing tooling—will often play a larger role than individual technical features.

Still, the scenarios that follow are useful thought exercises. They provide a lens for assessing how particular requirements might elevate or eliminate specific options from your shortlist. Use them to test your assumptions, pressure-check candidate solutions, and understand where compromises may be necessary. The goal is not to prescribe a one-size-fits-all answer, but to give you a practical model for aligning storage choices with your organizational priorities, constraints, and long-term Iceberg lakehouse strategy.

### **4.5.1 Performance requirements**

Performance is often the most visible and scrutinized attribute of a storage system, particularly when users experience delays in data access or query execution. However, "performance" is not a singular metric, it includes throughput, latency, concurrency, and consistency, all of which can affect how well your Iceberg lakehouse supports different workloads. Let's explore a few hypothetical performance-driven scenarios to illustrate how different needs might guide the selection process.

#### **SCENARIO: INTERACTIVE DASHBOARDS WITH HIGH CONCURRENCY**

An organization runs a suite of real-time dashboards powered by a BI engine querying Iceberg tables. Dozens of users interact with these dashboards during business hours, triggering many small, frequent reads. In this case, low-latency object retrieval and consistent performance under concurrent load are critical.

A storage solution like Pure Storage or Amazon S3 with intelligent tiering and request-based optimization may be ideal. Pure Storage's low-latency flash hardware offers deterministic access times, while Amazon S3 provides elastic scalability that can absorb peak usage patterns. On the other hand, a system like Ceph, while flexible, might require extensive tuning to meet these interactive latency needs.

#### **SCENARIO: LARGE-SCALE BATCH ETL PROCESSING OVERNIGHT**

A data engineering team processes hundreds of gigabytes of raw event data each night, transforming it into partitioned Iceberg tables. The process is throughput-bound and tolerant of latency spikes, provided that total job duration remains within a multi-hour window.

Here, systems that prioritize sustained throughput over per-object latency, such as HDFS, Google Cloud Storage, or MinIO, may be more than sufficient. HDFS remains performant for large, sequential reads in legacy environments, while GCS offers strong consistency and predictable throughput, especially when co-located with compute. Cost-efficiency might also be a factor in selecting MinIO for self-managed clusters with stable traffic patterns.

## SCENARIO: TIME-SENSITIVE MACHINE LEARNING FEATURE PIPELINES

A machine learning team needs to load features from Iceberg tables into training jobs that run on GPU clusters. Because these jobs are scheduled and expensive to idle, consistent, low-latency access to training data is essential.

In this scenario, Azure Data Lake Storage (ADLS) or Amazon S3 with caching strategies may be appropriate. Both support strong integration with cloud-native compute, and their scalable access models ensure that training jobs are not bottlenecked on storage retrieval. A system like Wasabi, while reliable, may not offer the same latency profile under concurrent access at scale, depending on geography and network setup.

## SCENARIO: REGULATORY QUERIES WITH NON-URGENT TIMING

A compliance team runs scheduled reports against archived data stored in Iceberg tables. These queries are batch-oriented and do not require rapid response, but the underlying data must remain available and consistent.

NetApp StorageGRID or Dell ECS may be suitable here, as both support large-scale object access with enterprise durability and policy-based archival. Their performance characteristics align with long-running scans, and their ability to enforce data governance policies aligns with compliance needs. Performance in this case is a secondary consideration, shaped by reliability and cost.

These examples underscore how performance goals, when broken down by workload type and user expectation, can guide you toward storage systems optimized for specific access patterns. Importantly, many environments contain a mix of such workloads. A solution that excels for one use case may fall short for another. This makes it essential to evaluate not only the average performance but also the performance profile under your most demanding conditions.

### 4.5.2 Security requirements

Security requirements often take precedence over performance or cost when dealing with sensitive data, regulated industries, or multi-tenant environments. These requirements can span access control, encryption, auditability, and support for compliance standards such as HIPAA, GDPR, or FINRA. Let's examine a few hypothetical scenarios that show how specific security concerns may influence the choice of storage system for your Iceberg lakehouse.

## SCENARIO: HEALTHCARE DATA SUBJECT TO HIPAA REGULATIONS

A healthcare analytics firm uses Iceberg tables to manage patient records and clinical trial data. Access must be tightly controlled, all data must be encrypted at rest, and audit trails must be retained for every access and modification.

In this context, enterprise-grade solutions like NetApp StorageGRID or Azure Data Lake Storage (ADLS) are well-positioned. Both offer WORM capabilities, granular access policies, and integration with identity providers for strict access enforcement. ADLS, in particular, benefits from integration with Azure Active Directory and fine-grained ACLs, which can be used to enforce security controls down to the file level.

## SCENARIO: INTERNAL R&D DATA WITH RESTRICTED USER ROLES

A large enterprise runs machine learning models using proprietary data stored in Iceberg tables. Access must be restricted by business unit, with different teams having different levels of visibility into the same datasets.

Ceph and MinIO, when configured correctly, support multi-tenant deployments with role-based access controls. Ceph can integrate with Keystone or LDAP to enforce centralized identity policies, while MinIO provides IAM-like capabilities and can be extended with external identity providers. These systems offer flexibility but may require more manual configuration than cloud-native platforms.

## SCENARIO: PUBLIC CLOUD DEPLOYMENT WITH STRONG AUDITABILITY NEEDS

A financial institution operates its Iceberg lakehouse in a public cloud but must demonstrate audit trails for all data access and changes, including encryption key management. Amazon S3 and Google Cloud Storage are well-suited to this requirement, offering server-side encryption with customer-managed keys, object-level access logging, and integration with audit services like AWS CloudTrail and GCP Audit Logs. These platforms allow administrators to enforce and verify compliance through automation, making them ideal for highly regulated sectors that also benefit from cloud scalability.

## SCENARIO: SECURE DATA SHARING ACROSS JURISDICTIONS

An international organization needs to store and access Iceberg tables across multiple countries, each with its own data residency requirements and legal constraints on cross-border transfers.

Dell ECS and StorageGRID provide geo-distributed object storage with policy-based data placement and retention. These systems allow organizations to maintain control over where data is stored and how it is replicated, aligning with jurisdictional compliance without relying on public cloud providers. Their support for encryption, access auditing, and lifecycle governance also strengthens the overall security posture.

Security requirements can be subtle and context-sensitive. The presence of encryption and access controls alone is not enough, what matters is how these capabilities align with your organization's policies, how easily they can be audited, and how seamlessly they integrate with your existing identity and compliance infrastructure. In many cases, the right decision is not only about technology, but also about legal obligations and internal risk tolerance.

### 4.5.3 Integrity requirements

Integrity requirements center on the durability, recoverability, and consistency of data stored in your Iceberg lakehouse. These considerations are vital when dealing with mission-critical workloads, compliance-driven retention needs, or environments with limited operational tolerance for data loss. While Iceberg ensures metadata consistency and supports rollback through its snapshot model, the reliability of the underlying storage system is what ensures those snapshots and files are still available when needed. The following scenarios illustrate how integrity concerns might guide your storage selection.

## **SCENARIO: DISASTER RECOVERY IN A GEOGRAPHICALLY DISTRIBUTED ENTERPRISE**

A global logistics company manages Iceberg tables across multiple regions. It requires multi-site redundancy and fast recovery in the event of site failure, without risking data inconsistency or loss of recent updates. NetApp StorageGRID and Dell ECS offer built-in geo-replication and policy-driven data protection, ensuring that object replicas are distributed across data centers. These platforms also support versioning, object locking, and lifecycle rules, which help preserve integrity over time while supporting legal retention requirements. Their support for audit logging and restore capabilities adds further resilience in regulated environments.

## **SCENARIO: LONG-TERM RETENTION WITH VERIFIABLE IMMUTABILITY**

A government agency maintains historical records in Iceberg tables that must be retained for over a decade, with strict requirements for immutability and tamper resistance. Amazon S3, Wasabi, and NetApp StorageGRID each support object locking and WORM capabilities. These features prevent deletions or overwrites for a defined retention period, supporting legal and archival mandates. S3 also allows organizations to enforce these policies at the bucket level with compliance mode locks, while Wasabi makes immutability a default part of its storage model, simplifying secure long-term storage.

## **SCENARIO: FREQUENT SCHEMA EVOLUTION AND METADATA UPDATES**

A startup is building an Iceberg-based feature store and rapidly iterates on schema definitions. With constant metadata commits and schema changes, the integrity of the metadata and the reliability of file references must be maintained under frequent churn.

Google Cloud Storage and Azure Data Lake Storage provide strong consistency guarantees, including atomic object overwrites and immediate object visibility. These properties reduce the risk of stale or partially visible metadata, ensuring that Iceberg operations like snapshot creation and commit rollback behave predictably even under high write frequency.

## **SCENARIO: EDGE DEPLOYMENT WITH LIMITED NETWORK RELIABILITY**

An industrial IoT platform stores Iceberg tables in a distributed edge architecture, where network connections between nodes are intermittent. The system must maintain integrity even when sync windows are infrequent.

MinIO and Ceph offer erasure coding and self-healing mechanisms that ensure local durability and consistency, even in environments with unstable connectivity. These features are essential when centralized replication isn't always possible, and data must remain intact until the next synchronization window.

Data integrity is foundational to trust in a data platform. While Iceberg's architecture provides safeguards at the metadata level, it is the durability and resilience of the storage backend that ensures your data remains intact, available, and recoverable under adverse conditions. Whether through geo-replication, WORM enforcement, or strong consistency guarantees, the right storage decision here can prevent silent errors from becoming costly failures.

#### 4.5.4 Cost and operational requirements

While performance, security, and integrity often take the spotlight in architectural planning, cost and operational overhead ultimately determine the long-term viability of your storage decisions. These requirements shape how efficiently your team can manage the system, respond to change, and stay within budget, factors that directly impact the success of your Iceberg lakehouse initiative. Let's explore a few hypothetical scenarios that reflect different cost and operational trade-offs.

##### **SCENARIO: STARTUP MANAGING GROWTH WITH LIMITED DEVOPS CAPACITY**

A fast-growing data team is building its Iceberg platform but lacks dedicated infrastructure engineers. The team prioritizes ease of deployment, minimal ongoing maintenance, and predictable monthly costs.

In this case, Wasabi and MinIO offer clear advantages. Wasabi's flat-rate pricing eliminates unpredictable egress or API costs, while its managed nature removes the need for hands-on infrastructure tuning. MinIO, though self-hosted, is easy to deploy in containerized environments and offers a lightweight operational footprint, especially in Kubernetes-based stacks. Both solutions enable the team to move fast without getting buried in platform operations.

##### **SCENARIO: LARGE ENTERPRISE CONSOLIDATING INFRASTRUCTURE ACROSS TEAMS**

An enterprise IT department is unifying analytics workloads from multiple business units. They require centralized management, multi-tenancy, and strong operational controls to meet internal SLAs. Dell ECS, NetApp StorageGRID, and Pure Storage are designed for this level of operational control. These platforms offer multi-tenant configurations, policy enforcement, centralized monitoring, and integration with enterprise-grade support systems. While more expensive, they deliver predictable performance and governance at scale, reducing the internal support burden.

##### **SCENARIO: RESEARCH INSTITUTION WITH GRANT-FUNDED WORKLOADS**

A university supports multiple research labs with occasional high-volume analytics projects. Usage spikes are unpredictable, and funding is often tied to short-term grants with fixed budgets. Google Cloud Storage and Amazon S3 offer pay-as-you-go pricing and easy provisioning, allowing the institution to scale up and down without committing to long-term contracts. These services also integrate with cloud cost monitoring and forecasting tools, which help labs track usage by project. For cost-conscious institutions, storage classes and lifecycle rules can further reduce expenses without compromising data availability.

## SCENARIO: REGIONAL GOVERNMENT REQUIRING ON-PREM DEPLOYMENT AND COST TRANSPARENCY

A government agency must deploy Iceberg on-premises due to legal constraints, with fixed annual budgets and strict procurement oversight. Ceph provides a cost-effective, open-source option for large-scale object storage with fine-grained control over hardware and deployment topology. While more complex to manage, its open nature aligns with procurement models that favor capital expenses over recurring operational costs. MinIO, deployed on commodity hardware, is another viable option where simplicity and cost-efficiency are key priorities.

When considering cost and operational requirements, the focus should not be on minimizing expense alone, but rather on maximizing value over time. A system that is cheap to deploy but difficult to manage may cost more in engineering time than a more expensive solution with automated controls and enterprise support. Similarly, a highly performant system that demands constant tuning may divert resources from higher-value initiatives. By aligning storage decisions with your team's capabilities, budget constraints, and lifecycle expectations, you set your lakehouse up for resilience, not just launch.

In the next chapter, we'll shift focus to the ingestion layer—exploring how data is moved into the lakehouse, what ingestion patterns matter most, and which systems can help streamline and scale that process. With your storage foundation in place, it's time to examine how data gets in.

### 4.6 Summary

- The storage layer is a foundational component of any Apache Iceberg lakehouse, influencing performance, scalability, cost, and compliance.
- Before selecting a storage solution, it's critical to define requirements based on file retrieval performance, security, data integrity, and operational costs, ideally as part of a structured audit.
- Storage systems generally fall into two categories: block storage, which offers low-latency performance but higher operational complexity, and object storage, which is more scalable and cloud-native.
- Apache Parquet is the default file format for Iceberg due to its columnar design and rich metadata support, enabling efficient pruning and high-performance queries.
- The S3 API has become the industry standard for object storage access, enabling broad interoperability across cloud and on-premises systems.
- A wide range of storage solutions support Iceberg deployments, including cloud services like Amazon S3, Google Cloud Storage, and Azure Blob/ADLS, as well as on-premises options such as MinIO, Ceph, NetApp StorageGRID, Dell ECS, Pure Storage, and Wasabi.
- Each storage platform offers trade-offs across cost, complexity, performance, and governance. No single solution fits all use cases.
- Real-world requirements, like interactive latency, legal compliance, disaster recovery, or budget constraints, should guide final selection. Hypothetical examples illustrate how different needs might map to different technologies.

- By aligning your storage layer to the specific requirements uncovered during your audit, you ensure that your Iceberg lakehouse is not only technically sound, but also operationally sustainable and aligned with business goals.

## ***5 Architecting the ingestion layer***

### **This chapter covers**

- Requirements for ingestion performance, reliability, and latency
- Comparing batch, micro-batch, and streaming ingestion strategies
- How Iceberg handles data writes, commits, and conflict resolution
- Ingestion technologies such as Spark, Flink, and Others
- Ingestion patterns for schema evolution, data quality, and auditability

The ingestion layer is the starting point of your Apache Iceberg lakehouse in practice. It is where raw data enters the system, whether from operational databases, message queues, cloud services, or external vendors. While the storage layer determines how data is preserved, the ingestion layer determines how data arrives—how fast, how clean, and how reliably.

Designing this layer requires more than just choosing an ETL tool. You must consider latency tolerance, throughput capacity, schema evolution, and fault recovery. These requirements can vary widely across use cases. Some pipelines deliver high-frequency transactions that must be processed in seconds. Others may batch up nightly logs or slowly changing dimensions. Your ingestion layer must support both without compromising performance or consistency.

This chapter builds on the foundation of storage design to help you assess ingestion strategies and technologies. We identify the key requirements that shape ingestion architectures, including performance, reliability, and operational complexity. Next, we compare ingestion models: batch, micro-batch, and streaming, to understand where each fits best. We then explore how Iceberg manages data writes, providing isolation and atomicity even in distributed environments.



Finally, we examine the ingestion tools that integrate well with Iceberg and how they support different architectural patterns. As with storage, the goal is not to prescribe a one-size-fits-all solution but to equip you with a framework for choosing tools and designing ingestion flows that align with your organization's needs and constraints.

## 5.1 Ingestion requirements

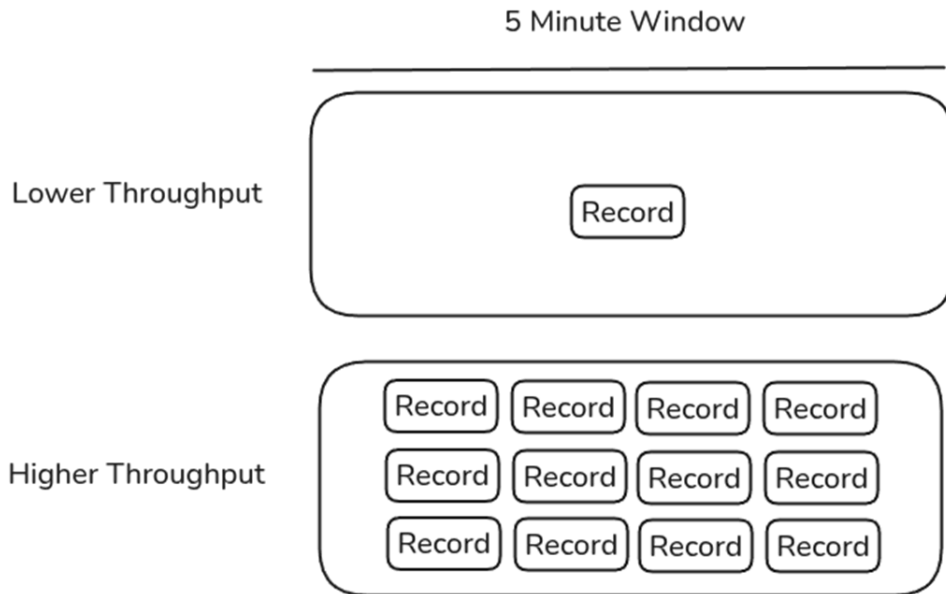
Before selecting tools or designing ingestion pipelines, defining what your ingestion layer needs to deliver is essential. Like the storage layer, ingestion is shaped by a range of technical, operational, and organizational factors. Some pipelines require high throughput and low latency, while others prioritize fault tolerance, schema evolution, or auditability. Without clear requirements, it's easy to over-engineer or misalign the ingestion architecture with business needs.

The audit process described in chapter 3 should have already revealed how data flows into your platform, how frequently it changes, and what constraints apply to its delivery. These insights are critical to shaping ingestion decisions. For example, real-time analytics may require streaming ingestion with low-latency writes and exactly-once guarantees. In contrast, batch jobs loading historical archives may tolerate higher latency but require robust handling of schema drift and file-level optimization.

This section explores the core requirement categories that most often drive ingestion design: throughput and latency, reliability and fault tolerance, schema management, and operational complexity. Understanding these dimensions will help you evaluate ingestion tools and patterns based on their features and how well they align with the realities of your data and platform.

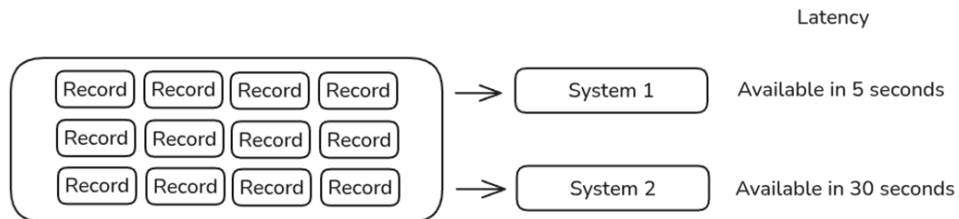
### 5.1.1 Ingestion throughput and latency

Every ingestion pipeline must balance throughput and data freshness. Throughput measures how much data the system can ingest over a set period. Data freshness reflects how quickly new data becomes available for querying after it is generated, which we illustrate in figure 5.1.



**Figure 5.1** The more records a system can process in the same time window, the higher the throughput it can handle.

Latency measures how quickly data becomes available for querying after it is generated, as illustrated in figure 5.2. These two metrics shape how well your lakehouse can respond to user expectations and operational demands.



**Figure 5.2** If two systems process the same data, the one that can deliver it in less time has lower latency.

Different use cases impose different requirements on data freshness. Real-time dashboards and fraud detection may need data to be queryable within seconds. Daily reports or regulatory exports can accept delays of several hours or more. Defining these expectations early helps align your ingestion architecture with actual needs. Streaming frameworks like Apache Flink and Kafka Connect support low-latency, continuous ingestion. Batch-oriented tools such as Apache Spark provide high throughput for large, scheduled jobs but with higher latency. Other options include integration frameworks like Airbyte or SaaS tools such as Amazon Data Firehose and Qlik, which can ingest data directly into Iceberg tables depending on the environment.

Throughput constraints are often more technical. They are shaped by the size and structure of your source data, the scalability of your compute resources, and the network bandwidth between source and destination systems. Bottlenecks can appear unexpectedly, from slow JDBC sources and network congestion to insufficient parallelism in your processing engine.

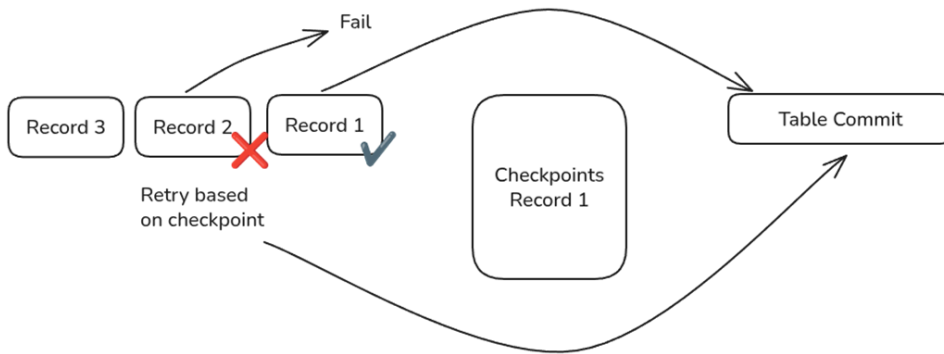
In Iceberg-based lakehouses, throughput and latency also interact with write semantics. If not managed carefully, high-frequency writes can lead to metadata bloat or file fragmentation. Techniques like micro-batching, compaction, and checkpointing can help mitigate these effects by grouping small writes and reducing metadata overhead.

When defining ingestion performance requirements, focus on scalability from the start. Benchmark both peak and sustained load expectations, and project how data volume and arrival patterns may grow. Your architecture should be able to scale throughput without increasing latency, especially under heavy or unpredictable loads. Building with these growth scenarios in mind helps prevent bottlenecks as demand rises.

### **5.1.2 Reliability and fault tolerance**

Reliability is a cornerstone of any ingestion system. It defines how confidently data pipelines can handle failures, retries, and interruptions without compromising data quality or system stability. In any architecture, unreliable ingestion can lead to data loss, inconsistent table states, and downstream analytics failures, all of which erode trust in the platform.

Fault tolerance begins with understanding the guarantees offered by your ingestion tools. Some systems provide at-least-once delivery, where data may be reprocessed in the event of a failure, requiring downstream deduplication logic. Others support exactly-once semantics, ensuring each record is written to Iceberg only once, even across retries or restarts. Apache Flink, for instance, achieves this through checkpointing and coordinated commits as illustrated in figure 5.3 with Iceberg tables.



**Figure 5.3** By using checkpointing to track which incoming records have been committed successfully, the system knows which records to retry when failure occurs.

Iceberg's write model supports atomic, snapshot-based commits, which helps ensure consistency even in distributed environments. If a job fails before committing, no partial writes are visible to readers, reducing the risk of producing incomplete or corrupt table states. However, the ingestion tool still has the responsibility to coordinate retries and avoid introducing conflicting changes or duplicates.

Another key aspect of reliability is resumability. Ingesting from external systems, such as databases, APIs, or message queues, often requires tracking offsets, timestamps, or change identifiers to prevent reprocessing old data or skipping new records during outages. Robust pipelines persist this state externally and resume ingestion from the last known good position after recovery. When possible, adopting the idempotent consumer pattern, designing ingestion to safely reprocess the same data without side effects, adds an additional layer of fault tolerance and simplifies recovery logic.

Finally, observability is essential for reliable ingestion. Pipelines should emit metrics and logs that reflect data volume, error rates, retry counts, and commit statuses. These indicators help teams detect anomalies quickly and take corrective action before data integrity is compromised.

Designing for reliability means planning for the inevitable: network failures, service restarts, bad records, and resource contention. A resilient ingestion layer anticipates these scenarios, recovers gracefully, and provides visibility throughout.

### 5.1.3 Schema management and evolution

Schema evolution is an everyday reality in modern data platforms. The incoming data structure often evolves as business requirements shift and source systems change. The ingestion layer must be able to handle these changes without breaking pipelines, introducing data inconsistencies, or requiring manual intervention.

Apache Iceberg supports schema evolution by design, enabling changes such as adding, renaming, reordering, or dropping columns without rewriting existing data files. This is possible because Iceberg uses stable field IDs instead of relying solely on column names, preserving the logical meaning of each field even as the schema evolves. However, this flexibility only works when ingestion tools handle schema changes consistently. Ingestion jobs must detect the incoming schema, validate it against the current Iceberg table definition, and apply safe mappings or transformations as needed. For example, if a new field appears in source data, the ingestion pipeline may need to update the table schema or cast the new field to match existing types. Failure to manage this process can result in rejected records, silent data corruption, or inconsistent query results. Tools that integrate with schema registries or metadata catalogs are often better equipped to detect and manage these changes automatically.

For batch and micro-batch ingestion, schema alignment typically occurs at the start of each job. The job reads the current Iceberg schema, inspects the incoming data, and applies any necessary transformations or mappings. In streaming contexts, this process becomes more complex. Systems like Apache Flink and Kafka Connect must reconcile schema differences continuously, often using schema registries or metadata catalogs to ensure compatibility.

Conflicts can arise when schemas evolve—for example, if data types change or field names are reused with different meanings. These changes can lead to subtle errors or data loss if not properly validated. Fortunately, Iceberg mitigates many of these risks by using stable field IDs rather than relying on field names for DDL operations. This allows columns to be safely added, removed, or reordered without breaking downstream queries. Still, ingestion pipelines must enforce schema compatibility through validation, whether using fail-fast checks, controlled casting, or both, to ensure that incoming data aligns with the current table definition.

### FAIL-FAST CHECKS AND CONTROLLED CASTING DURING INGESTION

One of the more important considerations in reliable data ingestion is how pipelines handle unexpected or malformed records. In Iceberg-based systems, this often involves managing data type mismatches, missing fields, or schema inconsistencies. There are several strategies available, each with tradeoffs around resilience, data quality, and operational complexity.

A strict approach is *fail-fast ingestion*, where pipelines reject or halt processing when data doesn't conform to the expected schema. This enforces high data quality by preventing invalid records from reaching Iceberg tables. It also aligns well with Iceberg's strong typing and can surface issues early in the data lifecycle. However, this strategy can lead to frequent failures and extended downtimes in environments where source data quality is variable. As such, fail-fast is rarely ideal in production unless used selectively on critical fields.

An alternative is *controlled casting*, where pipelines coerce values that don't exactly match the target schema. This might include converting strings to timestamps, truncating decimals, or inserting nulls for missing values. Some tools support only limited forms of promotion (e.g., int to long), while others offer more flexible casting capabilities. This method can improve resilience to upstream inconsistencies but risks introducing silent errors or unintended transformations that reduce trust in the data.

A more resilient and increasingly common pattern is the use of a *dead-letter queue (DLQ)*. Instead of failing or transforming problematic rows, the pipeline captures them in a side channel for later inspection or correction. This keeps ingestion running smoothly while preserving a record of rejected data for quality control. DLQs are particularly useful when working with semi-structured or externally sourced data where errors are expected but should not block ingestion entirely.

In practice, robust pipelines often combine these strategies. For example, you might apply fail-fast checks to essential fields, use controlled casting on optional or rapidly evolving ones, and redirect all unresolvable errors to a DLQ. This hybrid approach balances data quality enforcement with operational continuity and provides a safety net for handling edge cases without disruption.

While ingestion systems must accommodate schema changes, they are not typically responsible for tracking the full history of those changes. Storing and managing that level of metadata can add unnecessary overhead. Instead, a more practical approach is to leverage Iceberg's built-in metadata tables, which automatically capture schema evolution alongside table snapshots. These tables provide a reliable audit trail for debugging, governance, and impact analysis without burdening the ingestion layer. In regulated environments, this capability can be especially valuable, offering visibility into how data structures have changed over time.

Ultimately, strong schema management enables agility. It allows teams to adapt data models to evolving needs without disrupting downstream systems. A well-designed ingestion layer supports this evolution seamlessly, enabling Iceberg to deliver consistent, queryable data even as the shape of that data changes.

#### **5.1.4 Operational complexity and maintainability**

No matter how performant or reliable an ingestion system may be, its long-term value depends on how easily it can be operated, maintained, and extended. Operational complexity often becomes the hidden cost that slows innovation, consumes engineering time, and increases the risk of failure during change.

Several factors shape the operational footprint of the ingestion layer. One of the most important is pipeline orchestration. Batch jobs often depend on external schedulers such as Apache Airflow or managed orchestration services like AWS Step Functions. These tools coordinate task execution, retries, and dependencies. In contrast, streaming jobs run continuously and require persistent deployment, monitoring, and tuning. Managing stateful, fault-tolerant processing in real time typically demands deeper expertise and stronger operational discipline.

Second is automation and observability are key to simplifying operations. Well-designed pipelines emit metrics for data volume, processing time, and failure rates. They integrate with logging and alerting systems to enable proactive monitoring. Automated retries, backoff policies, and dead-letter queues help prevent transient errors from requiring manual intervention.

Configuration management is another consideration. As pipelines grow in number and scope, managing their configurations, source connections, schema mappings, and transformation logic can become burdensome. Solutions that support declarative pipeline definitions or templated deployments can reduce this overhead and improve reproducibility.

Additionally, your chosen ingestion framework's learning curve and tooling maturity affect operational efficiency. Open-source tools like Apache Spark or Flink offer flexibility but may require custom engineering to integrate and scale. Cloud-native tools such as AWS Glue, Azure Data Factory, or Google Dataflow offer managed services that reduce setup and maintenance but may impose constraints on customization.

Ultimately, the right ingestion architecture balances capability with maintainability. It should align with your team's skill set, integrate smoothly into your existing operations stack, and scale without requiring constant tuning. A manageable ingestion layer frees your team to focus on value-added work rather than firefighting routine pipeline issues.

## 5.2 Ingestion models and architectures

Once you've defined your ingestion requirements, the next step is to choose an architectural model that aligns with those needs. Ingestion approaches differ in how they handle data timing, failure recovery, and integration with downstream systems. The right choice depends on your goals for latency, freshness, reliability, and operational overhead.

Most ingestion models fall into three broad categories: batch, micro-batch, and streaming. Batch ingestion is valued for its simplicity and reliability, particularly its support for idempotent processing. If a job fails, it can be safely retried without concerns about state or partial writes. This makes batch ideal for historical loads and systems where predictability and recovery matter more than speed.

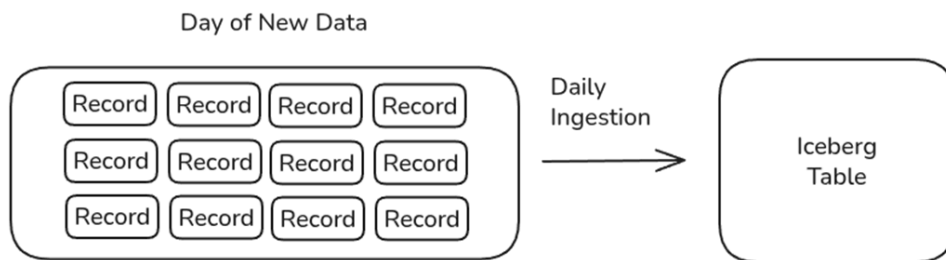
Micro-batch ingestion sits between batch and streaming. It delivers data at shorter intervals, often minutes rather than hours, providing faster access while maintaining high throughput. However, this comes with trade-offs: micro-batch systems are typically less idempotent and more operationally complex than batch pipelines. Failures are harder to recover from, and managing state across batches may require more sophisticated tooling.

Streaming ingestion continuously captures data as it arrives, offering the lowest possible latency for getting data into your lakehouse. A common misconception is that streaming ingestion always requires real-time processing. In fact, it simply enables real-time availability; consumers can still choose to process the data in batches, micro-batches, or in real time. The benefit is landing the data immediately, giving downstream systems the flexibility to consume it in whatever cadence makes sense for their use case.

This section will review each model, highlighting its strengths, trade-offs, and typical use cases in an Apache Iceberg lakehouse. By understanding these ingestion patterns, you can design pipelines that meet your requirements and evolve with your data and business needs.

### 5.2.1 Batch ingestion

Batch ingestion is the most established and widely used model for moving data into analytical platforms. It involves collecting data over a defined period- hourly, daily, or weekly and loading it into the lakehouse in discrete units, as illustrated in figure 5.4. This approach is efficient for large volumes of historical or slowly changing data, where latency is less critical than throughput, reliability, and cost control.



**Figure 5.4 Ingesting Data for the day once a day in a batch job.**

In a batch workflow, data is typically extracted from source systems like operational databases, flat files, or APIs, then transformed and written to Iceberg tables as Parquet files. Apache Spark, dbt, or Airflow orchestrate these processes through scheduled jobs or pipelines. Because all records are processed in a single run, batch ingestion simplifies coordination and often allows for more comprehensive data validation and transformation logic.

Apache Iceberg seamlessly supports batch ingestion. Jobs can write new data using the append mode or perform overwrites and upserts when needed. Each batch write results in a new Iceberg snapshot, preserving data version history and enabling time travel. Additionally, Iceberg's support for atomic commits ensures that batch jobs either succeed in full or leave no trace, eliminating the risk of partial writes or data corruption.



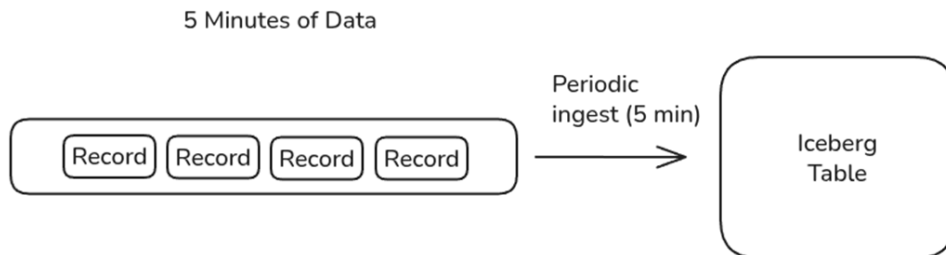
A major strength of batch ingestion is that it makes it easier to handle failures without causing problems in the data. Because batch jobs work on fixed chunks of data, they can be rerun without duplicating results or creating inconsistencies, depending on the source. This makes it simpler to recover from errors or reprocess data if needed. Batch pipelines are usually scheduled at regular intervals, which adds a level of predictability for operations teams, but that's more of a side benefit than the main reason to use batch processing. The downside is that data is only updated on a schedule, so users may have to wait to see the latest information.

Batch pipelines typically operate with fixed batch sizes, which can simplify tuning and resource allocation. However, if the volume of incoming data increases significantly and the batch size remains unchanged, it can create a growing backlog that delays data availability and stresses downstream systems. While these pipelines are less prone to failure from input fluctuations, they may still require adjustments to scheduling, parallelism, or job frequency to keep pace with evolving data loads and schema complexity.

Despite these limitations, batch ingestion remains a reliable foundation for many Iceberg lakehouse implementations. It's particularly well-suited for backfills, bulk imports, and analytics use cases where real-time visibility is not required. When paired with Iceberg's metadata optimizations and snapshot isolation, batch workflows deliver scale and consistency without adding unnecessary complexity.

### 5.2.2 Micro-batch and incremental ingestion

Micro-batch ingestion bridges the gap between traditional batch processing and real-time streaming. It involves processing data in small, frequent intervals, typically from a few minutes, allowing for lower latency while preserving many operational advantages of batch systems, as illustrated in figure 5.5. This model is beneficial for use cases that demand near-real-time updates without the complexity of full streaming architectures.



**Figure 5.5** Micro-batching data in an interval where an ingest job triggers to ingest all new data from the last 5 minutes.

In a micro-batch workflow, incoming data is collected over short time windows and ingested as discrete chunks. Tools like Apache Spark Structured Streaming treat each interval as a mini-batch, triggering a new execution cycle on a recurring schedule. These small jobs can append data to Iceberg tables, triggering new snapshots with each commit. Iceberg's metadata layer handles the incremental updates efficiently, and its snapshot isolation model ensures readers only see complete, committed states.

Micro-batching is well-suited for ingesting data from SaaS platforms, APIs, and periodic file uploads, sources that do not produce a continuous stream of records but can be polled or triggered on a regular schedule. It offers a middle ground between high-latency batch jobs and the complexity of continuous streaming, delivering fresher data without requiring fully stateful, long-running applications. By accumulating small volumes of data into timed batches, micro-batching provides predictable processing intervals and simplifies ingestion from semi-structured or event-driven external systems.

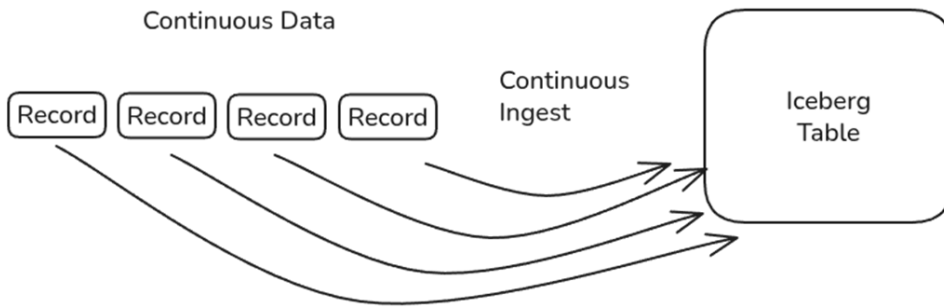
This model also plays well with Iceberg's maintenance features, which we will discuss further in a later chapter. Frequent small writes can lead to metadata growth and file fragmentation, which may degrade query performance over time. Iceberg supports background compaction processes that consolidate small files into optimized columnar blocks, ensuring that ingestion speed doesn't come at the cost of read efficiency.

One challenge with micro-batching is tuning the batch interval. Intervals that are too short may increase overhead from frequent job launches and metadata commits, while intervals that are too long may delay data availability. The correct configuration depends on your system's capabilities and users' expectations for fresh data.

Micro-batch ingestion offers a strong balance between throughput, latency, and maintainability. It is ideal for lakehouse environments where data needs to arrive continuously but with enough tolerance to simplify orchestration and resource allocation. When paired with Iceberg's transactional model and file-level optimizations, it provides an efficient and flexible ingestion approach for a wide range of real-time analytics needs.

### 5.2.3 Streaming ingestion

Streaming ingestion is designed for continuous, real-time data flow into the lakehouse, unlike batch or micro-batch models, which operate on defined time windows. Streaming pipelines process and deliver data as soon as it is generated. This model is essential for low-latency applications such as fraud detection, monitoring systems, personalized recommendations, and real-time dashboards.



**Figure 5.6 Streaming data is continuously ingested as it arrives at its destination.**

In a streaming workflow, data is ingested record by record or in small micro-batches with single-digit second intervals, as illustrated in figure 5.6. Systems like Apache Flink, Kafka Streams, and Spark Structured Streaming support complex stream processing with features such as event-time handling, watermarking, and stateful transformations. These tools can write directly to Apache Iceberg tables using connectors that buffer records, commit changes at safe intervals, and coordinate with Iceberg’s snapshot-based architecture.

Apache Iceberg integrates well with streaming paradigms, particularly its support for incremental writes and exactly-once semantics when paired with compatible engines. Streaming ingestion typically appends new data to a table, generating a new snapshot with each commit. Streaming writers often group records into small files to reduce overhead and commit them in batches, balancing freshness and performance.

Streaming ingestion introduces several unique challenges. One is commit frequency: committing too often can generate excessive metadata and degrade query performance, while committing too infrequently can delay data availability. Another is state and fault recovery. Because streaming jobs are long-running, they must persist processing state and restore it reliably after failures. Tools like Apache Flink address this through checkpointing and integration with Iceberg’s commit model to maintain consistency. Streaming pipelines must also handle out-of-order or late-arriving events, which can affect data accuracy if not properly sequenced or deduplicated. In addition, error handling in streaming systems is more complex than in batch jobs, requiring strategies to isolate bad records without halting continuous data flows.

Scalability is also a key concern. Streaming systems must handle fluctuating data rates, often with high peak loads. They require elastic resource allocation and robust backpressure handling to prevent data loss or lag.

Another key advantage of streaming ingestion is support for stateful transformations; operations that require maintaining context across multiple events. These include deduplication, sessionization, and change data capture (CDC) logic, all of which are difficult or inefficient to implement in traditional batch pipelines. Stateful transformations allow the pipeline to react to changes in real time while preserving data accuracy and consistency. For teams transitioning from batch workflows, this model may require a shift in mindset and tooling, but it unlocks powerful new patterns for building responsive and intelligent data applications.

Despite the complexity, streaming ingestion enables the highest data freshness and interactivity levels. It's a natural fit for event-driven architectures and operational intelligence use cases where minutes or seconds of delay could reduce business value. When configured correctly, a streaming pipeline into an Iceberg table provides a resilient, low-latency foundation for real-time analytics at scale.

## 5.3 How Iceberg manages writes

Choosing the right ingestion model is only part of the picture. Regardless of how data is delivered, whether through batch jobs, micro-batches, or streaming pipelines, it must ultimately be written into Iceberg tables. Understanding how Iceberg manages these writes is essential for building ingestion workflows that are both performant and reliable.

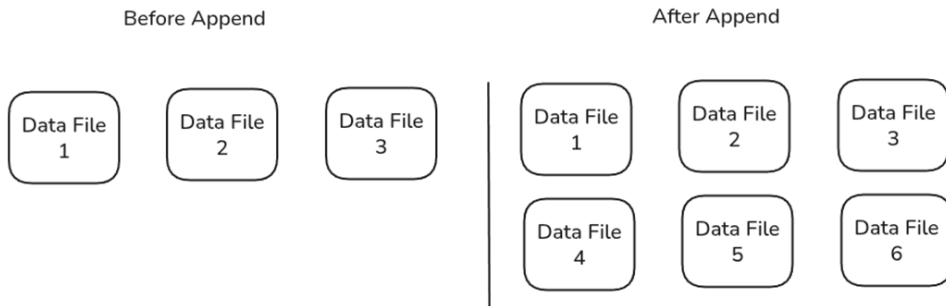
Iceberg is more than a file format. It is a transactional table format designed to bring database-like guarantees to data lakes. Every write operation in Iceberg creates a new table snapshot, updating metadata to reflect changes without modifying existing data files. This approach enables features like atomic commits, time travel, and concurrent write isolation, all critical for reliable data ingestion.

This section will examine how Iceberg handles different write operations, including appends, overwrites, upserts, and deletes. We'll also explore the role of Iceberg's commit protocol in ensuring consistency across distributed systems and how it integrates with standard ingestion engines to manage conflicts, retries, and incremental updates. These mechanics form the foundation for reliable, multi-writer data pipelines in a modern lakehouse.

### 5.3.1 Write semantics in Iceberg

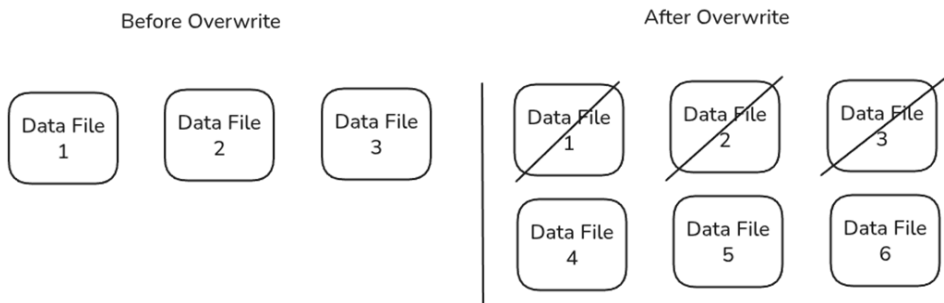
Apache Iceberg supports a range of write operations that allow data engineers to implement diverse ingestion patterns while maintaining consistency, scalability, and query performance. Each write creates a new snapshot, representing a consistent view of the table at a specific point in time. These snapshots are central to Iceberg's design, enabling atomicity, isolation, and time travel.

The most common write operation is append, where new data is added without modifying or removing existing files, as illustrated in figure 5.7. This is ideal for streaming or micro-batch ingestion, where new records are continuously written to the table. Since append operations don't alter previous data, they are highly efficient and can be performed concurrently by multiple writers, provided each batch lands in separate files.



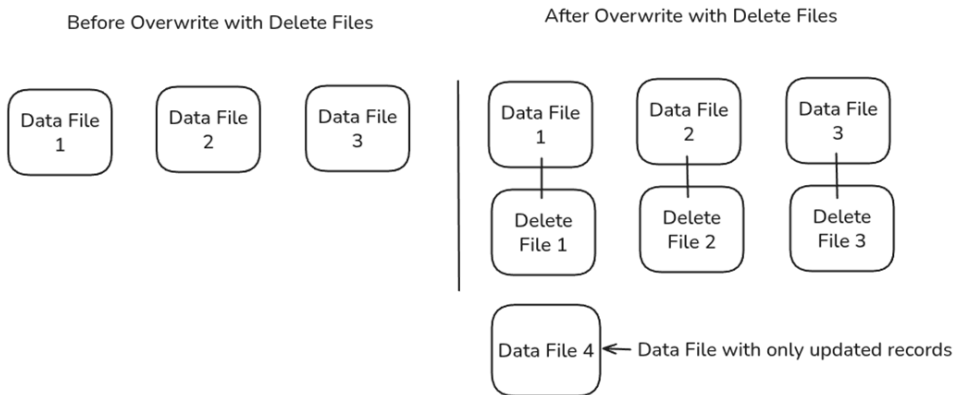
**Figure 5.7** After an append, the new snapshot includes all the previous datafiles and the new datafiles.

Iceberg supports overwrite operations for use cases involving data correction or periodic rebuilds, as illustrated in figure 5.8. An overwrite replaces a subset of the table's data based on a filter or partition specification. This is useful for pipelines that refresh data by partition (e.g., daily snapshots) or require deterministic reprocessing. Overwrites must be used carefully in multi-writer environments to avoid conflicts, as they can invalidate existing files within the targeted scope.



**Figure 5.8** In an overwrite, files with deleted or updated records are replaced with new data files.

Iceberg supports both *merge-on-read* and *copy-on-write* approaches for handling row-level updates and deletes, capabilities that go beyond traditional data lake workflows, as illustrated in figure 5.9. In a merge-on-read setup, delete files (such as position or equality deletes) are stored separately and applied dynamically during query time. In contrast, copy-on-write operations rewrite affected data files at the time of the update, applying changes immediately. Each approach offers trade-offs in latency, query performance, and storage costs. Iceberg also enables change data capture (CDC) use cases by tracking row-level changes between snapshots. This makes it suitable for scenarios like slowly changing dimensions, GDPR compliance, fine-grained corrections, and real-time synchronization, without requiring full rewrites of large datasets. For implementation details and trade-offs between these modes, refer to the Iceberg specification in Appendix C.



**Figure 5.9** When using delete files, updated/deleted records to ignore in existing data files tracked by delete files, while new data files with only the updated records are written.

## EQUALITY AND POSITION DELETES

Apache Iceberg v2 supports two primary mechanisms for expressing deletes: equality deletes and position deletes. These mechanisms allow data to be logically removed without rewriting entire data files, which can significantly improve performance and reduce operational overhead.

Equality deletes identify rows to delete based on column values. For example, to delete all records where `user_id = 123`, Iceberg writes a small file listing this condition. During query execution, engines apply this filter as they scan the data.

Pros: Efficient for removing many rows across large files; avoids the cost of rewriting data files.

Cons: Delete files must be checked at read time, which can increase query planning and execution latency; performance depends on engine support and file layout.

Position deletes mark specific row positions within a data file for deletion. This approach is more precise, typically used after a query engine identifies the exact locations of records to remove.

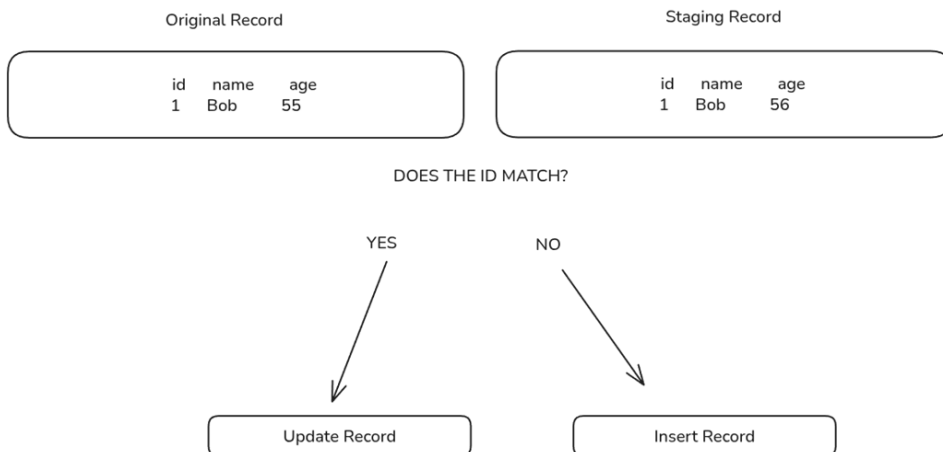
Pros: Enables highly selective deletes with minimal impact on unrelated rows; well-suited for update-heavy workloads.

Cons: Tightly coupled to file structure, which may limit file-level optimizations and require careful coordination across readers and writers.

Both delete types are powerful, but they also come with read-time trade-offs. Because the data files remain unchanged, query engines must merge base data with delete files during reads, which can add overhead in large or fragmented datasets.

Iceberg v3 introduced a new delete representation using binary deletion vectors stored in Puffin files. These vectors encode row-level deletions more compactly and efficiently, allowing faster application of deletes during query time with improved I/O locality. While not yet widely adopted, this advancement aims to reduce the performance gap between logical deletes and physical file rewrites, offering a more scalable approach to row-level mutability in lakehouse architectures. For more details on the different versions of the Apache Iceberg spec, please refer to Appendix C.

Another operation is MERGE, which allows conditional upserts and deletes based on user-defined logic, as illustrated in figure 5.10. MERGE statements are typically used in transactional ingestion patterns where new data may either insert fresh records or update existing ones. Engines like Apache Flink and Spark support MERGE operations through their Iceberg connectors, allowing sophisticated data transformation pipelines to run directly against Iceberg tables.



**Figure 5.10 Merge transactions allow you to compare records to a staging dataset to conditionally insert, update, and delete records.**

These write semantics give Iceberg the flexibility to support a wide range of ingestion needs, from immutable data appends to entirely mutable transactional updates, while maintaining consistency and enabling efficient reads. Choosing the right write strategy for each pipeline is crucial to balancing performance, cost, and correctness in your lakehouse architecture.

### 5.3.2 Commit protocols and conflict handling

Apache Iceberg's commit protocol is central to its ability to maintain consistency and isolation in distributed environments. Whether data is written through batch jobs, streaming pipelines, or interactive tools, every write operation concludes with an atomic commit that publishes a new table snapshot. This mechanism ensures readers see a consistent view of the table, even as multiple writers operate in parallel.

The commit process generates a new snapshot metadata file that references newly written data files and any associated deleted files. To publish this snapshot, Iceberg performs an atomic compare-and-swap operation on the table's metadata pointer (more on this in the catalog chapter). If no other writer has modified the table in the meantime, the new snapshot becomes the latest state. If another commit has occurred, the operation fails, and the writer must retry.

This optimistic concurrency control model allows high throughput and scalability without requiring distributed locks. It also ensures that concurrent writes do not interfere with one another, provided they operate on disjoint data sets. For example, two append operations to different partitions can succeed independently, while two overlapping overwrites may trigger a conflict.

Commit conflicts are a common consideration in ingestion workflows, especially in streaming or concurrent micro-batch jobs. Handling these conflicts involves a combination of retries, backoff strategies, and, in some cases, re-evaluating the current table state to determine whether a retry is appropriate. Most ingestion frameworks that support Iceberg (such as Apache Flink and Spark) implement built-in commit retry logic to simplify this process.

Commitment behavior also affects performance. Committing too frequently, especially in streaming pipelines, can increase metadata churn and slow query planning. Batching writes before committing helps amortize this cost and reduces pressure on the catalog and underlying storage. However, longer commit intervals introduce higher data freshness lag, so finding the right balance is essential.

Iceberg's snapshot isolation and atomic commits play a crucial role in scenarios requiring exactly-once delivery. By coordinating commit points with the ingestion engine's checkpointing mechanism, pipelines can ensure that data is written once and only once, even across failures or restarts. This coordination is essential for maintaining correctness in stateful streaming jobs.

Ultimately, Iceberg's commit protocol enables reliable ingestion across various deployment patterns. It abstracts the complexity of concurrency, conflict resolution, and transactional isolation, allowing teams to focus on building resilient pipelines without compromising on consistency or performance.

## 5.4 Tools and frameworks for ingestion

Once you've defined your ingestion requirements and selected an appropriate architectural model, the next step is choosing the tools to implement it. The modern data ecosystem offers various ingestion frameworks, each with different strengths, operational models, and degrees of integration with Apache Iceberg. The right tool depends on your latency needs, source systems, team expertise, and the scale and frequency of your data flows.



This section explores the most commonly used ingestion tools across open-source and commercial ecosystems. We'll begin with Apache Spark and Apache Flink, two of the most widely adopted engines for batch and streaming workloads that also offer deep native integration with Iceberg. From there, we'll look at Apache NiFi, a flexible, low-code tool for building data flows with minimal custom code.

We'll also examine popular commercial and open-source connectors and ELT platforms such as Fivetran, Airbyte, and Qlik (which acquired Upsolver), simplifying ingestion from SaaS applications and databases. For event-driven architectures, we'll consider stream processing platforms like Confluent and Redpanda, which provide Kafka-compatible capabilities and integration paths to Iceberg through connectors or sinks.

Finally, we'll touch on cloud-native alternatives such as AWS Glue, Azure Data Factory, and Google Dataflow, which offer managed ingestion services tightly coupled with their respective cloud ecosystems. While no single tool perfectly covers all use cases, this section aims to provide a clear picture of each option's strengths and how it fits within an Apache Iceberg lakehouse architecture.

### 5.4.1 Apache Spark

Apache Spark is a powerful distributed processing engine widely used for batch and micro-batch data pipelines. Its integration with Apache Iceberg makes it a foundational tool for many lakehouse ingestion workflows, particularly those that involve large-scale transformations, scheduled ETL jobs, or semi-real-time streaming.

Iceberg provides native support for Spark through the Iceberg Spark runtime module, enabling read and write operations across Spark SQL and DataFrame APIs. Spark is especially practical for writing partitioned data, performing compactions, and managing upserts or overwrite workflows. Here is an example of what appending some data to an Apache Iceberg table can look like in Apache Spark.

#### **Basic append operation with Spark:**

```
import org.apache.spark.sql.SparkSession #A
val spark = SparkSession.builder()
    [CA].appName("IcebergAppendExample")
    [CA].config("spark.sql.catalog.my_catalog",
    [CA]"org.apache.iceberg.spark.SparkCatalog")
    [CA].config("spark.sql.catalog.my_catalog.type", "hadoop")
    [CA].config("spark.sql.catalog.my_catalog.warehouse",
    [CA]"s3a://my-bucket/warehouse")
    .getOrCreate() #B
val df = spark.read.json(
    [CA]"s3a://input-bucket/new-data.json") #C
df.writeTo("my_catalog.db.my_table")
    [CA].append() #D
```

**#A Import Spark Session**

**#B Build a Spark Session with our Iceberg Catalog Configs**

**#C Read data from a JSON file into a dataframe**

**#D Write the data from the dataframe into a new append snapshot**

### Partitioned overwrite operation:

```
df.writeTo("my_catalog.db.my_table")
    .overwritePartitions()
```

This operation replaces only the affected partitions, preserving the rest of the table and minimizing write amplification.

### Merge (upsert) with Iceberg using Spark 3.3+:

```
MERGE INTO my_catalog.db.my_table t
USING my_catalog.db.staging_table s
ON t.id = s.id #A
WHEN MATCHED THEN UPDATE SET * #B
WHEN NOT MATCHED THEN INSERT * #C
```

**#A We first check if the ID of the records match**

**#B If they do, then update the record to match staging**

**#C If they don't, then insert the record into the table**

Spark's merge support allows users to implement slowly changing dimensions, handle late-arriving data, and simplify data reconciliation workflows.

Spark also supports streaming ingestion using Structured Streaming. In this model, micro-batches are written to Iceberg to support checkpointing and exactly-once semantics, provided the streaming source and processing logic are configured appropriately.

**Streaming write example:**

```

val inputStream = spark.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "broker:9092")
  .option("subscribe", "input-topic")
  .load() #A
val parsed = inputStream.selectExpr("CAST(value AS STRING) as json")
  .select(from_json($"json", schema).as("data"))
  .select("data.*") #B
parsed.writeStream
  .format("iceberg")
  .outputMode("append")
  .option("checkpointLocation", "s3a://checkpoints/my-table/")
  .start("my_catalog.db.my_table") #C

```

**#A** Connect to Kafka and read data from the desired topic

**#B** Turn topics into JSON and coerce them into the correct schema, and turn them into a dataframe

**#C** Write that dataframe to the table and update the checkpoint for exactly one semantics

Spark's flexibility, broad ecosystem support, and performance at scale make it a natural choice for various ingestion use cases. When combined with Iceberg's transactional model, Spark enables robust batch pipelines and flexible near-real-time data flows in a lakehouse environment.

**5.4.2 Apache Flink**

Apache Flink is a high-performance, stateful stream processing engine for low-latency, event-driven applications. Its native support for Iceberg, combined with powerful features like exactly-once semantics and event-time processing, makes it a strong choice for real-time ingestion into an Iceberg lakehouse.

Flink integrates with Iceberg through the Iceberg Flink Sink, which supports streaming and batch writes. It is particularly effective when data arrives continuously and needs to be committed incrementally with strong consistency guarantees.

**Flink SQL example for streaming to Iceberg:**

```

CREATE TABLE source_kafka (
  id STRING,
  event_time TIMESTAMP(3),
  data STRING,
  WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
) WITH (
  'connector' = 'kafka',
  'topic' = 'events',
  'properties.bootstrap.servers' = 'broker:9092',
  'format' = 'json'
); #A
CREATE TABLE sink_iceberg (
  id STRING,
  event_time TIMESTAMP(3),
  data STRING
) WITH (
  'connector' = 'iceberg',
  'catalog-name' = 'my_catalog',
  'catalog-type' = 'hadoop',
  'warehouse' = 's3a://my-bucket/warehouse',
  'format-version' = '2'
); #B
INSERT INTO sink_iceberg
SELECT * FROM source_kafka; #C

```

**#A Create the sources from the Kafka Topic**

**#B Create the destination in the table in an Iceberg catalog**

**#C Insert Data from the Source into the Destination**

Flink also supports complex processing such as filtering, enrichment, joins, and aggregations before ingestion. These transformations can be written using the `DataStream` API or SQL and committed to Iceberg in a streaming fashion.

### **DataStream API example with Iceberg:**

```

DataStream<Row> inputStream = env
    [CA].addSource(new FlinkKafkaConsumer<>("events",
    [CA]new SimpleStringSchema(), props))
    [CA].map(json -> parseJsonToRow(json)); #A
Table inputTable = tableEnv.fromDataStream(inputStream); #B
[CA] tableEnv.executeSql(
    "CREATE TABLE iceberg_sink (id STRING, data STRING) WITH (...)";
); #C
[CA] inputTable.executeInsert("iceberg_sink"); #D

```

**#A Create a Datastream Object pulling data from the Kafka Stream**

**#B Create a table from the datastream object**

**#C Defining the Iceberg to Receive the Data**

**#D Begin inserting data from stream into Iceberg Table**

Flink's checkpointing integrates with Iceberg's atomic snapshot system, enabling exactly-once delivery when configured correctly. This is crucial for mission-critical data flows where duplicates or data loss cannot be tolerated.

Because of its rich support for event-time semantics, Flink is particularly effective in out-of-order or late-arriving data scenarios. It also enables deduplication, windowed aggregations, and change data capture (CDC) processing, which can feed directly into Iceberg tables for incremental modeling.

In summary, Flink excels at building real-time ingestion pipelines that require fine-grained control, strong consistency, and low-latency delivery. It supports durable, versioned ingestion workflows that scale across streaming and batch use cases when paired with Iceberg.

### 5.4.3 Apache NiFi

Apache NiFi is a visual data flow tool designed to simplify the movement, transformation, and routing of data across diverse systems. Unlike engines such as Spark or Flink, which focus on high-throughput processing, NiFi excels at managing data ingestion pipelines with fine-grained flow control, backpressure handling, and lineage tracking through a user-friendly, drag-and-drop interface.

At its core, NiFi operates on a concept of processors, modular units that perform discrete tasks like fetching files, transforming formats, routing content, or invoking APIs. Users can visually connect these processors into complex pipelines to build ingestion flows tailored to their needs, often without writing custom code.

In the context of Apache Iceberg, NiFi serves as a flexible pre-ingestion and delivery layer. While NiFi 2.0.0 removed direct Iceberg support, the PutIceberg processor introduced in NiFi 1.19.0 and maintained through 1.23.1 enables writing to existing Iceberg tables. However, NiFi does not support table creation or compaction, which external engines like Spark or Trino must handle.

A standard NiFi-to-Iceberg flow might look like this:

- Monitor a local directory using GetFile.
- Normalize and transform data formats using ConvertRecord (e.g., CSV to Parquet).

- Route records using RouteOnAttribute based on metadata logic.
- Write files to an S3 bucket with PutS3Object.
- Downstream, an Iceberg-compatible engine (e.g., Spark or Flink) ingests this data into an Iceberg table.

For direct writes into Iceberg tables using the PutIceberg processor:

1. Use an Iceberg-compatible Hive Metastore configured via a HiveCatalogService.
2. Set up credentials and Hadoop configuration (e.g., core-site.xml) for your object storage.
3. Define the schema, table name, and file format (preferably Parquet or ORC).
4. Ensure the target table exists and has been created by Spark or another engine.

NiFi's operational features, such as backpressure, retries, and provenance tracking, make it suitable for edge ingestion and highly observable dataflows. While it defers heavy processing and table management to other tools, it remains a key orchestration component in an Apache Iceberg Lakehouse architecture.

#### 5.4.4 Fivetran

Fivetran is a fully managed ELT platform designed to simplify data movement from source systems, such as SaaS applications, operational databases, and event platforms, into cloud data warehouses and data lakes. Its Managed Data Lakes Service offers native support for Apache Iceberg, allowing users to ingest structured data directly into Iceberg tables across major cloud providers.

Depending on your configuration, Fivetran converts source data into Parquet files and writes them into Iceberg or Delta Lake tables. It supports Amazon S3, Azure Data Lake Storage (ADLS), and Google Cloud Storage (GCS) (in beta), enabling multi-cloud flexibility. A key feature of the service is that it maintains metadata for Iceberg tables using the Fivetran Iceberg REST Catalog, which is compatible with any engine that supports Iceberg's REST interface.

##### Key features of Fivetran's Iceberg integration include:

- Automated table maintenance includes compaction, cleanup of old snapshots, orphan files, and stale metadata.
- Generation of column-level statistics (min and max values) to enhance query performance.
- Built-in support for multiple query engines, including Spark, Dremio, Athena, and Snowflake.
- Support for reserved Iceberg field names through automatic column renaming.

##### Example architecture:

1. Fivetran pulls data from Salesforce, PostgreSQL, or many other services Fivetran supports.
2. It writes Parquet files into S3 in Iceberg format using the Fivetran Iceberg REST Catalog.

3. Query engines such as Snowflake or Dremio connect to the catalog and read the Iceberg tables.

Fivetran is particularly useful for organizations looking to integrate Iceberg into their data platform without manually building and maintaining ingestion pipelines. It abstracts the operational burden of schema mapping, transformation, metadata coordination, and table lifecycle management, making it a low-friction entry point for adopting Iceberg in production.

For use cases focused on near real-time syncs from business systems, SaaS platforms, and transactional databases, Fivetran offers a powerful combination of automation, governance integration, and Iceberg compatibility across cloud environments.

### 5.4.5 Qlik

Following its acquisition of Upsolver, Qlik now offers ingestion and optimization capabilities designed to support real-time data delivery into Apache Iceberg lakehouses. Qlik manages streaming and semi-structured data at scale, emphasizing integrating streaming pipelines into Iceberg table formats using declarative data flows.

The combined platform provides ingestion pipelines capable of capturing data from high-throughput sources such as Kafka, cloud object storage, and enterprise systems, and writing that data directly into Iceberg tables. This includes support for schema evolution, partitioning strategies, and background optimization routines to improve query performance and storage efficiency.

One of the distinguishing features of the Qlik approach is its automatic handling of compaction, small file management, and partition optimization. These capabilities help minimize the operational overhead typically associated with managing Iceberg tables under continuous ingestion workloads. The system supports ingestion rates measured in millions of events per second and simultaneously manages hundreds of Iceberg tables.

Qlik's platform integrates this ingestion layer with a broader ecosystem of data tools, including connectors for diverse data sources and compatibility with multiple query engines. The architecture is designed to support multi-cloud and hybrid deployments while preserving openness through open table formats like Iceberg.

Use cases best suited to this platform include large-scale behavioral event pipelines, real-time analytics, and streaming AI/ML model inputs, particularly in scenarios where automation and reduced operational complexity are priorities. While the integration is still evolving under Qlik's branding, the technical model remains focused on efficient ingestion into open table formats, emphasizing automation, scale, and integration flexibility.

### 5.4.6 Airbyte

Airbyte is an open-source data integration platform that simplifies ELT workflows through configurable connectors. One of its supported destinations is an S3 Data Lake connector that enables data to be written in the Apache Iceberg format for S3-compatible storage systems.

This connector allows users to ingest data from supported Airbyte sources directly into Iceberg tables, leveraging storage on Amazon S3 or self-hosted S3-compatible systems and integration with a supported Iceberg catalog such as REST, AWS Glue, or Nessie. Depending on sync mode and schema configuration, the connector is designed for append, overwrite, or merge-on-read write strategies.

Data is ingested as Parquet files, and metadata updates are coordinated through the chosen Iceberg catalog. When using merge-on-read mode, Airbyte supports deduplication through equality deletes and inserts, which rely on primary keys and cursor ordering from the source system. This strategy assumes the source emits changes consistently and in order, which may not hold for all APIs.

Airbyte supports schema evolution under specific conditions, such as adding or removing columns and widening types. More disruptive schema changes require a complete refresh or manual intervention to adjust table definitions. Nested or complex fields (e.g., arrays or objects) are stored as serialized JSON strings in the Iceberg tables.

Each sync in Airbyte creates a new snapshot in the Iceberg table. During truncate operations, data is staged in a temporary Iceberg branch before being atomically promoted to the main branch. This allows readers to continue accessing stable data during ingestion, a valuable property when dealing with large updates or unstable source systems.

Airbyte's S3 Data Lake destination offers a straightforward path to integrate structured data pipelines into Iceberg lakehouses, especially for teams seeking declarative, connector-based ingestion workflows without managing stream processing infrastructure. It is best suited for periodic syncs and batch-style ingestion into Iceberg tables hosted on cloud object storage.

## 5.4.7 Confluent

Confluent, the company behind Apache Kafka, offers Tableflow as part of its Confluent Cloud platform to simplify the integration of Kafka topics with Iceberg tables. Tableflow allows organizations to materialize streaming data from Kafka directly into Apache Iceberg or Delta Lake table formats without building custom ingestion pipelines.

Tableflow automates key tasks commonly required when bridging the operational (streaming) and analytical (lakehouse) estates. These include type conversions, schema evolution using the Confluent Schema Registry, change data capture (CDC) stream materialization, and background table maintenance such as file compaction. Tableflow can write to Iceberg tables using Confluent Managed Storage or Bring Your Storage (e.g., Amazon S3).

Materialized tables are exposed in Iceberg format and can be registered with catalog services such as AWS Glue, Iceberg REST Catalog, and Snowflake Open Catalog. This allows query engines like Athena, Snowflake, or Trino to access the data. These Iceberg tables are read-only from the perspective of external compute engines, simplifying stream data flow into analytics systems without duplication.

Tableflow integrates with Confluent Cloud for Apache Flink for data processing before ingestion, allowing real-time transformations, such as filtering, joins, or PII scrubbing, before materialization. This design supports a shift-left processing model, where stream processing logic is applied earlier in the pipeline, reducing the need for post-ingestion cleanup.



Tableflow supports data in Avro, Protobuf, or JSON Schema formats and manages schema evolution based on compatibility rules defined in the Schema Registry. It automatically publishes Iceberg table metadata and maintains several table snapshots to support versioning and rollback.

**Key considerations and limitations:**

- Tableflow supports external catalog integration only with Bring Your Storage.
- Materialized tables are append-only; upserts and retract changelog modes are not supported.
- Each Kafka cluster can integrate with only one catalog at a time.
- Tableflow is currently limited to Confluent Cloud on AWS.
- Some compute engines (e.g., Confluent Cloud for Flink) cannot query Iceberg tables directly.

Confluent Tableflow is best suited for teams already operating within the Confluent Cloud ecosystem who want to streamline the ingestion of Kafka streams into Iceberg tables with minimal custom infrastructure. Its catalog integration and automation features reduce the complexity of moving real-time operational data into analytical systems.

### 5.4.8 Redpanda

Redpanda is a Kafka-compatible streaming platform that supports direct integration with Apache Iceberg. This integration allows Redpanda topics to be materialized as Iceberg tables stored in cloud object storage, simplifying access to streaming data from analytical tools without the need for intermediate ETL pipelines.

When enabled, Redpanda writes topic data to Iceberg-compatible Parquet files and manages associated metadata, enabling downstream systems like Spark, Flink, Snowflake, and ClickHouse to treat streaming data as queryable tables. Redpanda supports version 2 of the Iceberg table format and provides options for catalog integration via REST endpoints or file system-based catalogs.

To enable Iceberg support, Redpanda users must configure the `iceberg_enabled` cluster setting and define the `redpanda.iceberg.mode` topic property. Three ingestion modes are available:

- `key_value`: Stores records as binary payloads along with metadata.
- `value_schema_id_prefix`: Writes structured data using schemas registered in the Schema Registry, requiring the wire format.
- `value_schema_latest`: Writes using the latest schema for a given subject, without requiring the wire format.

Schema evolution is supported for Avro and Protobuf, allowing changes such as adding fields or reordering columns. If schema conversion fails (e.g., due to invalid wire format), the failed records are routed to a dead-letter queue (DLQ) table for inspection and optional reprocessing.

Redpanda supports partitioning Iceberg tables using topic-level settings, enabling optimization of table layout based on frequently queried fields. This can improve query performance by reducing the number of files scanned during reads.

Redpanda also allows users to select between two types of Iceberg catalogs:

- **REST catalog:** Suitable for production environments, allowing metadata to be shared across tools.
- **Filesystem-based catalog:** Stores metadata alongside data in object storage, requiring manual table refresh by clients.

While JSON schemas are not supported natively, JSON data can still be ingested using `key_value` mode. Performance considerations include increased CPU usage for Iceberg translation tasks and potential backpressure on producers if cluster resources are insufficient.

This integration best suits users who want to simplify real-time data access in Iceberg-based architectures while maintaining compatibility with Kafka tooling. Redpanda can serve operational and analytical workloads from the same source without duplicating data.

### 5.4.9 Cloud-native ingestion services

Major cloud providers offer managed ingestion services, integrating with Apache Iceberg through their broader data lake and analytics ecosystems. These services often prioritize ease of use, scalability, and tight integration with the provider's storage and compute platforms. While they may differ in implementation, they typically support batch and streaming ingestion patterns and can serve as sources or transformation layers for Iceberg-based lakehouses.

AWS Glue supports writing to Iceberg tables via its Spark-based ETL engine. Glue jobs can be configured to use Iceberg as a table format and registered with the AWS Glue Data Catalog for discoverability. This is useful for building scheduled or event-driven pipelines that land curated data into Iceberg tables on Amazon S3.

Azure Data Factory (ADF) now supports Apache Iceberg as a sink format when writing to Azure Data Lake Storage Gen2. Using Copy activities, pipelines can write directly to Iceberg tables by specifying the dataset type as Iceberg and configuring appropriate format and store settings. This enables seamless integration with downstream tools such as Microsoft Fabric or Databricks on Azure, which can read and manage these Iceberg tables using external catalogs like Unity Catalog or through REST-compatible interfaces.

Google Cloud Dataflow, built on Apache Beam, supports structured data transformations and delivery to Google Cloud Storage. While it does not offer direct Iceberg integration, Dataflow pipelines can land Parquet data in GCS that is later registered as Iceberg tables using compatible engines like Spark or Trino. Metadata can be tracked using the BigQuery metastore or a REST catalog.

Each provider also supports stream-oriented ingestion tools:

- **Amazon Data Firehose** can deliver data to S3 in Parquet format or directly into Iceberg.
- **Azure Event Hubs** and **Google Pub/Sub** can be paired with stream processors like Spark Structured Streaming or Flink to write to Iceberg tables.

These services offer strong operational support and compliance alignment with their respective cloud environments. However, their native integration with Iceberg may vary in maturity. Organizations looking to adopt Iceberg in a cloud-native stack should evaluate not only storage and compute compatibility but also catalog integration, schema evolution support, and long-term maintenance implications. For many, these ingestion services bridge legacy or proprietary data formats and a more flexible, open lakehouse architecture centered on Iceberg.

#### 5.4.10 Tool selection considerations

The growing adoption of Apache Iceberg across organizations has led to various tools supporting its ingestion layer. These range from general-purpose engines like Apache Spark and Flink, to connector-based ELT platforms like Fivetran and Airbyte, to streaming platforms such as Confluent and Redpanda. Each tool approaches ingestion differently; some prioritize real-time streaming, others offer declarative syncs, and some focus on orchestration or transformation.

When selecting an ingestion tool, there are several practical considerations to guide your decision:

- **Ingestion model:** Choose tools aligned with your latency needs. Streaming engines like Flink or Redpanda are better suited for high-frequency event streams. Spark or micro-batch tools may be more appropriate for scheduled ETL or semi-real-time ingestion.
- **Catalog integration:** Ensure the tool supports your chosen Iceberg catalog type, whether it's a REST catalog, AWS Glue, or a file system-based implementation. Tools like Confluent and Fivetran offer integrated catalog support, simplifying discovery and governance.
- **Schema evolution support:** Look for tools that can effectively manage schema drift. Streaming tools with Schema Registry support (e.g., Redpanda or Confluent) typically offer more resilient evolution capabilities than file-based batch tools.
- **Data transformation needs:** If your pipeline includes joins, filtering, or enrichment, a stream processing engine (like Flink) or a general-purpose framework (like Spark) provides the necessary flexibility.
- **Operational maturity:** Consider the ease of deployment, monitoring, and scaling. Managed services like Fivetran or cloud-native ingestion layers often reduce operational burden but may trade off customization.
- **Cloud environment:** Many tools work best within their native cloud ecosystems. AWS-based environments will benefit from Glue and S3 integration, while Azure and GCP may favor solutions that interoperate well with their storage and metadata layers.

There is no one-size-fits-all ingestion tool. Most modern Iceberg deployments rely on various technologies to meet the varied needs of different data sources, use cases, and user teams. Selecting a small set of tools that complement your architecture and are well-supported within your organization's skill set can help ensure a sustainable and flexible ingestion layer for your Iceberg lakehouse.

## 5.5 Applying ingestion requirements in context

Having reviewed the primary ingestion models and the wide range of tools available to implement them, the next step is understanding how to apply these concepts in practice. This section describes a series of illustrative scenarios based on the requirement categories introduced earlier in the chapter: latency, throughput, transformation complexity, and schema evolution. Each scenario highlights how a specific ingestion need might influence your architectural choices.

These examples are not intended to be prescriptive. In most real-world deployments, you won't optimize your architecture for a single requirement in isolation. Instead, you must balance multiple priorities, such as minimizing latency while preserving data quality or reducing operational burden while supporting high schema variability. You must also account for existing constraints, such as your current cloud provider, team expertise, and governance model.

This section aims to provide a structured way to think about trade-offs by exploring how different requirements map to different ingestion models and tooling choices. You can better evaluate how to design a solution that aligns with your broader data platform strategy. Each subsection presents a sample use case, discusses the implications of its dominant requirement, and outlines potential approaches to meet that need using the ingestion tools introduced earlier.

### 5.5.1 Prioritizing low latency

When low latency is the primary concern, the ingestion pipeline must be designed to minimize the time between data generation and availability in the Iceberg table. This is common in fraud detection, personalization engines, or operational monitoring dashboards, where fresh data needs to be queried within seconds of arrival.

In these cases, streaming ingestion is the preferred model. Apache Flink, Redpanda, and Confluent Tableflow support streaming ingestion workflows that materialize Kafka or Redpanda topics into Iceberg tables with minimal delay. These tools can operate on event time, support watermarking, and apply real-time transformations.

**Illustrative scenario:** A fintech platform wants to monitor real-time customer transactions to detect suspicious activity. Each transaction event must be available to analytical systems within five seconds of being emitted.

#### Design considerations:

- Use Apache Flink to read from Kafka, apply filtering and enrichment, and write to Iceberg with micro-batch commits every few seconds.
- Redpanda could be used if tighter Kafka compatibility and lower overhead are needed, especially with its support for schema-based topic-to-Iceberg mapping.
- Catalog consistency and commit frequency must be tuned to ensure that Iceberg snapshots are committed fast enough to meet freshness requirements without overwhelming metadata.

#### Trade-offs:

- Streaming systems require careful tuning to avoid metadata bloat in Iceberg due to frequent commits.

- Monitoring and alerting are more critical due to the continuous nature of the pipeline.
- Performance tuning is necessary to manage ingestion at scale without impacting downstream query workloads.

This approach ensures that data is made available with minimal delay but requires higher operational maturity. Low-latency ingestion pipelines benefit from strong schema validation, controlled commit frequencies, and optimized file sizes to maintain Iceberg performance over time.

### 5.5.2 Managing high throughput

Ingestion pipelines designed for high throughput prioritize the ability to handle large volumes of data efficiently, often from multiple sources in parallel. These scenarios are common in user behavior tracking, telemetry pipelines, and large-scale application logging, where ingestion systems must process millions or billions of records daily.

Apache Spark, Apache Flink, and platforms like Redpanda and Qlik can sustain high ingestion volumes while writing to Iceberg tables. These tools support bulk writes, batching, and compaction strategies that help manage Iceberg's resulting file and metadata overhead.

**Illustrative scenario:** A media streaming company collects playback and interaction events from millions of users across devices. The system ingests over 100 billion events daily and stores them in Iceberg tables for downstream analysis.

#### Design considerations:

- Use Apache Flink with checkpointing to manage high-velocity Kafka streams, applying deduplication and partitioning before committing data to Iceberg.
- Employ Qlik or Redpanda to manage ingestion across hundreds of topics, with automatic compaction and schema enforcement to reduce manual overhead.
- Batch commits and optimize file sizes to limit snapshot growth and reduce read amplification during queries.

#### Trade-offs:

- High-throughput pipelines require careful control of file generation and commit frequency to avoid Iceberg performance degradation.
- Schema evolution and validation must be automated to prevent ingestion interruptions caused by source-side changes.
- Long-term operational planning is essential to manage partition sizes, metadata growth, and compaction strategies.

While the system architecture may emphasize throughput, it must also account for Iceberg's metadata sensitivity. Adaptive compaction, partition tuning, and snapshot expiration are essential for stabilizing performance as data volumes scale.

### 5.5.3 Supporting complex transformations

Some ingestion pipelines require extensive data transformation before writing to Iceberg tables. These include joining across streams, filtering, masking sensitive information, and deriving new fields. This category is common in pipelines that prepare data for analytics, reporting, or compliance workflows.

Apache Flink and Apache Spark are the most capable options in this space. They offer expressive APIs and SQL interfaces for building complex transformation logic. These tools can integrate with schema registries, support time-based joins, and manage stateful processing for tasks like deduplication or windowed aggregations.

**Illustrative scenario:** A healthcare provider ingests data from patient monitoring devices and merges it with historical records to track clinical metrics. The ingestion pipeline must clean, normalize, and join multiple data streams before writing into an Iceberg table for clinical reporting.

**Design considerations:**

- Use Apache Flink's SQL or DataStream API to perform multi-stream joins, data cleansing, and field derivation in-flight before writing to Iceberg.
- Leverage schema evolution support to accommodate changes in input formats over time.
- Apply partitioning and bucketing strategies in Iceberg to organize data for downstream query performance.

**Trade-offs:**

- Complex transformation logic introduces more moving parts, increasing the risk of pipeline failures or latency spikes.
- Stateful processing requires tuning of memory and checkpointing strategies, especially in long-running jobs.
- Greater developer expertise and testing infrastructure are needed to maintain correctness as business logic evolves.

This type of ingestion architecture emphasizes the importance of strong transformation capabilities close to the data source. Organizations reduce the cost and latency of downstream analytics by performing enrichment or filtering before storage while enforcing data quality and governance policies during ingestion.

### 5.5.4 Handling schema evolution

Ingestion pipelines often operate in environments where data schemas evolve. Source systems may introduce new fields, rename existing ones, or change types, especially in domains like application telemetry, SaaS integrations, or customer interaction data. Supporting these changes without disrupting downstream workflows is critical for reliable data ingestion.

Apache Iceberg provides native support for schema evolution, including adding and renaming fields, reordering columns, and changing field types where safe. Ingestion tools that integrate with schema registries or enforce structured typing, such as Redpanda, Confluent Tableflow, and Airbyte, are well-suited for handling schema drift during ingestion.

**Illustrative scenario:** A product analytics platform ingests event data from multiple mobile applications. As the app evolves, new fields are added to events, and the schema changes frequently. The ingestion system must gracefully handle these changes and reflect them in the Iceberg table structure without manual intervention.

**Design considerations:**

- Before writing, use a tool with built-in schema registry support (e.g., Redpanda or Confluent) to ensure that schema versions are tracked and compatible.
- Configure ingestion in append or merge-on-read mode to accommodate evolving schemas while avoiding file rewrites.
- Implement monitoring or alerts to flag incompatible changes or schema mismatches before they reach the storage layer.

**Trade-offs:**

- Tools that rely on static schemas or lack schema registry integration may require manual updates to accommodate changes.
- Schema evolution can introduce performance issues if not managed carefully, particularly when dealing with nested fields or high column counts.
- Metadata churn in Iceberg from frequent schema updates must be addressed through periodic compaction or snapshot pruning.

In environments where schema change is the norm, choosing ingestion tools that abstract and manage these changes automatically can reduce operational friction and improve data reliability. Aligning the ingestion pipeline with Iceberg's schema capabilities helps ensure that downstream systems continue functioning as expected, even as upstream data evolves.

## 5.5.5 Balancing operational overhead

Ingestion pipelines are shaped not only by technical requirements but also by the operational realities of the teams that manage them. Simpler pipelines that require less tuning, fewer dependencies, and lower ongoing maintenance can free up time and resources for higher-value work. This is especially important for smaller teams or for organizations that prioritize fast iteration and reduced infrastructure complexity.

Tools like Fivetran, Qlik (Upsolver), and Airbyte emphasize ease of deployment and managed operations. They are well-suited for teams that want ingestion pipelines that “just work,” without needing to manually manage stream processing infrastructure or implement complex ingestion logic.

**Illustrative scenario:** A growth-stage company needs to ingest marketing and sales data from SaaS tools like Salesforce, HubSpot, and Google Ads into Iceberg tables for dashboarding and ML feature generation. The data must be fresh but not real-time, and the team managing the pipeline is small.

**Design considerations:**

- A managed ingestion service like Fivetran or Airbyte can extract and land data into cloud object storage in Parquet format.
- Configure the service to write directly to Iceberg-compatible tables, or orchestrate a lightweight transformation step to register and compact data before making it queryable.
- Rely on built-in schema mapping and incremental sync modes to reduce manual schema management.

**Trade-offs:**

- These platforms offer less flexibility for advanced transformations or custom data logic.
- Schema limitations and sync models may not fit all source types or change patterns.
- Some tools may incur higher costs at scale compared to custom-built pipelines using Spark or Flink.

For many teams, reducing operational overhead is a strategic choice that aligns with business agility and cost control. While managed platforms may not meet every use case, they can serve as an effective default path for routine ingestion, allowing engineering resources to focus on pipeline monitoring, validation, and higher-order data modeling.

### 5.5.6 Considering existing cloud environments

While technical requirements often drive architectural decisions, in practice, many ingestion pipelines are shaped by existing cloud environments. Most organizations have already standardized on one or more cloud platforms, and the available tooling, identity and access management, data governance frameworks, and operational models all influence which ingestion solutions are practical.

Cloud-native services such as AWS Glue, Microsoft Fabric, and Google Cloud Dataflow offer seamless integration with their respective storage and compute platforms. Similarly, tools like Confluent (for AWS) or Databricks (on Azure) are often selected not solely for their capabilities, but because they fit cleanly into existing architectures, compliance boundaries, and support agreements.

**Illustrative scenario:** A data team working entirely in AWS wants to build a data lakehouse using Iceberg on Amazon S3. They already use AWS Glue for metadata management and prefer tools that work within their existing IAM and network policies.

**Design considerations:**



- Use AWS-native tools like Glue, Athena, or Amazon EMR to orchestrate ingestion and register Iceberg tables in the Glue Catalog.
- Evaluate managed ingestion solutions like Fivetran or Confluent Tableflow that support AWS S3 and integrate with Glue.
- Leverage IAM roles, bucket policies, and region-based configurations to streamline security and cost management.

#### **Trade-offs:**

- Cloud-native tools may lack some features found in more general-purpose open-source platforms.
- Switching cloud providers or adopting multi-cloud ingestion strategies may be more difficult if you rely heavily on provider-specific services.
- Some Iceberg integrations are still maturing across specific cloud-native tools, especially for schema evolution or streaming workloads.

Aligning ingestion tools with the surrounding cloud ecosystem improves integration, reduces friction, and often speeds up development cycles. While this may limit flexibility in tool selection, it simplifies governance and helps teams deliver value more quickly within existing operational frameworks. Most successful lakehouse architectures are built not in isolation, but as extensions of existing environments and constraints.

Each ingestion requirement category explored in this section, latency, throughput, transformation complexity, schema evolution, operational overhead, and cloud alignment, offers a different lens to evaluate architectural decisions. In isolation, each requirement suggests specific tools and patterns. In practice, however, ingestion pipelines are shaped by a balance of these priorities and organizational constraints such as team size, security policies, and cloud provider commitments.

The examples provided are not definitive blueprints but rather illustrations of how data architects might weigh trade-offs and navigate competing needs. A pipeline optimized for throughput might still need to gracefully support schema evolution. A system built for low operational overhead might require occasional tuning to handle bursts in data volume. Choosing the right ingestion strategy is ultimately about aligning capabilities with context.

By approaching ingestion design through the lens of functional requirements and constraints, rather than starting from a single tool or technology, teams can develop ingestion pipelines that are robust, maintainable, and well-suited to evolving data needs. As the Iceberg ecosystem continues to mature, the range of available tooling will expand, making it even more essential to evaluate ingestion options through structured, context-aware analysis.

## **5.6 Summary**

- Ingestion into an Iceberg lakehouse can follow batch, streaming, micro-batch, or hybrid models, each suited to different data latency and transformation needs.
- Apache Spark and Flink offer flexible and scalable ingestion pipelines with deep integration into Iceberg, making them ideal for complex or high-volume workflows.

- Tools like Fivetran, Airbyte, and Qlik (Upsolver) provide managed ingestion options that reduce operational overhead and support schema mapping and maintenance.
- Kafka-native platforms such as Confluent and Redpanda enable direct materialization of topic data into Iceberg tables, supporting low-latency ingestion and schema evolution.
- Cloud-native services (e.g., AWS Glue, Microsoft Fabric) offer ingestion capabilities aligned with their storage and catalog ecosystems, often simplifying governance and deployment.
- Each ingestion tool and approach should be evaluated against functional requirements such as latency tolerance, throughput demands, schema variability, and team capacity.
- Real-world ingestion architectures often involve trade-offs and require balancing priorities across multiple requirements and existing cloud or infrastructure constraints.
- Designing ingestion pipelines around Iceberg's strengths—such as schema evolution, snapshot isolation, and support for open formats—ensures long-term flexibility and maintainability.

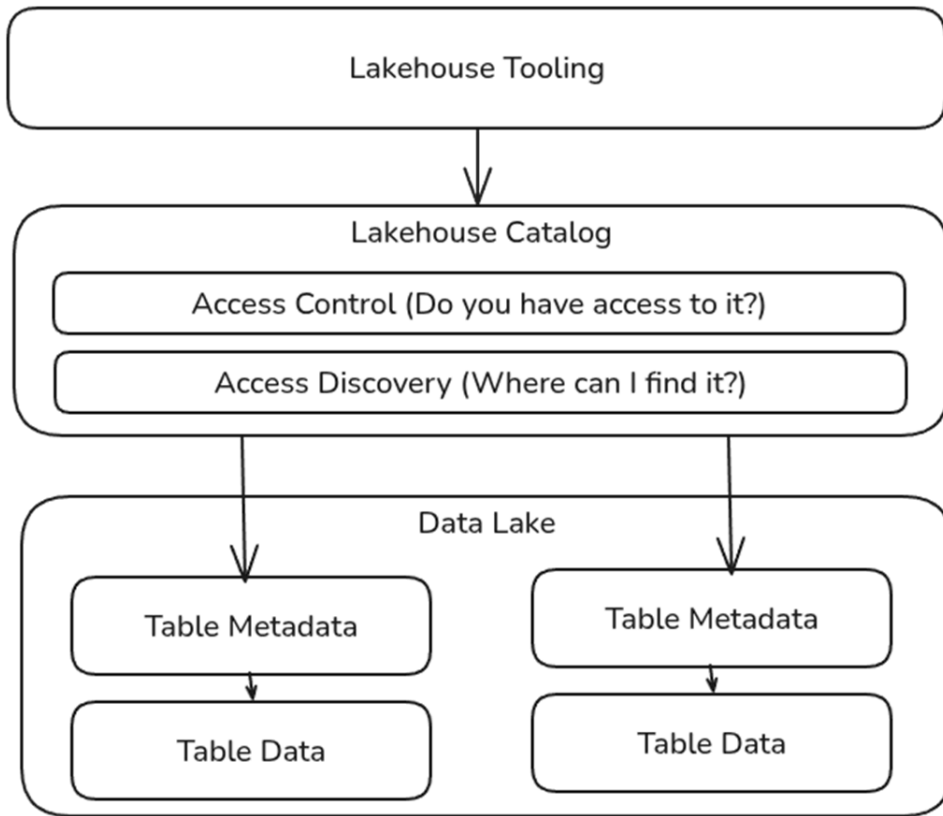
## 6 *Implementing the catalog layer*

### **This chapter covers**

- Defining catalog requirements from audit insights
- The role of the catalog layer in Apache Iceberg
- Evaluating Apache Iceberg catalog implementations
- Applying the REST Catalog specification for interoperability
- Selecting the right catalog for your organization

We've explored the foundational components of an Apache Iceberg lakehouse, including storage and ingestion. Now we turn our attention to the catalog layer, an essential part of any Iceberg deployment. While the storage layer manages physical data and the ingestion layer transforms and loads it, the catalog provides the metadata and coordination necessary for the entire system to function reliably and at scale.

The catalog layer is where Iceberg tables are registered, tracked, and organized. It tracks table metadata, manages namespaces, and serves as the point of coordination for data operations. Choosing the right catalog is not merely a technical decision; it is also a strategic one. It influences governance, interoperability, scalability, and integration with the broader ecosystem, as illustrated in figure 6.1.



**Figure 6.1** The catalog enables tools that access lakehouse tables to verify permissions and locate the corresponding data within the lake.

Each organization brings unique architectural constraints, compliance needs, and operational goals. This chapter begins by examining the requirements that should guide catalog selection, insights you will have determined during the audit process described in chapter 3. From there, we'll explore how Iceberg's catalog interface works, including support for a growing set of implementations such as Apache Polaris, Nessie, AWS Glue Data Catalog, Dremio Catalog, Snowflake Open Catalog, and more.

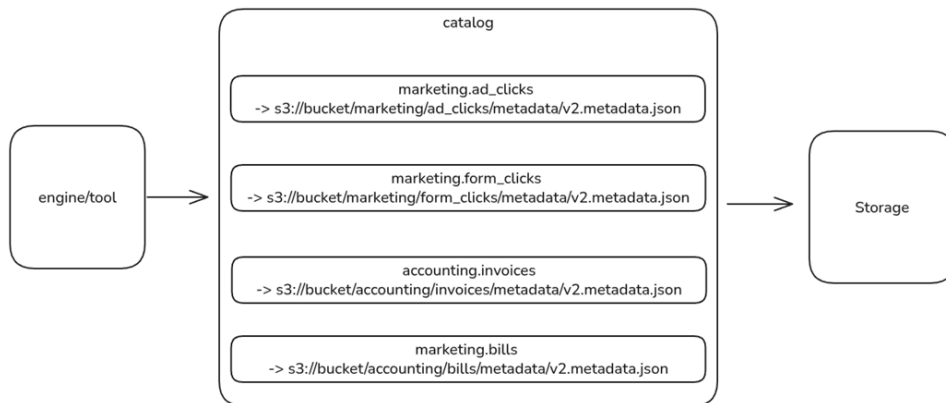
We'll also examine the Apache Iceberg REST Catalog specification, a recent development designed to unify how engines and tools connect to catalog services. Finally, we will walk through several realistic decision-making scenarios to demonstrate how different catalog features align with specific organizational priorities.

By the end of this chapter, you'll have the clarity to select and configure a catalog that aligns with your lakehouse's needs and the confidence to adapt as those needs evolve.

## 6.1 The role of the catalog in Apache Iceberg lakehouses

Apache Iceberg enables reliable and concurrent data access by using a snapshot-based model to manage table state. Each time a change is made to a table, whether it's a data insert, delete, or schema update, Iceberg creates a new snapshot that represents the complete metadata state of the table at that point in time. These snapshots are immutable and versioned, allowing users to safely time-travel, roll back to earlier states, and query consistent views of data without conflicts. This snapshot isolation is what makes Iceberg ACID-compliant, even when multiple tools are reading from and writing to the same table.

The catalog is what connects these snapshots into a cohesive system. It serves as the metadata backbone of the lakehouse, tracking Iceberg tables and pointing to the current snapshot that represents each table's latest state. While traditional databases tie metadata storage to a single engine, Iceberg abstracts this responsibility through the catalog, allowing diverse compute engines to access and modify table metadata in a consistent and transactional manner. As shown in figure 6.2, the catalog manages table locations, organizes tables into namespaces, and handles concurrency by coordinating snapshot updates. This separation of storage, compute, and metadata management makes Iceberg highly interoperable and scalable across tools, clouds, and use cases.



**Figure 6.2** An Apache Iceberg catalog maps each table name to the location of its latest metadata file, which serves as the entry point for tools to begin scanning and interacting with the table.

This section will break down the specific responsibilities of the catalog, highlight how it fits into Iceberg's architecture, and explain how engines rely on it to resolve tables, plan queries, and execute updates. Understanding these responsibilities is crucial before comparing catalog implementations, as it helps clarify which capabilities are essential for your use case.

### 6.1.1 Responsibilities of the catalog

The catalog in Apache Iceberg serves as the authoritative index for all table metadata. While data files live in cloud or distributed storage, and queries are executed by engines like Spark, Flink, or Trino, the catalog ties everything together. It enables engines to locate, interpret, and modify Iceberg tables safely and consistently.

The catalog handles four key responsibilities:

#### 1. Table registration and discovery:

The catalog tracks all registered Iceberg tables within a given namespace. When a user queries a table, the engine contacts the catalog to resolve the table's name into a metadata location. This allows multiple engines to share a common table catalog, ensuring consistent access across batch and streaming jobs.

#### 2. Metadata location management:

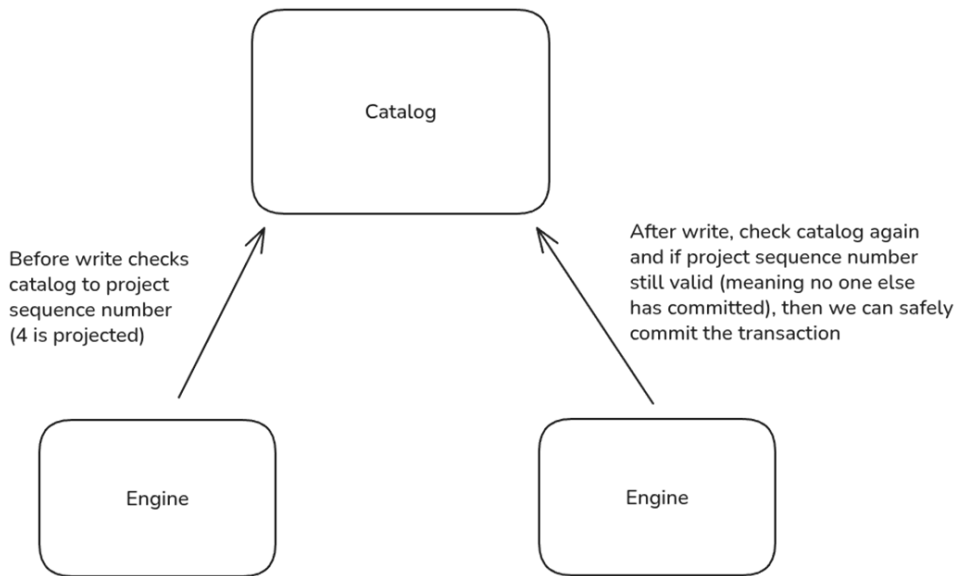
Each Iceberg table consists of multiple metadata files: the current snapshot, schema definitions, partition specs, and manifest lists. The catalog maintains a pointer to the current metadata root, which is updated transactionally during commits. This pointer is the foundation for snapshot isolation, enabling time travel and rollback features.

#### 3. Namespace organization:

Catalogs group tables using namespaces, which often reflect organizational or domain boundaries. For example, a table might be registered under `analytics.marketing.sales_data`. This hierarchy simplifies table discovery and access control, especially in large environments with hundreds or thousands of tables.

#### 4. Transaction coordination:

During write operations, the Iceberg engine performs an optimistic commit as illustrated in figure 6.3. It generates new metadata, then attempts to atomically update the catalog's metadata pointer. If the pointer has changed since the last read, indicating another job committed concurrently, the operation fails and must retry. This coordination ensures that concurrent writers do not corrupt the table state.



**Figure 6.3 Optimistic commits allow engines to write data and metadata and determine if the commit is successful by checking its place in the commit sequence before and after writing files and then committing.**

Some catalog implementations may also offer additional features such as audit logging or version history. However, these four core responsibilities define what all Iceberg-compatible catalogs must support.

Understanding these mechanics helps explain why catalog reliability and performance are so crucial. Every query, update, or schema change passes through this layer, making it one of the most critical components in the Iceberg architecture.

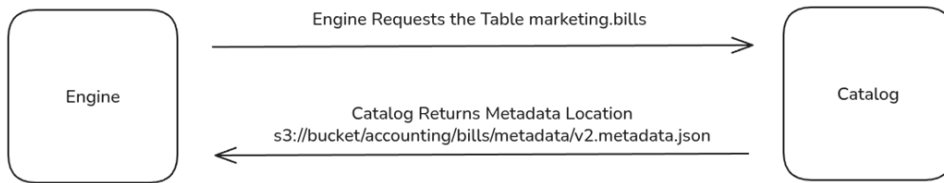
### 6.1.2 Catalog interactions with query and processing engines

The catalog is the first point of contact when a query engine or processing job begins working with an Iceberg table. Whether a user runs an interactive SQL query or submits a streaming ingestion job, the engine must consult the catalog to resolve the table name, load its metadata, and plan the operation accordingly.

This interaction typically follows a predictable sequence:

1. **Table resolution:**

The engine requests a table from the catalog by name. The catalog returns a reference to the latest metadata file for that table, as shown in figure 6.4. The table's metadata file includes schema, partitioning, snapshot information, and file manifests.

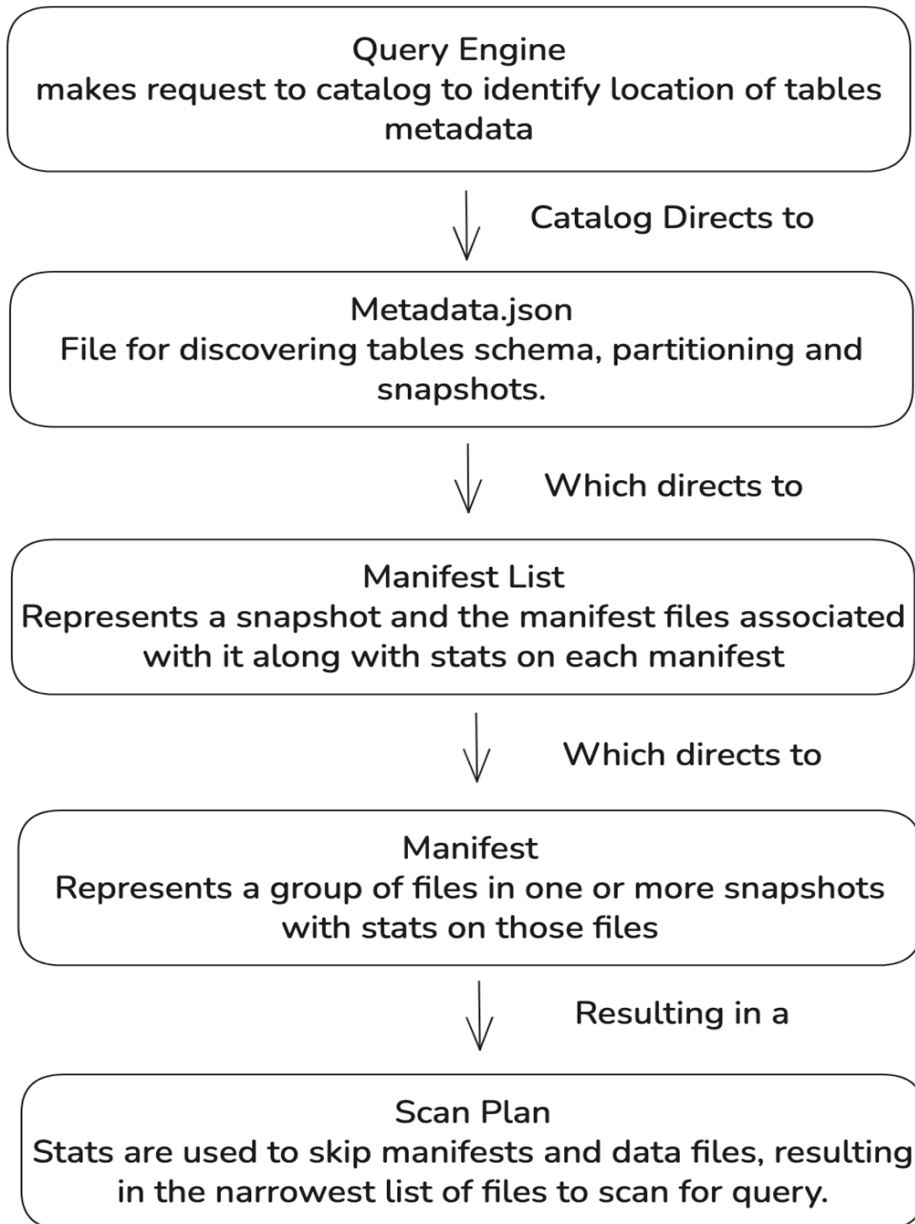


**Figure 6.4 Engines require tables from catalogs and return their metadata location.**

2. **Snapshot planning:**

Using the metadata as illustrated in figure 6.5, the engine determines which data files to read or write. Iceberg's metadata includes statistics and file-level metadata that support data skipping and predicate pushdown. This reduces the data scanned and improves performance..





**Figure 6.5** The metadata provides different levels of data for an engine to identify a snapshot and determine which data files in that snapshot should be scanned.

3. **Commit coordination:**

For write operations, engines construct new metadata, potentially including added or deleted files, updated schemas, and new snapshots. Once constructed, they attempt to commit this metadata by updating the catalog's pointer atomically. If another operation has updated the table in the meantime, the commit will fail, ensuring consistency and avoiding race conditions.

4. **Schema and evolution management:**

Engines also use the metadata discovered via the catalog to validate schemas, detect changes, and align transformations. For example, before inserting data, Spark may compare the job's write schema with the current table schema retrieved through the metadata file discovered via the catalog.

5. **Time travel and rollback:**

If a user requests a previous snapshot, the engine can retrieve that specific metadata version from the catalog's snapshot history in the metadata file. Catalogs that retain this history enable time-travel queries and data recovery workflows.

The consistency of these interactions is what allows Iceberg to function as a multi-engine, multi-purpose lakehouse format. Whether you're using Spark for ETL, Flink for streaming, or Trino for ad hoc analytics, the catalog provides a unified metadata view that keeps operations aligned.

## 6.2 Evaluating catalog requirements

Before selecting a catalog, it's essential to clarify what your organization needs from it. The catalog layer is more than a technical component; it defines how teams discover, access, and manage data across the lakehouse. A mismatch between catalog capabilities and organizational requirements can lead to governance gaps, integration issues, or operational inefficiencies.

In chapter 3, we outlined how stakeholder interviews and technical audits help surface platform needs. Those insights now come into play. This section examines the key requirements to consider when evaluating catalog options, including performance, availability, metadata governance, security, and compatibility with deployment environments.

We'll begin by framing these requirements around common enterprise concerns, latency expectations, regional availability, access controls, and schema tracking. From there, we'll look at how these concerns translate into concrete catalog features and how to weigh trade-offs based on your organization's architecture, compliance needs, and growth plans.

### 6.2.1 Performance, availability, and scale

At its core, a catalog enables query engines and processing tools to discover Iceberg tables and access their metadata. This interaction often occurs on the critical path of read and write operations, making catalog performance a key consideration. High-latency responses can slow down table planning and query execution, especially in workloads that involve frequent metadata lookups or concurrent writes.

Availability is equally important. If the catalog is unreachable, users may not be able to query or update tables, even if the underlying storage is fully operational. This makes catalog uptime a foundational dependency for the entire lakehouse. In production environments, high availability typically means achieving uptime targets of 99.9% (roughly 8.8 hours of downtime per year) or better. Mission-critical systems often target 99.99% availability, allowing less than an hour of annual downtime. To meet these expectations, evaluate whether your catalog supports fault-tolerant deployment patterns such as clustering, multi-region replication, or managed failover mechanisms.

Scalability also plays a critical role as your platform grows. In multi-tenant environments or organizations managing hundreds to thousands of Iceberg tables and supporting dozens or even hundreds of concurrent users, the catalog must sustain high metadata request volumes without becoming a bottleneck. This includes not only read throughput—where mature catalogs often handle thousands of metadata lookups per second—but also concurrent write operations, namespace expansion, and low-latency snapshot metadata resolution. Some production-grade deployments report handling over 10,000 tables and millions of daily catalog interactions.

When evaluating catalog options, ask the following:

- Can the catalog handle a high request rate with low latency responses??
- Does it offer a decoupled backend database for storing metadata?
- Can the catalog scale its ability to handle more requests by adding compute vs. deploying new catalog instances?
- Does it offer fault-tolerant deployment modes?
- Is it optimized for latency-sensitive workloads such as interactive queries or near-real-time/real-time ingestion?
- Is the catalog open and interoperable with the ecosystem of tools you use?

While no single metric captures catalog readiness, understanding and testing your expected workloads, such as batch versus interactive, development versus production, will help determine whether a candidate catalog can consistently meet performance, availability, and scalability targets.

## 6.2.2 Metadata governance and lineage

A catalog does more than store table locations; it becomes the foundation for data governance across the lakehouse. In most organizations, the catalog serves as the system of record for where data resides, its structure, and who has access to it. This makes metadata governance one of the most critical dimensions to evaluate when selecting a catalog.

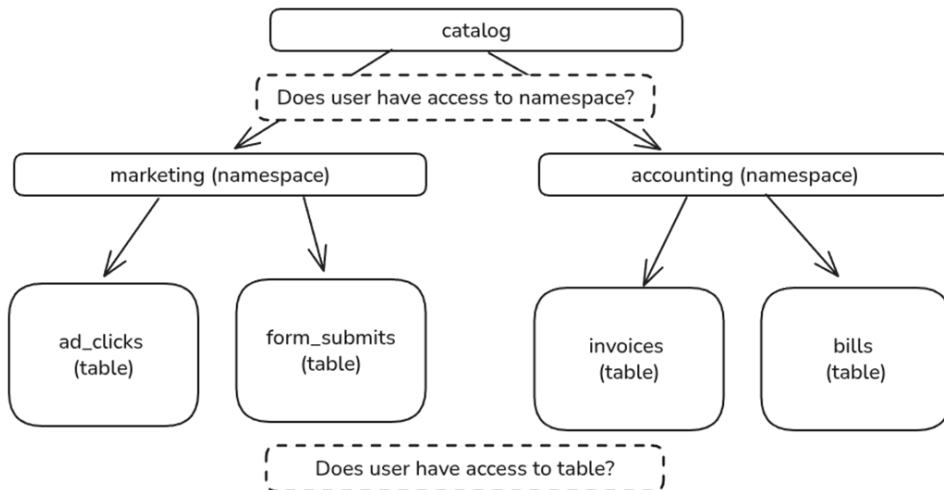
At a minimum, your catalog should support namespacing and access control as illustrated in figure 6.6. Namespaces allow related tables to be grouped logically, often aligning with business domains or organizational units. However, robust governance requires capabilities beyond basic access restrictions.

First, catalogs must integrate with external identity providers such as OAuth, LDAP, SAML, or enterprise SSO systems. These integrations allow the catalog to inherit user identities and access contexts, ensuring that platform permissions are consistent with broader enterprise policies.

Second, catalogs should support fine-grained authorization models like Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC). RBAC assigns permissions based on user roles (e.g., analyst, engineer), while ABAC makes decisions based on user attributes, such as department or location. These models are crucial for maintaining tight control over who can read, write, or administer specific tables or namespaces.

Third, some catalogs implement credential vending, a security pattern where the catalog issues short-lived, scoped-down credentials to querying clients. This mechanism ensures that clients access only the storage paths necessary for their query, reducing blast radius and simplifying auditing.

Together, these features enable the catalog to enforce security at the metadata and storage levels. But governance also requires lineage tracking, the ability to follow how data flows through systems over time. Lineage connects identities, permissions, and operations into a chain of custody that supports compliance, auditability, and root-cause analysis. For enterprises subject to regulatory frameworks such as GDPR, HIPAA, or SOC 2, this capability is not optional—it's essential.



**Figure 6.6 Catalogs can have different levels of access controls, allowing you to control who has access to what namespaces or tables.**

Catalogs should log metadata operations such as table creation, schema updates, and snapshot rollbacks. These logs help enforce internal controls and provide evidence during audits. Catalogs should also record which users or services initiated changes and from where they originated.

When comparing catalog implementations, consider:

- Does it support connecting to an external identity provider?
- Does it allow for defining fine-grained access controls?
- Does it offer credential vending to all of the storage systems you intend to use (S3, ADLS, GCS, HDFS)?
- Does it log and allow you to audit metadata operations?

- Does it offer metadata lineage capabilities?

Strong governance features not only fulfill external obligations, but they also enhance internal reliability, simplify debugging, and mitigate the risk of unauthorized changes that could compromise data integrity.

### 6.2.3 Security and compliance

Security is a fundamental requirement in any modern data platform. As the central control point for metadata and table discovery, the catalog plays a critical role in enforcing security policies and ensuring compliance. While Apache Iceberg provides guarantees around schema enforcement and transactional integrity, it is the catalog that determines who can see which tables, what actions they are allowed to perform, and how metadata is accessed and managed.

Most organizations already have established identity and access management (IAM) systems in place. Catalogs that integrate with these systems, such as Amazon IAM, LDAP, OAuth, or custom token services, simplify user management and align Iceberg with enterprise security practices. This integration also enables fine-grained permissions, ensuring different roles can manage, query, or alter metadata only as appropriate.

Encryption, both in transit and at rest, is another essential concern. While the storage layer typically encrypts data files, catalog metadata must also be protected, mainly when it includes snapshot history, partition values, or schema evolution records. Catalogs should use secure communication protocols (e.g., HTTPS, gRPC over TLS) and support storage encryption for any persisted state.

Compliance requirements often shape catalog architecture as well. Regulations may require data residency within specific geographic boundaries, mandatory logging of administrative actions, or formal controls over metadata retention and deletion. In regulated environments, the ability to produce audit trails and enforce metadata lifecycle policies is not just beneficial, it is necessary.

Key questions to ask include:

- Does the catalog support integration with your existing IAM or SSO systems?
- Are metadata operations encrypted and securely stored?
- Can the catalog provide the necessary audit trails and controls for regulatory compliance?

A catalog that meets your security and compliance requirements helps prevent unauthorized access, supports regulatory alignment, and strengthens the trustworthiness of your data platform from the foundation up.

## 6.2.4 Deployment flexibility and ecosystem compatibility

As data architectures grow more distributed and heterogeneous, catalog flexibility becomes essential. Whether your platform spans on-premises clusters, multiple cloud providers, or hybrid environments, the catalog must operate reliably across those boundaries. Deployment flexibility and ecosystem compatibility determine how easily a catalog fits into your infrastructure and interacts with other tools.

Deployment flexibility describes the ability to run your catalog in different environments based on your organization's infrastructure. Some catalogs are tightly integrated with specific compute engines or storage ecosystems, while others can operate as standalone services. These standalone catalogs may be self-hosted or delivered as managed services by vendors. REST-based catalogs are especially attractive because they expose a consistent interface, allowing tools and services to communicate with the catalog regardless of the programming language or cloud platform in use.

However, while REST interfaces are environment-agnostic, the catalog itself still runs somewhere physically; on-premises, in a cloud region, or across multiple locations. This matters because tools accessing the catalog need low-latency communication, especially for metadata-intensive operations. For example, if your catalog is deployed in an on-premises data center but your compute engine runs in the cloud, network latency can slow down operations like table discovery or snapshot reads. Therefore, deployment decisions must weigh not just compatibility and interface support, but also proximity to the systems that depend on the catalog's performance.

Compatibility with the rest of your stack is just as important. Your catalog must work seamlessly with processing engines like Apache Spark, Flink, Trino, or Dremio. It should support the same table resolution and transaction models across these engines, ensuring consistent behavior regardless of the execution environment. Catalogs that implement the Apache Iceberg REST specification generally offer broader compatibility across tools and vendors.

Another consideration is how well the catalog supports automation and integration with orchestration frameworks. CI/CD pipelines, schema registries, and metadata governance tools all benefit from catalogs that expose APIs, support declarative configuration, and emit machine-readable metadata.

Key evaluation questions include:

- Can the catalog be deployed in your preferred environment (cloud, on-prem, containerized)?
- Does it support all of the query and processing engines used by your teams?
- Is it REST-compliant or otherwise well-documented for integration with external systems?

Selecting a catalog that matches your deployment model and integrates cleanly with your ecosystem reduces operational friction and improves the long-term adaptability of your lakehouse architecture.

### 6.2.5 Cost and operational overhead

While performance, governance, and compatibility are often top of mind, cost and operational burden can quietly shape the long-term viability of a catalog. These factors include not only direct expenses like licensing or cloud usage but also the effort required to deploy, monitor, and maintain the catalog over time.

Self-managed catalogs, such as many OSS catalogs we will discuss in this chapter, may have no licensing fees, but they require engineering effort to deploy, configure, secure, and upgrade. This effort grows with scale, especially in environments that require high availability or multi-region deployment. Managed services, by contrast, reduce operational complexity but may introduce vendor lock-in or usage-based costs that grow over time.

Monitoring and observability also contribute to operational overhead. A catalog that provides clear metrics, structured logs, and integration with alerting systems allows teams to detect issues early and respond quickly. Catalogs that lack this visibility often require more manual troubleshooting, increasing support effort and time to resolution.

Documentation and community support are another part of the operational equation. Catalogs with mature documentation, active maintainers, and responsive forums can accelerate onboarding and reduce dependency on in-house experts. This support becomes especially important when custom integration or advanced tuning is needed.

To evaluate catalog costs and overhead, consider:

- What are the infrastructure and personnel costs to operate the catalog at scale?
- Does it offer monitoring, logging, and automated recovery features?
- How active is the development community or vendor support channel?

Balancing feature depth with operational simplicity is key. The most technically capable catalog may not be the right fit if it requires constant upkeep or exceeds your budget. Likewise, a lightweight catalog that integrates easily and runs efficiently may offer long-term value even if it lacks advanced features.

### 6.2.6 Catalog federation and mesh architectures

Organizational structures, compliance requirements, and regional deployment constraints frequently necessitate the use of multiple catalogs, each operating independently but still needing to participate in a cohesive ecosystem. This is where catalog federation and mesh architectures become essential.

A federated catalog architecture allows multiple catalog instances, deployed across clouds, geographies, or business units, to operate autonomously while still providing a unified, global view of Iceberg tables. In this model, each catalog manages its own namespace, access policies, and metadata operations, but exposes a common interface that enables discovery and access across the entire mesh.

This approach aligns well with data sovereignty mandates, allowing local catalogs to maintain control over data residency, encryption, and auditability. At the same time, it supports enterprise-wide analytics and governance by ensuring that users and tools can query across boundaries without needing to manually replicate metadata or coordinate between teams.

Common drivers for adopting a catalog mesh include:

- **Regulatory boundaries** that require data to remain within specific jurisdictions.
- **Decentralized teams** operating under separate governance models.
- **Cloud-specific deployments** where services are tied to provider-native catalogs (e.g., AWS Glue, Azure Purview).
- **Operational scale** where a single catalog becomes a bottleneck or a single point of failure.

To support federation, catalogs must implement:

- **Consistent APIs** (e.g., REST or gRPC) for metadata access and synchronization.
- **Namespace resolution strategies** to distinguish between local and remote resources.
- **Security and identity propagation** to maintain access control integrity across catalogs.
- **Change propagation mechanisms**, such as event-based sync or metadata replication, to keep federated catalogs aligned without introducing conflicts or stale states.

When evaluating catalog implementations, consider:

- Can it participate in a federation layer or integrate with external discovery services?
- How are conflicts, updates, and access controls managed across the mesh?

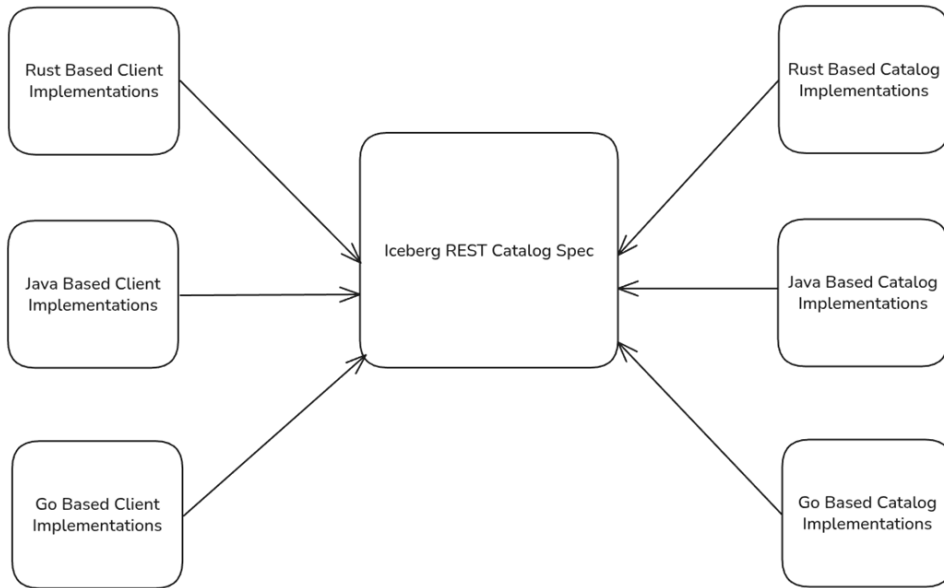
A well-architected catalog federation enables organizations to scale Iceberg lakehouses without sacrificing autonomy, compliance, or visibility. It reflects a shift from centralized control to distributed coordination, a necessary evolution as data ecosystems grow in complexity.

## 6.3 Apache Iceberg REST Catalog Spec

A central promise of the lakehouse architecture is openness, specifically, the ability for multiple engines to access and manage data through shared standards. Apache Iceberg delivers on this by standardizing the table format, but until recently, catalog interoperability remained inconsistent. Each catalog implementation relied on its own client libraries, often written for a specific language and maintained at varying levels of completeness. This created friction for developers and architects, who faced challenges integrating with multiple catalogs across languages and tools.



The introduction of the Apache Iceberg REST Catalog specification marked a major step forward. By defining a consistent HTTP-based interface for catalog operations it decouples the client from the catalog implementation. This makes it easier to mix and match query engines and catalog services, allowing organizations to evolve their architecture without being locked into a specific vendor or language ecosystem, as illustrated in figure 6.7.



**Figure 6.7 Using the REST Catalog spec, a client written in any language can talk to all catalogs regardless of what language the catalog server is written in, improving interoperability.**

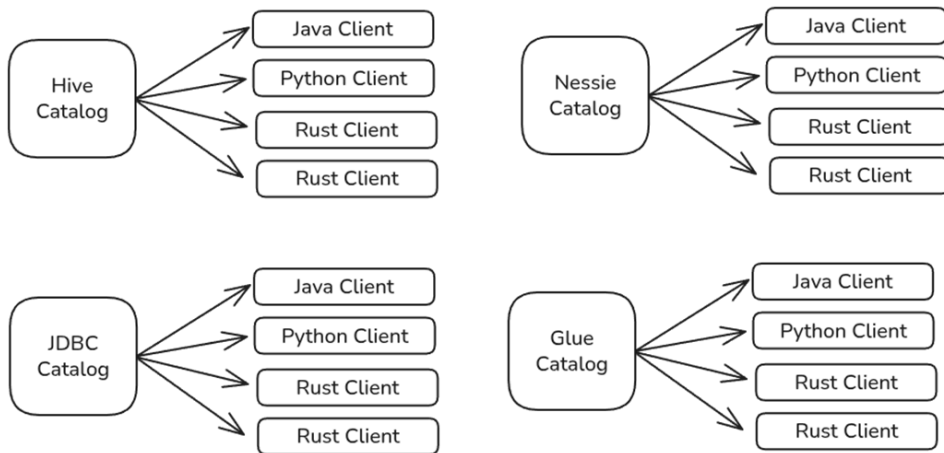
This section explains the fragmented environment that existed before the REST spec, the technical rationale for creating it, and the value it delivers to Iceberg users and catalog implementers alike. We'll also highlight the growing ecosystem of catalog services that support the REST interface and what this means for cross-platform lakehouse deployments.

### 6.3.1 Before the Apache Iceberg REST spec

Before the introduction of the Apache Iceberg REST Catalog specification, the landscape of catalog integrations was fragmented and inconsistent. Catalog functionality, such as listing namespaces, registering tables, and committing changes, was tightly coupled with language-specific client libraries. Each catalog provider typically had to implement and maintain client libraries in multiple languages, including Java, Python, Go, and Rust. However, the level of support and feature completeness varied widely between these implementations.

This inconsistency created a host of challenges for both catalog providers and Iceberg users. A catalog might have full support in Java but lack critical functionality in Python or Go. Developers integrating with Spark, for instance, had to ensure they were using the correct client library version, often managing complex classpaths and dependencies. Misconfiguration or mismatched libraries could easily lead to runtime errors or subtle bugs.

Moreover, the need to embed catalog logic into each client library duplicated effort and slowed innovation. Every new feature in a catalog had to be re-implemented and tested across all supported client languages, as illustrated in figure 6.8. This made it difficult for catalog vendors to ensure consistent behavior and performance across engines.



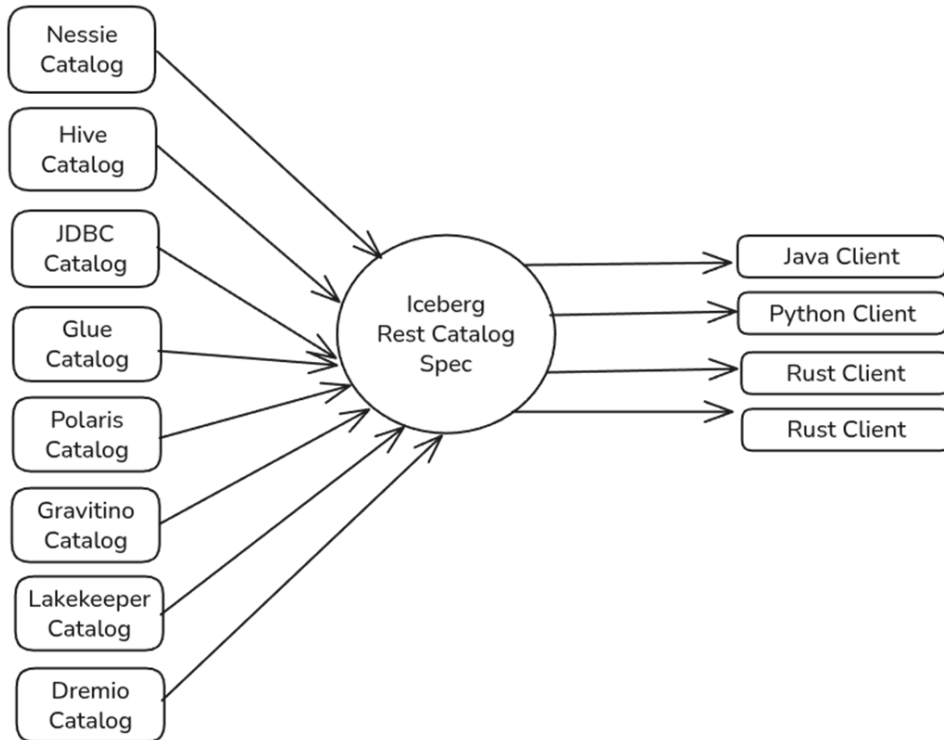
**Figure 6.8 Before the REST catalog spec, each catalog service would have to maintain its own client libraries in each language, causing a hard-to-maintain and fragmented environment.**

The lack of a standard communication protocol between engines and catalogs also hindered interoperability. Organizations looking to adopt multiple engines, such as Spark for batch processing, Flink for streaming, and Trino for interactive queries, faced difficulty if their chosen catalog only worked well with one of them. As the Iceberg ecosystem grew, so did the urgency for a more standardized, engine-agnostic approach to catalog integration.

### 6.3.2 The solution

To resolve the interoperability and maintenance challenges caused by fragmented catalog client libraries, the Iceberg community introduced the Apache Iceberg REST Catalog specification. This spec defines a consistent, engine-neutral HTTP-based API for interacting with catalogs. Instead of embedding logic in each client library, engines and applications can now communicate with any compliant catalog using a shared, language-independent protocol.

The REST spec standardizes operations such as retrieving table metadata, listing namespaces, registering new tables, and committing changes. This simplifies client implementations and ensures uniform behavior across different catalogs and engines. By adopting the REST interface, catalog services no longer need to build and maintain extensive language-specific client libraries, as illustrated in figure 6.9. Instead, engines like Spark, Flink, Trino, and others can integrate directly through a common interface.



**Figure 6.9 With the REST Catalog Spec, the same client library can be used for all catalogs, reducing the fragmentation and improving interoperability between iceberg catalogs.**

This approach brings immediate benefits to users and catalog providers. Users can choose the best-fit catalog for their use case without worrying about language or engine compatibility. Catalogs can focus on performance, governance, and availability, knowing that a well-defined REST interface will handle integration across the Iceberg ecosystem.

The REST spec has quickly gained traction, and several major catalogs now support it. These include AWS Glue Data Catalog, Apache Polaris, Apache Gravitino, Lakekeeper, Nessie, Dremio Catalog, and Snowflake Open Catalog. As adoption grows, organizations gain the flexibility to evolve their data infrastructure without reengineering the surrounding tools or pipelines.

In essence, the REST spec transforms the Iceberg catalog layer from a tightly-coupled integration point into an open, composable interface. This enhances the modularity and durability of lakehouse architectures and enables true interoperability at the metadata layer.

## 6.4 Catalog options: Exploring the ecosystem

Apache Iceberg supports a flexible catalog interface, and the ecosystem of catalog implementations continues to expand. These catalogs vary in architecture, operational model, and integration depth, but all aim to fulfill the core responsibilities described earlier: managing table metadata, coordinating transactions, and enabling multi-engine access to datasets.

Catalogs fall into several broad categories. Some, like the Hadoop Catalog and JDBC Catalog, are built into the Iceberg libraries and require no additional services other than a JDBC-compatible database. These are suitable for simpler deployments or development environments where external dependencies should be minimized.

Others, such as Apache Polaris, Project Nessie, Apache Gravitino, and Lakekeeper, are open source and self-managed. These catalogs are designed to support advanced metadata capabilities such as REST-based access, version control, and multi-tenant environments. They offer greater flexibility and extensibility, but require infrastructure planning and operational oversight.

Finally, several catalogs are provided and maintained by cloud and analytics vendors. These include AWS Glue Data Catalog, Dremio Catalog (based on Apache Polaris), and Snowflake's Snowflake Open Catalog (also Polaris-based). These managed services often integrate tightly with their respective platforms, offering simplified setup, built-in security, and scaling. However, they may impose limitations on customization or portability.

This section will walk through these catalog types, highlighting their architectures, capabilities, and trade-offs, some of which are summarized in table 6-1. By comparing their features and deployment models, you'll gain the context needed to align your catalog choice with the goals and constraints of your organization.

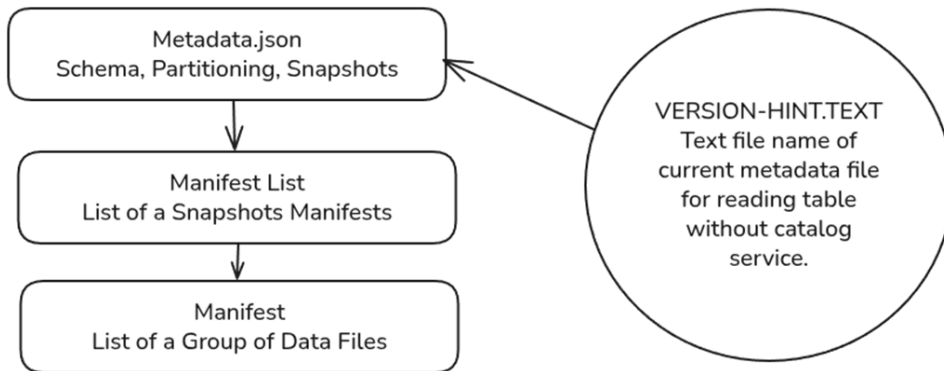
**Table 6.1 A comparison of Apache Iceberg catalogs**

| <b>Name</b>            | <b>OSS</b>                          | <b>REST Catalog</b> | <b>Features</b>                                                                                                                                        |
|------------------------|-------------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hadoop                 | Yes                                 | No                  | No Service Required, potential consistency issues                                                                                                      |
| JDBC                   | Yes                                 | No                  | Can use any JDBC Database as a catalog                                                                                                                 |
| Hive                   | Yes                                 | No                  | Leverage existing Hive catalog                                                                                                                         |
| Apache Polaris         | Yes                                 | Yes                 | RBAC, Catalog Federation, Multi-Tenancy                                                                                                                |
| Project Nessie         | Yes                                 | Yes                 | Git-Like Catalog Versioning                                                                                                                            |
| Apache Gravitino       | Yes                                 | Yes                 | Federation with Hive, Geo-Distributed Catalog                                                                                                          |
| Lakekeeper             | Yes                                 | Yes                 | Rust Based, Supports Data Contracts                                                                                                                    |
| AWS Glue Data Catalog  | No                                  | Yes                 | Integrated in AWS Tooling                                                                                                                              |
| Dremio Catalog         | Polaris Based                       | Yes                 | Commercial Service built on Apache Polaris with Automated Table Maintenance, paid based on compute used for maintenance, external requests not charged |
| Snowflake Open Catalog | Polaris Based                       | Yes                 | Commercial Service built on Polaris, charged on requests to the catalog                                                                                |
| Databricks Unity       | No (OSS version different codebase) | Yes                 | Supports Iceberg and Delta Lake                                                                                                                        |

### 6.4.1 Hadoop Catalog

The Hadoop Catalog is one of the simplest ways to manage Iceberg metadata. Built directly into the Apache Iceberg library, it does not require any external service or runtime component. Instead, it stores table metadata in a structured directory layout within a Hadoop-compatible file system, such as HDFS, Amazon S3, or Azure Data Lake Storage.

Each table's metadata is stored in a separate subdirectory under a common warehouse root. The catalog maps table names to these directories and manages their metadata references locally. Because the metadata is stored alongside the data itself, as illustrated in figure 6.10, this approach is straightforward to set up and easy to reason about.



**Figure 6.10** With Hadoop catalog, a file called **VERSION-HINT.TEXT** is used to discover the metadata file instead of a catalog service, which you would send a request to.

#### Key characteristics:

- *Local state management*—All catalog operations are filesystem-based. Table registration, updates, and deletions are handled by manipulating metadata files directly.
- *No external dependencies*—There is no server to deploy or service to manage. The catalog operates entirely within the same context as the client (e.g., Spark or Flink).
- *Namespace support*—Table names are organized into logical namespaces via directory hierarchies (e.g., warehouse/analytics/marketing/sales\_data).

#### Strengths:

- Fast to configure and useful for local development or isolated environments.
- Suitable for single-user or tightly controlled workloads.
- Minimal operational overhead with no need for a separate metadata service.

#### Limitations:

- Lacks centralized transaction coordination for concurrent writes across engines.
- Does not support remote access or REST APIs, making integration across environments or teams more difficult.
- Poor fit for multi-tenant or multi-region lakehouses where access control and lineage tracking are required.

The Hadoop Catalog remains a solid choice for teams exploring Iceberg, working in isolated environments, or maintaining smaller-scale workloads. However, for production-grade deployments that span tools, regions, or teams, a more service-oriented catalog is often required.

### 6.4.2 Hive Catalog

The **Hive Catalog** allows Apache Iceberg to use an existing Hive Metastore as the catalog backend. This makes it an attractive option for organizations that have invested heavily in Hive-based data platforms and want to transition incrementally to Iceberg without overhauling their metadata management infrastructure.

Using the Hive Catalog, Iceberg tables are registered in Hive Metastore, and engines like Spark, Trino, and Flink can interact with the catalog through their respective Hive integration layers. This compatibility makes it relatively straightforward to adopt Iceberg alongside legacy Hive tables during a gradual migration.

#### Advantages:

- Leverages existing Hive Metastore infrastructure
- Well-supported by popular engines like Spark, Flink, and Trino
- Familiar integration patterns for teams already using Hive

#### Trade-offs:

- Lacks support for some Iceberg features that depend on more modern catalog capabilities, such as namespace operations and fine-grained role-based access control
- Does not support the REST Catalog API, limiting interoperability with emerging tools
- Susceptible to limitations and scalability issues of the Hive Metastore

The Hive Catalog is best suited for teams that already run Hive Metastore and are looking for a low-friction path to Iceberg. It can serve as a transitional solution while planning a move to a more advanced catalog with stronger multi-engine support and governance features.

### 6.4.3 JDBC Catalog

The JDBC Catalog is another built-in option provided by Apache Iceberg. Instead of storing metadata pointers in the file system, it uses a relational database to manage catalog state. Table and namespace information is persisted in a set of catalog tables, and all operations, such as creating, updating, or deleting Iceberg tables, are executed through standard SQL transactions.

Supported databases include PostgreSQL, MySQL, and others that provide JDBC-compliant drivers. This approach brings stronger transactional guarantees than the Hadoop Catalog and introduces the ability to share a centralized metadata store across multiple compute environments.

**Key characteristics:**

- **SQL-backed metadata store:** Metadata pointers and namespace hierarchies are stored in structured tables within the database.
- **ACID guarantees:** SQL transactions ensure atomic updates and reduce the risk of metadata inconsistencies during concurrent operations.
- **Single-node architecture:** The catalog server is embedded within the client (e.g., Spark job or application), but the backing store enables shared state across jobs.

**Strengths:**

- Simple to configure if a JDBC-compatible database is already available.
- Easier to scale than Hadoop Catalog for multi-engine access.
- Supports concurrent writes with transactional integrity, making it more suitable for collaborative environments.

**Limitations:**

- The catalog client still runs inside each engine context, so scaling depends on the underlying database performance.
- No built-in REST interface, remote access, and cross-language integration must go through the Iceberg client library.

The JDBC Catalog provides a middle ground between file-based simplicity and full catalog services. It's a practical choice for teams that want centralized coordination without introducing a new catalog runtime, especially when a reliable relational database is already part of the stack.

## 6.4.4 Apache Polaris

Apache Polaris (Incubating) is an open source, REST-based catalog implementation for Apache Iceberg. Designed for multi-engine environments, Polaris acts as a centralized coordination layer for Iceberg table metadata, providing consistent, secure access across engines like Spark, Flink, Trino, and Snowflake.

Polaris builds on the Apache Iceberg REST Catalog specification, enabling decoupled metadata access that aligns with cloud-native architectures. It supports both internal and external catalogs and introduces advanced concepts such as service principals, RBAC-based access control, and temporary credential vending.

**Architecture and deployment model:**



Polaris operates as a standalone service, typically deployed in a secure environment with access to cloud object storage (e.g., S3, ADLS, or GCS). Each catalog configured within Polaris maps to a specific cloud storage location and manages metadata pointers for one or more Iceberg tables.

#### **Internal vs external catalogs:**

- *Internal catalogs* are fully managed within Polaris. Tables are registered, updated, and accessed through Polaris APIs, while data and metadata reside in your cloud storage.
- *External catalogs* allow Polaris to sync metadata from catalogs managed elsewhere (e.g., Glue or Dremio). These tables are read-only within Polaris, making this model useful for centralized visibility without assuming control of write operations.

#### **Security and access control:**

Polaris enforces security through a layered role-based access control (RBAC) system:

- *Service principals* represent users or systems (e.g., Spark jobs, BI dashboards).
- *Principal roles* group service principals with common access requirements.
- *Catalog roles* define privileges at the catalog, namespace, and table levels and are assigned to principal roles.

This structure allows fine-grained delegation of permissions, such as granting read-only access to data scientists while enabling full table management for data engineers. Credential vending ensures that query engines receive temporary access credentials at runtime, eliminating the need to expose cloud storage credentials directly.

#### **Integration and usage:**

Polaris is designed to be query-engine agnostic. Any engine that supports the Iceberg REST protocol can interact with it. Polaris also supports advanced namespace hierarchies, metadata introspection, and versioned access to Iceberg tables, enabling robust lifecycle management.

#### **Strengths:**

- Centralized, REST-based metadata coordination across engines.
- Built-in support for multi-cloud storage backends.
- Fine-grained access control via RBAC.
- External catalog support for unified metadata visibility.
- Secure credential vending reduces exposure risk.
- Supports different persistence backends (postgres, etc.)

#### **Limitations:**

- Requires infrastructure deployment and operational management.

Polaris is ideal for organizations seeking a centralized metadata plane that supports multiple compute engines and enforces consistent governance. Its standards-compliant interface and extensible design make it a strong candidate for production-grade Iceberg lakehouses that span teams and cloud environments.

### 6.4.5 Project Nessie

Project Nessie is an open source catalog implementation designed to bring Git-like version control semantics to data lakes. It functions as a metadata coordination layer for Apache Iceberg, enabling atomic, isolated, and auditable changes across one or more tables. Built as a REST service with client libraries in multiple languages, Nessie supports a variety of backends and integrates directly with tools like Spark, Flink, and Trino. Nessie also supports the Apache Iceberg REST Catalog interface.

#### Key architectural features:

At its core, Nessie maintains a versioned history of metadata operations. Every change to a table, such as appending files, modifying schemas, or deleting partitions, is recorded as a commit. These commits are grouped on branches, just like in Git, allowing for experimentation, staging, and production changes to coexist without conflict. Tags provide immutable references to specific points in time.

Nessie operates as a coordination layer only; it does not manage table metadata or data files directly. Instead, it delegates metadata storage and table operations to Iceberg, while maintaining a consistent view of their evolution through its own version log. This separation enables massive scalability and performance.

#### Branching and isolation:

One of Nessie's distinguishing features is its support for branching and merging of data changes at the catalog level. Analytics jobs or ingestion pipelines can operate in isolation by writing to a dedicated branch. Once the job completes successfully, all its changes can be merged atomically into the mainline. This model avoids exposing incomplete data and provides powerful rollback and debugging capabilities.

#### Multi-table transactions:

Nessie enables transactions that span multiple tables, a traditionally difficult task in distributed data lake environments. By grouping changes across tables into a single commit or a merge of a dedicated work branch, Nessie ensures consistency and atomicity. This is especially valuable in workflows where schema evolution or ingestion needs to occur across related datasets simultaneously.

#### Security and credential delegation:

When used with Iceberg's REST protocol, Nessie can push configuration and temporary credentials to client engines like Spark or Trino. This approach removes the need for those engines to be configured with direct access to cloud storage credentials, increasing security while simplifying deployment. For example, object store credentials can be securely vended via request signing during query execution.

**Scalability and deployment:**

Nessie is designed for high-throughput environments, supporting thousands of commits per second and large-scale metadata operations. It can be deployed in Docker or container-native platforms, with support for pluggable metadata backends like DynamoDB, PostgreSQL, and Cassandra. Nessie's REST API also enables integration with CI/CD pipelines or custom data management applications.

**Strengths:**

- Git-like version control for data lakes
- Branches and tags for isolated development and reproducibility
- Native support for cross-table transactions and rollback
- REST API and multi-language client support
- Compatible with Iceberg REST and integrates with engines like Spark and Trino

**Limitations:**

- Requires a separate deployment and operational footprint

Nessie is best suited for organizations that value data reproducibility, safe experimentation, and robust governance. Its model bridges the gap between traditional source control and the evolving needs of large-scale data lakes, making it a compelling option for modern Iceberg-based architectures.

**6.4.6 Apache Gravitino**

Apache Gravitino is a federated, geo-distributed metadata lake that provides unified access and governance for a wide range of data and AI assets. As an Apache project, Gravitino offers an extensible platform capable of managing metadata across various storage systems, regions, and data formats. It serves as both a metadata catalog and governance layer, integrating deeply with Apache Iceberg via its REST catalog interface.

**Architecture and metadata model:**

Gravitino introduces a multi-layered architecture for metadata management:

- *Metalake* is the top-level container, often used to scope metadata by organizational group or tenant.
- *Catalogs* connect to specific data sources (e.g., Iceberg, Hive, Kafka, HDFS) and expose metadata through Gravitino.
- *Schemas* and *Tables* represent structured metadata objects for tabular sources, while *Filesets*, *Models*, and *Topics* support unstructured and AI-related metadata.

This flexible object model allows Gravitino to manage metadata for data lakes, message systems, and machine learning artifacts within a single governance domain.

**REST and connector ecosystem:**

Gravitino implements the Apache Iceberg REST catalog interface, enabling compatibility with any Iceberg-aware engine. It also provides connectors for Hive, MySQL, PostgreSQL, and more, which allow it to act as a centralized metadata hub across hybrid environments.

Its REST API, Java SDK, and Python SDK make it accessible from a wide range of platforms, and its Web UI and CLI simplify administration. Gravitino can be run locally, in containers, or deployed across cloud regions with support for geo-replication.

### **Governance and security:**

Gravitino includes built-in capabilities for metadata governance:

- *Access control* supports role-based policies across all cataloged assets.
- *Tag management* enables classification and discovery.
- *Audit logging* and *security controls* integrate with enterprise authentication methods, including OAuth and Kerberos.

These features ensure consistent security and visibility across distributed metadata domains.

### **Use cases and strengths:**

Gravitino is particularly well-suited for organizations with:

- Heterogeneous storage systems require unified metadata access.
- Global operations that benefit from federated catalog deployment.
- Diverse assets that span tabular data, unstructured files, and AI models.

### **Strengths:**

- Unified cataloging of tabular, file-based, and AI model metadata.
- REST-compatible Iceberg catalog integration.
- Web UI, CLI, and SDK support for comprehensive administration.
- Extensible to hybrid cloud and multi-region use cases.

### **Limitations:**

- Still incubating and evolving, some enterprise features may be early-stage.
- Geo-replication and advanced access control features are still maturing.
- Operational complexity may be higher due to the broad scope and flexibility.

Gravitino fills a unique niche in the metadata catalog landscape, offering broad horizontal integration across formats and domains. For Iceberg deployments that coexist with a wide array of data systems, Gravitino provides a centralized plane for metadata access and policy enforcement, making it a valuable option for complex, federated data lakehouses.

## 6.4.7 Lakekeeper

Lakekeeper is a lightweight, secure, and extensible implementation of the Apache Iceberg REST Catalog specification. Built in Rust, Lakekeeper offers a fast, JVM-free deployment option for Iceberg environments. It supports fine-grained access control, storage credential vending, and integration with modern authentication systems such as OpenID Connect and Kubernetes service accounts. With native support for Trino, Spark, Pylceberg, and StarRocks, Lakekeeper delivers a production-ready, scalable catalog suitable for a wide range of data lakehouse scenarios.

### Core architecture and deployment:

Lakekeeper operates as a standalone binary, avoiding the overhead of JVM or Python environments. It is designed to be deployed in containerized environments using Docker or Helm and is easily integrated into Kubernetes-based infrastructure. At its core, Lakekeeper serves two APIs:

- The *Iceberg REST API* (under `/catalog`) for client query engines.
- The *Management API* (under `/management`) for configuring Lakekeeper entities such as projects, warehouses, roles, and permissions.

Lakekeeper uses PostgreSQL for catalog persistence, and it supports external secret stores (e.g., Vault) and event stores (e.g., Kafka, NATS) to enable secure and auditable operations.

### Multi-tenancy and access control:

Lakekeeper is designed to support multi-tenant architectures through the concepts of Projects and Warehouses. A single server can manage multiple projects, each with multiple warehouses. Each warehouse defines its own object store location and authentication configuration. Lakekeeper enforces that these locations do not overlap, preventing data leakage via credential misuse.

Fine-grained access control is provided through OpenFGA integration or custom authorizers. Users and roles are managed externally, and authentication is delegated to OpenID-compliant identity providers. Permissions are enforced on catalogs, namespaces, tables, and views. Lakekeeper also supports recursive and force deletion APIs to manage protected hierarchies safely and flexibly.

### Security and governance:

Lakekeeper includes several advanced governance features:

- *Soft deletion*—Tables can be marked for deletion but retained for a configurable expiration period, enabling recovery and safe rollbacks.
- *Change approval hooks*—Integrations can intercept and deny changes that violate data contracts or quality thresholds.
- *Remote signing and credential vending*—Clients receive temporary credentials at query time, reducing exposure of long-lived storage credentials.

These mechanisms improve operational safety and enforce best practices for secure, governed data lake usage.

**Strengths:**

- Lightweight, Rust-based deployment with no JVM requirement
- Full implementation of the Iceberg REST Catalog interface
- Integrated storage credential vending and soft deletion mechanisms
- Strong Kubernetes integration with OpenID-based authentication
- Highly customizable via pluggable interfaces and trait-based extensions

**Limitations:**

- Currently supports only PostgreSQL as a backend

Lakekeeper is ideal for teams seeking a secure, scalable, and cloud-native catalog service that is easy to deploy and tightly integrated with modern infrastructure. It is especially well-suited to organizations looking to embed strong governance practices and integrate with CI/CD workflows and custom approval systems.

**6.4.8 AWS Glue Data Catalog**

The AWS Glue Data Catalog is Amazon's managed metadata service and a popular choice for Iceberg catalog implementations in cloud-native lakehouse architectures. Each AWS account has one Glue Data Catalog per region, which serves as a centralized, scalable repository for metadata across databases and tables. With integrated support for Amazon services like Athena, Redshift Spectrum, SageMaker Unified Studio, S3 Tables, and EMR, Glue provides a seamless environment for discovering, transforming, and governing data.

**Glue as an Iceberg catalog:**

AWS Glue Data Catalog can serve as a metadata catalog for Apache Iceberg, allowing you to register and manage Iceberg tables using the Glue Catalog. This integration is made possible by configuring compute engines to connect to Glue's REST endpoint, enabling them to read from and write to Iceberg tables stored in AWS-managed metadata.

Separately, AWS Glue Spark engine versions 3.0 and 4.0 include native support for transactional table formats like Apache Iceberg. This means the engine is pre-configured with the necessary libraries to recognize and operate on Iceberg tables without requiring additional setup. For use cases that require specific Iceberg versions or advanced functionality, users can override the default behavior by supplying custom JARs or connectors as part of their job configuration.

To configure Iceberg with AWS Glue Engine, you typically:

- Set `--datalake-formats iceberg` to enable native support
- Specify the `org.apache.iceberg.aws.glue.GlueCatalog` implementation
- Use `org.apache.iceberg.aws.s3.S3FileIO` for optimized I/O on S3

These configurations can be applied using visual tools, notebooks, or script-based Glue jobs.

**Governance and security:**

AWS Glue Data Catalog integrates tightly with AWS Lake Formation and Identity and Access Management (IAM) to offer fine-grained access controls. With this model, enterprises can securely publish and share data across departments while restricting sensitive information. Combined with AWS CloudTrail, the Glue Data Catalog supports strong auditability for metadata and schema changes.

AWS Glue Data Catalog serves as the metadata backbone for several AWS analytics and machine learning services. It acts as the underlying catalog for the SageMaker Lakehouse service, enabling users in SageMaker Studio to create and interact with Iceberg tables. These tables can also be queried seamlessly through services like Amazon Athena, Amazon Redshift, and SageMaker itself, unifying access across analytics and ML workloads. Additionally, the S3 Table feature integrates directly with the Glue Catalog, allowing users to define Iceberg-compatible tables on Amazon S3 using familiar DDL commands, without needing to manage external metadata services.

**Strengths:**

- Fully managed metadata store with deep AWS ecosystem integration
- Native support for Iceberg in AWS Glue Data Catalog Spark environments
- Strong security, audit, and access control features via IAM and Lake Formation

**Limitations:**

- Regional scope limits metadata sharing across regions unless duplicated
- Glue is tightly coupled to the AWS ecosystem
- Less control over internals compared to self-managed catalogs

AWS Glue Data Catalog is well-suited for organizations already committed to AWS infrastructure. It simplifies setup and operations, especially for teams that want to leverage serverless data integration and access governance without managing infrastructure. However, teams requiring multi-cloud portability or complete control over catalog internals might prefer other options.

## 6.4.9 Dremio Catalog

Dremio Catalog is a managed Iceberg catalog embedded within Dremio's Enterprise platform, built atop Apache Polaris, an open-source implementation of the Iceberg REST Catalog specification. Designed to simplify the deployment of an enterprise-grade lakehouse, the Dremio Catalog removes the friction often associated with managing and operating standalone catalogs.

**Polaris foundation:**

Apache Polaris, introduced in 2024, is rapidly gaining adoption as a vendor-neutral, REST-compatible catalog for Apache Iceberg. As a core component of Dremio Catalog, Polaris enables Dremio to provide interoperable, secure access to Iceberg tables across query engines like Spark, Flink, and Trino. This open foundation allows users to read and write Iceberg tables using any compatible engine while managing the metadata centrally within Dremio.

**Enterprise-grade capabilities:**

Dremio extends Polaris with enterprise-focused features designed for production workloads:

- *Fine-grained access control*, Support for role-based access control (RBAC), row-level filters, and column masking ensure secure data access.
- *Metadata lineage and discovery*, Integrated tools help analysts understand data dependencies and trace transformations through built-in lineage graphs.
- *Natural language discovery*, Business users can explore and understand datasets using metadata annotations and descriptive labels.

### **Automation and integration:**

Dremio automates critical Iceberg maintenance operations such as compaction and vacuum. This reduces operational burden and ensures performance without requiring external orchestration. The catalog supports clustering keys for better data layout optimization and is fully integrated into Dremio's UI and query interface, making it easy to register, manage, and query Iceberg tables.

### **Operational simplicity:**

The Dremio Catalog is embedded in all Dremio Enterprise deployments. No separate infrastructure is needed for catalog operations. Each catalog subfolder can be mapped to different storage destinations, and Dremio's native access controls extend seamlessly to all managed tables and views.

### **Strengths:**

- Built-in and fully managed within Dremio deployments
- Powered by Apache Polaris, offering broad interoperability
- Enterprise-grade security and metadata governance
- Automated Iceberg maintenance and performance tuning

### **Limitations:**

- Requires adoption of the Dremio platform (can still use other engines with the catalog)
- Limited catalog flexibility outside the Dremio ecosystem

Dremio Catalog is well-suited for organizations standardizing on the Dremio platform and looking to minimize catalog management overhead. It provides a production-ready, interoperable catalog experience that simplifies Iceberg adoption and accelerates data product delivery.



### 6.4.10 Snowflake Open Catalog

Snowflake Open Catalog is a fully managed implementation of the Apache Iceberg REST Catalog specification, hosted on Snowflake's infrastructure and powered by Apache Polaris. While it is maintained by Snowflake, you are not limited to using Snowflake's compute engine. External query engines—such as Apache Spark, Flink, Trino, or Dremio—can connect to the Open Catalog via its REST interface to read from or write to Iceberg tables. This design enables secure, governed access to Iceberg tables stored in Amazon S3, Azure, or Google Cloud, making it a flexible option for hybrid and multi-engine environments.

#### REST-based and interoperable:

Snowflake Open Catalog supports Iceberg's REST protocol, allowing any compatible engine, including Apache Spark, Flink, and Trino, to access and manage tables. This aligns with the lakehouse principle of separating storage and compute while promoting multi-engine access to consistent metadata.

#### Catalog types and storage configuration:

Snowflake Open Catalog supports both internal and external catalogs:

- *Internal catalogs* are fully managed by Snowflake. Query engines read and write to these catalogs using credential-vended access.
- *External catalogs* are synchronized from other catalog implementations (e.g., Snowflake, Dremio Arctic). These are read-only from the perspective of non-Snowflake engines.

Each catalog is configured with cloud storage credentials and a base directory hierarchy. This directory structure enforces strict isolation between tables and supports consistent namespace mapping. Snowflake automatically prohibits overlapping storage paths in internal catalogs to prevent unauthorized access.

#### Security and governance:

Snowflake Open Catalog adopts a centralized role-based access control (RBAC) model:

- *Principal roles* are assigned to service principals for engines such as Spark or Flink.
- *Catalog roles* define permissions on catalog resources and are granted to principal roles.

This separation provides granular, centralized control over access to metadata and data locations. Credential vending further simplifies secure access by dynamically provisioning temporary credentials for query engines.

#### Service integration:

Snowflake Open Catalog is designed to serve data from Iceberg tables across a range of compute environments. Users configure service connections for each engine (e.g., Flink ingestion, Spark ETL, Trino BI), and assign them service principals with scoped privileges. This structure enables secure multi-engine interaction with a shared catalog.

**Strengths:**

- Fully managed service hosted by Snowflake, minimizing operational overhead
- REST-compatible for cross-engine interoperability
- Strong security posture with IAM integration and RBAC enforcement
- Multi-cloud storage support with dynamic credential provisioning

**Limitations:**

- Currently available only in commercial regions, not government clouds
- Limited write access for external catalogs synced from other providers
- Catalog metadata API requests are billed

Snowflake Open Catalog is a strong option for enterprises invested in Snowflake that want to unify metadata across tools while maintaining control over data stored in their own cloud infrastructure. Its REST-based openness, Polaris foundation, and built-in security controls make it well-suited for scalable, governed Iceberg lakehouse deployments.

### 6.4.11 Databricks Unity Catalog

Databricks Unity Catalog is Databricks' unified governance solution for managing data, metadata, and access policies across its platform. Initially designed to centralize control over Delta Lake tables, Unity Catalog now supports reading from and writing to Apache Iceberg tables, offering a new degree of interoperability for organizations using Databricks while standardizing on Iceberg.

With Unity Catalog, Databricks users benefit from a uniform interface for managing structured and unstructured assets across all workspaces. It centralizes access controls, auditing, lineage, and data discovery through a shared catalog layer, enabling a consistent governance experience for data engineers, analysts, and machine learning teams. This includes fine-grained access control based on identities and roles, enforced across SQL, Python, and other supported languages.

The recent addition of native Iceberg support extends Unity Catalog's capabilities beyond Delta Lake, allowing organizations to register external Iceberg tables stored in cloud object storage (e.g., S3 or ADLS) or to create new Iceberg tables directly within Databricks notebooks and jobs. This enables teams to operate on a shared Iceberg lakehouse from within the Databricks environment, while maintaining compatibility with external tools that also support the Iceberg format.

Although Unity Catalog's Iceberg capabilities are relatively new, they are rapidly evolving. This positions Unity Catalog as a viable option for organizations looking to standardize governance and security across multiple data formats, including Delta and Iceberg, without fragmenting their platform operations.

When considering Unity Catalog for Iceberg, organizations should assess:

- Does your existing platform strategy already center on Databricks?

- Do you require centralized governance and lineage tracking across both Delta and Iceberg?
- Are you working within a single-cloud environment where Unity Catalog's hosted nature aligns with your deployment model?

Unity Catalog's support for Iceberg reflects a broader industry trend toward open table formats and interoperability. For Databricks users, it offers a streamlined path to adopt Iceberg while maintaining unified access control, cataloging, and auditing capabilities.

## 6.5 Choosing the right catalog: Evaluating options through scenarios

With a wide range of catalog implementations available, ranging from vendor-managed services to open source projects to lightweight embedded libraries, selecting the right catalog for your Iceberg deployment can be a nuanced decision. While feature checklists are useful, they rarely capture the full scope of operational, architectural, and strategic considerations that come into play when designing a lakehouse.

Rather than prescribing a single "best" catalog, this section walks through a set of illustrative scenarios. These examples are designed to help you frame the kinds of questions and trade-offs that arise in real-world catalog selection. They highlight how factors such as governance requirements, infrastructure preferences, team skills, cloud environment, and cost sensitivity may influence your choice.

Importantly, these scenarios are not recommendations or endorsements of any specific catalog solution. The catalog landscape continues to evolve rapidly, and the best option for your needs today may differ tomorrow. Use the examples that follow as models for how to think about your own requirements and how to evaluate the strengths and limitations of each catalog offering.

As you review these scenarios, consider the following:

- *Interoperability*—Does the catalog support the Iceberg REST spec? Can it be used across engines and languages?
- *Governance and Access Control*—What tools does it provide for securing and auditing metadata?
- *Operational Overhead*—Will your team need to self-manage the catalog, or can it be offloaded to a vendor?
- *Performance and Scale*—Can it support your concurrency and latency needs at scale?
- *Community and Support*—How active is the ecosystem, and what level of support is available?

The following subsections provide concrete examples of how different catalog choices align with different architectural contexts and operational goals.

### 6.5.1 Scenario: A mid-sized data team migrating from Hive

A mid-sized analytics team at a retail company is migrating from a legacy Hadoop data warehouse that uses Hive Metastore. The team is experienced with Spark and comfortable managing infrastructure. They currently run an on-prem Hadoop cluster and plan to transition incrementally to a cloud-based lakehouse over the next 12 to 18 months.

#### Requirements:

- Seamless compatibility with the existing Hive Metastore
- Support for Iceberg tables across Spark and Flink
- Self-managed option during initial migration
- Minimized disruption to current data pipelines
- Path to evolve toward cloud-native and governed architecture

#### Evaluation:

Initially, the team considers both the Hive Catalog and the Hadoop Catalog. The Hive Catalog offers the benefit of integrating directly with their current Hive Metastore, enabling them to begin using Iceberg without modifying their metadata layer. This reduces friction and risk during the transition.

Alternatively, the Hadoop Catalog offers a simple, self-contained approach that stores metadata directly in the file system. It avoids Hive Metastore dependencies but would require more changes to their metadata management processes.

#### Outcome:

The team chooses to start with the Hive Catalog. This allows them to use their existing metadata infrastructure and immediately integrate Iceberg with minimal disruption. As they progress with their cloud migration, they plan to reevaluate their catalog choice, considering options like Nessie or Polaris for more robust governance, multi-engine support, and REST API interoperability. This phased approach provides them with flexibility while demonstrating how catalog selection can evolve in response to an organization's architecture and governance needs.

### 6.5.2 Scenario: A rapidly scaling cloud-native startup

A fast-growing fintech startup is building its data platform from the ground up on a cloud-native stack. The company is all-in on managed services and prioritizes speed, scalability, and governance. They use AWS extensively, rely on Spark and Flink for processing, and plan to incorporate AI workloads using SageMaker and large language models.

#### Requirements:

- Fully managed, cloud-native metadata solution
- Seamless integration with AWS services
- Support for multi-engine read and write access
- Fine-grained access control and auditability
- REST compatibility for future interoperability

**Evaluation:**

The team considers AWS Glue Data Catalog and Polaris-based catalogs like Dremio Catalog. AWS Glue Data Catalog provides deep integration with AWS IAM, Lake Formation, and services like Athena, EMR, and Redshift. Its serverless model and support for Apache Iceberg make it a natural fit for a startup that wants minimal operational burden.

They also explore Polaris-based catalogs for their adherence to the Iceberg REST specification and alignment with open standards. While AWS Glue Data Catalog offers strong fine-grained access control through Lake Formation and supports multi-engine interoperability, it is tied to the AWS ecosystem. Polaris, by contrast, provides a cloud-agnostic and fully managed REST-compliant catalog, making it attractive for organizations that need consistent metadata management across multiple cloud providers or anticipate expanding beyond AWS in the future.

**Outcome:**

The team selects AWS Glue Data Catalog for its native integration with the AWS ecosystem, built-in support for Iceberg, and fully managed, serverless operation. This enables them to focus on delivering data products without managing catalog infrastructure. However, they also recognize the importance of cloud portability and anticipate the need to support customers across multiple cloud providers. With that in mind, they plan to revisit cloud-agnostic options like Polaris or other REST-compliant catalogs as their platform scales and diversifies.

**6.5.3 Scenario: A multinational enterprise with strict data governance**

A multinational enterprise in the healthcare sector operates under strict data residency, privacy, and compliance regulations. Its infrastructure spans multiple regions and cloud providers, with some workloads on-premises and others in the cloud. It uses a diverse mix of engines, including Spark, Trino, and Flink, and needs consistent metadata governance across all environments.

**Requirements:**

- Multi-region, geo-distributed metadata management
- Strong access controls and auditing
- Support for multiple engines and data types
- Open APIs and extensibility for custom integrations
- Federated access to diverse metadata sources

**Evaluation:**

The organization evaluates Apache Gravitino, which provides a geo-distributed, federated metadata lake that supports metadata for relational, file-based, messaging, and AI assets. Its unified governance layer and REST catalog implementation make it well-suited for complex enterprise environments. They also consider Lakekeeper and Nessie, but Gravitino's broad metadata model and connector ecosystem make it a better fit for federated governance.

**Outcome:**

They choose Apache Gravitino for its federated capabilities, unified access model, and extensible governance framework. Gravitino enables them to consolidate metadata across regions and enforce consistent policies, while supporting Iceberg tables through the REST spec. Its support for various metadata domains (including models and streams) helps unify the management of data and AI assets.

#### **6.5.4 Scenario: SaaS startup prioritizing operational simplicity**

A SaaS startup with a small data team is building a greenfield analytics platform on AWS. They want to use Apache Iceberg for its performance and data management features, but have limited resources for managing infrastructure. Their workloads primarily involve batch ingestion with Spark and SQL-based analytics through Athena.

##### **Requirements:**

- Fully managed catalog service
- Integration with AWS-native tools (Athena, Redshift, Glue ETL)
- Minimal setup and operational overhead
- Native support for Apache Iceberg
- Secure, scalable access control

##### **Evaluation:**

They evaluate AWS Glue Data Catalog as the default choice since it's deeply integrated into the AWS ecosystem and now supports Apache Iceberg via its native Data Catalog and Glue ETL jobs. It simplifies metadata management, supports serverless ETL, and integrates seamlessly with Athena and Redshift Spectrum. They briefly consider Lakekeeper and JDBC Catalog, but these would require more infrastructure management.

##### **Outcome:**

They select AWS Glue Data Catalog to leverage its managed catalog and ETL capabilities, enabling them to focus on building data products rather than managing metadata infrastructure. The team appreciates the centralized governance, crawler-based schema discovery, and seamless query access via Athena.

#### **6.5.5 Scenario: A large enterprise with multi-cloud and federated governance needs**

A global financial services firm operates in multiple regions and under diverse regulatory jurisdictions. They are adopting a lakehouse strategy based on Apache Iceberg to modernize their analytics platform while ensuring tight governance and compliance. Their workloads span Spark, Flink, and Trino, and they require a catalog solution that supports distributed deployments, unified governance, and access by various engines across clouds.

##### **Requirements:**

- Federated metadata management across regions and clouds
- Support for multiple query engines and metadata types

- Centralized security and governance
- Interoperability through Iceberg REST support
- Open source and extensibility

#### **Evaluation:**

They assess Apache Gravitino for its federated architecture and support for multi-regional metadata management. Gravitino's REST-compatible catalog and rich governance features, including access control, tags, and unified interfaces, allow them to manage data and AI assets uniformly. Its support for various connectors (Hive, Hudi, Iceberg, Kafka, and more) and query engines aligns with their heterogeneous environment. They also evaluate Nessie and Polaris, but Gravitino's federated metadata model and extensibility make it more suitable for complex regulatory and organizational requirements.

#### **Outcome:**

They adopt Apache Gravitino to establish a federated metadata lake that supports consistent governance and discovery across global teams. Gravitino's REST API allows integration with all their engines, and its support for model and messaging metadata positions them well for future AI initiatives.

### **6.5.6 Scenario: Financial firm requiring daily environment cloning for stress testing**

A financial institution runs daily regulatory stress tests that simulate adverse market conditions and analyze the firm's resilience. To ensure accuracy and isolation, these simulations must operate on an exact snapshot of production data without interfering with live operations. The team needs a way to rapidly create and manage test environments without physically copying large datasets, which would be time-consuming and expensive.

#### **Requirements:**

- Ability to clone the entire production environment without data duplication
- Time-travel and versioning capabilities for consistent reproducibility
- Lightweight, branch-based metadata isolation
- Compatibility with their existing Spark-based pipeline
- Support for continuous integration with version-controlled data

#### **Evaluation:**

They consider several catalog options. While AWS Glue Data Catalog and Polaris support REST integration, neither offers first-class branching or tagging semantics that meet their zero-copy cloning needs. Apache Gravitino and Lakekeeper provide metadata federation and REST compatibility, but don't focus on branching semantics.

Nessie stands out for its Git-like branching and tagging capabilities, which allow the team to create isolated metadata views without duplicating data. Using Nessie, they can branch from production data at the start of each test cycle, run simulations on the branch, and discard or merge results as needed. This allows for precise, cost-effective stress testing and experimentation.

**Outcome:**

The firm adopts Nessie to manage its Iceberg catalog. They automate the creation of daily branches for stress testing and integrate branch lifecycles into their CI/CD workflows. This allows them to maintain consistent test environments, comply with auditability requirements, and iterate on testing procedures without impacting production data.

---

**NOTE:** This scenario highlights how data versioning and branch-based isolation can be critical for applications where reproducibility, isolation, and speed are paramount. Nessie provides an elegant solution for these needs.

---

### 6.5.7 Scenario: Phased Iceberg migration with query federation across legacy systems

A large enterprise is undertaking a strategic migration from a mix of legacy data warehouses and Hadoop-based data lakes to an Apache Iceberg-based lakehouse. Their data ecosystem includes Hive, Teradata, and Oracle systems, and the migration is planned in phases to minimize business disruption. During this transition, the company must maintain analytics continuity by enabling users to query data across both legacy and Iceberg systems.

**Requirements:**

- Query federation across multiple systems, including Hive, Oracle, and Iceberg
- Central catalog that simplifies metadata governance during the transition
- Ability to manage and surface data products built on Iceberg
- Seamless access for data consumers during the migration phases
- Minimal operational overhead with built-in governance features

**Evaluation:**

They examine several catalog options. Hadoop and JDBC catalogs are insufficient due to a lack of query federation and governance tooling. Polaris and Gravitino offer strong REST compatibility but require additional integration work to meet query federation needs. AWS Glue Data Catalog offers ETL integration but lacks built-in federation across heterogeneous sources.

The Dremio Catalog, powered by Apache Polaris, emerges as a practical fit. It supports REST-based interoperability with Iceberg and also integrates tightly with Dremio's query engine, which excels at federating queries across disparate sources. This enables analysts to create unified views that span both legacy systems and Iceberg tables, all without needing to move or duplicate data during the early stages of migration.

**Outcome:**



The enterprise adopts the Dremio Catalog as the foundation of its lakehouse migration. Dremio allows them to incrementally onboard Iceberg tables while still providing analysts access to legacy systems through a single SQL interface. With built-in governance features such as row-level security, data lineage, and clustering key support, the Dremio Catalog ensures compliance and maintainability throughout the transition.

### 6.5.8 Scenario: Lightweight lakehouse adoption with Hadoop catalog and Python

A growing startup has been operating primarily with application-level databases like PostgreSQL and SQLite. As their datasets begin to exceed what these systems can handle efficiently, particularly for analytics use cases, they explore options for scaling their data infrastructure. The team debates adopting a traditional cloud data warehouse but is wary of the vendor lock-in, rising costs, and rigid tooling.

#### Requirements:

- Low-friction, low-cost entry into scalable analytics
- Full control over metadata and storage
- Ability to operate directly from developer machines
- Prefer open standards and tooling that align with their engineering culture
- Strong Python support, as most of their data engineering is Python-based

#### Evaluation:

They explore cloud data warehouses, but the pricing models and limited transparency of storage and compute separation do not align with their goals. While enterprise-grade catalogs like Polaris or Glue offer rich features, they are more than what the team needs initially. REST catalogs would also require more infrastructure to deploy or configure.

Instead, the team discovers that the Hadoop Catalog, built into Apache Iceberg, allows direct access to Iceberg tables using just the file system paths. Combined with Pylceberg, they can begin creating and managing Iceberg tables stored on Amazon S3 without provisioning additional services.

They use simple Python scripts on their local laptops to run data transformations and metadata operations, storing results in Iceberg format directly in S3. This setup gives them scalable, open table format capabilities with minimal operational complexity.

#### Outcome:

The startup adopts a lakehouse architecture using Hadoop Catalog and Pylceberg as its initial catalog and access method. They store data in Iceberg format in their existing cloud storage and query it using Python-based tools and libraries. This decision allows them to scale incrementally, and when needed, they can later migrate to a REST-based or enterprise-grade catalog without changing their table layout or rewriting pipelines.

## 6.6 Summary

- The catalog layer serves as the metadata backbone of an Apache Iceberg lakehouse, enabling reliable data discovery, governance, and interoperability.

- Built-in catalogs like Hadoop and JDBC offer simplicity but limited flexibility for scaling or integration.
- Open-source and self-managed catalogs such as Polaris, Gravitino, Nessie, and Lakekeeper provide customization and advanced features for complex deployments.
- Vendor-managed catalogs like AWS Glue Data Catalog, Dremio Catalog, and Snowflake Open Catalog deliver turnkey infrastructure with integrated security and enterprise support.
- The Apache Iceberg REST Catalog specification enables standardized communication between compute engines and catalogs, improving multi-engine compatibility.
- Catalog selection should align with your platform's operational model, compliance requirements, cloud strategy, and future growth plans.
- Cloud-agnostic catalogs are increasingly important for organizations planning multi-cloud or hybrid deployments.
- Catalog technologies and integrations are evolving rapidly, so revisiting decisions as your architecture matures is essential.

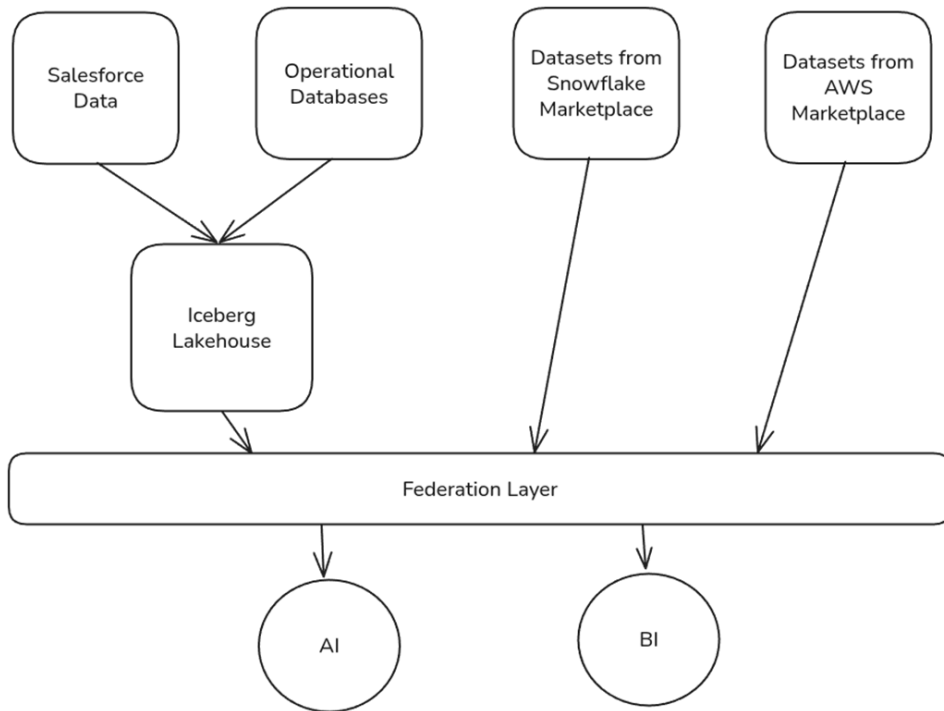
## 7 *Designing the federation layer*

### **This chapter covers**

- Evaluating requirements for data federation
- Designing the federation layer components
- Comparing Dremio and Trino for federated querying
- Self-managed and cloud-managed federation options
- Selecting a federation platform based on use cases

As your Apache Iceberg lakehouse takes shape, it is important to recognize that not all data will reside within Iceberg tables. Despite best efforts to centralize and standardize, some datasets will remain scattered, locked in third-party systems, legacy databases, and SaaS applications, or simply not worth the effort of extracting, transforming, and loading into your lakehouse. These realities make it essential to extend your architecture with a federation layer.

The federation layer acts as both a bridge and a harmonizer. It enables your analytics platform to access data across multiple systems without physically consolidating it. At the same time, it possibly introduces a semantic layer that standardizes business logic, ensuring consistency in metrics and datasets regardless of their origin. Whether your analysts query data through notebooks, BI dashboards, or custom applications, the federation layer that possibly provides a unified and governed interface to the underlying data ecosystem, as illustrated in figure 7.1.



**Figure 7.1** When you have third-party data, a federation layer can help you make use of those datasets without having to replicate them into your lakehouse so they can play the role of Lakehouse enrichment by sending your data to your favorite AI or BI tool.

We will explore the components that make up the federation layer, the key requirements for building it effectively, and the leading platforms that support it. We will compare Dremio and Trino, two of the most prominent tools in this space, discuss their deployment options, and walk through scenarios that illustrate how to choose the right platform for your needs. By the end, you will understand how to integrate a federation layer that complements your Iceberg lakehouse and empowers your organization to work with data wherever it lives.

## 7.1 What data federation is and why it matters

In a perfect world, all your organizational data would reside in Iceberg tables, centralized, optimized, and uniformly governed. In reality, the modern data landscape is more fragmented. Many valuable datasets live outside the lakehouse in systems that are too costly, risky, or complex to ingest. These might include SaaS platforms, legacy databases, real-time services, or externally managed storage. Yet business users still need to access and analyze this information to generate meaningful insights.

In most architectures, the Iceberg lakehouse serves as the central hub where the majority of queries, often over 90%, are executed. This core provides a unified, high-performance foundation for governed analytics. Federation extends this foundation by connecting the lakehouse to external systems, enabling users to enrich lakehouse queries with just-in-time data from remote sources. These federated sources may account for a smaller percentage of queries, but they fill critical gaps without duplicating or relocating data.

This hub-and-spoke approach keeps the lakehouse at the center of your data architecture while enabling agility at the edges. Federation allows your analytics platform to query external systems as if they were part of the same environment. By joining Iceberg tables with remote data, applying consistent business logic through a semantic layer, and enabling access through familiar tools, federation ensures continuity without compromising governance or performance.

This section explores what federation means in the context of an Apache Iceberg lakehouse, why it has become increasingly important, and how it supports goals like agility, accessibility, and data unification. With a clear understanding of its purpose, you'll be better equipped to design a lakehouse that meets both technical realities and user expectations.

### **7.1.1 Common use cases and challenges driving federation needs**

The need for data federation often arises from the reality of fragmented data ecosystems. As organizations scale, they accumulate systems, each with its storage model, performance profile, and operational constraints. Despite efforts to centralize data into a lakehouse, certain use cases continue to benefit from federation due to cost, complexity, or governance concerns.

## **COMBINING OPERATIONAL AND ANALYTICAL DATA**

Many analytics workflows require joining historical data in Iceberg with live operational data from transactional databases. For example, a fraud detection team may need to blend user behavior logs stored in Iceberg with real-time payment activity from an OLTP system. Rather than replicate the operational data into Iceberg, federation allows secure, real-time access to it.

## **INTEGRATING THIRD-PARTY AND SAAS PLATFORMS**

Data from marketing automation, CRM, finance, and other SaaS platforms often resides in vendor-controlled environments. While ETL can replicate some of this data into the lakehouse, not all of it is needed persistently. Federation allows users to query this external data directly, making it available for analysis without full pipeline investments.

## **SUPPORTING EXPLORATORY ANALYSIS AND PROTOTYPING**

Analysts and data scientists often need to explore new datasets quickly before a formal ETL pipeline is justified. Federation supports this by enabling ad hoc access to source systems. If the data proves valuable, it can later be modeled into Iceberg; if not, the cost of experimentation remains low.

## ENABLING HYBRID AND MULTI-CLOUD STRATEGIES

Organizations with both on-premises systems and cloud platforms often face logistical and compliance barriers to full data consolidation. Federation allows these systems to remain where they are while still participating in enterprise analytics. It also supports gradual migrations, where data moves incrementally to Iceberg over time.

## MINIMIZING DUPLICATION AND STORAGE COSTS

Not all data merits long-term storage in the lakehouse. Some are ephemeral, high-volume, or simply accessed too infrequently to justify replication. Federation makes this data queryable without storing it twice, helping reduce costs and complexity.

However, federation also introduces challenges. Query performance can suffer when accessing systems with varying response times or large data volumes. Governance becomes more difficult when interacting with platforms not designed for centralized oversight. Inconsistencies in schema, data freshness, or access interfaces can introduce friction for end-users. Additionally, federated queries can place unexpected load on source systems, triggering performance degradation or even outages. These systems may not be optimized for analytical query patterns, and scaling them to support federated access can introduce significant cost and architectural complexity.

Despite these challenges, federation has become a strategic necessity in the modern data architecture. It allows organizations to unify access, accelerate experimentation, and serve a broader range of use cases without overextending their ETL pipelines or sacrificing governance.

### 7.1.2 How federation aligns with agility and accessibility

One of the defining goals of lakehouse architecture is to enable fast and reliable access to data for a wide range of users, including data engineers, analysts, scientists, and business stakeholders. Federation plays a critical role in fulfilling this vision by removing barriers to access and reducing the time it takes to act on new data.

## AGILITY THROUGH REDUCED DATA MOVEMENT

Traditional ETL workflows require careful planning, infrastructure provisioning, and transformation logic, all of which introduce delays between data generation and availability. Federation eliminates much of this lag by enabling direct access to source systems. New data sources can be queried immediately, and iterative discovery can begin without waiting for ingestion pipelines to be built. This is especially useful in fast-moving environments where data relevance is time-sensitive.

## ON-DEMAND ACCESS FOR DIVERSE USER PERSONAS

Federation extends self-service capabilities to a broader audience. Data analysts can connect BI tools to federated views and build dashboards without engaging engineering teams. Data scientists can prototype models using live datasets from multiple systems. Product teams can validate assumptions by directly querying external sources. This accessibility shortens feedback loops and fosters a data-driven culture across the organization.

## STANDARDIZATION THROUGH THE SEMANTIC LAYER

While federation makes data more accessible, it also requires mechanisms to maintain consistency. A semantic layer built into or externally paired with the federation platform allows organizations to define common metrics, dimensions, and business logic once and enforce them across all tools and users. This guards against conflicting interpretations and supports trust in data-driven decisions, even when the data comes from disparate sources.

By aligning with the principles of agility and accessibility, federation makes the lakehouse more than just a storage system. It becomes a flexible, user-friendly platform for discovery, innovation, and enterprise-wide collaboration.

## 7.2 Key requirements for federation

Building a successful federation layer begins with a clear understanding of your organization's goals, constraints, and data landscape. In chapter 3, we introduced the platform audit as a foundational step in architecting your Iceberg lakehouse. The audit revealed a wide range of technical and organizational needs, including fragmented data systems and inconsistent metrics, as well as demands for faster access and greater autonomy for business users.

These findings directly inform the design of your federation layer. Federation is not a generic solution; it must be tailored to the specific needs identified in your audit. Whether your users are struggling with duplicative pipelines, inconsistent definitions, or limited visibility into external systems, the federation layer must address these concerns while reinforcing the lakehouse's broader architectural goals.

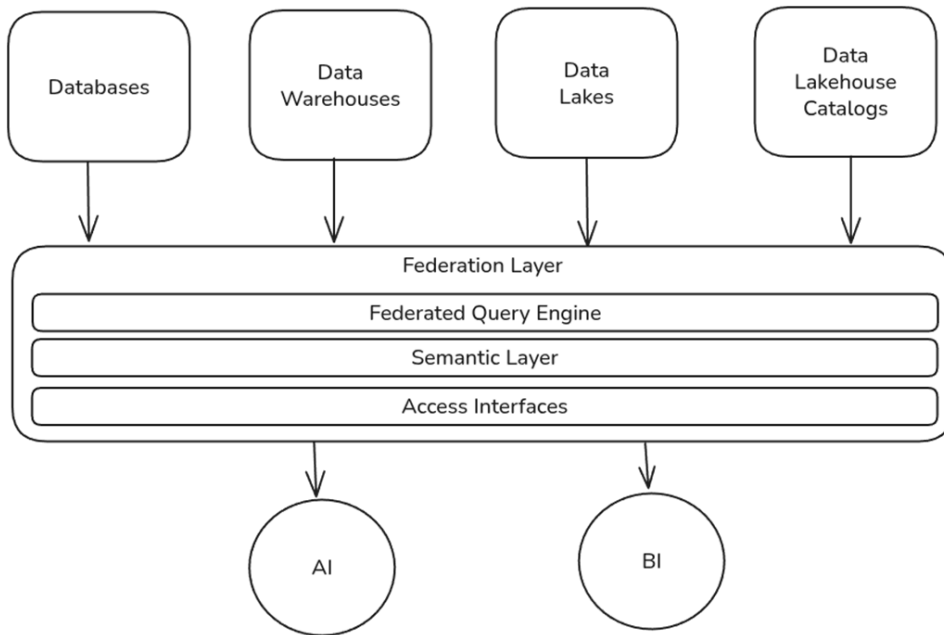
Three core requirements consistently emerge from this analysis:

1. **Support for connecting to external data sources** that are impractical to ingest into the lakehouse, such as those with short-lived value or limited ETL feasibility.
2. **A semantic layer** providing shared definitions, governance, and reusability across teams and tools.
3. **Flexible, tool-friendly interfaces** ensure data can be accessed from wherever users work, without bottlenecks or specialized skills.

Let's explore each of these requirements in depth. Together, they provide a framework for evaluating federation platforms, guiding implementation choices, and ensuring your lakehouse delivers both reach and reliability.

### 7.2.1 Supporting diverse data sources without duplication

A fundamental requirement of any federation layer is the ability to connect to and query data across various systems. During your platform audit, you likely encountered a mix of data sources, cloud-based storage, relational databases, and on-premises systems. Federation helps accommodate this diversity without demanding a uniform storage format or centralized ingestion, as illustrated in figure 7.2.



**Figure 7.2** The federation layer should be able to directly connect to your data ecosystem, including databases, data warehouses, data lakes, and data lakehouse catalogs, to be the primary access point for AI and BI.

This requirement reflects a key architectural principle: not all data needs to be moved. In some cases—particularly when data has a short-lived analytical value—copying it into the lakehouse introduces unnecessary cost, latency, and operational overhead. Federation addresses this by allowing queries to be executed in place, fetching only the data needed at the moment of analysis. This minimizes storage duplication while ensuring that source systems remain the authoritative source of truth.

To support this, your federation engine must:

- Provide a wide array of native connectors or adapters for common data platforms.
- Support secure access mechanisms, including credential delegation and role-based access control.
- Enable pushdown capabilities so filters, projections, and aggregations are executed as close to the source as possible.
- Manage varying latency and concurrency demands across systems to ensure consistent performance and a smooth user experience, even under high query loads.



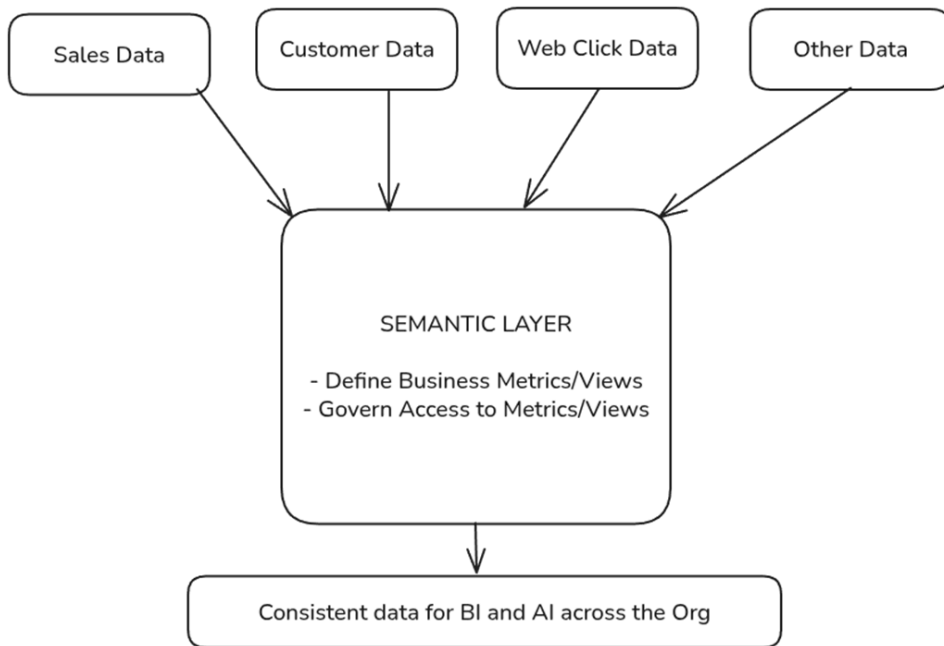
This approach not only expands the reach of your lakehouse but also improves data freshness. Users can interact with the most up-to-date information without waiting for nightly ETL jobs or manual exports. And because the data remains in its original location, compliance and auditability requirements tied to source systems can be more easily preserved.

By supporting diverse sources without duplication, the federation layer fulfills a frequent demand uncovered during stakeholder interviews: “We need to use data where it already lives.” Meeting this requirement helps your lakehouse evolve from a centralized system into a truly federated analytics platform.

### **7.2.2 Ensuring consistent semantics and business logic**

One of the most common pain points uncovered during the platform audit was inconsistency in how data is defined, interpreted, and shared across teams. Analysts in different departments may calculate metrics differently. Data scientists may use raw attributes without applying standard business logic. Even well-meaning teams can arrive at conflicting insights simply because they operate on different assumptions. The federation layer can address this by introducing a semantic layer that unifies business logic across the organization, whether it is coupled with your federated query engine or acts as separate components.

The semantic layer serves as a bridge between raw data and business understanding, as illustrated in figure 7.3. It provides a centralized location to define key metrics, dimensions, and calculations, such as revenue, customer churn, or daily active users, ensuring consistency regardless of the data source or consumption tool. This eliminates duplication of logic, reduces errors, and increases trust in the results.



**Figure 7.3 By taking the data from across the organization and creating universally defined metrics and views, you can enable consistent use of the data for AI and BI.**

In practical terms, this means your federation platform should:

- Allow creation of reusable views or virtual datasets that encapsulate business logic.
- Support versioning and governance of these semantic definitions.
- Integrate with access controls to ensure the right users see the right data.
- Make these standardized datasets discoverable through catalogs or metadata layers.

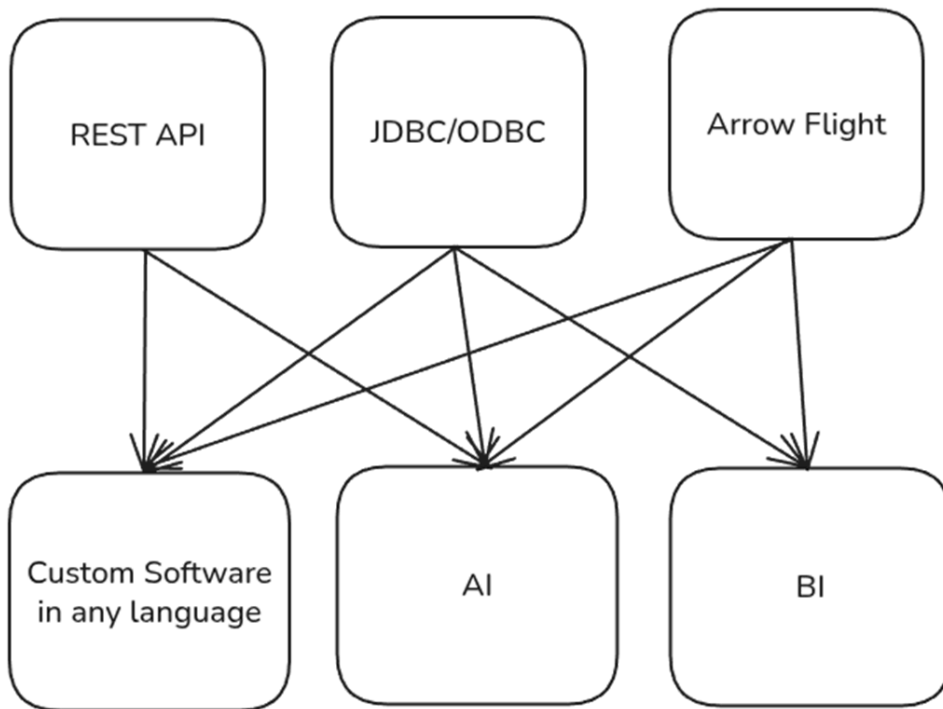
For example, a revenue calculation may involve joining multiple datasets, applying currency conversions, and filtering out returns. Defining this once in the semantic layer ensures that every dashboard, notebook, or report using it reflects the same definition. It also means changes, such as adjusting for a new tax rule, can be implemented centrally and propagated consistently.

This capability is especially important in federated environments, where underlying data may come from multiple systems with differing schemas, freshness, and levels of granularity. The semantic layer provides a stable, governed abstraction that smooths out these differences, giving end users a consistent interface to work with.

Ultimately, consistent semantics transform federation from a convenience into a strategic enabler. It ensures that insights derived from federated queries are as reliable and actionable as those sourced from your curated Iceberg tables, supporting scalable, trustworthy analytics across the entire organization.

### 7.2.3 Providing seamless connectivity for analytics tools

A federation layer's value isn't measured solely by how many data sources it can connect to; it's also defined by how easily users and tools can interact with that data, as illustrated in figure 7.4. This interaction typically occurs through standard interfaces such as JDBC and ODBC for BI tools, REST APIs for application integration, and Arrow Flight for high-performance, language-agnostic data transport. During the audit process, many stakeholders emphasized the need for broad tool compatibility and intuitive access patterns. Data analysts wanted to build dashboards in their preferred BI environments. Data scientists wanted to query live datasets directly from notebooks. Engineers sought to automate data workflows via APIs. Meeting these needs requires a federation layer that supports seamless, standards-based connectivity through these interfaces.



**Figure 7.4** By using industry-standard interfaces like JDBC/ODBC/Arrow Flight and Platform Specific REST APIs, customer software, AI, and BI can communicate with all the data accessible in your federation layer.

At a minimum, this means offering traditional interfaces such as:

- **JDBC and ODBC** for compatibility with mainstream BI platforms like Tableau, Power BI, and Looker.
- **REST APIs** for integrating with custom applications, lightweight tools, and scripting environments.

However, modern data platforms are increasingly adopting next-generation interfaces that provide richer semantics, faster performance, and new architectural possibilities.

## ADBC AND ARROW FLIGHT

The Apache Arrow project was created to address performance bottlenecks in data analytics by introducing a standardized, in-memory columnar format for efficient data exchange between systems and programming languages. By enabling zero-copy reads and minimizing serialization overhead, Arrow has become a foundational layer for high-performance analytics. Building on this foundation, the Arrow ecosystem introduced Arrow Flight, a gRPC-based protocol designed for fast and efficient transport of Arrow-formatted data between processes. Arrow Flight SQL and ADBC (Arrow Database Connectivity) extend this effort by offering modern alternatives to JDBC and ODBC. ADBC provides a cross-language API standard for low-latency columnar access to databases, while Arrow Flight SQL enables high-speed query execution and data transfer. Together, these interfaces deliver substantial performance improvements and reduced memory overhead, particularly for large-scale analytics workloads.

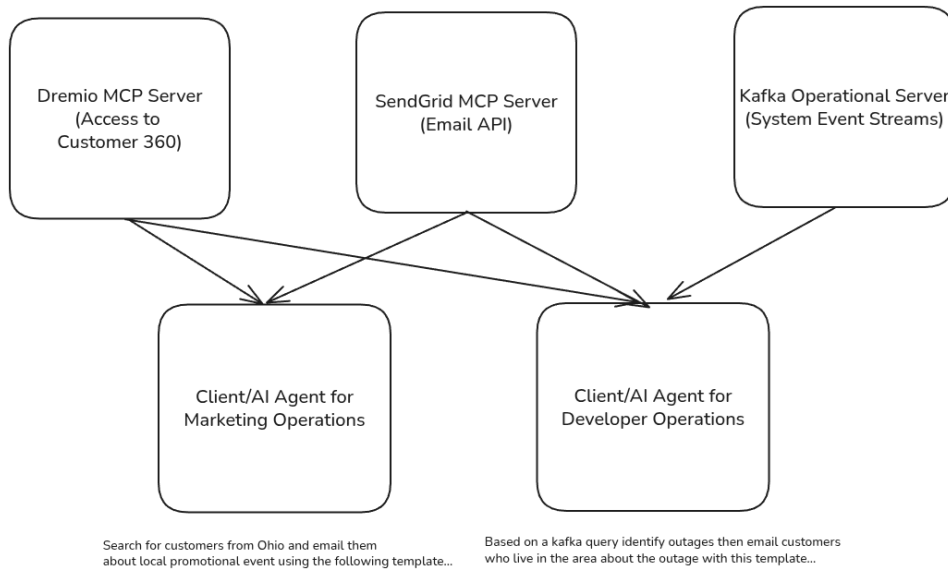
In a federated lakehouse, ADBC and Arrow Flight can be used to:

- Retrieve large datasets with reduced serialization costs.
- Enable partitioned and parallelized query results.
- Improve inter-system query execution across distributed query engines.

Platforms that support ADBC/Flight, such as Dremio and parts of the Arrow ecosystem, allow your federation layer to serve as both a source and a consumer of Arrow-native data, resulting in better performance and more scalable compute.

## MODEL CONTEXT PROTOCOL (MCP)

As AI assistants and autonomous agents become more central to enterprise workflows, a longstanding limitation has come into focus: these systems have historically operated in isolation, disconnected from live enterprise data and governed systems. The Model Context Protocol (MCP) addresses this gap by providing a standardized, open interface for connecting AI models to real-time enterprise data. Developed to solve the NxM integration problem, where each model and tool pairing previously required bespoke integration, MCP acts as a universal layer that allows AI applications to securely access structured data and services without duplicative effort, as illustrated in figure 7.5



**Figure 7.5 With MCP, you can give different Agentic AI clients access to different platforms without rebuilding the integration, making it easier to design AI agents for a specific workflow. In this case, using an MCP layer to discover federated customer 360 data.**

In a federated Iceberg lakehouse, MCP unlocks several capabilities:

- *Standardized data access*—AI agents can query federated Iceberg datasets using a common protocol, avoiding model-specific integration complexity.
- *Two-way interoperability*—Tools like Claude, ChatGPT, or custom LLM-based agents can not only retrieve but also act upon data, initiating workflows or generating insights on demand.
- *Agentic autonomy*—With MCP, AI agents can explore datasets, interpret metadata, and construct contextual queries, enabling use cases such as automated root cause analysis, dynamic reporting, or ML pipeline orchestration.

By exposing your federation layer through a Metadata Control Plane (MCP), you empower AI agents to become informed collaborators. They can interpret business logic, enforce governance policies, and operate within the same access controls as human users, bridging the gap between LLM intelligence and enterprise trust boundaries.

It's important to note that MCP does not replace traditional access interfaces like JDBC, ODBC, or ADBC. Instead, it complements them. While standard interfaces ensure broad compatibility with BI tools, notebooks, and applications, the MCP provides a structured way to expose metadata, semantics, and policies that guide responsible and context-aware data usage. Together, they enable both human and machine actors to work effectively within the same federated environment—each using the interface best suited to its role.

## 7.2.4 Introducing Dremio and Trino

As the demand for federated querying grows across industries, two platforms stand out as lakehouse-focused platforms for federated queries: Dremio and Trino. Both engines provide high-performance, distributed SQL querying across diverse data sources, but with distinct architectures, features, and use cases. Understanding these differences is crucial when choosing the right foundation for your federation layer.

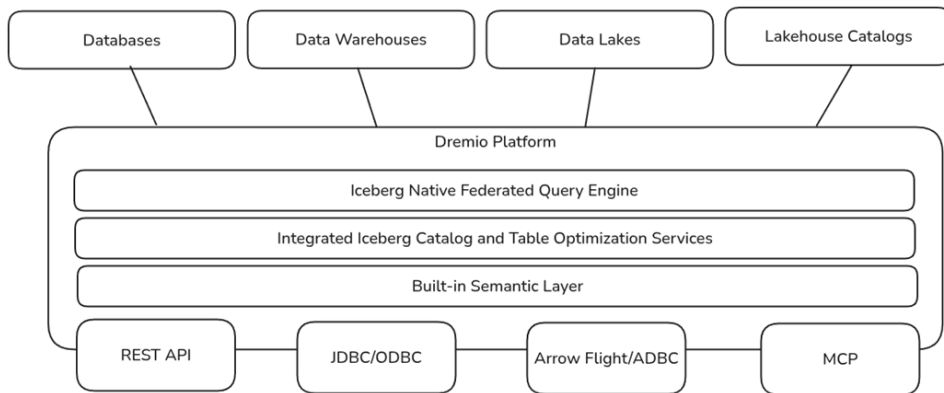
At a high level:

- **Dremio** positions itself as a lakehouse query platform with deep integration into Apache Iceberg and a built-in semantic layer. It emphasizes self-service, strong governance, and performance acceleration through techniques such as reflections (Dremio's unique acceleration technology we'll discuss shortly) and caching. Dremio is designed to make data accessible to both technical and non-technical users while supporting hybrid cloud deployments and interactive analytics.
- **Trino** (a fork of PrestoSQL) is a general-purpose federated SQL engine that excels at querying many sources quickly. It's prized for its extensive connector ecosystem, pluggable architecture, and ability to handle complex, distributed queries across massive datasets. Trino is especially popular in environments where flexibility and scale are more important than out-of-the-box governance or UI features.

While both support federated SQL, their strengths reflect different priorities: Dremio focuses on building a seamless Iceberg lakehouse experience with governance and ease of use, while Trino emphasizes broad connectivity and raw federated query power.

## 7.3 Dremio

Dremio is a federated query engine designed to enable fast, SQL-based access to data across a wide range of sources, including cloud storage, relational databases, NoSQL systems, and modern table formats such as Apache Iceberg. Unlike traditional databases that store and manage their own data, Dremio acts as a query layer over existing data infrastructure, allowing users to query data in place without requiring it to be moved or transformed first, as illustrated in figure 7.6



**Figure 7.6 Dremio can connect to databases, data warehouses, data lakes, and lakehouse catalogs so you can model your data in its semantic layer and manage Iceberg tables in its built-in catalog, then use common interfaces to connect it to your favorite tooling.**

Originally built to address the performance and usability limitations of tools like Apache Drill and Hive, Dremio combines a columnar execution engine with Apache Arrow and several query acceleration features, including reflections, caching, and predictive optimization. These capabilities are especially valuable in a federated architecture, where query performance can suffer due to latency, data fragmentation, and remote system constraints. By accelerating queries and minimizing redundant scans, Dremio helps mitigate the performance overhead of querying multiple systems in real time, making it well-suited for interactive analytics and dashboard workloads in a federated environment. The result is a more responsive, self-service experience for data analysts and engineers working across distributed data sources.

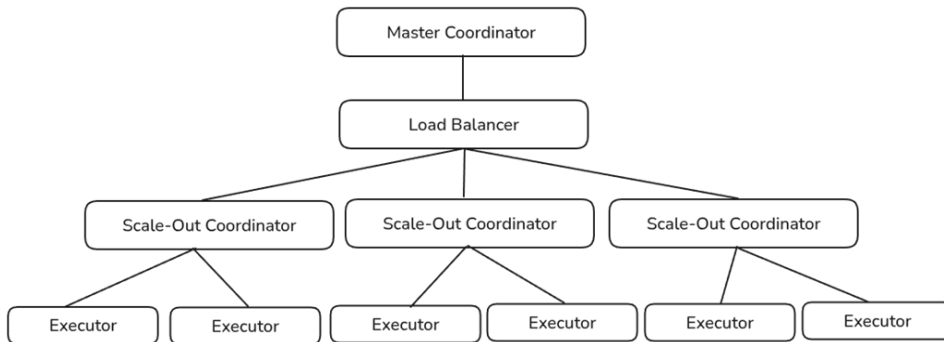
Dremio supports a wide range of communication protocols, including JDBC, ODBC, REST, and Apache Arrow Flight, enabling integration with common BI tools, notebooks, and custom applications. These interfaces are essential in a federated architecture because they allow users to access and analyze data across distributed systems using familiar tools without needing to re-platform or learn new query patterns. While SQL is the primary language interface, Dremio also supports Python through client libraries like pyDremio and dremio-simple-query, making it accessible to data scientists and engineers building custom workflows. Additionally, Dremio supports the Model Context Protocol (MCP), enabling AI agents to connect securely and perform context-aware operations. This combination of human- and machine-friendly interfaces ensures that federated data remains broadly accessible and consistently governed across a variety of use cases.

Let's explore Dremio's architecture, catalog model, performance characteristics, and connectivity features in greater depth, providing a comprehensive view of how the engine operates and its role within a modern data platform.

### 7.3.1 Dremio architecture

Dremio's architecture is purpose-built for high-performance, self-service analytics on data residing across diverse sources and formats. Rather than requiring data movement or transformation before analysis, Dremio provides a distributed, SQL-based query engine that operates directly on cloud storage, relational databases, NoSQL systems, and open table formats such as Apache Iceberg. This is especially beneficial in a federated context, where performance can be impacted by the need to access remote and heterogeneous systems. By combining in-memory processing via Apache Arrow with a modular execution framework, intelligent caching, and query acceleration features like reflections, Dremio minimizes latency and reduces query overhead across federated sources.

At the center of Dremio's design are its core services, including the master coordinator, scale-out coordinators, and execution engines, as illustrated in figure 7.7. The master coordinator handles query parsing, planning, optimization, metadata management, and cluster orchestration. For scalability and high availability, additional coordinators can be deployed behind a load balancer to distribute planning workloads. Dremio's execution layer, composed of engines and their constituent executors, efficiently executes distributed query plans and DML operations. This architecture enables the federation layer to scale with user demand and query complexity, ensuring responsive access even when sourcing data from multiple backends.



**Figure 7.7** Dremio can manage multiple coordinators to parse and plan query execution, then distribute the execution among multiple executors available to those coordinators, enabling scalability of workloads and availability.

To support high concurrency and workload isolation, Dremio allows administrators to configure and scale engines independently. Each engine can be tailored to serve specific workloads, ensuring compute resources are aligned with business priorities and usage patterns.

Performance is further enhanced by the Cloud Columnar Cache, a local, disk-based cache on each executor. This cache stores frequently accessed data in a columnar format optimized for Dremio's execution engine, reducing reliance on remote cloud storage and minimizing latency and egress costs.



Dremio also integrates an Apache Iceberg-based catalog, allowing it to manage Iceberg tables natively. This catalog supports cross-engine interoperability, schema evolution, time travel, and fine-grained access control, forming the foundation of Dremio's lakehouse capabilities.

Supporting services, such as the metastore, secured telemetry, distributed storage, and high availability features in Kubernetes deployments, ensure that Dremio is resilient, observable, and scalable across various environments. Additional capabilities, such as AI-enabled semantic search and comprehensive namespace management, make the platform suitable for complex, multi-tenant data ecosystems.

### 7.3.2 Dremio's connector ecosystem and Iceberg-centric focus

Dremio provides a broad connector ecosystem that allows users to federate queries across an array of data sources, from object storage and databases to catalog services and cloud warehouses. The platform is designed to let organizations query data where it lives, minimizing the need for costly and time-consuming ETL pipelines. Unlike general-purpose federated query engines such as Trino, which take a format-neutral stance and aim to provide broad, but relatively uniform, support for multiple table formats (Iceberg, Delta Lake, Hudi), Dremio has deliberately aligned its architecture around Apache Iceberg.

Dremio's native integration with Iceberg goes beyond compatibility; it includes a built-in Iceberg catalog (powered by Apache Polaris) that supports Iceberg's REST Catalog API, enabling interoperability across tools and engines that support the standard. This enables Dremio to manage Iceberg tables directly, offering advanced features such as automatic data compaction, vacuuming, and metadata pruning, without requiring third-party tools or orchestration. In contrast, engines like Trino, while capable of reading Iceberg tables via plugin connectors, generally rely on external catalog services (e.g., Hive, Nessie, AWS Glue) and do not offer native write support or built-in optimization tooling to the same depth.

In addition to its Iceberg-first approach, Dremio also supports a wide range of other source types, including:

- *Lakehouse catalogs*—Snowflake Open Catalog, Unity Catalog, Iceberg REST Catalog, AWS Glue, Hive, and Nessie.
- *Object storage*—Amazon S3 (including S3 Compatible Storage), Azure Data Lake Storage, Google Cloud Storage, HDFS, and NAS.
- *Relational and NoSQL databases*—PostgreSQL, MySQL, SQL Server, Oracle, MongoDB, Teradata, Snowflake, SAP HANA, IBM Db2, Apache Druid, and more.
- *Data warehouses and search platforms*—Amazon Redshift, BigQuery, Azure Synapse Analytics, Amazon OpenSearch Service, and Elasticsearch.
- *Other Dremio clusters*—Users can also federate across multiple Dremio deployments, enabling cross-cluster queries for complex, distributed environments.

To support advanced use cases, Dremio includes capabilities such as external query pushdown, which allows SQL written in the native dialect of the underlying source to be executed directly at the source when Dremio cannot process it natively. Runtime filtering enhances performance by dynamically applying filters across joined datasets from relational databases, improving efficiency without compromising transparency.

While Dremio's connector catalog is extensive, its strategy emphasizes deep optimization over breadth. The philosophy is to offer high-quality, tightly integrated support for the most strategic systems, especially Apache Iceberg, while allowing extensibility through REST-compatible catalogs and external query features. For gaps in connector coverage, Dremio users have the option to either create custom connectors or prioritize ingestion into Iceberg tables managed by Dremio's built-in catalog.

### **7.3.3 Dremio's performance enhancements**

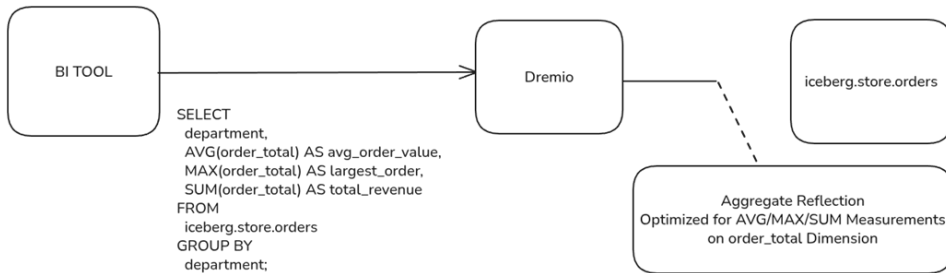
Dremio is engineered to deliver high-performance analytics across cloud and on-prem data, whether you're querying raw files, federated databases, or Iceberg tables. A major differentiator for Dremio is its layered performance enhancement architecture, which reduces latency, minimizes compute costs, and accelerates user experience. These optimizations are especially impactful in a federated environment, where performance can otherwise suffer due to network hops, lack of indexing, or remote storage.

## **APACHE ARROW AND COLUMNAR EXECUTION**

At the core of Dremio's performance model is its use of Apache Arrow, a columnar in-memory format designed for efficient analytical processing. Dremio's query engine operates entirely on Arrow, allowing vectorized execution and reduced memory overhead compared to row-based processing. This format also enables zero-copy data exchange between systems that support Arrow, such as BI tools and data science notebooks connected via Arrow Flight.

## **DATA REFLECTIONS: MATERIALIZED ACCELERATION**

One of Dremio's most powerful performance tools is Data Reflections: pre-computed data structures based on views or tables that act like intelligent caches. Unlike static materialized views, reflections are transparent to users and do not require query rewrites or maintenance, as illustrated by figure 7.8. When a query is submitted, the query planner automatically determines if a reflection can satisfy it, fully or partially, and rewrites the plan for optimal performance. Reflections can accelerate both raw file queries and Iceberg table scans and are particularly effective in serving repeated queries across BI dashboards or large-scale joins.



**Figure 7.8** When an aggregate query gets sent to Dremio, it will see if any existing reflections would optimize the query and transparently substitute them in the query. In this case, Dremio detects an aggregate reflection that contains the exact measurements/dimensions that this query needs and diverts the query to the optimized data structure.

Reflections come in multiple types:

- *Raw Reflections*—Optimize queries by accelerating table scans.
- *Aggregation Reflections*—Pre-aggregate data to speed up analytical queries.

Because Dremio's planner is cost-based, it uses reflection metadata, like statistics, cardinality, and physical layout, to determine when a reflection should be used. Administrators can configure reflections at both physical and virtual dataset levels, providing control over what gets accelerated and how. This reflection management can be autonomously managed for Iceberg tables, allowing Dremio to create and remove reflections based on your query patterns without manual intervention.

## CLOUD COLUMNAR CACHE

For deployments that query data in cloud object storage, such as Amazon S3 or Azure Data Lake Storage, Dremio includes a Cloud Columnar Cache. This is a local, disk-based cache on each executor node that stores frequently accessed data blocks in a compressed columnar format. Subsequent queries on the same data can read directly from this cache, dramatically reducing cloud storage reads and latency. It also lowers egress costs by avoiding repeated downloads of large files. The cache is managed automatically using a Least Recently Used (LRU) policy.

## RUNTIME FILTERING

When querying data across federated systems, minimizing data movement is critical to maintaining performance. Dremio addresses this with runtime filtering—a technique that reduces the amount of data retrieved from external sources by applying filters as early and selectively as possible during query execution. For example, if a query joins a small table with a much larger one, Dremio can first identify the relevant values from the smaller table and use those values to filter the larger dataset before retrieving it. This significantly reduces the data scanned, transmitted, and processed, leading to faster query times and more efficient use of resources.

In federated scenarios, this optimization is especially valuable because it reduces the burden on remote systems and network infrastructure. While runtime filtering can work independently, its effectiveness improves when paired with a semantic layer. The semantic layer helps Dremio understand relationships and join logic in advance, making it easier to identify where and how filters should be applied for optimal performance. This synergy between runtime filtering and semantic modeling enhances the efficiency and responsiveness of federated queries.

## WORKLOAD MANAGEMENT AND ENGINE ISOLATION

Dremio provides granular control over query execution through its multi-engine architecture, allowing organizations to create, isolate, and configure separate execution engines tailored to specific workload types. Each engine operates as an independent compute cluster, meaning it can be sized, scaled, or paused without affecting others. This enables workload isolation, a key capability for preventing “noisy neighbor” effects where resource-intensive queries in one workload degrade performance for others. For example, a dedicated engine can be provisioned for interactive ad hoc analysis, optimized for low-latency response times, while another engine handles scheduled data refresh jobs with higher throughput and relaxed latency requirements. Similarly, BI dashboards with predictable patterns can run on engines tuned for concurrency and stability.

Administrators can configure each engine’s executor count, memory footprint, and concurrency limits to align with the demands of the assigned workload. Engines can be resized or scaled out horizontally as usage grows or paused entirely during periods of inactivity to reduce compute costs in cloud environments. This flexible execution model not only improves performance and reliability but also allows organizations to optimize infrastructure spend while ensuring critical business processes have guaranteed resources. Through policies and routing rules, queries can be automatically directed to the appropriate engine based on user groups, data sources, or query type, enabling fine-grained governance and performance management across the lakehouse.

## 7.4 Trino

While Dremio offers a tightly integrated platform with built-in acceleration features, it’s not the only engine designed to support federated query workloads. Another widely adopted option is Trino—an open-source, distributed SQL query engine purpose-built for high-performance analytics across diverse and large-scale data environments. Although often referred to as a database, Trino differs from traditional transactional systems like MySQL or PostgreSQL. It is not designed for OLTP (Online Transaction Processing) but rather excels in OLAP (Online Analytical Processing) scenarios, making it well-suited for interactive, read-heavy workloads that span multiple data sources.

Originally conceived as a faster alternative to MapReduce-based engines like Hive and Pig for querying HDFS, Trino’s architecture has since evolved into a general-purpose query layer capable of federating across a broad array of data sources. It can query object storage-based lakehouses, relational databases, NoSQL stores, and analytic systems, all from a single SQL interface. Thanks to its plugin-based connector model, Trino can operate on nearly any data platform, making it highly adaptable for heterogeneous enterprise environments.

The core of Trino's performance comes from its distributed architecture. A Trino cluster is composed of one coordinator and multiple workers. The coordinator parses and plans the query, distributing work across the workers, which in turn fetches and processes data in parallel. Each query is transformed into a series of stages, tasks, and splits, executed concurrently for maximum efficiency.

Trino's key strength lies in its connector-rich ecosystem, which allows organizations to query data in place without needing to move or transform it first. Whether querying Iceberg tables in S3, dimensions in MySQL, or events in Kafka, Trino provides a unified SQL interface over all of it. This makes it particularly attractive to organizations with highly fragmented data architectures or hybrid environments.

However, Trino focuses solely on query execution and source connectivity; it does not include a native semantic layer, data curation tools, or UI for business users out of the box. These features must be implemented through complementary tools or provided by managed offerings like AWS Athena, Cloudera, Starburst, or others, which package Trino with additional governance, security, and usability layers.

In this section, we'll explore Trino's core design principles, strengths, and trade-offs. We'll examine how Trino's flexible architecture and wide connector support make it well-suited for certain federation scenarios, and what infrastructure and tooling may be required to get the most out of it in production environments.

## THE HISTORY OF TRINO AND PRESTO

The story of Trino begins with Presto, a distributed SQL query engine originally developed at Facebook in 2012. Presto was designed to solve the problem of running interactive analytics across large-scale data distributed across multiple systems, such as HDFS, S3, and relational databases, without the need to move or duplicate data.

As Presto grew in popularity, it attracted a vibrant open-source community and was adopted by major tech companies like Netflix, Uber, and LinkedIn. However, in 2019, a significant shift occurred: the creators of Presto at Facebook, Martin Traverso, Dain Sundstrom, and David Phillips, forked the project due to disagreements over governance and trademark.

They launched the fork under a new name: PrestoSQL, which later became Trino in late 2020, to establish a distinct identity. Trino inherited the original vision and momentum of Presto, but with a stronger community-driven ethos, frequent updates, and expanded ecosystem support.

Meanwhile, Facebook retained the PrestoDB name and continues to develop it independently. Both engines share a common ancestry but have diverged in terms of features, development pace, and community structure. Today, Trino is considered the more actively developed and widely adopted version, especially among open-source users and commercial vendors such as Starburst, who offer enterprise distributions and cloud-native platforms based on Trino.

In short, if you're exploring the world of federated SQL engines, understanding the Presto–Trino lineage helps clarify why the term "Presto" still appears in some tools or docs, and why Trino has become the preferred choice for modern federated query architectures.

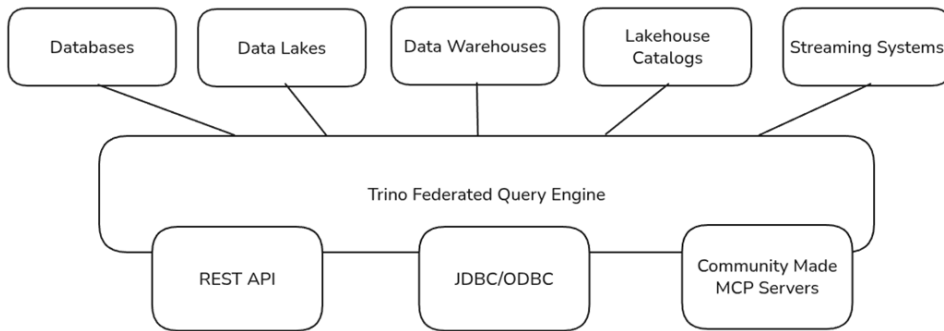
### 7.4.1 Modular architecture for wide-source support

One of Trino's defining architectural strengths is its modular and extensible connector system. Rather than simply offering a large number of prebuilt connectors, Trino provides a Service Provider Interface (SPI) that enables developers to build and plug in new connectors easily. Each data source is treated as a catalog, configured independently and accessed using standard SQL. This design allows organizations to mix and match data sources across clusters, and to join across them seamlessly—without requiring users to understand the underlying configurations. By leveraging its distributed query engine, Trino can execute federated queries across remote systems with strong performance, making it an effective unifying layer for a wide variety of analytical workloads.

Out of the box, Trino includes connectors for:

- *Data Lake & Lakehouse Formats*—Iceberg, Delta Lake, Hudi, Hive
- *Cloud Storage & Platforms*—Amazon S3, Google Cloud Storage, Azure Data Lake, Google Sheets
- *Databases & Data Warehouses*—PostgreSQL, MySQL, Oracle, SQL Server, Db2, MariaDB, Redshift, Snowflake, Vertica, Exasol, SingleStore, DuckDB
- *NoSQL & Key-Value Stores*—Cassandra, MongoDB, Redis, Ignite, OpenSearch
- *Streaming & Analytical Systems*—Kafka, Pinot, Druid, ClickHouse, Elasticsearch, Prometheus, Loki
- *Utilities & Miscellaneous*—BigQuery, Thrift, JMX, Memory, System, Faker, TPC-H, TPC-DS, Black Hole

This allows Trino to serve as a hub for federated analytics, ideal for enterprises with fragmented data landscapes that span cloud, on-prem, and multi-vendor ecosystems, illustrated in figure 7.9.



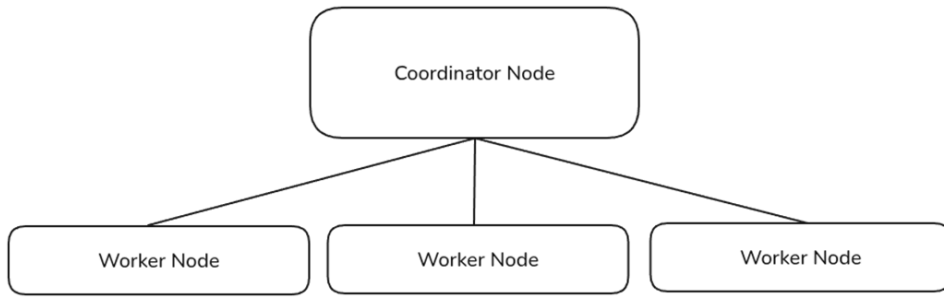
**Figure 7.9** Trino can connect to databases, data lakes, data warehouses, lakehouse catalogs, and streaming systems, making the results of federated queries accessible via REST API, JDBC, ODBC, and a variety of community-developed MCP servers.

Ultimately, Trino’s modular architecture ensures broad data access with minimal duplication, making it a flexible and powerful engine for distributed analytics and query federation, especially in complex enterprise environments.

#### 7.4.2 Flexibility and configurability for complex environments

Trino is designed for flexibility, but what sets it apart is the power of its distributed query engine. Its architecture is well-suited for querying large datasets across both data lakes and federated sources in a cost-effective and performant manner. At the heart of this design is Trino’s hierarchical execution model—composed of queries, stages, tasks, and splits—which breaks down complex analytical queries into small, independent units of work. These units can then be executed in parallel across a scalable cluster of machines, significantly reducing query latency even when dealing with massive datasets or multiple remote sources.

This distributed processing model is especially advantageous for federated environments, where Trino can parallelize reads and aggregations across different systems. Whether accessing object storage, relational databases, or other connected sources, Trino efficiently pushes down filters, minimizes data movement, and maximizes concurrency. Its decoupled coordinator-worker architecture further enhances scalability, allowing compute resources to be adjusted based on workload demands. Combined with fine-grained controls over memory, concurrency, and resource isolation, Trino provides the performance and adaptability required for modern, multi-tenant data platforms.



**Figure 7.10** Trino clusters are made up of a coordinator node that distributes work among worker nodes, which can be horizontally scaled, adding more workers to handle more queries.

### TRINO GATEWAY, LOAD BALANCING, AND ROUTING FOR TRINO CLUSTERS

Trino Gateway is an open-source extension that provides load balancing, proxying, and intelligent routing across multiple Trino clusters. It addresses many of the same scaling and workload isolation needs that Dremio handles through its scale-out coordinator architecture.

#### Use Cases and Advantages

##### Unified Access Point

Trino Gateway allows users and BI tools to connect through a single connection URL, while queries are transparently distributed to one of many backend Trino clusters.

##### Workload-Aware Routing

Administrators can define rules to route queries based on query type, user group, or data source. This enables isolating specific workloads, such as reporting or ad hoc analysis, to dedicated clusters.

##### Seamless Upgrades

Trino Gateway supports blue/green and canary deployment models, allowing you to test or roll out new versions of Trino clusters without disrupting users.

##### Dynamic Cluster Management

Clusters can be added, removed, or scaled with no interruption to users. The gateway handles routing automatically based on available capacity and health checks.

#### Key Features

- Centralized query routing and workload distribution
- Rule-based configuration for routing policies



- Support for high availability and failover
- Monitoring and logging hooks
- Integration with access control and identity systems

### High-Level Architecture

Trino Gateway operates between clients and Trino clusters, intercepting incoming queries and routing them to the appropriate cluster. It monitors cluster health and uses routing rules to ensure that queries are handled efficiently and reliably. This design makes it easier to manage multi-cluster environments and ensures that compute resources can be scaled or upgraded independently of end-user connections.

Trino also provides a cost-based optimizer (CBO) and supports strategies like dynamic filtering, broadcast joins, and partitioned joins, helping improve performance across varying query shapes and data distributions. These settings can be controlled via session properties, offering users and administrators the ability to tailor query plans based on real-time conditions or known bottlenecks.

When working with large-scale or performance-sensitive environments, Trino allows for:

- **Query routing and workload isolation** through catalog- or session-based rules.
- **Integration with external security systems**, such as LDAP, Kerberos, or OAuth.
- **Caching layers**, like Alluxio or RubiX, to reduce I/O from remote sources.
- **Support for custom extensions**, such as adding UDFs (user-defined functions) or writing new connectors using Trino's plugin SPI.

Because Trino is storage-independent and doesn't manage data directly, it can be used in tandem with existing infrastructure and doesn't impose strict requirements on how or where data resides. This makes it suitable for hybrid deployments where data may be split between on-premise systems and cloud platforms or across multiple cloud vendors.

While Trino's flexibility provides many opportunities for optimization and customization, it also requires a higher level of operational expertise to manage effectively. Tuning the system for specific workloads, maintaining connector configurations, and ensuring consistent governance across federated sources typically falls to platform or infrastructure teams.

Trino's design makes it adaptable to a broad range of use cases and deployment models, particularly in environments where architectural complexity, diverse sources, and performance tuning are expected considerations.

### 7.4.3 Community-led evolution and vendor extensions

Trino's development is primarily driven by an active open-source community, with a broad base of contributors and users shaping its roadmap and features. This community-led model enables the engine to evolve rapidly in response to emerging data needs, encouraging transparency in how new capabilities are introduced and adopted.

The open governance model of Trino ensures that enhancements, such as support for new connectors, query optimizations, or language features, are reviewed and developed in collaboration with a wide range of stakeholders. As a result, Trino often introduces early support for popular data platforms and formats, benefiting from real-world feedback and diverse usage scenarios.

Alongside the core open-source project, several vendors offer commercial distributions and managed services based on Trino. These vendors, such as Starburst, AWS (via Athena), and Cloudera, typically extend the base engine with additional enterprise-focused features.

These vendor offerings aim to lower the operational burden of deploying and maintaining Trino clusters while adding capabilities such as security, caching, observability, and integration with enterprise infrastructure.

The coexistence of an open-source core and commercial enhancements offers users flexibility in how they adopt Trino. Organizations can start with the open-source engine for initial experimentation or internal workloads and later transition to a managed or enterprise version as their needs evolve.

This ecosystem model, anchored in community development but enriched by vendor-led innovation, provides a balance between openness and enterprise readiness, allowing organizations to tailor their Trino usage according to their technical requirements, governance standards, and operational preferences.

### 7.4.4 Semantic layer considerations in Trino

While Trino excels in federated query performance and broad connector support, it does not include a built-in semantic layer like Dremio. This distinction becomes especially important when building governed, business-friendly data experiences where standardized metrics, consistent definitions, and curated datasets are critical for data consumers.

In contrast, organizations using Trino must adopt one of the following strategies to incorporate semantic capabilities:

*Pairing with a dedicated semantics layer tool*—Technologies like Cube, AtScale, Select Star, and dbt Metrics can be layered on top of Trino to define consistent, reusable business metrics and semantic models.

*Leveraging enterprise-grade managed services*—Some commercial platforms built around Trino, such as Starburst Galaxy or Cloudera's Trino deployments, may include integrated or add-on semantic modeling features as part of their enterprise offerings.

These solutions help bridge the gap by offering features like governed data access, metadata cataloging, metric standardization, and integration with BI tools. However, they introduce additional deployment, licensing, and integration considerations that must be evaluated based on your team's goals and infrastructure.

When choosing a semantic layer strategy for Trino, it's important to align with the needs of your data consumers, the complexity of your governance requirements, and the broader tooling ecosystem you plan to support.

## 7.5 Deployment models

As organizations adopt data federation platforms to unify access across diverse systems, choosing the right deployment model becomes a strategic decision that balances performance, cost, scalability, and operational complexity. Both Dremio and Trino offer flexible deployment options, ranging from self-managed clusters running on-premises or in the cloud to fully managed services provided by commercial vendors.

These models cater to different organizational needs. Some teams may require tight control over infrastructure due to regulatory or cost considerations, while others prefer the agility and reduced overhead of a cloud-native SaaS solution. In many cases, companies evolve through these deployment stages, starting with on-prem or hybrid configurations before gradually shifting to managed services as part of a modernization initiative.

In this section, we'll explore the deployment models available for both Dremio and Trino, examining their respective approaches to self-managed and managed environments. We'll also highlight the unique capabilities and trade-offs associated with each, helping you better align deployment choices with your organization's architectural and operational goals.

### 7.5.1 Deployment with Dremio

Dremio offers two primary deployment models, self-managed and cloud-managed, allowing organizations to choose the operational approach that best aligns with their infrastructure preferences and governance requirements.

## SELF-MANAGED DEPLOYMENT

Dremio can be deployed as a self-managed solution in two forms:

- *Dremio Community Edition*—This free edition provides core capabilities and can be installed in virtually any environment, including on-premises, within virtual machines, or basic container orchestration setups. It's an excellent entry point for teams exploring Dremio's federated query capabilities.
- *Dremio Enterprise Edition*—This version is designed for production-grade, mission-critical environments and requires deployment within a Kubernetes cluster. In exchange, it delivers several advanced features not available in the Community Edition, including:
  - Fine-grained role-based access control (RBAC) and field-level access control (FGAC).
  - An integrated, Iceberg-native catalog for transactional table management.
  - Autonomous Iceberg table maintenance, including compaction and cleanup.

- Autonomous Reflection management for hands-off query acceleration.
- Elastic scaling of query executors based on workload demand.

These capabilities allow Dremio Enterprise to operate as a powerful, scalable lakehouse engine that can unify governance, performance, and cost management in self-hosted environments.

## **CLOUD-MANAGED DEPLOYMENT: DREMIO CLOUD**

For teams seeking a fully managed solution, Dremio Cloud offers a SaaS experience deployed directly into AWS or Azure environments. In this model, Dremio handles the management of the control plane, while the customer retains control over data sources and compute infrastructure via secure connectivity and delegated execution.

Dremio Cloud includes all the capabilities of Enterprise Edition, including the integrated Iceberg catalog, elastic scaling, and reflection management, without the operational overhead of maintaining infrastructure. It is particularly well-suited for organizations pursuing cloud-native strategies or modernizing their data platforms with minimal DevOps burden.

### **7.5.2 Deployment with Trino**

Trino, as an open-source distributed SQL engine, offers significant flexibility in its deployment, ranging from fully self-managed environments to a growing number of cloud-managed offerings. This versatility allows organizations to choose a deployment strategy that aligns with their infrastructure maturity, resourcing model, and scalability needs.

## **SELF-MANAGED DEPLOYMENT**

At its core, Trino is a free and open-source project that can be deployed in any environment, on-premises, on virtual machines, or across Kubernetes clusters. Trino's modular architecture and pluggable connector ecosystem make it highly extensible, allowing users to tailor deployments for specific data sources, security configurations, and query performance goals.

In self-managed setups, organizations are responsible for deploying, configuring, orchestrating clusters, optimizing queries, and tuning performance. However, Trino benefits from an active community that maintains a rich library of open-source plugins and extensions, ranging from authentication modules to cost-based optimizers, enabling deep customization of the platform. This makes Trino particularly attractive to teams with strong engineering capabilities that want to retain control over their deployment and tuning strategies.

## **CLOUD-MANAGED TRINO OFFERINGS**

For organizations looking to reduce operational overhead or accelerate time-to-value, several vendors offer managed Trino deployments, each bringing unique value propositions:

- *Amazon Athena*—Athena is a fully serverless, Trino-based query engine integrated into the AWS ecosystem. It enables interactive SQL querying directly over Amazon S3, with automatic scaling, managed infrastructure, and native integration with services like AWS Glue and QuickSight. Athena abstracts away cluster management entirely and is priced per query, making it ideal for ad hoc or low-frequency access patterns.
- *Cloudera*—Cloudera offers Trino as a managed component within its hybrid data platform, positioning it as an alternative to Impala for federated SQL analytics. With Cloudera's Customized Professional Services (CPS), customers can benefit from tailored deployments, optimized cluster sizing, advanced security configurations, and long-term operational support, which is especially valuable for enterprises transitioning from legacy systems or managing large-scale, multi-source environments.
- *Starburst*—As a commercial steward of Trino, Starburst offers two deployment options: Starburst Enterprise and Starburst Galaxy, its fully managed SaaS version. Starburst builds on the open-source core by adding enterprise features like advanced query observability, built-in security integrations, autoscaling, and federated access controls. Galaxy is particularly well-suited for teams looking to operationalize Trino quickly in the cloud with minimal infrastructure overhead.
- *Other Providers*—Regional and niche vendors such as Yandex, Pandio, and others also offer Trino as a managed service, often tuned for specific use cases or geographies. These platforms offer alternative pathways for organizations seeking localized support, regulatory compliance, or integrations tailored to specific industries.

Whether deployed independently or through a managed provider, Trino's deployment ecosystem emphasizes flexibility, extensibility, and broad connector support. Self-managed deployments offer granular control and customizability, while cloud-managed services lower the barrier to adoption by handling infrastructure complexity. Depending on organizational goals, whether it's rapid deployment, integration with cloud-native services, or support for highly specialized use cases, Trino's varied deployment options can be matched to a wide range of federation needs.

## 7.6 Federation platform decision scenarios

Choosing the right data federation platform, whether Dremio, Trino, or another engine, is rarely a one-size-fits-all decision. Rather than prescribe a specific tool for every situation, this section presents a set of illustrative scenarios to show how different organizations might weigh their options based on real-world needs. These examples are intended to serve as guidance for framing your decision-making process, not as fixed rules.

Every organization has unique priorities, including the distribution of its data, the skills of its teams, regulatory requirements, and long-term architectural goals. The goal here is to help you think through the trade-offs and capabilities that matter most in your own context.

We'll now walk through a few hypothetical but realistic decision scenarios to show how certain platform characteristics, such as connector breadth, Iceberg support, semantic modeling, or cloud compatibility, can drive alignment with specific goals. As always, the best choice is the one that most directly supports your users, your data landscape, and your strategic outcomes.

### **7.6.1 Fragmented multi-source environment: Trino for connector breadth**

Organizations with highly fragmented data environments, where operational data is spread across multiple cloud platforms, relational databases, NoSQL systems, and even flat files, often find Trino's broad connector ecosystem to be a practical fit. Trino supports dozens of connectors out of the box, from traditional data warehouses like Teradata and Oracle to cloud-native stores like Amazon S3, Delta Lake, and MongoDB.

In this scenario, a company might be undergoing a digital transformation and isn't yet ready to centralize its data into a single lakehouse architecture. With a mandate to enable analytics across existing silos without disrupting ongoing operations, they chose Trino for its ability to federate queries across diverse systems with minimal data movement.

The decision isn't necessarily about long-term architectural vision but rather about enabling quick, read-time access to existing systems. While the company may eventually evolve toward a more consolidated model using Apache Iceberg or a centralized lakehouse, Trino enables them to bridge the present reality and future goals without forcing premature ETL or replication decisions.

This example illustrates how connector breadth and extensibility can serve as short- to mid-term enablers for complex environments, particularly when agility and minimal disruption are top priorities.

### **7.6.2 Building a native Iceberg lakehouse: Dremio for Iceberg-native features**

For organizations that have committed to a lakehouse architecture, particularly one centered on Apache Iceberg, Dremio offers a compelling fit due to its deep, native integration with Iceberg. Unlike general-purpose federation engines, Dremio is built from the ground up with Iceberg in mind, offering features like automatic table optimization, autonomous reflection management, and a fully integrated Iceberg catalog powered by Apache Polaris.

In this scenario, a company has begun consolidating its data into a data lake, with Iceberg as the foundational table format. Their goal is to reduce cloud storage costs, simplify operations, and future-proof their architecture for advanced use cases such as data versioning, time travel, and streaming ingest. Choosing Dremio allows them to not only query Iceberg tables with high performance but also to manage and maintain them with minimal manual intervention.

Because Dremio supports Iceberg read/write capabilities natively and includes Iceberg-aware performance enhancements, such as partition pruning, reflection acceleration, and clustering, it allows teams to operationalize the lakehouse with confidence, without needing to stitch together multiple tools or services.

This example demonstrates how tight integration with your core table format and catalog architecture can be a critical deciding factor, especially when Iceberg plays a central role in your data strategy.

### 7.6.3 Empowering business users with UI and governed datasets: Dremio

Organizations seeking to enhance data accessibility for business users typically prioritize platforms that provide intuitive interfaces, governed semantic layers, and self-service capabilities. In this context, Dremio stands out for its native semantic layer, user-friendly UI, and integrated access controls, all of which are designed to enable non-technical users to discover, understand, and explore data with confidence.

Consider a company where business analysts heavily rely on dashboards and ad hoc querying but struggle with inconsistencies across departments due to disconnected, BI tool-specific semantic layers. By adopting Dremio, the company is able to centralize metric definitions and business logic in the semantic layer of the federation engine itself, independent of downstream BI tools. This reduces duplication, enforces consistency, and streamlines collaboration across data consumers.

Dremio's UI allows users to browse data sources, curate virtual datasets, and define joins or filters, all without writing SQL. At the same time, IT and data engineering teams maintain fine-grained access control and governance policies, ensuring compliance without bottlenecks.

This scenario highlights the critical importance of usability and governance in empowering data consumers at scale. For teams focused on enabling broader access while maintaining trust in the data, Dremio provides a cohesive self-service experience out of the box.

### 7.6.4 Lightweight querying of Hudi datasets: Trino via AWS Athena

When the primary goal is to run lightweight, ad hoc queries on existing Apache Hudi datasets, particularly those stored in Amazon S3, Trino, via AWS Athena, offers a practical and low-friction solution. Athena is a fully managed, serverless query service powered by Trino, enabling users to analyze data directly in S3 using standard SQL without the need to provision or manage infrastructure.

In this scenario, a company stores transactional log data in Hudi format for ingestion efficiency and incremental data updates. While the data is important for investigation and occasional analysis, it's not part of a heavily curated or performance-sensitive pipeline. Rather than setting up and managing a dedicated engine, the company opts for Athena's pay-per-query model, leveraging Trino's native support for Hudi to quickly explore the data without overhead.

A key factor here is that Dremio does not support Apache Hudi, which excludes it from consideration in this specific use case. By contrast, Trino's broad format compatibility, including support for Hudi, Iceberg, and Delta Lake, makes it a flexible choice for teams working across mixed table formats or transitioning between them.

This example emphasizes how format compatibility and infrastructure overhead influence technology selection, especially when the querying needs are occasional, the datasets are already in a specific format, and the desired experience is serverless and usage-based.

### 7.6.5 On-prem Cloudera modernization: Trino replacing Impala for performance

Many enterprises with long-standing Cloudera deployments find themselves reevaluating their query engines as workloads grow and performance expectations rise. In environments where Apache Impala no longer meets performance or extensibility needs, Cloudera's managed Trino offering presents a natural upgrade path within the same ecosystem.

In this scenario, an enterprise operates a large on-premise Cloudera cluster and uses Impala to query data stored in HDFS and Hive. However, they encounter increasing limitations, such as a lack of support for modern table formats, slower performance for complex federated queries, or difficulty scaling analytics workloads. Since Cloudera now offers Trino as a first-class, managed component within its platform, the organization can shift to Trino without leaving its current environment or disrupting existing infrastructure.

Trino provides broader connector support, improved performance for federated and ad hoc queries, and a more modern architecture. Additionally, Cloudera's Customized Professional Services (CPS) ensures that the Trino deployment is tuned and integrated with existing metadata, security, and governance layers.

This example shows how platform-native upgrade paths, especially when supported by vendor-managed services, can provide a smooth transition while unlocking new capabilities. When the goal is to modernize query performance without abandoning existing investments in Cloudera, managed Trino becomes a compelling option.

### 7.6.6 Hybrid cloud Iceberg strategy: Dremio bridging on-prem and ADLS

Organizations modernizing their data infrastructure often adopt a hybrid architecture, maintaining on-prem systems while incrementally adopting cloud services. This phased approach is especially common in industries with strict data residency, latency, or compliance requirements. For teams pursuing an Iceberg-based lakehouse strategy, Dremio offers a compelling solution due to its first-class Apache Iceberg support, integrated catalog, and hybrid deployment flexibility.

Consider a company that currently operates an on-prem Hadoop cluster but aims to migrate data and workloads to Azure Data Lake Storage (ADLS) over time. Rather than executing a disruptive lift-and-shift migration, the team adopts Dremio to bridge both environments. Using Dremio's support for querying both on-prem and cloud-based data sources, they can curate, optimize, and accelerate Iceberg tables wherever they reside, gradually transitioning storage and compute to the cloud.

Dremio's ability to deploy in either self-managed or cloud-managed modes, combined with its support for federating across environments, allows the organization to maintain continuity while embracing cloud-native capabilities. Features such as autonomous reflection management, elastic execution scaling, and semantic-layer-based data governance enhance both performance and user experience across the hybrid architecture.

This example illustrates how platform-native Iceberg capabilities, combined with hybrid query federation, enable organizations to evolve at their own pace, minimizing risk while maximizing long-term agility.



## 7.7 Federation Alternatives

While traditional federation engines like Dremio and Trino provide powerful mechanisms for querying distributed data sources in real time, they are not the only architectural path for enabling cross-system analytics. Several alternative approaches are emerging that address similar goals, reducing data duplication, simplifying integration, and enabling broader data access, while taking different architectural routes.

### 7.7.1 Virtualization via shortcuts in OneLake

One notable alternative is the virtualization strategy used in Microsoft OneLake. Rather than pushing full SQL queries into external systems as in conventional federation, OneLake creates *shortcuts*—lightweight links to external storage locations such as Amazon S3, Azure Data Lake Storage (ADLS), Google Cloud Storage (GCS), and S3-compatible services like NetApp and MinIO.

These shortcuts allow external data to appear as native tables within the OneLake environment. When accessed, only the underlying data requests (such as filters or column projections) are pushed down to the remote storage layer—not the entire SQL query. This distinction matters: instead of executing distributed joins or aggregations across systems, OneLake optimizes access patterns and keeps most compute operations local to the querying engine. While this limits the expressiveness of cross-system querying, it offers operational simplicity and predictable performance, particularly when external datasets are large or frequently accessed.

### 7.7.2 AI-native data virtualization with Spice.ai

Another emerging approach comes from the AI application domain. The Spice.ai Cloud Platform blends data federation with AI application enablement by providing composable building blocks for SQL query, model inference, vector search, and retrieval-augmented generation (RAG). Spice supports federated SQL queries across sources like PostgreSQL, MySQL, Databricks, Snowflake, BigQuery, and object storage systems such as S3 and MinIO.

What sets Spice.ai apart is its orientation toward *virtualized data views* for AI and application use cases. Rather than querying remote sources on demand for every user request, Spice.ai allows developers to create small, fast materialized views that serve APIs, dashboards, or agents. These views can be cached, replicated, or optimized for AI workloads, improving responsiveness and resilience in production environments.

Under the hood, Spice.ai uses Apache DataFusion and the Apache Arrow Flight protocol for high-performance query execution. Its cloud platform integrates observability, access control, and resource management—making it well-suited as a backend for AI-enabled applications that rely on diverse data sources.

### 7.7.3 Choosing the right fit

While these alternatives do not replace general-purpose federation engines, they provide specialized value in scenarios where operational simplicity, caching, or AI integration are prioritized. Shortcut-based virtualization may be ideal in environments focused on storage unification and performance predictability. Platforms like Spice.ai are better aligned with teams building AI applications that require federated access along with acceleration and model orchestration.

As the landscape evolves, organizations may choose to combine these approaches—using a federation engine for broad analytical access, shortcuts for repeatable external access patterns, and virtualization platforms for application-centric performance. The right fit depends on the specific mix of users, data sources, and workloads your lakehouse must support.

## 7.8 Summary

- Not all data belongs in the lakehouse; federation allows selective, real-time access to external systems while keeping Iceberg at the architectural core.
- Effective federation strategies prioritize minimizing data movement, managing performance variability, and maintaining semantic consistency across sources.
- A robust federation layer integrates three critical components: a distributed query engine, a semantic layer for standardizing logic, and accessible interfaces for both humans and AI agents.
- Dremio and Trino illustrate two distinct approaches to federation—Dremio emphasizing built-in acceleration and semantic control, and Trino offering flexible, modular federation with wide connector support.
- Trino's hierarchical query execution and Dremio's runtime filtering are optimized for parallelizing workloads and reducing latency in federated environments.
- Deployment choices—including self-managed or cloud-native models—impact cost, scalability, and operational complexity; selecting the right platform depends on your team's capabilities and data landscape.
- Real-world federation success hinges on aligning tool capabilities with business needs—balancing access, governance, and performance in an increasingly distributed data ecosystem.

## 8 *Understanding the consumption layer*

### **This chapter covers**

- Semantic consistency across tools
- Open interfaces such as JDBC, ODBC, Arrow Flight, and MCP
- Evaluating BI tools, notebook environments, and AI platforms for integration
- How to choose the right consumption tools

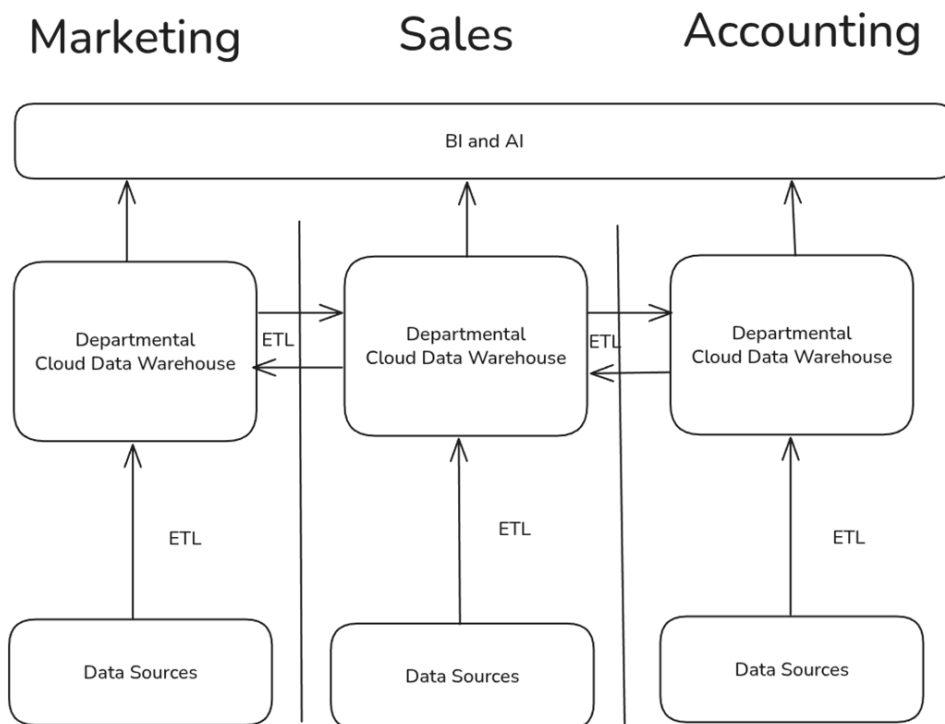
Now that your lakehouse has a solid foundation, from storage and ingestion to catalog and federation, it's time to focus on where data creates value: consumption. This is where your lakehouse architecture begins to yield insights, drive decisions, and power innovation. Whether you're enabling real-time dashboards, supporting ad hoc data exploration in Python notebooks, or training large-scale machine learning models, the consumption layer bridges your technical investment with practical outcomes.

In traditional data architectures, consumption was often bound by the limitations of data movement, format compatibility, and tool lock-in. Accessing data meant replicating it into specialized databases, BI tools, etc., each with its own constraints. Apache Iceberg's emphasis on openness and portability of table formats has reshaped this paradigm. Now, the data remains in place, and tools can come to the data, rather than the other way around. This shift dramatically reduces friction, empowering teams to bring their tool of choice without compromising governance, consistency, or performance.

We'll explore how to design an effective consumption layer in an Iceberg-based lakehouse. We'll revisit the benefits of the lakehouse model, examine the diverse requirements different teams bring to the table, and unpack the interfaces and tools that enable seamless data access. From BI dashboards and ad-hoc queries to analytics pipelines, ML training, and AI agents, we'll explore how your architecture supports diverse consumption patterns and how to align them with your organizational goals. By the end of this chapter, you'll understand how to complete the loop of your data architecture, enabling each layer to support the next while keeping your system agile, open, and ready to evolve.

## 8.1 Revisiting the benefits of the lakehouse for consumption

Many organizations struggle with data silos, not because they lack tools, but because those tools are fragmented across teams. Marketing might build dashboards on one cloud warehouse, while finance pulls reports from another. This creates data duplication, complex ETL pipelines, and inconsistent results, as illustrated in figure 8.1. Data is copied and reshaped repeatedly, which not only increases costs but also introduces opportunities for bad quality data and downtime.



**Figure 8.1** In traditional architectures, different departments replicate data not only into their departmental warehouses but also between them, increasing latency and inconsistency in the data.

Even when data is shared, definitions often are not. In legacy BI architectures, business metrics like “customer lifetime value” or “gross margin” are often redefined in each dashboarding tool. Without a shared semantic layer, the same metric can have multiple interpretations depending on which team created the report or which tool is rendering it.

And beyond your core datasets, the challenge grows. Valuable data from third-party platforms, SaaS tools, and legacy systems often sits outside your primary warehouse. Without query federation, integrating this “long tail” of data requires additional ETL jobs, further delaying access and creating more points of failure.

The lakehouse architecture, particularly when built on Apache Iceberg, flips this model. Instead of pushing data into isolated systems, it establishes an open and modular foundation. Data is stored once in Iceberg tables, governed centrally, and accessed through interoperable systems and tools. This architecture enables the support of diverse consumption patterns without compromising consistency and data quality. BI tools, notebooks, and ML platforms all query the same data, with shared definitions and without unnecessary duplication.

The result is a system that empowers teams. Analysts gain flexibility, engineers reduce duplication, and the organization as a whole becomes more aligned. With the right consumption layer on top of your lakehouse, you replace friction with flow.

## 8.2 Connecting the lakehouse to the people

The true value of a lakehouse isn't just in how it stores or organizes data; it's in how that data is ultimately consumed. With an Apache Iceberg-based architecture, organizations unlock an open, flexible, and future-proof data foundation that supports a wide range of consumption patterns. Whether it's BI dashboards, ad hoc notebooks, ML pipelines, or AI agents, the same curated Iceberg tables can serve every team through standardized interfaces and shared semantics, enabling seamless integration across all applications.

This architecture removes many of the historical friction points that have plagued enterprise analytics. Teams no longer need to replicate data into separate silos or define metrics independently within each BI tool. Instead, a central semantic layer and query federation engine provide a common contract for accessing and understanding data. Tools become interchangeable, and teams gain the freedom to choose the right interface for their specific needs, without breaking consistency or governance.

By aligning consumption tools with the lakehouse's core components, organizations gain both agility and control. Analytics becomes more accessible, collaborative, and trustworthy. In this chapter, we've explored not just the tooling landscape but also the principles and patterns that ensure effective integration with the lakehouse.

## 8.3 Revisiting requirements from our audit

In chapter 3, we laid the foundation for architecting a lakehouse by emphasizing the importance of understanding your organizational requirements. These requirements, spanning performance, compliance, accessibility, governance, and cost, shape the choices made across every layer of the architecture. As we now focus on the consumption layer, it's essential to revisit those same considerations through the lens of data access and usage.

Consumption is where all earlier architectural decisions converge. The tools chosen to analyze, visualize, or model data must reflect not just user preferences but also the operational realities and priorities identified during the platform audit. For example, if your analysts prioritize fast access to curated datasets, you need to ensure low-latency query paths and reliable semantic layers. If compliance teams require data masking or audit trails, consumption tools must enforce those rules without exception.

This section will reconnect each major category of requirement discussed in chapter 3 with the design of your consumption layer. We'll examine how to interpret those needs from the perspective of data consumers, ensuring that the systems you build continue to deliver value through the final stages of the data journey.

### 8.3.1 Interpreting requirements for consumption

Each requirement surfaced during your initial audit, whether technical, organizational, or regulatory, has implications for how users consume data. Understanding these implications helps you match the right tools and interfaces to the right use cases.

**Performance and Latency:** For teams building dashboards or running interactive analyses, query latency is paramount. A high-performing consumption layer should minimize the time between question and answer. This places importance on selecting query engines that support predicate pushdown, caching, and incremental results. Tools must also strike a balance between performance and cost, especially in environments where compute usage is tied to billing.

**Accessibility and Flexibility:** Not every data consumer is a SQL expert. Some prefer drag-and-drop interfaces, while others rely on Python or R. Your consumption layer should support a range of access methods, including visual tools, notebooks, and APIs, to meet users where they are. This includes ensuring that interfaces like JDBC, ODBC, and REST are available across environments, and that authentication and permissions are consistently enforced.

**Governance and Compliance:** Data usage is not just about who can access what; it's also about ensuring that access is controlled, logged, and aligned with policy. The consumption layer must enforce row-level security, column masking, and lineage tracking as defined in your governance model. Ideally, these controls should be inherited from the catalog layer to avoid duplication and drift.

**Semantic Consistency:** The presence of a shared semantic layer helps avoid metric fragmentation. When consumption tools access data through a unified layer that standardizes key metrics and definitions, they ensure that different teams speak the same language. This improves trust, accelerates decision-making, and reduces time spent reconciling conflicting reports.

**Cost Control and Resource Management:** Certain workloads, particularly those involving large-scale data science and machine learning, consume substantial compute resources. The consumption layer should provide observability into usage patterns and support workload isolation where needed. Integration with query planners and cost estimators can further help teams make efficient use of resources.

By grounding your consumption layer in these real-world requirements, you can ensure that it not only supports access to data but also reinforces the architectural principles established throughout the lakehouse.

### 8.3.2 Requirements for BI tools

Business Intelligence (BI) tools are often the most visible part of a data platform. They enable stakeholders across the organization to explore data, track KPIs, and make informed decisions. But for BI to be effective, the underlying lakehouse must be designed with specific consumption requirements in mind.

**Performance and responsiveness** are critical. Executives and business users expect dashboards to load quickly and reflect up-to-date information. This means the data platform must support efficient query execution, often through caching, materialized views, or federation engines that optimize access paths to Iceberg tables.

**Semantic consistency** is another core requirement. BI tools should not define their own logic for calculating metrics like revenue or churn. Instead, these definitions should reside in a centralized semantic layer that all tools can query. This ensures that dashboards across different departments reflect the same truth, minimizing confusion and misalignment.

**Interoperability and open interfaces** also matter. Many organizations use a mix of BI platforms, often due to team preferences, licensing constraints, or legacy adoption. Supporting JDBC, ODBC, and emerging standards like Arrow Flight ensures that multiple BI tools can access the same datasets without needing to duplicate or reshape the data.

**Access control and governance** are essential for maintaining trust. BI users should only see the data they're authorized to view. This requires the consumption layer to respect role-based access controls defined in the catalog or semantic layer and to enforce policies such as row-level security or column masking.

Finally, **collaboration and flexibility** must be supported. Modern BI workflows involve not just consuming static dashboards but also sharing insights, embedding visualizations in other tools, and exploring data through drag-and-drop interfaces. A lakehouse that exposes data in consistent, well-documented schemas makes it easier for BI teams to build confidently without relying heavily on engineering support.

### 8.3.3 Requirements for interactive notebook environments

Interactive notebooks, such as those built in JupyterLab or integrated into platforms like Databricks and Deepnote, are essential tools for data exploration, prototyping, and ad hoc analysis. While BI tools focus on visual consumption and predefined metrics, notebooks offer data practitioners full programmability and freedom. Supporting these environments in a lakehouse architecture introduces its own set of requirements.

**Direct, low-latency access to raw and transformed data** is foundational. Data scientists and analysts working in notebooks often need to iterate rapidly over queries, aggregations, or transformations. If every cell execution triggers a slow query, productivity suffers. A consumption layer that supports efficient access patterns through predicate pushdown, column pruning, and query planning, especially when using Iceberg tables, can significantly improve the development experience.

**Rich support for open data access interfaces** is also essential. Notebook users rely on libraries such as Pandas, PyArrow, DuckDB, and SQLAlchemy to interact with data. Your lakehouse should support standard interfaces, such as JDBC, ODBC, and Arrow Flight, to ensure these libraries can access data seamlessly. For in-memory performance and efficient data interchange, integration with Arrow or native Iceberg connectors can provide substantial advantages.

**Interoperability with multiple compute engines** is often required. Notebooks are used for more than querying; they may include lightweight Spark jobs, embedded SQL engines like DuckDB, or machine learning pipelines using Scikit-learn or TensorFlow. A flexible consumption layer must allow notebook users to choose their preferred compute frameworks without forcing unnecessary data movement or re-ingestion. Just as importantly, these notebooks can be operationalized—scheduled to run automatically—turning exploratory work into repeatable production workflows.

**Access to semantic definitions** is another requirement. While notebook users may write their own transformations, many will benefit from reusing canonical metrics defined in the semantic layer. Exposing those definitions through accessible APIs or queryable views helps ensure consistency across tooling, even in a highly flexible programming environment.

Finally, **security and reproducibility** cannot be overlooked. Notebooks may access sensitive data, and organizations must enforce access controls, audit logs, and environment isolation to prevent data leakage. At the same time, notebooks often serve as prototypes for production pipelines. Ensuring that the data consumed in a notebook is versioned, queryable by snapshot, and reproducible over time is crucial for turning exploratory work into operational workflows.

Supporting Python notebooks as first-class consumers ensures that your lakehouse can serve not just executives and analysts, but also data engineers, scientists, and developers who turn data into models, services, and products.

### 8.3.4 Requirements for AI and specialized data consumption tools

Beyond BI dashboards and notebooks, many organizations rely on tools that consume data to power machine learning (ML), artificial intelligence (AI), and operational systems. These workloads often introduce unique access patterns, integration requirements, and performance expectations. Supporting them effectively requires designing a consumption layer that accommodates these diverse needs while maintaining consistency and performance.

**Machine learning platforms** such as MLflow, Amazon SageMaker, and TensorFlow Extended (TFX) depend on structured, clean, and versioned data. Model training typically involves reading large datasets repeatedly and at scale. To ensure reproducibility, your lakehouse must provide high-throughput access and stable snapshots that are immune to mid-process schema or data changes. Apache Iceberg's snapshot isolation and schema evolution capabilities make it particularly well-suited for these workloads, especially when paired with efficient data interfaces such as Arrow Flight or native connectors.



**AI agents and intelligent applications** are increasingly leveraging structured data to make real-time decisions, generate content, or provide dynamic user experiences. These agents often issue high-frequency, context-aware queries that must be fulfilled with low latency and high concurrency. Supporting such interactions requires query engines and APIs that can access Iceberg tables quickly, respect user-level permissions, and return consistent, filtered results. Caching strategies, metadata pruning, and optimized scan planning all play a role in enabling AI agents to operate effectively against lakehouse data.

**Streaming applications and real-time systems**, such as fraud detection engines or recommendation services, require timely access to the freshest data. While ingestion is handled upstream, the consumption layer must enable up-to-the-moment views via engines or APIs that support incremental querying and continuous refresh. This may involve integrating with materialized views, event-driven processing, or low-latency federated query engines capable of surfacing Iceberg updates promptly.

**Embedded analytics and APIs** are also key components of modern data consumption. Internal dashboards and customer-facing applications often require real-time access to structured insights. These endpoints must be secure, scalable, and capable of row-level filtering. REST APIs built atop the federation layer are commonly used for this purpose, provided they can resolve queries against Iceberg datasets while enforcing authorization policies aligned with business rules.

Finally, data sharing and external collaboration introduce additional requirements. When sharing data with partners, vendors, or regulatory entities, your architecture should support read-only access, exportable snapshots, and open, portable formats that preserve schema integrity. Iceberg's engine-agnostic metadata and REST-friendly design allow organizations to expose datasets confidently, without tying consumers to specific processing tools.

By accommodating these broader consumption patterns—including ML training, AI agents, real-time applications, embedded analytics, and external sharing—the lakehouse becomes more than an analytics solution. It becomes a unified backbone for data-driven operations, intelligent products, and secure collaboration at scale.

## 8.4 Open interfaces for seamless consumption

As we explored in chapter 7, the ability to connect tools through open interfaces is essential for effective data federation. The same principle applies to the consumption layer. Whether you're serving dashboards, notebooks, or machine learning pipelines, the flexibility and usability of your lakehouse depend heavily on the interfaces it exposes.

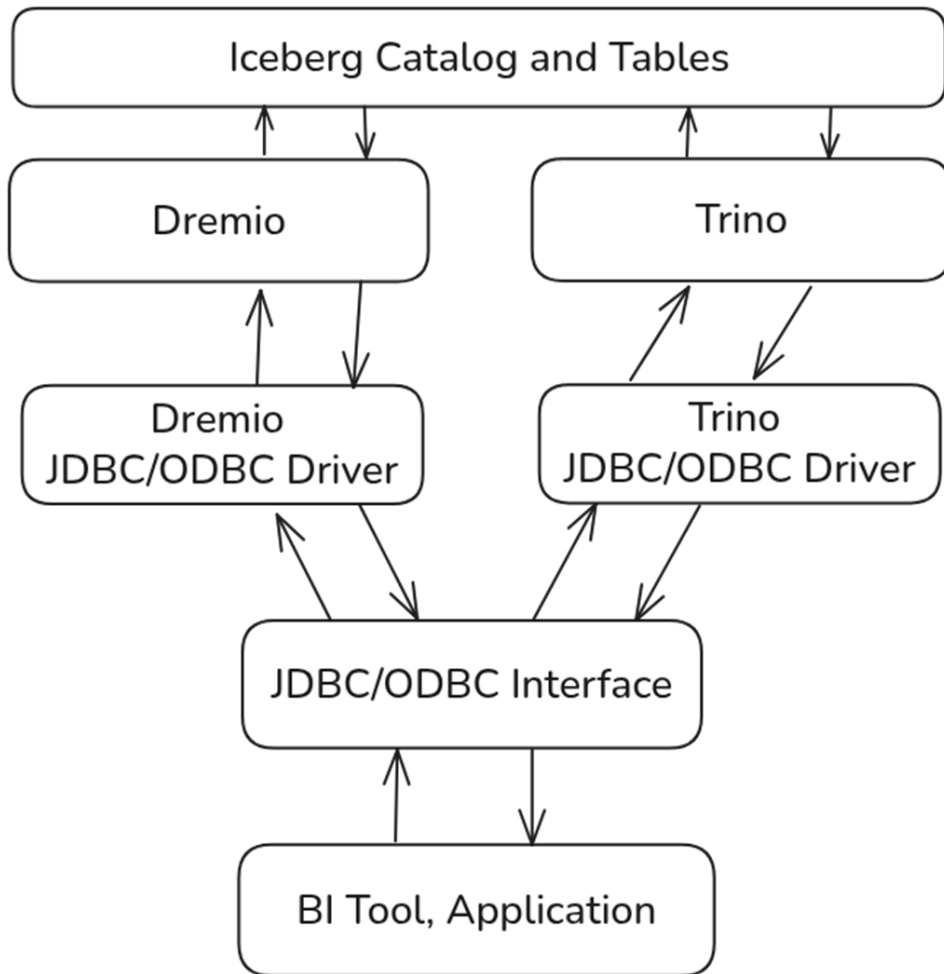
Open interfaces, such as JDBC, ODBC, Arrow Flight, and the emerging Model Context Protocol (MCP), act as bridges between the data platform and the tools that rely on it. When consumption tools can connect through these standardized protocols, they gain reliable, performant access to governed data without requiring bespoke integrations or brittle data extracts. This not only simplifies tooling decisions but also future-proofs your architecture as new platforms emerge or organizational needs evolve.

These interfaces also support consistency. By routing all consumption through common access layers, often mediated by the federation or semantic layer, you ensure that business logic, access policies, and data definitions remain centralized and enforced. This creates a more reliable and trustworthy experience for end users, regardless of the tools they use.

In the following subsections, we'll examine each of these open interfaces from the consumer's perspective. We'll focus on what they enable, how they work in practice, and what to consider when choosing or prioritizing support for them in your architecture.

### **8.4.1 JDBC and ODBC**

JDBC (Java Database Connectivity) and ODBC (Open Database Connectivity) remain the most widely adopted standards for connecting analytical and business applications to structured data sources, as illustrated in figure 8.2. Despite their age, these interfaces are still foundational to modern data consumption, particularly for BI tools, reporting applications, and spreadsheet integrations.



**Figure 8.2 JDBC/ODBC has a large ecosystem of drivers for systems from which you can pull data. This allows you to connect to tools like Dremio and Trino to pull data from your Apache Iceberg Lakehouse.**

From a lakehouse perspective, JDBC and ODBC are the most common connection interfaces between consumption tools and query engines. Engines that support Iceberg, such as Trino, Dremio, and Spark SQL, typically expose these endpoints, allowing tools to interact with Iceberg tables as if they were traditional relational databases. In some cases, a federation layer acts as a proxy, routing queries across multiple engines or sources while presenting a unified interface. While semantic layers may sit above these systems to provide business-friendly views, the core interaction for consumption still flows through the query and federation engines.

Thanks to standard, well-established JDBC and ODBC connectors, tools like Excel, Looker, and many others can query your lakehouse with minimal configuration. These interfaces require little management overhead, provide a clear upgrade path, and integrate easily with enterprise access controls. When routed through a centralized federation engine, they also enable consistent enforcement of access policies, metric definitions, and query optimizations across diverse consumption tools.

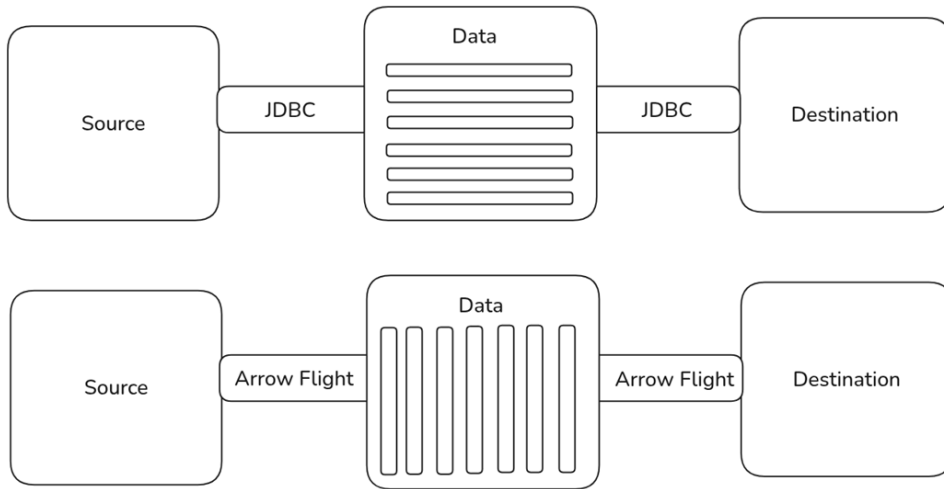
However, these interfaces also have limitations. They were designed primarily for synchronous, tabular workloads and can struggle with high-volume data transfers or streaming use cases. They also require configuration and driver management, which can complicate deployment in distributed environments.

For most organizations, ensuring strong JDBC and ODBC support remains essential. However, when high-performance analytics or modern developer tooling is involved, additional interfaces, such as Arrow Flight, may offer compelling advantages.

### **8.4.2 Arrow Flight**

Arrow Flight is a modern RPC-based protocol designed for high-performance data transfer. Built on Apache Arrow's columnar memory format, it is optimized for large-scale analytical workloads and interactive applications that require low-latency access to high-volume datasets.

Unlike JDBC and ODBC, which serialize and deserialize data row by row, Arrow Flight uses efficient binary transport over gRPC to stream columnar data directly between systems, as illustrated in figure 8.3. This significantly reduces overhead and enables data to be processed with minimal copying and transformation. For consumption tools that need to retrieve large datasets, such as notebooks, AI platforms, or custom applications, Arrow Flight offers a significant performance advantage.



**Figure 8.3 JDBC transfers data in a row-based format, while Apache Arrow Flight transfers data in a columnar format ideal for analytics.**

In the context of a lakehouse, Arrow Flight plays a strategic role in bridging consumption tools to the query engine. Engines like Dremio and tools in the Arrow ecosystem natively support Flight, making it easier to deliver governed data to developers and data scientists working in Python, R, or other programming environments. When integrated into the architecture, Flight enables scalable, interactive access to Iceberg tables without the bottlenecks of legacy interfaces.

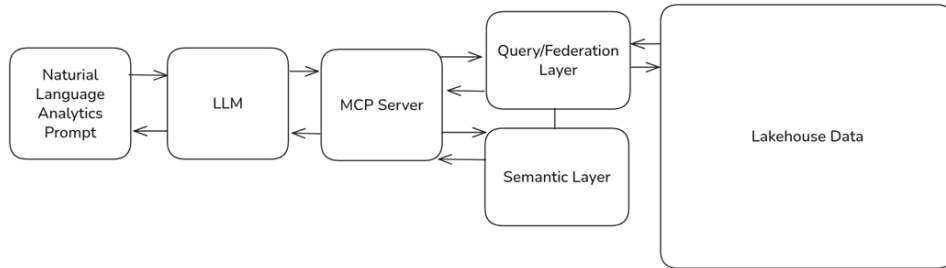
Another advantage of Arrow Flight is its extensibility. It can carry authentication tokens, handle parallel streams, and interoperate with emerging standards such as Arrow Flight SQL, which introduces SQL semantics into the protocol for broader tool compatibility.

While adoption of Arrow Flight is still growing, its performance characteristics and tight integration with the broader Arrow ecosystem make it an increasingly valuable interface for modern consumption patterns. It complements, rather than replaces, traditional interfaces, providing a fast path for data-intensive tools without sacrificing the interoperability that JDBC and ODBC provide.

### 8.4.3 Model Context Protocol (MCP)

As AI applications become central to data consumption, traditional interfaces like JDBC or Arrow Flight are no longer sufficient on their own. These protocols enable access to structured data, but they don't address the broader challenge of enabling AI agents to act on that data, retrieve real-time insights, or automate business workflows. The Model Context Protocol (MCP), introduced in late 2024, addresses this gap by creating a standardized way for large language models (LLMs) to interact with external data, tools, and services.

MCP enables AI systems to move beyond static prompts and become dynamic actors within a data platform. Instead of hallucinating answers or relying on outdated training data, a data platform equipped with MCP can query live Iceberg datasets, trigger actions through APIs, or retrieve governed metrics from a semantic layer, as illustrated in figure 8.4. This positions MCP not just as another interface, but as a protocol that activates the consumption layer for AI-native workflows.



**Figure 8.4 A natural language prompt can have more power to query and transform data using MCP servers that allow LLMs to take actions on your lakehouse.**

In the context of a lakehouse, MCP enables LLMs to integrate with the query engine, access Iceberg tables, and use shared definitions through standardized, secure function calls. For example, an AI assistant could utilize MCP to identify available datasets, issue a query via a federation engine like Dremio, and summarize the results, all while adhering to role-based access controls and semantic definitions.

Unlike retrieval-based systems that simply augment prompts with external text, MCP supports two-way interaction. An LLM can ask for real-time data, invoke specific functions (such as generating a forecast or triggering a downstream process), and receive structured responses. This enables organizations to integrate AI into operational systems, workflows, and decision-making processes while maintaining full governance and transparency.

For Lakehouse architects, supporting MCP means preparing your platform to expose capabilities, queries, metrics, or actions as callable tools. These tools must be registered with MCP servers and designed to respect access policies, monitor usage, and ensure data privacy. Like with other interfaces, this may involve wrapping existing APIs, exposing semantic-layer endpoints, or instrumenting federation engines to respond to MCP calls.

As adoption of LLMs accelerates, the ability to support MCP will define whether a lakehouse can meet the demands of AI-powered consumption. It offers a forward-looking, extensible path for integrating intelligent agents directly into the data platform, unlocking new possibilities for automation, self-service, and insight generation.

## 8.5 Business intelligence tools in the lakehouse

Business Intelligence (BI) tools remain one of the most important access points to data in modern organizations. Executives, analysts, and operational teams all rely on dashboards, reports, and visualizations to monitor performance, track KPIs, and make strategic decisions. In traditional architectures, BI tools were often tightly coupled to specific data warehouses or required extensive ETL (Extract, Transform, Load) pipelines to stay up to date. With a well-architected lakehouse, that relationship changes.

By centralizing data in Iceberg tables and exposing it through open, federated interfaces, the lakehouse enables the decoupling of BI tools from specific storage systems. Dashboards can query data in place without replication. Semantic layers can enforce shared definitions of metrics, ensuring consistent results across teams. Access controls, caching, and query optimization can all be managed centrally, regardless of the visualization platform in use.

This section explores the evolving role of BI tools in a lakehouse context. We'll examine the categories of tools available, starting with open source and moving through commercial and embedded offerings. Along the way, we'll highlight how each class of tool for consumption can connect to the lakehouse query engine, what capabilities to evaluate, and how to make BI consumption scalable, governed, and user-friendly.

### 8.5.1 Open source BI tools

Open-source BI tools offer organizations flexibility, transparency, and cost control, making them an appealing choice for teams building on open data architectures, such as the lakehouse. While they may lack some of the enterprise polish of commercial platforms, many have matured significantly and provide robust functionality for dashboarding, visualization, and data exploration.

**Apache Superset** is one of the most prominent open-source BI tools. It supports a wide range of SQL-speaking backends and connects easily to query engines like Trino or Dremio through standard interfaces such as JDBC. Superset offers an intuitive UI for creating charts and dashboards, along with role-based access controls and lightweight semantic modeling. When used in a lakehouse, Superset can directly query Iceberg tables through the federation layer, benefiting from caching and centralized metric definitions.

**Metabase** is another popular option, particularly valued for its user-friendly experience and ease of deployment. It enables teams to build dashboards with minimal SQL knowledge, making it ideal for non-technical users. Metabase supports embedding, alerts, and parameterized dashboards, which extend its usefulness across different business units. Like Superset, Metabase connects to federated query engines, allowing it to serve real-time data from Iceberg tables without complex ETL workflows.

**Lightdash** and **Redash** also contribute to the open-source BI landscape, each with strengths in collaborative analytics and lightweight deployment. These tools typically emphasize direct SQL authoring and transparency, enabling analysts to understand and own the logic behind dashboards.

When evaluating open source BI tools in the context of a lakehouse, consider:

- **Interface compatibility:** Ensure the tool supports JDBC or other open interfaces exposed by your Apache Iceberg supporting Query Engine/Federation Layer.

- **Security and governance:** Verify that the tool can integrate with authentication systems and honor row- and column-level permissions.
- **Caching and performance:** Evaluate whether the tool can leverage query acceleration features offered by the federation engine.
- **Extensibility:** Consider how easily the tool can be extended, embedded, or customized to meet your organization's needs.

Open source BI tools provide a strong foundation for analytics in a lakehouse architecture. Their modular design and community-driven development align well with the open standards and flexibility that define the modern data platform.

## 8.5.2 Commercial BI tools

Commercial BI platforms provide polished user experiences, extensive enterprise features, and integrated support, making them a popular choice for large organizations with extensive analytics requirements. When paired with a lakehouse architecture, these tools can use open interfaces and federation layers to access governed Iceberg tables without requiring data duplication or complex integration.

**Tableau**, long known for its powerful visualization capabilities, integrates with a wide range of data sources using JDBC and ODBC. In a lakehouse context, Tableau can connect to query engines like Trino, Dremio, or Snowflake to retrieve data from Iceberg tables. Organizations can use Tableau's rich charting features, dashboard interactivity, and user-level permissions while relying on the federation layer to apply semantic consistency and enforce access controls.

**Power BI**, Microsoft's analytics platform, offers deep integration with the broader Microsoft ecosystem and supports direct querying of Iceberg tables via compatible connectors. Through integration with services like Azure Data Lake and tools such as Microsoft Fabric, Power BI can access federated datasets and incorporate real-time analytics. Power BI also supports semantic modeling and row-level security, making it suitable for scenarios requiring detailed governance and report standardization.

**Looker**, now part of Google Cloud, offers a different model centered around its LookML language, a semantic modeling layer that defines metrics and dimensions in a shared repository. When used with a lakehouse, Looker can connect to federated query engines and access Iceberg data while ensuring that all metrics are defined consistently. This approach aligns closely with the goals of a semantic layer, helping to reduce metric sprawl across teams.

**Qlik Sense**, **ThoughtSpot**, and **Sigma Computing** are additional commercial platforms that bring unique strengths, ranging from natural language querying to spreadsheet-style interfaces and real-time data exploration. Each can connect to federated engines through standard interfaces and deliver fast, governed access to Iceberg-backed datasets.

When incorporating commercial BI tools into a lakehouse, consider:

- **Native connector availability** for your federation or semantic layer.
- **Support for centralized semantic definitions** and governed metric layers.
- **Licensing and deployment flexibility**, particularly in hybrid or multi-cloud environments.
- **Scalability and concurrency**, to ensure smooth performance under load.



Commercial BI tools can deliver high impact when paired with a lakehouse architecture that centralizes data, defines shared logic, and exposes clean interfaces. They offer the performance and reliability expected by business users while benefiting from the openness and extensibility of the lakehouse model.

## 8.6 Tools for AI and machine learning workloads

As organizations seek to operationalize data science and AI, the lakehouse serves as a unifying foundation for both experimentation and production. Unlike traditional pipelines that require exporting data to bespoke ML platforms, a well-architected lakehouse keeps data centralized and accessible, streamlining the path from raw data to intelligent applications.

Modern AI workflows typically begin in notebook environments where data scientists explore, clean, and engineer features. These workflows then progress to model training, validation, and deployment using ML frameworks like TensorFlow, PyTorch, or Scikit-learn. Lakehouses make these steps more efficient by exposing Iceberg tables through open interfaces and ensuring that data versioning, schema evolution, and time travel are built in from the start.

Several categories of tools support AI workloads on a lakehouse:

- **Notebook Platforms** such as JupyterLab, Deepnote, and Databricks Notebooks provide interactive environments for exploration. These tools often support direct connections to federated engines or can query Iceberg tables via connectors like DuckDB or Pylceberg, reducing the need for data export or duplication.
- **Feature Stores**, including Feast and Tecton, manage engineered features across the model lifecycle. When integrated with a lakehouse, feature stores can read directly from Iceberg tables, publish versioned feature sets, and maintain lineage, critical for reproducibility and governance.
- **Model Training and Orchestration Platforms** such as Vertex AI, SageMaker, and MLflow coordinate training at scale and monitor model performance. These platforms benefit from the consistency and efficiency of querying curated Iceberg data, especially when paired with federation engines that support predicate pushdown and partition pruning.
- **Vector Stores**, such as FAISS, Weaviate, and Pinecone, store and retrieve high-dimensional embeddings for similarity search and retrieval-augmented generation (RAG). When integrated with a lakehouse, vector stores can link embedding metadata to Iceberg tables, enabling hybrid queries that combine structured filtering with semantic search, essential for powering AI applications, such as recommendation systems and LLM-driven agents.
- **AI Agents and LLM Integrations** are an emerging category. With tools like LangChain, Haystack, and applications built on the Model Context Protocol (MCP), AI agents can now retrieve live data from Iceberg tables, issue dynamic queries, and interact with business systems, enabling everything from automated reporting to data-driven decision-making.

To support these tools, the lakehouse must ensure:

- **Access via standard interfaces** like JDBC, Arrow Flight, and REST APIs.
- **Fine-grained security and observability**, especially when models access sensitive data.
- **Reliable, reproducible data states** via snapshot and schema control.
- **Interoperability with compute environments**, including Spark, Ray, and Kubernetes-based ML stacks.

By meeting these requirements, the lakehouse enables not just data storage but end-to-end AI development. Teams can build more innovative applications, deploy faster experiments, and automate decision-making, all while maintaining a single source of truth.

## 8.7 Choosing the right consumption tools: Ten illustrated scenarios

Selecting the right tools for data consumption is not a one-size-fits-all decision. Each organization brings its own set of priorities, constraints, and users to the table, ranging from startup data science teams working entirely in notebooks to large enterprises that balance the needs of multiple departments and compliance requirements. Rather than prescribing a universal solution, this section presents a series of realistic scenarios designed to illustrate how different consumption tools can align with varying needs within a lakehouse architecture.

These ten examples do not represent idealized blueprints, but rather practical decision paths shaped by context. Each scenario highlights how specific business goals, technical requirements, and team preferences lead to different tool selections. Whether it's a retail company building customer-facing dashboards or a biotech startup training models on massive datasets, the underlying theme is the same: flexibility, openness, and alignment with the broader lakehouse architecture are key.

As you read through these examples, consider how they reflect or differ from your own environment. The goal is to help you think critically about your options, not just which tool to choose, but how to ensure it integrates effectively with the semantic, federation, and catalog layers already in place.

Let's explore the first scenario.

### 8.7.1 Startup with a data science focus

A small AI startup is building a predictive analytics product based on customer behavioral data. The core team comprises machine learning engineers and data scientists who primarily work in Python, utilizing Jupyter notebooks and libraries such as Pandas, Scikit-learn, and PyTorch. The company has chosen Apache Iceberg as its core table format and uses DuckDB and Spark for processing.

#### KEY REQUIREMENTS:

- Interactive, low-latency access to curated and raw datasets
- Programmatic control from within notebooks
- Reproducibility of training datasets

- Minimal infrastructure overhead

### **CONSUMPTION TOOL SELECTION:**

Instead of adopting a traditional BI tool, the team builds its consumption layer around JupyterLab and integrates DuckDB and Pylceberg directly within the notebooks. This allows them to query Iceberg tables using SQL and manipulate data in-memory with Arrow-backed dataframes. For model development and tracking, they add MLflow to capture metadata and version outputs.

### **LAKEHOUSE INTEGRATION:**

The notebook environments access Iceberg tables through direct catalog-based reads and Spark sessions orchestrated via lightweight jobs. Versioned access to data is enforced using Iceberg's snapshot capabilities, ensuring that training workflows remain reproducible. The notebooks access the catalog to allow dynamic discovery of datasets and security.

### **WHY THIS WORKS:**

This setup gives the team full flexibility without over-engineering the stack. They avoid building dashboards or defining metrics prematurely and instead focus on iterating quickly with the tools they already use. The lakehouse provides a structured foundation, while their lightweight consumption layer aligns with their size, skill set, and pace of development.

Next, we'll examine a very different case: a large financial institution dealing with strict governance and wide tool adoption.

## **8.7.2 Large financial institution with strict governance**

A global bank has dozens of teams across various departments, including compliance, trading, and customer analytics. Each team uses its preferred BI tool, such as Tableau, Power BI, or Excel, and strict regulatory requirements mandate comprehensive auditing, access control, and metric consistency across the organization.

### **KEY REQUIREMENTS:**

- Centralized governance of data definitions and user access
- Interoperability across many BI platforms
- Auditable, traceable data access for compliance
- Scalability for hundreds of concurrent users

### **CONSUMPTION TOOL SELECTION:**

Instead of standardizing on a single BI platform, the institution adopts a federated semantic layer using a tool like AtScale or dbt Semantic Layer. This semantic layer exposes consistent metrics and definitions across all BI tools through standard JDBC/ODBC connections. Consumption tools are selected based on team preferences and license availability, while metric governance is handled centrally.

## LAKEHOUSE INTEGRATION:

The semantic layer queries Iceberg tables through a query engine such as Trino or Dremio, which enforces role-based access control and supports row- and column-level security. Iceberg's time travel and schema evolution features ensure data integrity over time, while catalog policies govern who can discover and query specific datasets.

## WHY THIS WORKS:

This architecture balances flexibility and control. Different teams can continue using familiar tools without compromising consistency or compliance. The semantic layer ensures that a KPI, such as "Net Exposure," means the same thing across Tableau and Excel, and audit logs track exactly who accessed what, when, and how. The lakehouse underpins this with strong governance and interoperability, making the system robust and extensible.

In the next scenario, we'll shift to a mid-sized e-commerce company focused on building embedded analytics for its customers.

### 8.7.3 Mid-sized e-commerce platform building embedded analytics

A growing e-commerce company aims to provide embedded analytics to its merchants, enabling them to view product performance, inventory levels, and customer trends directly within their merchant dashboards. The analytics must be responsive, multi-tenant, and aligned with the company's data governance policies.

## KEY REQUIREMENTS:

- Embedded analytics with real-time responsiveness
- Multi-tenant data isolation
- Ability to customize dashboards per merchant
- Low operational complexity for scaling across customers

## CONSUMPTION TOOL SELECTION:

The company selects a BI platform with strong embedded analytics capabilities, such as Metabase or Sigma. These tools support iframe or API-based embedding, allow white-label customization, and expose multi-tenant controls. Metabase is chosen for its open-source core and strong SQL-based dashboarding, with each merchant's data filtered via row-level security controls.

## LAKEHOUSE INTEGRATION:

All merchant data is ingested into Iceberg tables and organized using partitioning and metadata tagging to support tenant isolation. The federation layer, powered by Dremio, applies access filters and authenticates dashboard queries. JDBC connectors allow Metabase to serve interactive dashboards while caching common queries for performance. All metrics are defined in a lightweight semantic layer to ensure consistency across tenant dashboards.

## WHY THIS WORKS:

This architecture supports embedded use cases with full data isolation and performance optimization. Iceberg ensures consistency, version control, and scalability as new merchants are onboarded. The BI layer provides a clean and intuitive user experience, while developers retain full control over access, styling, and data governance. The result is a seamless extension of the platform's core functionality through embedded analytics.

Next, we'll explore the case of a media company enabling self-service analytics across a highly decentralized organization.

### 8.7.4 Decentralized media organization enabling self-service analytics

A multinational media company operates dozens of brands, each with its own independent editorial, marketing, and operations teams. While central IT governs infrastructure and compliance, each brand maintains autonomy over its analytics practices. The company's goal is to enable self-service analytics across all brands while maintaining consistent data definitions and observability.

## KEY REQUIREMENTS:

- Support for diverse BI tool preferences across teams
- Central governance over metrics and access controls
- Easy onboarding for non-technical users
- Visibility into query usage and data access patterns

## CONSUMPTION TOOL SELECTION:

To support diversity and autonomy, the company enables each brand to choose its preferred BI tool, ranging from Looker and Tableau to open-source options like Superset, while centralizing metric logic through a semantic layer using dbt. This layer defines canonical metrics that can be queried via SQL or exposed through federation engines, such as Dremio, which serve as the common query interface for all consumption tools.

## LAKEHOUSE INTEGRATION:

Iceberg is used as the central storage format for all brand data, partitioned and cataloged with lineage metadata. The federation layer applies brand-specific access policies, ensuring that teams only see their data while drawing from a shared infrastructure. Query engines expose JDBC and ODBC endpoints for BI tools, maintaining query logs for centralized observability and optimization.

## WHY THIS WORKS:

By allowing brands to retain their preferred tooling while enforcing common governance, the architecture supports both flexibility and alignment. Teams can move quickly and tailor insights to their workflows, while central IT ensures that key metrics and security controls are consistent. The lakehouse provides the necessary infrastructure for scalable, governed self-service analytics.

Next, we'll examine a government agency that must balance public data transparency with stringent internal controls.

### **8.7.5 Government agency balancing public transparency and internal control**

A national government agency manages a wide range of data, including census statistics, economic indicators, and public infrastructure records. It must provide open access to certain datasets for public transparency, while ensuring strict internal access controls for sensitive data related to national operations and citizen information.

#### **KEY REQUIREMENTS:**

- Public data access via dashboards and APIs
- Strong internal security and auditing capabilities
- Clear separation between public and private data domains
- Long-term data preservation and reproducibility

#### **CONSUMPTION TOOL SELECTION:**

The agency adopts Apache Superset for internal analytics and Looker Studio for publicly accessible dashboards. Superset connects directly to the internal federation engine through a secured network, while public dashboards are powered by cached extracts or controlled query endpoints that expose non-sensitive Iceberg data. Additionally, a custom REST API layer is built for developers and researchers to programmatically access curated public datasets.

#### **LAKEHOUSE INTEGRATION:**

Data is ingested and stored in Iceberg tables across clearly defined domains: "internal," "public," and "restricted." The catalog maintains strict metadata tagging, and access policies are enforced by the federation layer using Trino. All internal queries are logged and auditable, and public-facing queries route through pre-approved views or materialized datasets refreshed on a controlled schedule. Time travel and snapshot isolation ensure that historical views are preserved for policy reporting and the reproducibility of research.

#### **WHY THIS WORKS:**

This architecture allows the agency to fulfill its dual mission of transparency and security. Public users can explore relevant datasets without compromising sensitive data, while internal users benefit from robust tooling and governance. Iceberg provides the necessary data integrity and auditability, and the consumption tools are configured to reflect the agency's clear data boundaries.

In the next scenario, we'll examine a healthcare provider with strict compliance needs and data locality constraints.

### 8.7.6 Healthcare provider with compliance and data locality constraints

A regional healthcare provider operates across multiple jurisdictions, each with unique data residency and compliance regulations. Patient data must be securely stored, with restricted access for analytics and AI. The organization aims to utilize analytics to enhance clinical workflows, optimize operations, and facilitate research, all while adhering to data governance policies.

#### KEY REQUIREMENTS:

- Strict data residency and patient privacy controls
- HIPAA and regional compliance alignment
- Role-based access to sensitive metrics
- Support for both internal dashboards and ML workloads

#### CONSUMPTION TOOL SELECTION:

The provider selects Power BI for clinical dashboards and JupyterHub for research notebooks. Power BI is deployed in a secure, region-specific cloud environment and connects via a compliant federation engine. JupyterHub environments are containerized and region-bound, providing isolated access to authorized datasets. Both tools rely on federated semantic definitions for clinical and operational KPIs.

#### LAKEHOUSE INTEGRATION:

Patient data is stored in Iceberg tables partitioned by geography and department. A region-aware catalog layer tags data based on locality requirements, and the federation engine (Dremio) enforces these policies during query planning. The semantic layer ensures that sensitive metrics, like readmission rates or treatment durations, are defined consistently and exposed only to authorized personnel. Query logging, data masking, and snapshot tracking provide a full audit trail.

#### WHY THIS WORKS:

By integrating Power BI and JupyterHub within a compliant lakehouse architecture, the healthcare provider enables data-driven decision-making while maintaining strict governance and control. Data remains localized, queries are centrally controlled, and all tools consume from the same versioned, policy-enforced datasets. This ensures trust, transparency, and flexibility across analytics and research workflows.

Next, we'll explore a logistics company looking to unify real-time operations with historical trend analysis through its consumption layer.

### 8.7.7 Logistics company unifying real-time operations and historical analysis

A global logistics company manages fleet tracking, inventory systems, and delivery optimization across continents. Operations teams need real-time dashboards to monitor delays and bottlenecks, while analytics and planning teams require access to historical trends to model demand and refine routing algorithms.

## KEY REQUIREMENTS:

- Low-latency dashboards for operational monitoring
- Access to historical data for planning and forecasting
- Consistent data definitions across time horizons
- Integration with geospatial and streaming data sources

## CONSUMPTION TOOL SELECTION:

The company adopts a dual-layered consumption strategy. For real-time dashboards, it utilizes a commercial BI tool with robust live query capabilities, such as Sigma or ThoughtSpot. For historical analysis and planning, it uses Looker connected to a semantic layer that defines standard logistics metrics. In addition, internal geospatial tools and notebook environments access federated Iceberg data via Arrow Flight and DuckDB.

## LAKEHOUSE INTEGRATION:

Operational data is streamed into Iceberg tables using a streaming ingestion framework (e.g., Apache Flink or Kafka Connect with Iceberg integration). Recent data is queryable with low latency, while older partitions are optimized for scan efficiency. The federation layer handles queries from both real-time dashboards and offline planners, exposing a consistent set of views.

## WHY THIS WORKS:

This approach supports both real-time responsiveness and long-term analytical depth. Teams operate from a single source of truth, with Iceberg handling temporal consistency and schema evolution. The use of open interfaces ensures interoperability across the company's diverse tooling, while the semantic layer bridges live operations and strategic planning.

Next, we'll consider the case of a SaaS company enabling end-user reporting through customizable data exports and APIs.

### 8.7.8 SaaS company offering customizable data access to clients

A SaaS company provides a platform for marketing automation and campaign analytics. Enterprise customers expect flexible access to their data, both through self-service dashboards and programmatic APIs for integration into internal systems. The company must deliver a secure, multi-tenant reporting layer without duplicating datasets or building bespoke ETL pipelines for each client.

## KEY REQUIREMENTS:

- Secure, tenant-isolated access to client-specific data
- Support for self-service dashboards and developer APIs
- Scalable delivery of large query results
- Data export options with governance and auditing



## CONSUMPTION TOOL SELECTION:

The company implements Superset as the embedded dashboard interface for business users and builds a RESTful API layer for programmatic access using technologies like FastAPI and Arrow Flight. Both interfaces connect to a federated query engine (e.g., Dremio) that enforces tenant filters and retrieves data from Iceberg tables. Dashboards are pre-configured per client, while developers can query data directly via API using secure tokens.

## LAKEHOUSE INTEGRATION:

Iceberg tables are structured with tenant-aware partitioning and metadata tagging. A central catalog tracks all customer datasets, while the federation layer restricts queries based on the authenticated user's tenant ID. Exports are batched using query result caching or persisted to cloud storage for large dataset retrieval. API usage is logged for auditability and rate-limited to ensure platform stability.

## WHY THIS WORKS:

This configuration provides flexibility and control. Clients can generate dashboards or extract data on their terms without introducing governance risks. The lakehouse ensures that all clients query against the same governed, versioned source of truth, while the delivery layer separates concerns between visualization and automation. It's a modern SaaS architecture that scales with the customer base and the complexity of their data needs.

Next, we'll examine a nonprofit organization that collaborates with external research partners and requires access to auditable, collaborative data.

### 8.7.9 Nonprofit organization supporting collaborative research

A nonprofit focused on climate research partners with universities, government agencies, and private sponsors to analyze environmental data. It must provide collaborators with access to large, shared datasets while ensuring transparency, reproducibility, and careful stewardship of sensitive or unpublished findings.

## KEY REQUIREMENTS:

- Shared, collaborative access to curated datasets
- Transparent and reproducible analysis workflows
- Clear audit trails for data access and transformations
- Flexible support for varied tooling across academic partners

## CONSUMPTION TOOL SELECTION:

The organization standardizes on JupyterHub for notebooks and Apache Superset for internal dashboards. For external researchers, it offers read-only access to Iceberg-backed datasets via Arrow Flight and DuckDB, along with hosted notebooks in a controlled environment. Each project team is assigned access to specific views or snapshots to support versioned, auditable workflows. Git-based workflow tools and MLflow are also integrated to track model and data lineage.

## LAKEHOUSE INTEGRATION:

All research data is ingested into Iceberg tables organized by project and subject domain. A shared catalog manages access policies and supports time-based queries. The federation layer enables researchers to join new datasets with historic versions for longitudinal analysis. Audit logs record when and how datasets are accessed, while metadata tags flag unpublished or embargoed content. Superset provides curated visualizations for broader communication within the organization.

## WHY THIS WORKS:

This design supports open collaboration without sacrificing governance. By relying on open formats and interoperable tools, researchers with diverse skill sets and environments can collaborate productively. Iceberg ensures that findings are reproducible over time, while the catalog and query layers enforce appropriate access. It's an architecture that promotes transparency, reuse, and scientific rigor.

Next, we'll explore the final scenario: a manufacturing company operationalizing predictive maintenance across its plants with the help of a unified consumption layer.

### 8.7.10 Manufacturing company enabling predictive maintenance

A multinational manufacturer operates several production facilities with extensive sensor instrumentation. The company is implementing predictive maintenance initiatives to minimize downtime and enhance equipment efficiency. This requires unifying time-series sensor data with maintenance logs, production schedules, and supply chain information, while delivering insights to engineers, analysts, and machine learning teams.

## KEY REQUIREMENTS:

- Integration of real-time and historical data
- Support for time-series analytics and model development
- Data access for operations teams and data scientists
- Governance to ensure data quality and lineage tracking

## CONSUMPTION TOOL SELECTION:

The company adopts a hybrid model. Grafana is deployed to monitor real-time equipment metrics with alerting capabilities. Apache Superset is used for exploratory dashboards and reports, while data scientists use JupyterHub for developing models to predict failures and optimize schedules. All tools connect to a federated engine that provides governed access to Iceberg-managed sensor and maintenance datasets.

## LAKEHOUSE INTEGRATION:

Sensor data streams into Iceberg tables using Flink or Kafka integrations, with schema evolution and partitioning configured for time-series analysis. Historical logs and reference datasets are ingested through batch pipelines. The catalog organizes all assets with metadata describing equipment types, data quality, and update frequency. Feature engineering occurs in notebooks connected to the same data layer as operational dashboards, ensuring alignment across domains.

## WHY THIS WORKS:

This architecture supports the end-to-end lifecycle of predictive maintenance, from real-time detection to long-term trend modeling. Engineers gain visibility through Grafana dashboards, while data teams work with consistent, reproducible datasets using familiar tooling. The lakehouse ensures that all consumers interact with high-quality, versioned data while enabling innovation through open interfaces and interoperable formats.

With these ten scenarios, we've demonstrated how diverse organizational needs can be met through careful selection of consumption tools aligned with the core lakehouse architecture. In the next section, we'll summarize how the pieces of the lakehouse come together and how consumption tools play a pivotal role in delivering value to data users.

As we move into the next chapters, we'll begin to explore how to operate and scale this lakehouse environment, ensuring not only that your data is accessible but also secure, performant, and sustainable for the long haul.

## 8.8 Summary

- The consumption layer connects your data to your people, where insights, decisions, and automation take place.
- With Iceberg as the table format, you avoid the need to duplicate data across tools and platforms.
- Open interfaces (JDBC, ODBC, Arrow Flight, MCP) ensure that tools can connect to your federation layer reliably and consistently.
- BI tools, notebook environments, ML platforms, and APIs each have unique roles; the lakehouse allows them to coexist on a single data foundation.
- A semantic layer enables shared metric definitions across tools, supporting governance and reducing inconsistency.
- Scenarios across industries show how flexible the consumption layer can be when anchored in an open data infrastructure.

- Iceberg's time travel, schema evolution, and partitioning features help maintain performance and reproducibility across all consumption modes.

## 9 *Maintaining an Iceberg lakehouse*

### **This chapter covers**

- Identifying and resolving performance issues from suboptimal data and metadata files
- Running compaction jobs to optimize file layout and improve query speed
- Managing snapshot retention to reduce storage footprint and meet compliance needs
- Using Iceberg metadata tables to monitor table health and guide maintenance

Designing and deploying a lakehouse is only the beginning. Long-term value comes from keeping the platform performant, governed, and resilient over time. Apache Iceberg provides powerful capabilities for data organization, schema evolution, and transaction isolation, but without proactive maintenance, those strengths can erode. As datasets grow, write patterns evolve, and business needs change, the lakehouse must adapt.

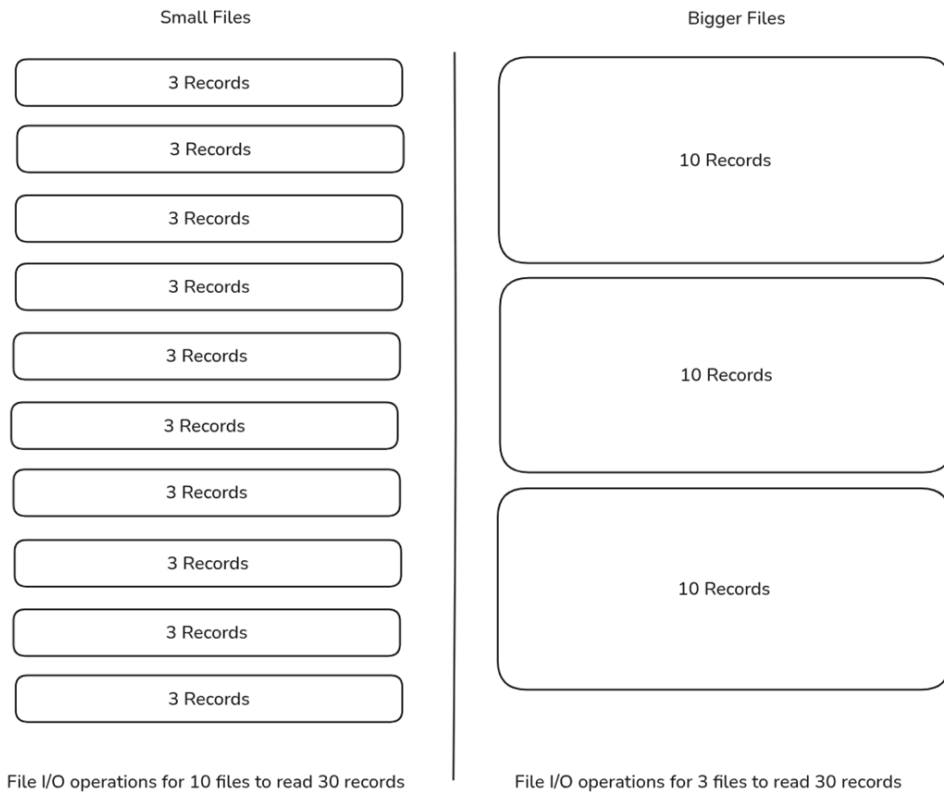
This chapter focuses on the operational practices that keep your Apache Iceberg lakehouse healthy and performant over time. Unlike traditional data warehouses, where many optimization tasks are handled automatically by the engine, Iceberg-based lakehouses place more responsibility on engineers to manage data and metadata layout. Without proactive maintenance, performance can degrade, storage costs can grow unnecessarily, and governance requirements may be harder to enforce. We begin by identifying common issues that cause this degradation, such as small files, scattered partitions, and metadata sprawl, and explain why these are especially important in the context of Iceberg's table format. From there, we explore techniques for compaction, snapshot management, and resource optimization.

By the end of this chapter, you'll have a practical framework for maintaining your Iceberg lakehouse, one that helps you avoid pitfalls, meet SLAs, and support ongoing innovation without technical debt.

## 9.1 Problem: Suboptimal data files

Even the most carefully architected Iceberg lakehouse can degrade over time if data files are not managed properly. As new records are ingested, updated, or deleted, the layout and organization of files evolve, often in ways that introduce inefficiencies. These inefficiencies may not cause immediate failures, but they can significantly affect query performance, metadata overhead, and system stability.

This section outlines the most common data layout issues that arise in production Iceberg tables. You'll learn how small files increase file I/O as illustrated in figure 9.1, how poorly colocated data leads to inefficient scans, and how excessive metadata accumulation can strain the catalog. We also examine the impact of Merge-on-Read (MOR) patterns on read performance as delete files accumulate. Recognizing these problems is the first step toward maintaining a fast and cost-effective lakehouse.



**Figure 9.1** More file I/O operations are needed when the same data is stretched across a bigger number of files, resulting in slower performance. The fewer IO operations, the faster the query can complete.

### 9.1.1 Small files

One of the most common performance pitfalls in an Iceberg lakehouse is the accumulation of small data files. These typically result from frequent streaming or micro-batch ingestion jobs that write data in small increments. While Iceberg handles these writes efficiently and transactionally, the long-term consequence is a table composed of many small Parquet, ORC, or Avro files rather than a few well-sized ones. You can detect this condition by inspecting the Iceberg metadata tables, which we cover in detail later in this chapter. For example, querying the files metadata table can reveal the distribution of file sizes across your table. Tables with a high count of files under 128 MB may be experiencing this issue and would benefit from compaction.

This matters because most query engines, such as Spark, Trino, and Dremio, must open and scan each file individually. The overhead of locating, opening, and processing a large number of small files can quickly dominate the total query time, especially in cloud storage systems where each file read incurs a network, API and metadata cost. Instead of reading a few large files efficiently, the engine must spend time managing thousands of small ones.

Moreover, small files also contribute to metadata bloat. Iceberg tracks each data file's metadata, including its path, statistics, and partition information. A table with tens of thousands of files will require the catalog to manage and scan proportionally larger manifest and metadata files, increasing planning latency and coordination costs.

Small files are not inherently bad; low-latency ingestion workloads often require them, but they must be actively managed. In later sections, we'll discuss compaction as a key strategy for consolidating these files into optimized segments that preserve performance without disrupting data freshness or consistency.

### 9.1.2 Poorly colocated data

Efficient queries depend not just on the size of individual files, but also on how relevant data is physically grouped. When a table's rows are poorly colocated, meaning that related records are scattered across many files, as illustrated in figure 9.2, query engines must scan more files and process more irrelevant data to retrieve results. This phenomenon is widespread when partitioning, sorting or clustering strategies are misaligned with query patterns.



**Figure 9.2** If you were to search all “yellow” records, you’d have to read all three files, which contain other unnecessary data for the query.

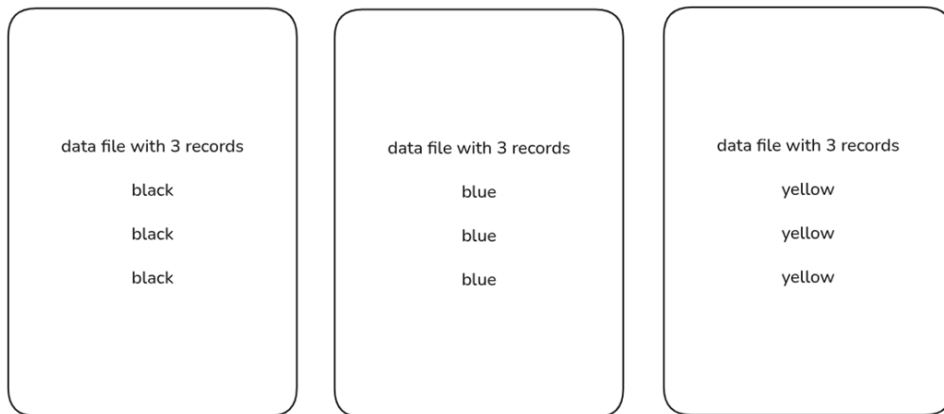
For example, consider a sales dataset partitioned by region but frequently queried by customer\_id or product\_category. Because the partition column does not match the filter condition, query engines cannot effectively prune irrelevant files. Instead, they must read from many partitions to locate records associated with a specific customer or product. This increases file I/O, CPU usage, and overall query latency.

Even within a single partition, poorly colocated data can lead to inefficiencies. Ingested records may arrive in a semi-random order, especially in streaming or distributed write scenarios, resulting in a lack of locality. This disrupts column-level compression and impairs the query engine's ability to skip data using file or row-group statistics. The outcome is predictable: longer queries, higher costs, and wasted resources.



Colocation problems often emerge gradually. A table may start out well-partitioned, but evolving data volumes and access patterns shift over time. Additionally, updates and deletes can fragment records further, especially if performed via Merge-on-Read semantics that append delete files rather than rewriting data.

Fixing colocation issues requires rethinking how data is grouped during ingestion and leveraging Iceberg's flexible partition evolution. Iceberg allows you to change partition strategies over time, applying more appropriate partitioning schemes without rewriting historical data. Combined with compaction jobs that cluster similar records into fewer files, you can dramatically improve data locality and minimize unnecessary scans through better colocation of data, as illustrated in figure 9.3.

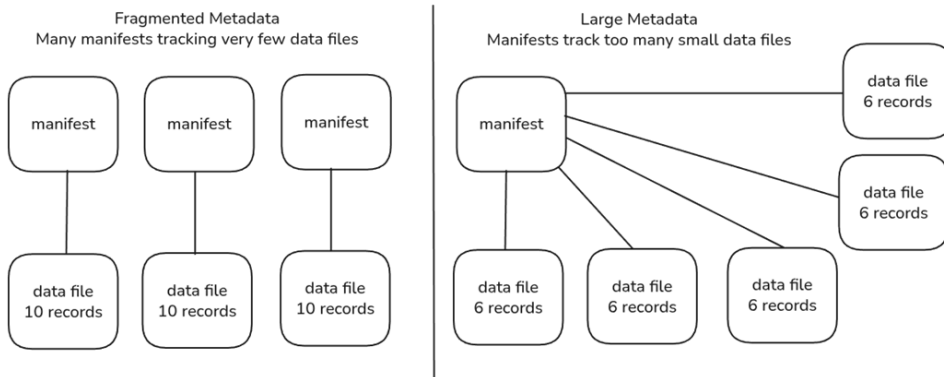


**Figure 9.3 with better colocation of data, a query for all “yellow” records only needs to read one file instead of all three files. Partitioning allows records with unique values to be written to distinct data files.**

The key is to align physical data layout with logical access patterns. When data that is queried together is also stored together, performance gains can be substantial. Monitoring query behavior and adjusting data organization accordingly is a core part of long-term lakehouse maintenance.

### 9.1.3 Metadata sprawl

Even when data files are properly sized and reasonably colocated, metadata sprawl can silently degrade performance. This condition arises when the metadata layer, specifically the manifest files that track data files, becomes fragmented or excessively large, as illustrated in figure 9.4. Over time, this slows down query planning, and increases coordination overhead for both reads and writes.



**Figure 9.4 Metadata bloat occurs when you have more manifests than necessary relative to the number of data files, or you have manifests tracking too many small data files. Both of these will result in delays of query planning.**

To understand how metadata sprawl develops, and when it becomes a concern, it helps to first look at how Apache Iceberg structures table metadata. Each snapshot in Iceberg points to one or more *manifest lists*, which reference a set of *manifest files*. Each manifest file, in turn, tracks a group of data files, storing details like partition values, statistics, and file-level metadata. In a healthy scenario, manifest files are reused across multiple snapshots, and each manifest covers a substantial number of data files. This layered structure keeps metadata compact and efficient. However, not all metadata growth is harmful. When tables grow in size or experience legitimate schema or partition evolution, an increase in metadata is expected. Metadata becomes problematic when the number of manifest files expands disproportionately, often due to high-frequency ingestion jobs that write only a few files at a time or frequent commits that generate redundant manifests. In these cases, metadata expansion no longer reflects meaningful table growth but instead signals fragmentation, which increases query planning time and catalog overhead.

However, certain write patterns can produce too many small manifest files that can effect performance. For example, high-frequency writes that append just a few data files per job often result in new manifest files for each write. Similarly, streaming or near-real-time pipelines may commit changes incrementally, leaving behind a trail of manifests that each describe a small portion of the table. Over time, thousands of manifest files may accumulate, with many containing overlapping data file references.

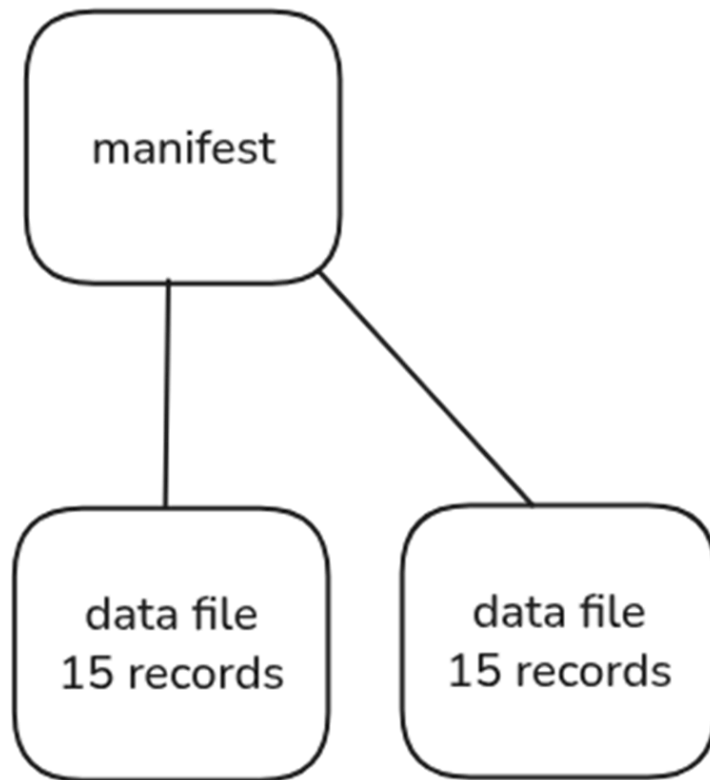
This sprawl inflates the planning cost of every query. When a query engine wants to scan a table, it must load the latest snapshot metadata, traverse all relevant manifest lists, read each manifest file, and finally determine which data files match the query filters. As the number of manifests grows, so does the work needed to plan each query, even if the data file layout remains efficient.

Metadata sprawl can also affect table operations like snapshot expiration, rollback, or schema evolution. Each of these actions must navigate the manifest structure, and bloated metadata increases the cost of these operations. In large tables, the impact may be measured in seconds or minutes rather than milliseconds, adding friction to routine tasks.

Addressing metadata sprawl involves more than just compacting data files. You must also periodically rewrite manifests to consolidate redundant or fragmented metadata. Iceberg provides built-in procedures for this purpose, such as `RewriteManifests` in Spark or implicit metadata rewriting via the `OPTIMIZE` command in tools like Dremio. These procedures help prune excess manifests and align metadata boundaries with logical partitions. Refer to figure 9.5 for a more ideal metadata layout.

## Ideal

### Less Manifests tracking less data files



**Figure 9.5** Ideally, you have fewer manifest files tracking fewer data files containing a greater number of records per file, minimizing the cost of metadata traversal and data file IO.

## WHY ICEBERG USES A MANIFEST-BASED METADATA STRUCTURE

At first glance, the layered metadata structure in Iceberg—manifest lists pointing to manifest files, which in turn point to data files, may seem like an unnecessary source of complexity or overhead. But this design plays a central role in enabling key features of Iceberg: ACID guarantees, time travel, and compatibility across engines.

Each snapshot in Iceberg is immutable and fully self-describing, which allows operations like rollback, branching, and concurrent reads and writes. To support this, metadata must capture the full state of the table at each point in time. Rather than rewriting all metadata from scratch, Iceberg builds snapshots from reusable manifest files that describe subsets of the data. This reuse keeps metadata growth manageable for most workloads and enables snapshot-level consistency.

However, this structure does come with trade-offs. In workloads with frequent writes or streaming ingestion, new manifests can accumulate quickly, even if the data changes only incrementally. This leads to metadata sprawl and increased planning costs, which is why Iceberg includes tools like `RewriteManifests` and `RewriteDataFiles` to clean up and consolidate data and metadata periodically.

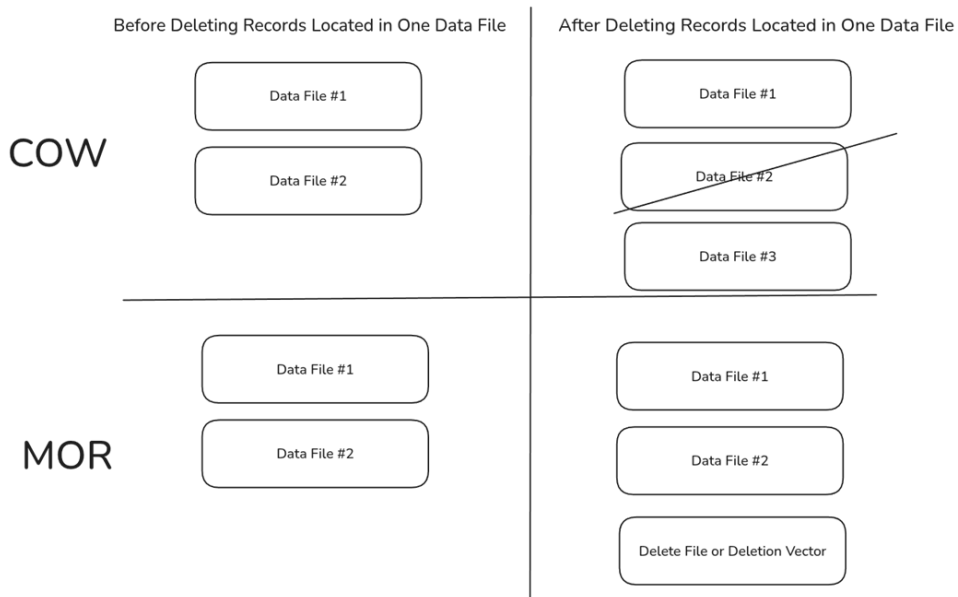
The Iceberg community is actively exploring improvements in this area. For example, the upcoming v4 specification includes proposals to restructure metadata handling so that only modified parts of the metadata tree are recreated during a commit. These enhancements aim to reduce commit latency, minimize catalog writes, and better support high-frequency and streaming workloads.

Iceberg's manifest structure reflects a careful balance: strong transactional guarantees and openness come at a cost, but ongoing innovations continue to reduce that cost while preserving its architectural strengths.

Regular metadata compaction, combined with sensible ingestion practices, helps maintain a balanced metadata structure. Auditing and managing manifests ensures that table operations remain fast, even as the dataset grows. As with data file optimization, the goal is to reduce unnecessary work for query engines and catalog clients, keeping your lakehouse agile, responsive, and cost-effective.

### 9.1.4 Merge-on-read (MOR) performance hits

Merge-on-Read (MOR) is a powerful feature in Apache Iceberg that enables efficient handling of updates and deletes by writing changes as incremental “delete files” rather than rewriting entire data files. While this approach provides high write throughput and minimizes data churn, it comes with a tradeoff: as delete files accumulate, read performance can degrade significantly. To see the behavior of MOR vs the default of Copy-on-Write (COW), refer to figure 9.6.



**Figure 9.6** In Copy-on-Write, when you delete records, the data file with those records is rewritten without them. In Merge-on-Write, instead of rewriting a whole file, the records deleted are tracked in a delete file or deletion vector, which can be used to reconcile when the table is read, which enables faster writing to the table but slows down reading.

In an MOR table, each read operation must scan the base data files and then apply all relevant delete files to reconstruct the current view of the data. This dynamic merging increases query complexity and slows execution, particularly when the number of delete files per data file grows. In high-churn workloads, such as frequent upserts or time-sensitive updates, this overhead can become a major bottleneck for both read and writing operations.

---

**NOTE** As Iceberg V3 support grows, the same dynamic will apply, but using Deletion Vectors instead of Delete files. Delete Files applies to Iceberg V2 tables.

---

The problem compounds over time. Delete files are typically written in an append-only fashion, meaning older base files may be associated with dozens or even hundreds of delete files. Each of these files must be loaded, parsed, and applied during read execution, straining the query engine and reducing the effectiveness of data skipping and predicate pushdown.

A common misconception is that avoiding MOR and using COW mode will solve these issues. In COW mode, updates and deletes result in rewriting entire data files, eliminating the need for delete files altogether. While this does simplify reads, it shifts the cost to write operations, which can become significantly more expensive, especially in large tables with frequent changes. In fact, the choice between MOR and COW should be made based on workload characteristics, not as a shortcut to avoid maintenance.

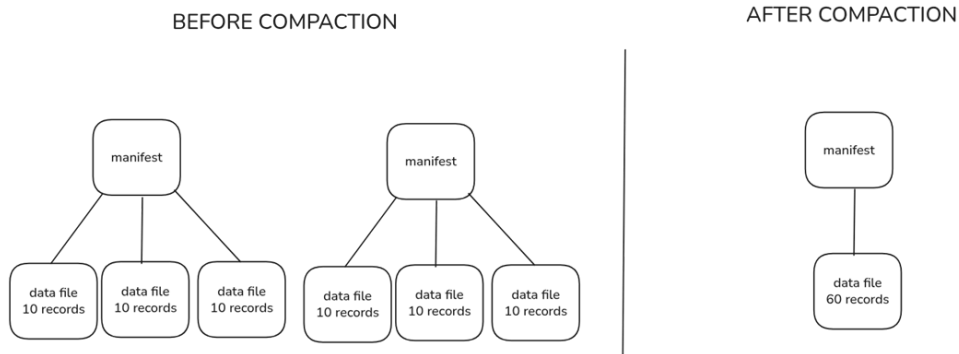
The proper solution for managing MOR performance is compaction. Iceberg supports rewriting data and delete files into fresh, optimized base files using procedures like `RewriteDataFiles`. During compaction, delete files are applied to their corresponding data files, and the result is written as new base files that no longer require dynamic merging. This restores read performance without abandoning the efficiency of MOR for incoming writes.

Effective compaction strategies consider both the volume of delete files and the freshness of the data. For example, you may choose to compact partitions with high delete activity more frequently, while deferring less active regions to conserve resources. Tools like Apache Spark and Dremio offer mechanisms to target compaction based on partition, file size, or timestamp, allowing for incremental cleanup without disrupting active workloads.

MOR enables scalable, write-efficient tables, but must be paired with a compaction strategy to maintain read performance. Ignoring compaction leads to growing latency and wasted resources. Recognizing the signs of Merge-on-Read (MOR) performance degradation, such as slower query execution, elevated query planning latency, a growing number of uncompactable delete files, or rapid table size expansion, is essential for maintaining a high-performing Iceberg lakehouse.

## 9.2 Solution: Compaction

Compaction addresses the problems of small files, scattered data, fragmented metadata, and accumulated delete files by directly reorganizing data and metadata into more efficient structures. Compaction is the process of rewriting Iceberg data files and manifest files to consolidate them, reduce overhead, and restore optimal performance. During compaction, small files are merged into larger ones, delete files are applied to base files in Merge-on-Read tables, rows are sorted within files, and excess manifest files are consolidated. These operations result in a more query-efficient layout with fewer files, better data colocation, and more manageable metadata, as illustrated in figure 9.7.



**Figure 9.7** During compaction, data files will be written so there are fewer larger files, and manifests will be rewritten so fewer manifests are tracking more files. This will improve the performance of tables that have seen degraded performance after many writes.

While compaction improves performance, it also introduces trade-offs. Rewriting files consumes compute resources, may impact concurrent workloads, and must be managed carefully to align with SLAs and throughput constraints. Choosing which files to compact, how large to make the outputs, and when to run compaction jobs are all critical decisions that influence lakehouse efficiency.

In the sections that follow, we will explore the key parameters of compaction: how to determine target file sizes, how to select files for rewriting, and how to use filters and partition-level targeting to run incremental compaction jobs. Together, these strategies form the foundation for a healthy, maintainable Iceberg deployment.

### 9.2.1 What is compaction?

At its core, compaction is a rewriting process that optimizes how data is physically stored to improve read performance and reduce metadata overhead. During compaction, Apache Iceberg can perform several enhancements: merging small or oversized files into more balanced segments, applying better compression, sorting records to improve data locality, consolidating delete files in Merge-on-Read (MOR) tables, and updating file-level statistics. These transformations reduce the number of files that query engines must scan, enhance predicate pushdown, and lower query planning latency, making compaction a key maintenance task for any Iceberg lakehouse.

Iceberg OSS libraries support two main procedures for compaction: `rewrite_data_files` and `rewrite_manifests`. These procedures are available when using Apache Spark with the Iceberg Spark runtime. Other engines may implement these procedures like Trino or similar procedures like Dremio with `OPTIMIZE`. The `rewrite_data_files` procedure supports both bin-pack and sort strategies:

- **Bin-pack** is the default strategy and aims to combine small files into new files of a target size, avoiding costly file opens and metadata overhead.

- **Sort** compaction goes a step further by reordering records based on one or more columns. This improves locality and can accelerate filtered queries. For example, sorting by `event_date` or using Z-order on multiple dimensions improves pruning effectiveness.

Each compaction procedure can be fine-tuned using options such as:

- `target-file-size-bytes` to control the size of output files.
- `min-input-files` and `max-file-size-bytes` to select which files are eligible for rewrite.
- where to limit compaction to specific partitions or filtered data subsets.
- `remove-dangling-deletes` to clean up unused delete files from prior updates.

Here's an example of bin-pack compaction on an Iceberg table using Spark SQL:

```
CALL catalog_name.system.rewrite_data_files('db.sales');
```

Or, using a sort-based compaction with z-ordering:

```
CALL catalog_name.system.rewrite_data_files(
  table => 'db.sales',
  strategy => 'sort',
  sort_order => 'zorder(customer_id, product_id)'
);
```

For environments using Dremio, compaction is exposed through the `OPTIMIZE TABLE` SQL command. This command supports similar tuning parameters:

```
OPTIMIZE TABLE demo.example_table
  REWRITE DATA USING BIN_PACK (TARGET_FILE_SIZE_MB = 512);
```

In both engines, compaction can also include metadata optimization by appending `REWRITE MANIFESTS` in Dremio or calling the `rewrite_manifests` procedure in Spark. While compaction is essential, it's not always necessary to rewrite all files. Selective compaction—such as compacting only a heavily queried partition or files below a certain size, enables incremental optimization that aligns with service-level agreement (SLA) requirements and available cluster capacity. When applied routinely, compaction ensures your Iceberg tables remain lean, query-friendly, and resource-efficient, even as data volume and complexity grow.

```
CALL catalog_name.system.rewrite_manifests('db.sales');
```



## 9.2.2 Target file size

A central decision in compaction planning is determining the target size and structure of the output files, particularly the size of row groups within those files. While reducing the number of files lowers query planning and I/O overhead, oversized row groups can cause query engines to run out of memory or spill to disk, since row groups are not splittable during execution. Files themselves are splittable, but poorly sized row groups limit parallelism and increase resource pressure. Choosing appropriate file and row group sizes directly influences the performance, scalability, and stability of your Iceberg tables.

Apache Iceberg's default target file size is 512 MB, configured via the `write.target-file-size-bytes` table property. While this may be suitable for some large-scale workloads, many production environments find better results targeting smaller files, typically in the 10 to 20 MB range. Smaller files reduce the cost of repeated reads and writes during compaction and align better with the reality that parallelism is driven by row group size, not file size. Overly large files can also slow down compaction and increase memory pressure during reads. Selecting a file size that balances metadata overhead with efficient rewrites is critical to achieving sustainable performance over time.

However, this default is not a one-size-fits-all solution. The ideal target file size depends on several factors:

- **Query engine characteristics:** Some engines handle large files better than others. For example, Dremio and Trino can efficiently scan files up to 1 GB or more, while others may benefit from smaller files that better fit into memory.
- **Compression:** The adequate size of a file on disk depends on compression. Highly compressible data may result in smaller file sizes even when a large number of records are written.
- **Table width and data shape:** Wide tables with many columns or nested structures may benefit from slightly smaller files to avoid excessive memory usage during query execution.
- **Sorting:** On systems with limited executor memory or I/O bandwidth, setting a smaller target file size (e.g., 128–256 MB) may help avoid performance bottlenecks or query failures.

You can explicitly set the target file size for compaction in Spark with:

```
CALL catalog_name.system.rewrite_data_files(
  table => 'db.sales',
  options => map('target-file-size-bytes', '268435456')
);
```

Or in Dremio with:

```
OPTIMIZE TABLE db.sales
  REWRITE DATA USING BIN_PACK (TARGET_FILE_SIZE_MB = 256);
```

It's also important to understand that this target value guides, but does not strictly enforce, output sizes. Factors like record boundaries, compression, and partitioning can cause slight variations.

Monitoring query engine behavior, such as parallelism, I/O throughput, and query execution plans, is the most effective way to validate whether your compaction strategy and file sizes are delivering real performance gains. While Iceberg metadata tables can show whether files are converging on the intended size, this alone doesn't confirm that the engine is benefiting from those changes. Instead, reviewing query plans from engines like Spark, Trino, or Dremio provides direct insight into whether file layout is improving scan efficiency, reducing spill, and maximizing task parallelism.

In practice, it's common to iterate. Start with a default like 10 MB, evaluate query performance and storage efficiency, and adjust the target as needed based on observed results and operational constraints. Maintaining a consistent and well-reasoned file size standard helps your Iceberg lakehouse scale efficiently as data volumes grow.

### 9.2.3 Files to be included

Not all files need to be rewritten during every compaction job. Indiscriminate rewriting can waste resources, increase job duration, and disrupt concurrent writes. Apache Iceberg allows fine-grained control over which files are eligible for compaction by configuring thresholds for file size and file count. These parameters enable compaction to focus effort where it's most needed, striking a balance between optimization and operational efficiency.

Compaction procedures in engines like Spark and Dremio use a file grouping strategy to plan compaction jobs. Rather than evaluating individual files for benefit, the engine first enumerates all eligible files and then groups them into file sets based on configuration limits such as `max-file-group-size-bytes`. These file groups become the units of work for compaction, processed independently to improve parallelism and control metadata growth.

You can influence how these file groups are formed using parameters like:

- `min-input-files` **or** `MIN_INPUT_FILES`: This sets the minimum number of files that must be present in a group for it to be compacted. It primarily serves as a way to control the granularity and parallelism of compaction jobs, not to assess whether compaction is necessary. The actual decision about which files to include is made earlier in the planning phase.
- `max-file-size-bytes` **and** `min-file-size-bytes`: These thresholds are typically used during the file selection phase to determine which files are eligible for inclusion in compaction. For instance, files that are too small may be included to reduce file count and metadata overhead, while very large files may be broken down to improve memory efficiency and scan parallelism.
- `rewrite-job-order`: This configuration controls the order in which file groups are compacted. You can prioritize by smallest files, largest files, or based on group size (fewest or most files), depending on whether your goal is to reduce metadata overhead, improve I/O balance, or distribute compaction cost across multiple runs.

These settings provide flexibility to balance efficiency, throughput, and operational cost when compacting Iceberg tables.

For example, in Spark, you might use:

```
CALL catalog_name.system.rewrite_data_files(
  table => 'db.sales',
  options => map(
    'min-file-size-bytes', '134217728',      -- 128 MB
    'max-file-size-bytes', '1073741824',     -- 1 GB
    'min-input-files', '5'
  )
);
```

In Dremio, the equivalent would be:

```
OPTIMIZE TABLE db.sales
  REWRITE DATA USING BIN_PACK (
    MIN_FILE_SIZE_MB = 128,
    MAX_FILE_SIZE_MB = 1024,
    MIN_INPUT_FILES = 5
  );
```

Choosing a narrower range between the minimum and maximum file sizes—for example, setting a relatively high min-file-size and low max-file-size, broadens the pool of files eligible for compaction by capturing more files that fall outside the optimal size boundaries. This increases the likelihood of comprehensive optimization but can also lead to larger, longer-running compaction jobs that consume more resources and potentially impact system availability or downstream tasks.

In contrast, a wider range, where the min-file-size is low and the max-file-size is high, will exclude many files from compaction, focusing only on the smallest and largest outliers. This reduces runtime and resource usage, making it suitable for environments with tight operational windows or where compaction runs alongside continuous ingestion. However, it may also leave some inefficiencies unaddressed. Striking the right balance is especially important in streaming use cases, where compaction must be both effective and non-disruptive.

To manage this, it's common to schedule incremental compaction jobs that execute frequently with narrower thresholds. These jobs gradually bring the table back to an optimized state over time without requiring full-table rewrites. When paired with metadata table queries to identify compaction candidates, this strategy becomes a powerful mechanism for maintaining performance without disrupting SLAs.

Ultimately, decisions about which files to include in compaction should be guided by factors such as query patterns, table size, observed query performance, and available compute resources. Iceberg provides the flexibility to tailor compaction to these considerations, enabling a more targeted and efficient approach to lakehouse maintenance.

## 9.2.4 Using filters to scope compaction

Compaction jobs can be resource-intensive, especially in large tables with thousands of files spread across many partitions. Rewriting all eligible files at once may not be feasible during normal business hours or on shared infrastructure. To manage this, Iceberg supports filtering mechanisms that allow you to target specific subsets of data for compaction, whether by partition, time window, or business logic.

Both Spark and Dremio support filtering compaction jobs using SQL-like predicates. This enables incremental optimization strategies that localize the impact of a rewrite job, keeping execution time short and compute costs manageable.

For example, in Spark, you can use the `where` parameter to target a specific partition:

```
CALL catalog_name.system.rewrite_data_files(  
  table => 'db.sales',  
  where => "region = 'us-west'"  
);
```

This command scopes the compaction job to only rewrite data files in the `us-west` partition, leaving other partitions untouched. You can also use compound filters, such as:

```
where => "region = 'us-west' AND sale_date >= '2024-01-01'"
```

Dremio offers similar functionality using the `FOR PARTITIONS` clause:

```
OPTIMIZE TABLE db.sales  
  REWRITE DATA USING BIN_PACK  
  FOR PARTITIONS region = 'us-west'
```

These filtering options are particularly useful when:

- You want to focus compaction efforts on hot partitions with frequent writes or updates.
- You're applying maintenance in stages to avoid long-running jobs.
- You're working with partitioned tables and need to respect system SLAs during business-critical periods.

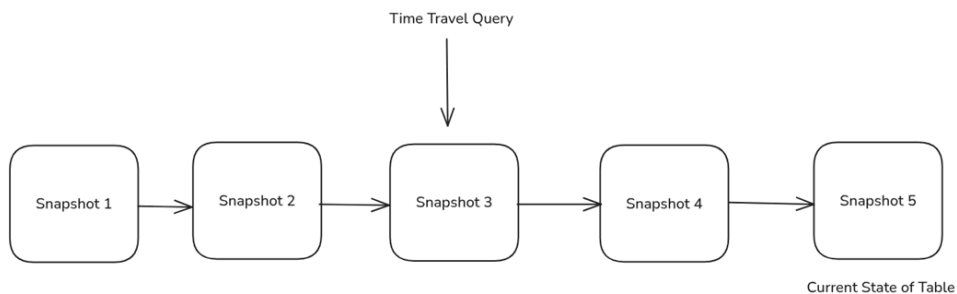
Filtering also enables adaptive compaction. For example, if monitoring reveals a particular partition has an influx of small files or accumulated delete files, you can trigger a compaction job scoped to just that segment. This approach aligns well with automation strategies that rely on metadata tables or external monitoring systems to detect table health signals and trigger maintenance workflows.

A key advantage of scoped compaction is the ability to tailor optimization strategies to the specific characteristics of each table. Rather than applying a one-size-fits-all process across the lakehouse, you can define compaction logic that reflects the needs of individual workloads. For example, one table might benefit from sorting by a regional dimension to improve query pruning, while another may only require file consolidation without sorting. This flexibility allows compaction to be more aligned with data access patterns, improving performance where it matters most.

When combined with size-based file inclusion thresholds, filters give you fine-grained control over compaction. Instead of rewriting the entire table indiscriminately, you can iteratively maintain performance where it matters most, keeping your Iceberg lakehouse optimized without sacrificing stability or efficiency.

### 9.3 Storage footprint management and data retention

Apache Iceberg's support for time travel and snapshot isolation is one of its most powerful features, allowing users to query historical table states and roll back unwanted changes, as illustrated in figure 9.8. However, this functionality comes with a tradeoff: data and metadata are retained across snapshots, which increases the table's storage footprint over time. Files that have been logically deleted may still physically exist until they are explicitly cleaned up.



**Figure 9.8 Iceberg tables have snapshot isolation, allowing you to query older versions of the table. The catch is that all the historical data is still being stored along with the current data.**

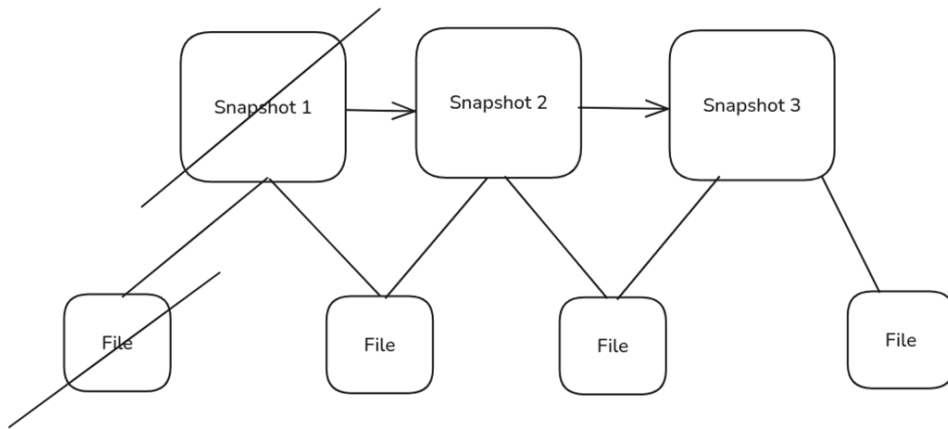
This section explains how Iceberg handles data retention, what mechanisms are available to remove unneeded files, and how to manage the balance between regulatory compliance and cost efficiency. Without regular cleanup, the accumulation of old snapshots and unreferenced files can lead to escalating storage costs and degraded table performance.

We'll begin by examining snapshot expiration, the primary mechanism for pruning obsolete data and metadata. We'll then explore how the chosen write mode, Copy-on-Write or Merge-on-Read, affects file lifetimes and deletion behavior. Finally, we'll review considerations for regulatory compliance, including GDPR and other data retention mandates that require the permanent removal of user data.

### 9.3.1 Running snapshot expiration

To support time travel and snapshot isolation, Apache Iceberg retains a full lineage of metadata and associated data files across table operations. Over time, these historical snapshots accumulate, consuming both storage space and metadata processing time. While these snapshots are useful for rollback and auditing, they must eventually be pruned to maintain a healthy lakehouse environment. This is where snapshot expiration comes into play.

Snapshot expiration removes old table snapshots and the data and metadata files that are uniquely associated with them, as illustrated in figure 9.9. Crucially, Iceberg ensures that no file required by an active or retained snapshot is deleted. This guarantees that queries remain reliable and reproducible even as history is trimmed.



**Figure 9.9** When Snapshot 1 is expired, only metadata or data files that aren't associated with the remaining snapshots are deleted.

### SNAPSHOT EXPIRATION IN SPARK

In Spark, the `expire_snapshots` stored procedure handles this task. You can invoke it with various parameters to control how aggressively snapshots are expired. The most common usage pattern sets a time boundary and retains a minimum number of recent snapshots:

```
CALL catalog_name.system.expire_snapshots(
  table => 'db.sales',
  older_than => TIMESTAMP '2024-06-01 00:00:00.000',
  retain_last => 20
);
```

This call removes all snapshots older than June 1, 2024, while ensuring that at least the last 20 snapshots are retained. For production systems, it is common to configure this procedure to run periodically, either based on time windows or triggered by metadata table monitoring.

Additional options include:

- `snapshot_ids`: directly expire specific snapshots.
- `clean_expired_metadata`: remove unreferenced partition specs and schemas.
- `max_concurrent_deletes`: parallelize delete operations to speed up cleanup.
- `stream_results`: stream deletion results to avoid Spark driver memory issues.

These options offer operational flexibility and allow snapshot expiration to scale with table size and system capacity.

## SNAPSHOT EXPIRATION IN DREMIO

In Dremio, snapshot expiration is performed using the `VACUUM TABLE` command:

```
VACUUM TABLE s3.sales EXPIRE SNAPSHOTS older_than '2024-06-01 00:00:00.000'  
retain_last 20;
```

This has the same effect as the Spark procedure: it removes snapshots older than a specific date while preserving a baseline number of recent snapshots. If no parameters are provided, Dremio uses a default policy of retaining the last snapshot and removing snapshots older than five days.

The result of these operations includes counts of deleted data files, delete files, manifest files, and statistics files, helping teams understand the impact of expiration on storage usage.

## CONSIDERATIONS

Snapshot expiration is safe and non-destructive concerning active or recently used data. However, it does mark the point at which time travel to older table states becomes unavailable. It's essential to align expiration schedules with organizational policies around data auditing, compliance, and recovery windows.

In environments with high transaction volumes, frequent expiration helps prevent storage bloat and keeps metadata operations fast. For slower-changing tables, expiration may occur less frequently, but it should still be included as a regular maintenance task. Used in combination with orphan file removal (described later), snapshot expiration is an essential step in maintaining a cost-efficient and performant Iceberg lakehouse.

### SNAPSHOT RETENTION STRATEGIES FOR NEAR REAL-TIME AND CDC INGESTION

Change Data Capture (CDC) and other near real-time ingestion patterns create a unique challenge for Iceberg maintenance. Because these pipelines commit new data frequently, sometimes thousands of times per day, the table's snapshot history can grow extremely quickly. Each commit generates a new snapshot, and without intervention, this can bloat metadata, increase query planning times, and strain catalog performance.

To manage this, organizations often need to expire snapshots more aggressively than they would for batch or micro-batch workloads. While aggressive expiration reduces storage and metadata overhead, it also minimizes the benefit of time travel, since fewer historical states are retained. Additionally, configuration options like `retain_last`, which keeps a fixed number of recent snapshots, may be less effective in high-frequency scenarios because even a small number of snapshots can represent only minutes or hours of history.

In these cases, it's essential to tune snapshot retention policies around workload realities. For example, you might set a time-based expiration window (such as "retain snapshots for only the last 2 hours") rather than relying solely on snapshot counts. Balancing retention with performance is especially critical in CDC-heavy tables to ensure both efficient reads and manageable metadata without losing all rollback capability.

### 9.3.2 COW vs MOR: Implications for data retention

Apache Iceberg supports two primary write modes: Copy-on-Write (COW) and Merge-on-Read (MOR). While both enable ACID operations and snapshot isolation, they differ significantly in how updates and deletes are physically applied to data. These differences have direct implications for file retention and storage cleanup strategies.

#### COPY-ON-WRITE (COW)

Copy-on-Write (COW) is the default write mode in Apache Iceberg. In this model, update and delete operations result in the creation of new data files that reflect the desired changes, such as removed or modified rows. Instead of rewriting existing files, the engine writes new ones with the changes applied. The table's metadata is then updated to reference the new files in a new snapshot, while the original files remain referenced in the previous snapshot to preserve time travel capabilities.

Because deletions are physically applied during the write, COW tables do not accumulate separate delete files as Merge-on-Read tables do. Instead, they accumulate *replaced* data files, files that are no longer needed once the snapshots referencing them have expired. This tight coupling between data files and manifests ensures that only files actively referenced by metadata are retained.

For example, a delete operation in COW will:

- Create a new data file with the deleted rows omitted.
- Record this file in a new manifest, referenced by the latest snapshot.



- Leave the original file referenced in the previous snapshot until it is explicitly expired.

This eager materialization simplifies retention management. Once older snapshots are expired, the replaced files they uniquely reference become eligible for physical cleanup, making COW tables more predictable and easier to manage from a storage perspective.

## MERGE-ON-READ (MOR)

Merge-on-Read (MOR) mode is often used to support late-arriving changes or high-frequency update workloads, where low-latency writes are prioritized over immediate data reorganization. Instead of rewriting base data files on every update or delete, MOR tables capture changes separately in delete files, either position deletes (which mark specific row positions within a file) or equality deletes (which mark rows based on column values). At query time, these deletes are applied dynamically to filter out the affected rows.

This design improves write throughput but comes with trade-offs. Over time, the volume of metadata that must be scanned and maintained grows, especially when many delete files accumulate across snapshots. In particular, position deletes, which are being phased out in favor of delete vectors in the evolving Iceberg spec, present retention challenges. A base data file cannot be deleted until all snapshots referencing any of its associated delete files have also been expired. If a delete file is reused across many snapshots, it can prolong the retention of otherwise obsolete data files, even if most earlier snapshots have already been expired.

For example, if a position delete file introduced in snapshot 103 is still referenced in snapshot 110, expiring snapshots 101 through 105 won't trigger cleanup of the base file or the delete file. This can result in growing storage usage, even as snapshot expiration proceeds.

To mitigate this, Iceberg provides the `rewrite_position_delete_files` procedure, which can compact and restructure delete files. This process helps eliminate dangling deletes, delete files that no longer match any live data files, thereby reducing storage overhead and improving planning performance.

---

**NOTE** As of the V3 Iceberg specification updates, delete vectors are expected to replace position deletes, providing more efficient tracking and better integration with modern query engines. These improvements will simplify retention and reduce the long-term storage footprint of MOR tables.

---

## OPERATIONAL GUIDANCE

Effective snapshot and file retention strategies vary based on write mode and workload patterns. Here are some practical recommendations to help keep your Iceberg tables efficient and storage under control:

1. **Use regular snapshot expiration, daily for COW, hourly for high-change MOR tables.**

For Copy-on-Write (COW) tables, daily snapshot expiration is generally sufficient to remove unreferenced files and maintain a lean storage footprint. In contrast, Merge-on-Read (MOR) tables with high-frequency updates, such as Change Data Capture (CDC) workloads generating hundreds of changes per minute, should expire snapshots more frequently, ideally on an hourly basis. This reduces the volume of retained delete files and shortens the retention window for stale data.

2. **Compact MOR tables frequently to manage delete file growth.**  
MOR tables benefit from more aggressive compaction. In high-change environments, running compaction every few minutes (e.g., every 5 minutes) helps consolidate delete files and maintain manageable table and partition sizes. This ensures that the volume of metadata and file I/O does not grow unchecked, which could otherwise degrade query performance over time.
3. **Monitor table size and delete file trends—not just individual file counts.**  
Rather than trying to track delete file growth in isolation, monitor overall table size and partition-level storage trends. A healthy compaction cycle will cause the number of delete files to grow and shrink in a sawtooth pattern. If delete file counts only grow, this may indicate that compaction jobs are misconfigured or failing to keep up with ingestion.
4. **Be cautious with long-lived branches or tags.**  
In Iceberg, features like branches and tags preserve specific snapshots for time travel or reproducibility. However, they can also prevent snapshot expiration, effectively blocking deletion of older data and metadata files.

Understanding how write mode affects retention, and tuning expiration and compaction frequency accordingly, is critical to maintaining a performant and cost-efficient lakehouse—especially under high-ingestion conditions.

### 9.3.3 Regulatory considerations for data deletion

As data volumes grow, so do the legal obligations for handling personal and sensitive information. Regulations such as the General Data Protection Regulation (GDPR), California Consumer Privacy Act (CCPA), and others require organizations to guarantee that certain user data can be permanently deleted upon request. This presents unique challenges in a system like Apache Iceberg, which is designed to preserve data history for snapshot isolation and time travel.

In Iceberg, delete operations do not immediately remove files from storage. Instead, they update the table metadata to exclude certain records or files from future snapshots. The physical removal of data only occurs after the relevant snapshots have been expired and data files compacted. This deferred deletion model, while beneficial for performance and auditability, complicates compliance with regulations that require timely and irrevocable erasure.

## ALIGNING SNAPSHOT EXPIRATION WITH REGULATORY REQUIREMENTS

To comply with data deletion regulations such as GDPR or CCPA, it's not enough to logically mark data as deleted, you must also ensure that all physical references to that data are removed. In Apache Iceberg, this requires a multi-step approach:

- **Expire snapshots** that reference data subject records. Snapshots preserve references to data files, so as long as a snapshot exists, the data it references remains accessible.
- **Compact frequently**, especially in Merge-on-Read (MOR) tables. Delete files in MOR mode contain pointers to rows in base data files, not the data itself, so compaction is required to physically rewrite those base files and exclude the deleted rows.
- **Ensure no retained snapshots reference the old files.** Even if compaction rewrites a file, it won't be deleted until all snapshots referencing the original version have been expired.
- **Run orphan file cleanup** after expiration and compaction to remove unreferenced files from storage.

It's important to understand that Iceberg snapshots do not partition data logically, for instance, one snapshot can contain only aggregates while another contains personal data. All snapshots represent consistent views of the same table, referencing shared data files unless changes are committed. Therefore, removing personal data means identifying and expiring every snapshot that references the original files, not just the ones where changes were made.

## CONSIDERATIONS FOR MERGE-ON-READ TABLES

In Merge-on-Read (MOR) tables, delete operations generate separate delete files rather than modifying the underlying data files directly. These delete files can persist across many snapshots, making deleted records more durable than intended unless cleanup procedures are run. To ensure regulatory compliance, especially for data deletion requests, it's important to:

- Run Iceberg's built-in procedures (such as `rewrite_position_delete_files` and `expire_snapshots`) to remove obsolete delete records and their references.
- Ensure that compaction has rewritten base files so that deleted rows are physically removed, not just logically filtered at query time.
- Confirm that no retained data files still contain the deleted rows and that all relevant snapshots have been expired.

These steps are handled automatically by Iceberg's maintenance procedures, but it's up to the user to run them at appropriate intervals. Regular execution is essential for making sure that deletions are complete and compliant from both a logical and physical standpoint.

## BRANCHES, TAGS, AND SNAPSHOT RETENTION

Branches and tags in Iceberg allow users to preserve alternate views of a table, such as for audit or rollback purposes. However, snapshots referenced by branches or tags do not expire by default. This can inadvertently preserve data long past its intended retention period.

To ensure compliance:

- Audit and manage custom branches and tags regularly.
- Use the `history.expire.max-ref-age-ms` property to set automatic expiration policies on branches and tags where appropriate.

## RECOMMENDED PRACTICES

- **Automate expiration workflows** for snapshots containing regulated data.
- **Avoid using long-lived branches** for data tied to personal or sensitive information.
- **Integrate data subject management systems** with your data lakehouse tooling to orchestrate deletions across metadata and storage.
- **Monitor and log expiration events** to demonstrate compliance during audits.

By designing retention and deletion strategies with regulatory mandates in mind, teams can ensure that Iceberg's powerful time travel capabilities do not conflict with the imperative to protect user privacy.

## 9.4 Exploring Apache Iceberg's metadata tables

Apache Iceberg captures detailed metadata to support its transactional model, schema evolution, partitioning, and snapshot-based time travel. Beyond serving internal table mechanics, this metadata is also exposed to users through a set of *metadata tables*. These special tables provide a transparent view into the internal state of an Iceberg table and are accessible using standard SQL queries in engines such as Spark, Trino, Flink, and Dremio.

Iceberg's metadata tables cover a range of operational dimensions:

- **File-level insights:** Understand how data files are distributed across partitions, their sizes, and associated statistics.
- **Manifest and snapshot visibility:** Examine how data files are grouped and versioned across snapshots.
- **Delete file tracking:** Monitor the accumulation of delete files in Merge-on-Read tables and identify when compaction is needed.
- **Partition evolution and layout:** Audit partition structures and detect fragmentation or skew.
- **History and snapshot lineage:** Track how and when data changed over time, supporting governance, auditing, and rollback.

These metadata tables serve as a powerful tool for proactive table maintenance. They allow engineers to detect small file accumulation, track snapshot growth, identify unoptimized partitions, and trigger corrective actions such as compaction or snapshot expiration. By integrating metadata table queries into monitoring pipelines or scheduled jobs, you can automate much of the operational upkeep required to maintain an efficient lakehouse.

For a complete reference to all Iceberg metadata tables, including schemas and usage examples, see **Appendix A**.

## 9.5 Access controls in an Iceberg lakehouse

As Iceberg tables become central to enterprise data platforms, managing secure access across multiple engines, users, and roles is critical. Unlike traditional data warehouses, a lakehouse introduces complexity with its decoupled architecture, where storage, catalog, and compute may be managed separately. This necessitates a layered approach to access control that balances flexibility with governance. In this section, we explore the different layers where access control can be applied in an Iceberg-based lakehouse and offer guidance on where specific policies are best enforced.

### 9.5.1 Storage layer controls

The foundational layer of access control in an Iceberg lakehouse is the object store, typically Amazon S3, Azure Data Lake Storage, or Google Cloud Storage. Because all Iceberg tables ultimately reside as files in cloud storage, securing access to these files is the first line of defense.

#### **IAM POLICY EXAMPLE: READ-ONLY ACCESS**

For users who only need to query data, an AWS IAM policy might look like this:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::your-data-lake-bucket",
        "arn:aws:s3:::your-data-lake-bucket/*"
      ]
    }
  ]
}
```

## IAM POLICY EXAMPLE: READ-WRITE ACCESS

For compute engines (like Spark or Dremio) that perform writes:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::your-data-lake-bucket",
        "arn:aws:s3:::your-data-lake-bucket/*"
      ]
    }
  ]
}
```

## BEST PRACTICES

- **Least Privilege:** Always grant the minimum set of permissions required for a user or service.
- **Path-Based Scoping:** Use prefixes (e.g., `arn:aws:s3:::bucket/tableA/*`) to control access to specific tables.
- **Immutable Writes:** For audit trails, consider storing committed data in a write-once path and using object versioning.
- **Separation of Duties:** Segment administrative and compute roles to minimize accidental or malicious changes.

While storage layer controls provide strong isolation, they operate at the file level and lack awareness of Iceberg semantics like snapshots or schema evolution. Therefore, they are best used in combination with catalog and engine-layer access controls, which we'll explore in the next sections.

## 9.5.2 Catalog-level controls

While cloud storage permissions provide foundational access control, they lack awareness of data context, schemas, and roles. This is where catalog-level controls shine, offering a centralized and semantically rich access control layer that governs how users and compute engines interact with Iceberg tables.

Catalogs like Apache Polaris, Apache Gravitino and others implement robust Role-Based Access Control (RBAC) frameworks to govern access across catalogs, namespaces, tables, and views. In this model:

- **Catalog Roles** define a set of privileges (e.g., table read/write, view creation).
- **Principal Roles** group users or service principals, such as data engineers or scientists.
- **Assignments** connect catalog roles to principal roles, delegating access across teams.

For example, in Polaris:

- A **Catalog Contributor** role might grant `TABLE_READ_DATA` and `TABLE_WRITE_DATA` privileges.
- A **Data Engineer** principal role would be granted this contributor role to enable read-write operations across relevant tables.

This indirection provides flexibility. Changing a role's privileges affects all users with that role, simplifying administration. Here's an example of granting read access via the Polaris CLI.

```
polaris privileges \
  table \
  grant \
  --catalog prod_catalog \
  --catalog-role catalog_reader \
  --namespace finance \
  --table transactions \
  TABLE_READ_DATA
```

This command enables all users with the `catalog_reader` role to read data from the `finance.transactions` table.

## CREDENTIAL VENDING: ALIGNING CATALOG AND STORAGE ACCESS

A major advantage of catalog-centric control is the concept of credential vending, where the catalog dynamically issues short-lived credentials to access underlying cloud storage. This abstracts storage layer complexity and ensures that:

- Permissions defined in the catalog (e.g., Polaris) directly control access to data in S3 or ADLS.

- There is no need to maintain complex IAM mappings per user or group.
- Access audits can be centralized through catalog activity logs.

With credential vending, catalog policies become the source of truth for both logical and physical data access.

## BEST PRACTICES

- **Use Vended Credentials** wherever supported. They reduce the risk of misconfigured IAM roles.
- **Group users by job function** (e.g., `data_engineer`, `data_scientist`) via principal roles to simplify role assignments.
- **Delegate privileges via catalog roles**, not individual users, to keep permissions maintainable and auditable.
- **Regularly audit privileges** across roles to ensure access is aligned with data governance policies.

Catalog-level access control, especially when coupled with vended credentials, forms the backbone of a secure and scalable Iceberg deployment.

### 9.5.3 Engine-level access controls

While catalog-level access control provides a centralized mechanism for coarse-grained authorization, the enforcement of fine-grained access control (FGAC), such as row- and column-level security, currently depends heavily on the capabilities of individual engines. Unlike catalog privileges, which can be standardized across tools, FGAC lacks a universally accepted specification, limiting true interoperability.

## THE STATE OF FGAC TODAY

At present, no cross-engine standard for FGAC exists in the Iceberg ecosystem. This means that when fine-grained policies, like masking a salary column or filtering employee records by department, are needed, the logic must be implemented within each engine independently. While this gives engines flexibility, it leads to fragmented enforcement, duplicated effort, and inconsistency in security guarantees.

For example, one engine might support SQL-based row filters while another uses a Java-based UDF mechanism. If these implementations are not synchronized with a central policy, inconsistencies emerge: two users querying the same table via different engines might receive different results, undermining trust and control.



## TOWARD A STANDARD: THE INTEROPERABLE FGAC PROPOSAL

To address this gap, a group of contributors from the Iceberg community has proposed a standard for interoperable FGAC enforcement across engines and catalogs via Iceberg REST. This design allows catalogs to store row-level filters and column-level transformations, referred to as protection instructions, that can be enforced consistently across trusted engines.

At a high level, the proposal suggests:

- **Catalogs define FGAC policies** with rules for row filtering and column masking.
- **Trusted engines retrieve and apply protection instructions** from the catalog at query time.
- **Policies are expressed in interoperable formats**, such as Substrait or SQL92/99, to ensure engine compatibility.
- **Contextual attributes** (like user role or department) are retrieved securely by catalogs and embedded into query plans as filters or expressions.

Here is an example use case: consider a table `employees(department, salary)` with a policy that restricts access by department. A department manager querying this table should only see records where `department = 'marketing'`, while a top-level manager sees all rows. The catalog provides this logic as a row filter, and the engine enforces it during query execution.

## CHALLENGES AND LIMITATIONS

This approach introduces several open questions:

- How is **trust** established between catalogs and engines?
- Should **policy definitions** be stored in catalogs or external systems?
- How are **authorization systems** (OAuth, Kerberos, etc.) mapped to attributes for policy evaluation?

Additionally, write operations like INSERT, UPDATE, and DELETE introduce complexity. Should column masking apply to writes? If a user sees a masked value, can they overwrite it? The current consensus leans toward enforcing row-level checks for writes while deferring column-level transformations to read operations only.

## BEST PRACTICES AND FORWARD-LOOKING GUIDANCE

Until this proposal or a similar standard is implemented, data platform teams should adopt the following best practices:

- **Use catalogs for centralized identity and credential management.** Vending temporary credentials through the catalog ensures that downstream engines are bound to the same security context.
- **Replicate FGAC logic consistently across engines** using configuration management or CI/CD automation.

- **Monitor engine-specific security capabilities** and favor those that offer built-in FGAC enforcement, such as Apache Spark with DataFrame filters or Flink's SQL-based row-level filtering.
- **Prepare for transition** by structuring current FGAC logic in reusable, declarative formats (e.g., SQL expressions), which can be easily migrated to a catalog-based policy model in the future.

As the ecosystem evolves toward interoperable fine-grained access control (FGAC), catalogs are increasingly positioned not just as metadata stores but as centralized authorities for defining and enforcing security policies. This shift enables query engines to delegate access control enforcement while adhering to a shared, catalog-driven access model. While Section 4.5 introduced the basics of access controls in Iceberg, there are broader governance topics worth exploring beyond its scope. These include integration with data classification frameworks, auditing and lineage tracking, policy versioning, and support for row- and column-level security across multiple engines.

Additionally, organizations should consider how Iceberg fits into larger security architectures, such as identity federation, role-based access controls, and catalog-layer encryption strategies. These dimensions are critical for building a secure and compliant data lakehouse, particularly in regulated environments.

In the next chapter, we'll step beyond core platform features to explore topics like Iceberg's growing Python ecosystem and other operational and architectural considerations that round out a modern Iceberg deployment.

## 9.6 Summary

- Iceberg manages physical files through structured, linked metadata files, making maintenance a critical operation.
- Compaction is essential for managing metadata overhead, merging changes, removing deleted rows and improving read performance, especially with Merge-on-Read tables.
- Snapshot expiration and garbage collection prevent unbounded metadata and file storage growth.
- Effective file sizing and clustering strategies help optimize query performance and reduce scan costs.
- Cloud-native IAM policies should be scoped to temporary credentials for read/write access and tightly controlled with IAM conditions.
- Credential vending enables catalogs to serve as the authoritative access control layer for both metadata and data storage.
- Polaris-style RBAC models separate users (principals) from permissions (catalog roles), providing scalable access control patterns.
- Engines currently lack a standardized mechanism for enforcing fine-grained access control (FGAC).
- A proposed FGAC model for Iceberg introduces catalog-issued row filters and column transformations that engines can enforce at runtime.
- Future interoperability for FGAC across engines depends on standardized protection expression formats like SQL or Substrait.

## 10 *Operationalizing Apache Iceberg*

### **This chapter covers**

- Automating Iceberg maintenance
- Using metadata for health monitoring
- Enforcing retention and compliance
- Tracking changes for governance
- Planning for disaster recovery

Building an Apache Iceberg lakehouse is only the beginning. Once data is flowing and tables are live, the real challenge begins: keeping the system healthy, secure, compliant, and resilient amid constant change. Operationalization is what transforms a functional data platform into a sustainable one. It ensures that the architecture you designed in the earlier chapters and the maintenance workflows you implemented in chapter 9 continue to support business needs reliably over time.

Apache Iceberg is built for scale, but scale brings complexity. As snapshots accumulate, delete files grow, and ingestion patterns shift, your Iceberg tables evolve in ways that require regular intervention. Compaction, snapshot expiration, and orphan file cleanup are not just technical procedures. They are operational commitments that must be executed consistently and monitored for effectiveness. Without automation and visibility, even a well-designed table can silently degrade, leading to increased query latency, rising storage costs, or worse, compliance violations.

This chapter explores what it takes to operationalize an Iceberg deployment in production. It begins with orchestration: how to schedule, trigger, and manage recurring maintenance tasks using workflow engines like Apache Airflow or dbt, and how to leverage Iceberg's metadata tables to detect when intervention is needed. These tasks are table-specific and context-aware, requiring policies tailored to the size, volatility, and business role of each dataset.

Next, we address auditing and governance. Unlike traditional warehouses, Iceberg provides fine-grained versioning and structural metadata that can be used for schema tracking, access logging via the catalog, and regulatory enforcement. Using these features effectively requires embedding them into a broader governance framework. This includes enforcing data retention through snapshot management, coordinating delete file compaction to meet regulatory deletion requirements, and maintaining access controls that align across multiple query engines and catalog systems.

Finally, we cover disaster recovery. Iceberg's append-only model and snapshot lineage provide built-in mechanisms for rollback and state recovery, but full resilience also requires planning for catalog failure, storage layer outages, and human error. Recovery in Iceberg is not just about restoring from backups; it's about using time travel, schema history, and storage versioning to respond precisely and confidently in the face of incidents.

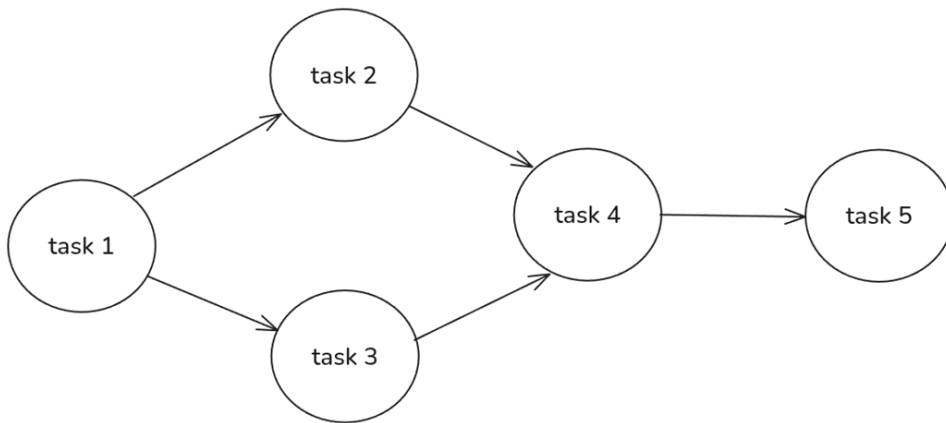
Together, these practices form the operational foundation of a modern Iceberg lakehouse. By the end of this chapter, you'll know not just how to maintain your data platform, but how to run it with the predictability and discipline required in production. Operational excellence in Apache Iceberg begins where architecture ends: when every maintenance task, governance rule, and recovery step must be automated, observable, and reliable.

## 10.1 Orchestrating the lakehouse

Running Apache Iceberg in production means much more than executing manual maintenance commands or inspecting table statistics by hand. At scale, operational integrity relies on automation: scheduled jobs, metadata-aware triggers, and table-specific logic that ensure the lakehouse remains healthy without constant human oversight. Orchestration is the discipline that connects all these pieces, making maintenance workflows reliable, repeatable, and responsive to changing data conditions.

Apache Iceberg provides the procedural foundation for a wide range of maintenance tasks: compaction, snapshot expiration, delete file rewriting, and orphan file cleanup, among others. Each of these operations contributes to the long-term performance and efficiency of your tables. In practice, they are not one-time actions. They must be executed repeatedly and often conditionally, based on how each table behaves over time. Without orchestration, these procedures either run too frequently, wasting resources, or too rarely, allowing performance to degrade silently.

Orchestration tools such as Apache Airflow, dbt (data build tool), Dagster, or custom workflow engines allow teams to manage these procedures in a structured, programmatic manner. They support the definition of maintenance pipelines that run on a schedule, in response to events, or based on conditions derived from Iceberg's metadata. For example, you might define a Directed Acyclic Graph (DAG), as illustrated in figure 10.1, that checks for excessive snapshot growth or small file accumulation and only triggers compaction when thresholds are breached. These pipelines can also be configured to behave differently per table, reflecting differences in ingestion frequency, query volume, and compliance requirements.



**Figure 10.1 A Direct Acyclic Graph (DAG) illustrates a non-looping order of tasks. In this example, tasks 2 & 3 can't begin until task 1 is completed, and task 4 can't begin until tasks 2 & 3 have been completed. This ensures a task doesn't begin until all other dependent tasks are complete.**

#### WHAT IS A DAG? .

A DAG, or Directed Acyclic Graph, is a structure used to define workflows where tasks have clear dependencies. "Directed" means each connection flows in one direction, from one task to the next. "Acyclic" means there are no loops: a task can't eventually lead back to itself.

In orchestration tools like Apache Airflow, DAGs represent pipelines as a series of tasks that must run in a specific order. For example, a DAG might define that a table compaction must run only after a metadata check confirms the need, and that snapshot expiration can only follow a successful compaction.

By modeling workflows as DAGs, teams can control execution order, define retries, and visualize complex maintenance pipelines in a structured way. DAGs make orchestration predictable, auditable, and easy to debug.

Unlike traditional data warehouse platforms, where operations are tightly coupled to the engine, Iceberg's open architecture gives you flexibility in how orchestration is implemented. Maintenance logic can run through Spark, Trino, Dremio, or other engines that support Iceberg's SQL procedures. This decoupling enables organizations to centralize maintenance workflows even in heterogeneous environments where different teams or workloads rely on different compute platforms.

Let's explore the tools and practices that make orchestration work in an Apache Iceberg lakehouse. We will examine how to build and schedule workflows, define maintenance triggers based on table metadata, and apply differentiated policies across staging and production environments. The goal is to move beyond manual intervention and toward a model where operational stability is achieved through proactive, automated action—guided by metadata, governed by policy, and tailored to the specific needs of each table.

### 10.1.1 Choosing orchestration tools and patterns

Selecting the right orchestration framework is foundational to operationalizing an Apache Iceberg deployment. While Iceberg offers a rich set of built-in procedures for table maintenance, it does not provide a native job scheduler or workflow engine. As a result, integrating Iceberg into your orchestration layer is a design decision that will shape how reliably and flexibly your lakehouse can be maintained at scale.

Most Iceberg maintenance operations are procedural and can be invoked through SQL-compatible engines such as Apache Spark, Trino, or Dremio. These engines act as execution layers, running SQL commands like `CALL expire_snapshots`, `CALL rewrite_data_files`, or `CALL remove_orphan_files`. But without a higher-level system to schedule and manage these operations, you're left with the burden of manually triggering jobs or writing brittle scripts. This is where orchestration frameworks come into play.

Apache Airflow is a common choice for orchestrating Iceberg maintenance because of its flexibility and extensibility. Airflow's DAG-based structure allows you to define multi-step workflows with dependency management, retries, alerting, and conditional logic. For example, a daily Airflow DAG might begin by querying a metadata table to assess the number of small files in a given Iceberg table. If a threshold is exceeded, the DAG could trigger a compaction job, which is followed by snapshot expiration and orphan file removal. This sequence ensures that maintenance is both reactive and automated, adapting to the actual state of the table rather than a static schedule.

Other teams may adopt dbt (data build tool), especially when Iceberg is integrated into a broader analytics stack that already uses dbt for transformations. dbt's model-driven approach makes it easy to incorporate maintenance logic into existing data pipelines, particularly when using dbt's support for run-operation macros to invoke custom SQL commands. For example, a dbt model might process new data, and a follow-up operation could trigger snapshot expiration to keep the table lean after ingest. While dbt does not offer the full scheduling and DAG management features of Airflow, it provides a lightweight option for teams that want to keep orchestration close to transformation logic.

For environments with strong DevOps practices or cloud-native infrastructure, containerized orchestration using Kubernetes-native tools like Argo Workflows or Prefect is also a compelling option. These tools allow you to build declarative, container-based pipelines where each maintenance task runs in isolation with clearly defined resources. This is particularly useful for isolating maintenance workloads from production ingestion and query workloads, avoiding resource contention and improving reliability.

In some cases, a combination of tools may be appropriate. For example, Airflow may be used to schedule daily compaction and retention workflows, while dbt is used for transformation-aware snapshot management. In hybrid environments where both batch and streaming pipelines coexist, orchestration patterns may differ by workload: streaming ingestion might trigger compaction jobs at regular intervals, while batch tables rely on daily or weekly maintenance cycles.

It's also worth noting that orchestration patterns evolve with maturity. Early deployments often begin with manual job execution or basic cron jobs. Over time, as more tables are added and data volumes grow, teams transition to parameterized, metadata-aware DAGs that manage dozens or hundreds of tables based on standardized policies. The key is to design orchestration workflows that are idempotent, transparent, and observable so they can be trusted to run autonomously, while still offering insight into their behavior.

### **THE VALUE OF BEING IDEMPOTENT, TRANSPARENT, AND OBSERVABLE**

These three qualities define well-behaved operational workflows in any data platform:

Idempotent means that a task can be run multiple times with the same effect as running it once. If a compaction job runs twice due to a retry or scheduler glitch, the result should be the same as if it ran once. This protects against accidental duplication or corruption.

Transparent workflows are easy to understand and explain. Each step has a clear purpose, documented inputs and outputs, and minimal hidden logic. This makes it easier for new team members to trace what's happening and for experienced engineers to debug issues quickly.

Observable systems expose enough information to track what is happening in real time and to reconstruct what happened in the past. Logs, metrics, and alerts should give you insight into the success, failure, duration, and side effects of each operation.

Together, these principles make automation safer, debugging faster, and operations more reliable at scale.

Ultimately, the orchestration layer is what turns Iceberg's procedural capabilities into a sustainable system. By choosing the right tools and structuring workflows to respond to the actual needs of your lakehouse, you move from reactive table maintenance to proactive operational control.

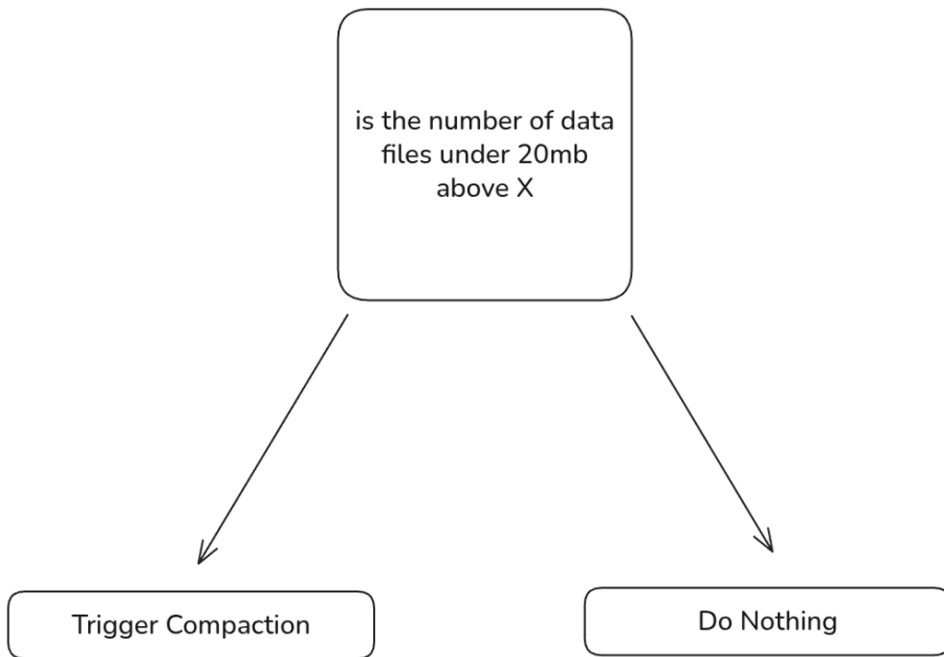
### 10.1.2 Metadata-driven triggers for proactive maintenance

One of Apache Iceberg's greatest strengths is its detailed and structured metadata layer. Each table tracks snapshots, manifests, partition statistics, and file-level metadata in a format designed for direct access. Although this metadata is not stored in separate physical tables, it is exposed through metadata views that can be queried like tables. These views are backed by the Iceberg SDK, which reads and parses metadata files to present them as rows. When used effectively, these views provide the foundation for a proactive, intelligent maintenance strategy.

Rather than scheduling compaction, snapshot expiration, or delete file rewriting on fixed intervals alone, sophisticated Iceberg deployments can respond dynamically to the state of the table itself. For instance, compaction may be unnecessary for a table that sees infrequent updates and already contains well-sized files. Conversely, a streaming ingestion table might accumulate thousands of small files each hour, requiring near-continuous optimization. By querying metadata tables regularly, orchestration tools can determine exactly when intervention is needed and act accordingly.

This approach is beneficial for triggering compaction based on small file accumulation. Iceberg's files metadata table exposes every active data and delete file along with its size, partition, and file path. A scheduled query can count how many files in a given table fall below a target threshold (such as 20 MB). If that number exceeds a predefined limit, a compaction job can be triggered automatically, as illustrated in figure 10.2. This ensures that resources are spent compacting only when there is actual fragmentation, reducing unnecessary I/O while keeping query performance stable.





**Figure 10.2** By having conditional logic based on data we can track using the metadata tables, we can avoid running maintenance work when the table doesn't need it based on metrics like the size of files in the table's current snapshot.

Similarly, metadata-driven triggers can monitor the growth of delete files in Merge-on-Read tables. Since delete files are stored separately and applied at query time, excessive accumulation can degrade read performance and inflate storage costs. The files table again provides a view into these files, allowing teams to detect when the number or size of delete files exceeds healthy limits. When this happens, a `rewrite_position_delete_files` operation can be invoked to eliminate redundant or obsolete entries and merge deletes into base files.

Another use case is snapshot sprawl. While snapshots enable time travel and rollback, excessive snapshot growth can slow metadata operations and increase storage consumption. Iceberg's snapshots metadata table shows the creation time and lineage of each snapshot. By querying this table, you can track whether snapshot expiration policies are working as expected or whether specific tables are retaining snapshots longer than intended. If too many old snapshots are still active, a maintenance job can expire them based on a timestamp or retention policy, keeping the metadata tree lean and responsive.

In production environments, this metadata-driven approach is often embedded into orchestration workflows. For example, an Apache Airflow DAG might begin by querying the metadata tables to determine file count and snapshot age. Based on the results, the DAG branches to invoke only the necessary procedures, such as `expire_snapshots` or `rewrite_data_files`. This branching logic enables per-table customization and ensures that maintenance operations scale with data volume and usage patterns, rather than relying on rigid schedules.

Importantly, metadata-driven triggers are not limited to operational use. They also support observability and alerting. Teams can expose metadata metrics to systems like Prometheus or CloudWatch, allowing them to build dashboards and set alerts on key indicators such as rising snapshot counts, uneven partition sizes, or frequent compaction failures. This observability layer makes it easier to identify systemic issues early and intervene before they escalate into production problems.

Appendix A provides a comprehensive overview of Iceberg's metadata tables and the fields they expose. While this section focuses on practical usage patterns, a deeper understanding of the available metadata will enable more sophisticated and nuanced maintenance strategies.

Metadata-aware orchestration is the key to sustainable Iceberg operations. By grounding maintenance in actual table behavior rather than assumptions or fixed schedules, you can reduce waste, improve performance, and build confidence in the long-term health of your lakehouse.

### 10.1.3 Per-table maintenance policies

Not all Iceberg tables are created equal. Some ingest high-velocity CDC streams that generate thousands of small files and frequent snapshots. Others store slowly changing fact tables updated once a day or less. Some are mission-critical assets serving executive dashboards, while others are temporary staging areas that exist only to support downstream transformation jobs. Each of these tables has different operational needs, and treating them all the same risks wasting compute, overloading the catalog, or missing optimization opportunities.

Apache Iceberg's flexibility makes it possible to define maintenance policies at the table level. This allows you to tailor how and when you perform operations like compaction, snapshot expiration, delete file rewriting, and orphan file cleanup. Instead of applying the same schedule and logic to every table, you can align maintenance practices with the unique characteristics of each dataset. This approach not only improves efficiency but also ensures that the lakehouse responds appropriately to diverse business and technical requirements.

Consider, for example, a table that ingests change data capture events every few minutes. This table will generate a high volume of small files and delete files, particularly in Merge-on-Read mode. For such a workload, compaction may need to run multiple times an hour, and snapshot expiration may need to occur daily to prevent metadata bloat. You may also need to rewrite delete files more aggressively to avoid compounding query-time merge costs. The maintenance strategy here must be tightly coupled to the ingestion pattern, and the procedures must run frequently and predictably.

Contrast that with a table used for monthly reporting. This table may receive one large batch of new data at the start of each month and remain static otherwise. In this case, frequent compaction or snapshot expiration is unnecessary. A lightweight policy that runs once per month, shortly after the ingestion job completes, would suffice. Applying daily or hourly procedures to this table would provide little benefit and might even introduce unnecessary catalog updates.

Operational policies should also account for table size and query access patterns. Large fact tables that serve interactive dashboards may need to be compacted more often to maintain low-latency query performance. Dimension tables that change rarely can tolerate longer retention intervals. Regulatory tables containing personally identifiable information may require more aggressive snapshot expiration and delete file rewriting to comply with data protection laws. These distinctions are not just operational preferences; they are critical to delivering reliable and compliant data products.

Most orchestration frameworks support parameterization, making it easy to encode per-table policies into scheduled jobs. For example, an Airflow DAG might read a configuration file or metadata catalog to determine how often each table should be compacted, how many snapshots to retain, and what file size thresholds to use. These parameters can be passed into the DAG dynamically, ensuring that each table is managed according to its defined policy. This design enables centralized management with decentralized execution logic, a key pattern for scaling operational control.

```

from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from datetime import datetime
import json

def load_config():
    with open('/configs/table_policies.json') as f:
        return json.load(f)

def run_compaction(table_name, file_size_threshold, snapshot_retention):
    print(f"Running compaction for {table_name}")
    print(f"File size threshold: {file_size_threshold}MB")      #A
    print(f"Retaining {snapshot_retention} snapshots")          #B
    # Placeholder for actual compaction logic
    # For example: spark.sql(f"CALL ...")

default_args = {'start_date': datetime(2023, 1, 1)}

with DAG('table_maintenance',
        default_args=default_args,
        schedule_interval='@daily',
        catchup=False) as dag:

    config = load_config()                                     #C

    for table, policy in config.items():
        task = PythonOperator(
            task_id=f"compact_{table}",
            python_callable=run_compaction,
            op_kwargs={
                'table_name': table,
                'file_size_threshold': policy['file_size_threshold'],
                'snapshot_retention': policy['snapshot_retention']
            }
        )                                                       #D

```

**#A** Reads the file size threshold from the config to apply compaction selectively per table.

**#B** Enforces snapshot retention according to each table's policy for compliance and cost control.

**#C** Loads centralized configuration to drive dynamic DAG behavior.

**#D** Passes policy parameters into the task, enabling table-specific logic without hardcoding.

Iceberg supports table-level properties that influence maintenance behavior. For example, the `write.target-file-size-bytes` property can be set per table to control file sizing during compaction or ingestion. Retention settings such as `history.expire.min-snapshots-to-keep` or `history.expire.max-snapshot-age-ms` can be tuned per table to control how long historical metadata is preserved. These properties provide a native mechanism to encode table-specific policies directly within the table definition, which can then be interpreted and enforced by your orchestration layer.

As the number of tables in a lakehouse grows, per-table policies become increasingly important. Uniform schedules may work for small-scale deployments, but they begin to break down in larger environments where data freshness, cost, and compliance must be balanced table by table. Establishing thoughtful policies and embedding them into your orchestration framework allows you to maintain consistent performance, predictable behavior, and operational discipline across a diverse ecosystem of datasets.

In practice, per-table maintenance aligns technical operations with the real data usage. It ensures that Iceberg's powerful features are applied where they matter most, conserving resources and reinforcing the integrity of your lakehouse platform.

#### 10.1.4 Monitoring and alerting integration

Even the best-designed maintenance workflows can fail silently without proper observability. In a production Apache Iceberg lakehouse, knowing that a compaction job ran is not enough; you need to know it was necessary, had the intended effect, and did not disrupt downstream processes. Monitoring and alerting transform passive automation into active oversight.

Unlike traditional databases, where performance can often be tracked by querying a single engine, Iceberg operates in a multi-layered, multi-engine environment. Storage, metadata, and compute are decoupled, introducing more components that must be monitored independently. Maintenance tasks such as snapshot expiration or delete file rewriting often occur outside the main query path. Without observability into these background activities, problems may go unnoticed until performance degrades or storage costs rise.

The first and most immediate source of monitoring data is Iceberg's metadata tables. These tables expose the current state of the system, including file counts, file sizes, snapshot history, and the accumulation of deleted files. Regular queries against the files, snapshots, and manifests tables can surface indicators such as rising small file counts, uneven partition sizes, or unexpectedly long snapshot chains. These are leading signals of deeper performance or operational issues. Although this metadata is covered in depth in Appendix A, its operational use comes from integrating the results into your monitoring systems.

Orchestration tools can query these metadata tables and emit custom metrics to external systems such as Prometheus, CloudWatch, or Datadog. These metrics might include the number of snapshots older than a configured retention window, the number of delete files per partition, or the total size of uncompact files. Once emitted, these metrics can be visualized in dashboards, tracked over time, and used to drive alerts.

### WHAT ARE PROMETHEUS, CLOUDWATCH, AND DATADOG?

Prometheus, CloudWatch, and Datadog are observability tools used to collect, store, and visualize metrics and logs. Each helps track system health and diagnose problems, but they serve slightly different roles:

Prometheus is an open-source monitoring system often used in Kubernetes and containerized environments. It collects time-series metrics via a pull model, stores them locally, and integrates with alerting tools like Alertmanager.

CloudWatch is Amazon's native observability platform. It aggregates metrics, logs, and events from AWS services, including S3 and Glue, making it a natural choice for Iceberg deployments in the AWS cloud.

Datadog is a commercial SaaS platform that provides unified observability across infrastructure, applications, and services. It combines logs, metrics, traces, and dashboards with strong support for alerting and correlation.

For Apache Iceberg, these tools are valuable because much of the platform's state and behavior, like file counts, snapshot sizes, query latency, or compaction frequency, can be surfaced through metrics or logs. Whether you're tracking ingestion health, compaction job status, or catalog access patterns, tools like Prometheus, CloudWatch, and Datadog provide the visibility needed to monitor trends and respond to anomalies before they become incidents.

Consider a scenario where a Merge-on-Read table accumulates delete files faster than they are being rewritten. Over time, this degrades query performance, since each query must merge an increasing number of deletes at runtime. By monitoring the count or size of delete files per partition, you can set an alert to notify the operations team when compaction thresholds are breached, allowing for targeted intervention before users notice the lag.

Query performance metrics are another critical layer of observability. Engines like Dremio, Trino, or Spark typically expose query planning latency, scan time, and task execution metrics. These engine-level metrics should be monitored alongside Iceberg metadata metrics to provide a complete view of system health. For example, if query planning time rises while snapshot counts remain unchanged, the issue might be engine configuration or resource contention. If planning time increases in tandem with snapshot growth, it could indicate overdue snapshot expiration or excessive table branching.

Alerting policies should reflect meaningful risk. Alert fatigue is common in operational environments, especially when thresholds are poorly calibrated or metrics are too noisy. Instead of triggering alerts based on any fluctuation in file count, monitor for patterns that signal true degradation, such as consistent growth in delete files without a corresponding drop after compaction, or data files that remain unreferenced across multiple snapshots. These are symptoms of deeper issues that require human attention or a reconfiguration of automated processes.

Monitoring should also include the orchestration layer. A failed maintenance DAG, a skipped compaction job, or a misconfigured parameter can all compromise table health. Logs and success metrics from tools like Airflow or dbt should be collected and tracked just like any other operational signal. This visibility ensures that maintenance procedures are not only running on schedule, but also completing successfully and producing the expected results.

This log shows a failed compaction task due to a system-level file descriptor limit. The SQL call to `rewrite_data_files` is logged correctly, but the task crashes before completion. In a healthy orchestration system, such failures should be detected, alerted on, and retried or escalated. When monitoring orchestration logs, look for error patterns like repeated task failures, system exceptions (like “too many open files”), or inconsistent task durations. These signs often point to deeper infrastructure or configuration issues that can silently degrade table health if left unresolved.

```
[2025-10-01 02:00:12,345] {taskinstance.py:1064} INFO - Starting task:
compact_orders_table
[2025-10-01 02:00:13,678] {sql.py:76} INFO - Running SQL: CALL
iceberg.system.rewrite_data_files('db.orders')
[2025-10-01 02:00:20,982] {sql.py:81} ERROR - Compaction failed: Too many open
files
[2025-10-01 02:00:21,104] {taskinstance.py:1234} ERROR - Task failed with
exception: OSError: [Errno 24] Too many open files
[2025-10-01 02:00:21,105] {taskinstance.py:1300} INFO - Marking task as FAILED
[2025-10-01 02:00:21,200] {taskinstance.py:1421} INFO - Triggering alert:
maintenance.compaction_failure
```

Integrating monitoring and alerting into your Iceberg deployment is not optional; it is a core requirement for running data infrastructure at scale. By building a feedback loop between metadata, orchestration, and observability, you can detect issues early, respond quickly, and operate with confidence. An Iceberg lakehouse is at its most resilient when its operational state is transparent, measurable, and actively managed.

### 10.1.5 Putting orchestration into practice

To make these concepts more concrete, let's walk through examples of implementing Apache Iceberg maintenance workflows in real orchestration environments. These examples show how to trigger procedures conditionally using metadata, parameterize per-table logic, and emit observability metrics as part of the workflow.

#### EXAMPLE 1: APACHE AIRFLOW DAG FOR METADATA-AWARE COMPACTION

This Airflow example defines a DAG that inspects an Iceberg table for small files and triggers compaction only if the file count exceeds a threshold. The query engine used is Apache Spark, invoked through a Bash operator for simplicity.

```

from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator
from datetime import datetime
import subprocess

default_args = {
    'start_date': datetime(2023, 1, 1),
    'retries': 1,
}

dag = DAG(
    'iceberg_compaction_dag',
    schedule_interval='@daily',
    default_args=default_args,
    catchup=False,
)

def should_compact():
    result = subprocess.check_output([
        "spark-sql",
        "-e",
        "SELECT count(*) FROM mydb.mytable.files WHERE file_size_in_bytes <
20000000;" #A
    ])
    return int(result.strip()) > 500 #B

check_compaction = PythonOperator(
    task_id='check_file_fragmentation',
    python_callable=should_compact,
    dag=dag,
)

compact_table = BashOperator(
    task_id='run_compaction',
    bash_command='spark-sql -e "CALL mydb.system.rewrite_data_files(table =>
\'mydb.mytable\')"' #C
    dag=dag,
)

check_compaction >> compact_table #D

```



- #A Queries Iceberg's files metadata table to count small files below 20 MB
- #B Compaction is triggered only if more than 500 small files are present
- #C Runs the Iceberg compaction procedure using Spark SQL
- #D Defines DAG flow: check condition, then run compaction if needed

## EXAMPLE 2: DBT MODEL WITH SNAPSHOT EXPIRATION POST-PROCESSING

This dbt example chains Iceberg snapshot expiration after a transformation model using run-operation. It assumes the dbt project is configured for a SQL engine that supports Iceberg procedures.

```
-- models/transform_customer_data.sql
SELECT *
FROM raw.customers
WHERE updated_at > CURRENT_DATE - INTERVAL '30 days'

-- macros/expire_snapshots.sql
{% macro expire_snapshots(table_name) %}
    CALL {{ table_name }}.system.expire_snapshots(
        older_than => CURRENT_TIMESTAMP - INTERVAL '7 days'
    );
{% endmacro %}
```

Then, in the job runner:

```
dbt run --select transform_customer_data
dbt run-operation expire_snapshots --args '{"table_name": "warehouse.customers"}'
#A
```

#A Expires snapshots older than 7 days after data transformation is complete

## EXAMPLE 3: EMITTING METADATA METRICS TO PROMETHEUS

In this final example, a Python script queries Iceberg metadata and pushes custom metrics to a Prometheus pushgateway.

```

import requests
import subprocess

def get_snapshot_count():
    result = subprocess.check_output([
        "spark-sql",
        "-e",
        "SELECT count(*) FROM mydb.mytable.snapshots"
    ])
    return int(result.strip())

snapshot_count = get_snapshot_count() #A

requests.post(
    "http://localhost:9091/metrics/job/iceberg_table_monitoring",
    data=f"iceberg_snapshot_count{{table=\"mydb.mytable\"}} {snapshot_count}\n"
    #B
)

```

**#A** Retrieves the current snapshot count from the Iceberg snapshots metadata table

**#B** Sends metric to Prometheus Pushgateway for observability

Each example demonstrates a different layer of orchestration: conditional execution, post-transformation cleanup, and integration with monitoring systems. In practice, these techniques can be combined and extended to form end-to-end operational pipelines tailored to each table's requirements. With Iceberg's SQL-accessible metadata and procedural interfaces, orchestration is not just feasible but fundamental to maintaining a scalable and sustainable lakehouse.

## 10.2 Auditing the lakehouse

Auditing in a modern data lakehouse is more than a regulatory checkbox. It is an essential practice for ensuring operational transparency, enforcing accountability, and providing teams with the confidence that their data ecosystem is behaving as expected. In traditional databases, auditing typically involves transaction logs or limited metadata snapshots. But in Apache Iceberg, the architecture itself is versioned, immutable, and traceable by design, offering rich opportunities for both compliance and observability.

Iceberg tables capture every write operation as a new snapshot, maintaining a complete lineage of changes over time. This snapshot history includes the exact files added or removed, the timestamp of the operation, and the engine or process that performed the commit. This built-in temporal structure enables a detailed audit trail without requiring additional instrumentation or external logging systems. You can answer questions like “Who changed this table?”, “What data was removed and when?”, or “Which files were added by a given job?” with direct queries to the Iceberg metadata tables.

Auditing in Iceberg also extends beyond raw data changes. Because schema evolution is a first-class feature, Iceberg captures when and how schemas are modified. You can trace the addition of new columns, the removal of deprecated ones, and the evolution of data types, all with an eye toward both backward compatibility and governance. This is especially important in environments where data models must align with external specifications or contractual obligations.

Another layer of auditability comes from integration with catalog systems such as AWS Glue, Nessie, or custom REST catalogs. These systems manage access to Iceberg tables and track metadata commits across branches and tags. When paired with access controls and usage logging from query engines like Trino, Spark, or Dremio, they form a comprehensive picture of both how data is being used and who is using it. This enables fine-grained lineage tracking, policy enforcement, and retrospective analysis of issues or anomalies.

The rise of data privacy regulations such as GDPR and CCPA has further elevated the importance of audit readiness. Teams must be able to prove that personal data was deleted when required, that sensitive fields were not accessed inappropriately, and that changes to retention policies were consistently applied. Iceberg's metadata model, combined with thoughtful orchestration and monitoring, supports these requirements through explicit, queryable records of change. For example, you can verify that a file containing sensitive data was removed by querying the DELETE entries in the `metadata_log_entries` metadata table, and cross-referencing them with the files metadata table:

```
SELECT f.file_path
FROM my_table.files f
LEFT JOIN my_table.metadata_log_entries m
  ON f.file_path = m.file
WHERE m.operation = 'DELETE'
  AND f.partition = 'region=EU'
  AND f.last_modified < DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY);
```

This query identifies deleted files that were stored in the EU partition and had not been modified for at least 30 days, a typical GDPR compliance threshold. While the metadata layer does not store the data itself, it provides traceability of deletions and changes.

Let's explore how to build a practical auditing framework around Apache Iceberg. We will examine the tools and practices that surface change history, expose usage patterns, and support data governance objectives. Whether your goal is compliance, operational control, or internal quality assurance, Iceberg provides the foundation for auditing at scale, without sacrificing performance or flexibility.

### 10.2.1 Leveraging snapshot history for change tracking

One of the most powerful and underutilized features of Apache Iceberg is its built-in versioning system. Every time data is written to an Iceberg table, whether through an insert, update, delete, or schema change, a new snapshot is created. These snapshots are not just technical artifacts. They are structured records of change, capturing a complete picture of what was modified, when the change occurred, and which files were involved.

This time-ordered sequence of snapshots enables detailed change tracking at a level of precision that few other data systems offer. Unlike traditional append-only logs or limited audit tables, Iceberg's snapshot history is deeply integrated into its metadata model. You can query snapshots directly using the snapshots metadata table, which includes fields such as the snapshot ID, parent snapshot (if any), timestamp, operation type, and summary statistics about the data files added or removed. These summaries can be used to trace both the lineage of data and the evolution of the table structure over time.

For example, suppose a data engineer wants to investigate an unexpected change in row counts for a critical fact table. In that case, they can start by querying the snapshots table to identify recent writes. Each snapshot contains a summary map, which may include metrics like the number of records added, removed, or rewritten. By comparing these values across snapshots, the engineer can quickly narrow down the specific commit that introduced the discrepancy. From there, they can drill deeper using the manifest\_list and manifests metadata tables to identify the exact files added or removed.

For example, if you run this query:

```
SELECT snapshot_id, committed_at, operation, summary
FROM "db"."critical_fact_table.snapshots"
ORDER BY committed_at DESC;
```

And get back the following data:

| snapshot_id         | committed_at             | operation | summary                                                 |
|---------------------|--------------------------|-----------|---------------------------------------------------------|
| 6583920471938451112 | 2025-10-10T02:01:12.321Z | append    | {"added-records": "120000", "total-records": "1120000"} |
| 6583920471938451111 | 2025-10-09T02:01:11.876Z | overwrite | {"removed-records": "50000", "added-records": "48000"}  |
| 6583920471938451110 | 2025-10-08T02:01:10.452Z | append    | {"added-records": "100000", "total-records": "1102000"} |
| 6583920471938451109 | 2025-10-07T02:01:09.201Z | append    | {"added-records": "90000", "total-records": "1002000"}  |

This table shows a history of recent commits made to a fact table. By examining the summary column, a data engineer can see how many records were added or removed in each snapshot. The entry for October 9 stands out: an overwrite operation removed 50,000 records but only added back 48,000. This two-thousand-record discrepancy might explain a drop in row counts. Investigating this snapshot further via the `manifest_list` or `manifests` metadata tables can help identify which files were involved in the overwrite, narrowing the scope of the audit and pinpointing potential data quality issues.

This level of visibility is particularly valuable in collaborative environments, where many teams may be writing to the same table. Snapshots make it possible to isolate the changes made by a specific job or user, even in the absence of external logs or metadata. In environments that use branching, such as with Project Nessie or other versioned catalog systems, snapshot tracking can also help differentiate between changes made on isolated development branches versus those committed to the main production lineage.

Change tracking via snapshots is not limited to data alone. Schema modifications also generate new snapshots. For instance, adding a new column or changing a column's data type results in a metadata update and a new snapshot. This makes it possible to build a complete audit trail of schema evolution by examining the snapshot sequence over time. Teams can use this to enforce data modeling conventions, validate backward compatibility, or simply understand how the data structure has changed as the application has evolved.

From an operational standpoint, snapshots also provide a practical foundation for time travel queries. If a problematic change is identified, analysts can query the table as of a previous snapshot ID to see what the data looked like before the issue occurred. This capability supports debugging, data validation, and even rollback scenarios. It also reinforces trust in the data platform by making changes transparent and reversible.

To make use of these capabilities in production, organizations should consider regular snapshot inspections as part of their audit routines. Orchestration tools can be configured to log snapshot summaries after each major write, or to raise alerts when unusual patterns are detected, such as large spikes in removed records or rapid increases in snapshot frequency. These signals can help detect pipeline issues, misconfigurations, or even malicious activity early, before they cause downstream impact.

Apache Iceberg's snapshot history is not just a technical detail; it is a built-in ledger of the lakehouse. By leveraging it effectively, teams gain the ability to trace data changes with confidence, improve accountability, and support robust audit and compliance processes across their data estate.

## 10.2.2 Using branching and tagging for governance

Governance in a data platform often centers on the ability to isolate changes, control visibility, and track lineage. In traditional systems, this is typically managed through environment separation, such as dev, test, and prod databases, or through manual export-import cycles that carry the risk of drift and human error. Apache Iceberg introduces a more elegant approach: branching and tagging. These features offer a version control-inspired model for managing data evolution. Still, their usage differs depending on whether you're referring to Iceberg's native table-level features or catalog-level capabilities provided by systems like Project Nessie.

At the table level, Apache Iceberg supports snapshot references in the form of named branches and tags. These references allow you to assign a stable, user-defined name to a specific snapshot. For example, tag a snapshot as `release_2023_01` to capture a known good state of a table used for monthly reporting. Tags are immutable and serve as bookmarks in time, while branches are mutable and can advance to newer snapshots, making them suitable for scenarios like development pipelines where the state evolves.

For example, let's assume we query a table's "ref" table to see its listing of branches and tags:

```
SELECT ref_name, type, snapshot_id, max_reference_age_in_ms
FROM "db"."critical_fact_table.refs";
```

We may get back the following:

| ref_name        | type   | snapshot_id         | max_reference_age_in_ms |
|-----------------|--------|---------------------|-------------------------|
| main            | branch | 6583920471938451112 | NULL                    |
| dev             | branch | 6583920471938451111 | 604800000               |
| release_2023_09 | tag    | 6583920471938451107 | NULL                    |

Here's how to interpret it:

- `main` points to the latest stable snapshot in production.
- `dev` is a branch that advances automatically as new snapshots are added, often used for QA or analyst testing. Its max age (7 days) means it may be garbage collected after that window.
- `release_2023_09` is a fixed tag that always points to the same snapshot, ensuring that any downstream job can rerun against a consistent data version, even if new data has since been appended.

Using the `refs` table, teams can manage snapshot lifecycles with precision, supporting reproducibility, isolating development changes, and enforcing retention policies tied to data references rather than raw timestamps.

However, these snapshot references are scoped to individual tables. They allow for snapshot isolation and rollback at the table level, but they do not provide isolation across an entire namespace or a multi-table workflow. That is where catalog-level branching, as implemented in Project Nessie, comes into play.

Nessie builds on Iceberg's snapshot model by extending versioning to the catalog level. In Nessie, a branch or tag applies to the entire set of tables within a catalog namespace. This enables you to treat your data platform more like a Git repository. A developer can create a branch of the entire environment, make changes to multiple Iceberg tables, test them in isolation, and then merge the branch back into main once the changes are validated. This model dramatically improves the governance and safety of data workflows, especially in multi-team environments where changes to schemas or data can have cascading effects.

Using catalog-level branching also enables a new class of testing and approval workflows. For instance, a data modeling team could create a `feature/extended_customer_schema` branch, apply schema updates, and populate test data, then share that branch with the analytics team to validate their dashboards before any change is promoted to production. Tags can be used to lock down historical states for regulatory or auditing purposes, ensuring that previously reported metrics remain reproducible, even as data continues to evolve.

Governance policies can leverage these references to control retention and lifecycle behavior. Iceberg does not allow snapshots to be expired while they are still referenced by a tag or branch. This means that long-lived tags, such as those created for compliance or reporting checkpoints, can effectively pin data in place, protecting it from deletion until explicitly released. On the other hand, short-lived development branches can be expired regularly to avoid bloating the metadata store. Understanding this distinction is essential for managing storage costs and ensuring auditability without creating long-term data sprawl.

Integration with access control systems is another advantage. Some catalog implementations can restrict access to specific branches or tags, enabling multi-tenant environments where users can interact only with their own isolated environments. For example, a machine learning team might have access to a dedicated experimental branch, while analysts see only the validated and merged production view. These controls help enforce the separation of duties and reduce the risk of accidental corruption or data leakage.

It is essential to recognize, however, that branching and tagging require discipline. Just as with source code, branches can diverge and drift over time. Without regular cleanup, the metadata associated with stale branches can accumulate, affecting performance and increasing storage overhead. Teams should establish governance policies for branch naming, usage, and expiration to ensure that the benefits of version isolation do not come at the cost of operational complexity.

Iceberg's snapshot-level references and Nessie's catalog-level branching together create a strong foundation for data governance. They offer a consistent and reliable way to track changes, isolate experiments, support audit requirements, and enforce organizational policies. By integrating these features into your data workflows, you gain not just version control but a framework for managing data responsibly across the entire lifecycle.

### 10.2.3 Implementing file and snapshot retention policies

Retention policies in Apache Iceberg serve a dual purpose. First, they help control storage growth by pruning outdated snapshots and unreferenced files. Second, they support data governance by ensuring that expired data is actually removed, whether to enforce business lifecycle rules or meet compliance obligations. While Iceberg provides the foundational mechanisms to expire snapshots and clean up orphan files, effectively implementing these operations requires thoughtful coordination and an understanding of how different workloads behave.

At the heart of Iceberg's retention model is the concept of snapshot expiration. Every Iceberg table consists of a linear or branched history of snapshots, each capturing the state of the table at a point in time. When a snapshot is expired using the `expire_snapshots` procedure, it becomes eligible for cleanup, along with any data or metadata files that are no longer referenced by any other active snapshot, tag, or branch. This makes snapshot expiration the primary mechanism for controlling data version retention. However, simply calling `expire_snapshots` is not enough. To free up physical storage, orphaned files must also be identified and removed using procedures like `remove_orphan_files`. Let's look at an example:



```

from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from datetime import datetime, timedelta
import subprocess

default_args = {
    'owner': 'dataops',
    'start_date': datetime(2025, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

def expire_snapshots():
    subprocess.run([
        "spark-submit",
        "--class", "org.apache.iceberg.actions.ExpireSnapshots",
        "--master", "local",
        "maintenance.jar",
        "--table", "db.sales",
        "--older-than", "7d"
    ])

def remove_orphans():
    subprocess.run([
        "spark-submit",
        "--class", "org.apache.iceberg.actions.RemoveOrphanFiles",
        "--master", "local",
        "maintenance.jar",
        "--table", "db.sales",
        "--older-than", "3d"
    ])

with DAG('iceberg_retention_policy',
        default_args=default_args,
        schedule_interval='@daily',
        catchup=False) as dag:

    expire = PythonOperator(
        task_id='expire_old_snapshots',
        python_callable=expire_snapshots
    )

    cleanup = PythonOperator(

```

```

        task_id='remove_orphan_files',
        python_callable=remove_orphans
    )

    expire >> cleanup #C

```

**#A** Calls the `expire_snapshots` procedure on the `db.sales` table, removing snapshots older than 7 days to enforce retention policy.

**#B** Executes `remove_orphan_files` to clean up unreferenced data and metadata files older than 3 days, ensuring storage is actually reclaimed.

**#C** Enforces task order: orphan cleanup runs only after snapshot expiration, ensuring no active references are mistakenly deleted.

This DAG ensures that snapshot history is trimmed based on policy and storage remains optimized, while protecting against premature deletion of in-use files.

In practice, coordinating snapshot expiration and orphan file cleanup is essential to avoid the accumulation of unused data. This is especially true in Merge-on-Read (MOR) tables, where delete files can span multiple snapshots and are often reused by Iceberg's commit logic. If not managed carefully, MOR tables can retain large volumes of delete files and outdated data files even after the relevant data has been logically deleted. For Copy-on-Write (COW) tables, the retention model is simpler, but still depends on regularly expiring old snapshots to ensure dereferenced files are eligible for deletion.

Retention policies must also account for the nature of the workload. Transient workloads, such as staging tables, temporary job outputs, or testing environments, may benefit from aggressive snapshot expiration, sometimes within hours of creation. In contrast, regulated datasets often require more extended retention periods to meet audit, legal, or contractual obligations. These tables may retain snapshots for weeks or months, and in some cases may be pinned indefinitely using tags or branches. Iceberg supports both use cases through flexible expiration options: snapshots can be expired based on age, timestamp, or number of retained versions, using the `retain_last` parameter or explicit time cutoffs.

To support automated retention across varied workloads, teams can define policies that reflect the purpose of each table. For example, tables used in experimentation may include orchestration logic to expire snapshots hourly and clean up orphan files daily. Production tables subject to compliance rules may instead retain the last 10 versions and rely on manual review before expiration. These policies should be encoded as part of the orchestration framework discussed earlier in this chapter, ideally using metadata-aware triggers to enforce consistency.

One helpful technique is to embed retention policies directly into the metadata of the table or catalog, such as via table properties or catalog annotations. For instance, a policy may specify that a table should retain 30 days of snapshots or that orphan file cleanup should be skipped for datasets under legal hold. These annotations can be referenced by orchestration jobs to ensure that the correct actions are taken per table, without hardcoding behavior into the workflow itself.

Ultimately, retention in Apache Iceberg is not a one-size-fits-all setting. It is a dynamic process that must adapt to the table's role, the organization's compliance requirements, and the behavior of upstream ingestion pipelines. By aligning expiration policies with business context and automating their enforcement, teams can maintain a lakehouse that is both efficient and compliant, with data lifecycles that are transparent, traceable, and well-governed.

#### 10.2.4 Practical retention policy orchestration

Designing a robust retention policy is only half the challenge in maintaining a healthy Apache Iceberg environment. The other half lies in operationalizing it, turning those rules into repeatable, reliable, and safe procedures that execute automatically across your lakehouse. This is where orchestration and procedural automation play a key role. In Iceberg, retention operations are exposed as native SQL procedures that can be invoked from Spark, Dremio, Trino, or any engine integrated with the Iceberg catalog. These procedures form the backbone of automated lifecycle management for snapshots, data files, and metadata.

At the core of any retention workflow is the `expire_snapshots` procedure. This operation removes snapshots that are no longer needed and, crucially, deletes any data or metadata files that are not referenced by remaining snapshots, tags, or branches. Because Iceberg's snapshot architecture provides strong isolation and guarantees that no active snapshot references expired data, this procedure is both safe and efficient. It not only trims the version history but also physically reclaims storage space tied to obsolete data files. This makes `expire_snapshots` the primary mechanism for enforcing retention boundaries across tables, ensuring that old data is purged in a controlled and consistent manner.

In contrast, `remove_orphan_files` serves a more specialized role. This procedure cleans up files that were never correctly linked to any snapshot, typically leftovers from failed or aborted write operations, uncommitted transactions, or external processes that wrote data directly to the table's storage path. Because Iceberg manages all table files through its metadata tree, any file not referenced by that tree is effectively invisible to the table but can still occupy space in object storage. While `expire_snapshots` removes valid but outdated data, `remove_orphan_files` deals with invalid artifacts that never belonged to a valid table state. Together, the two procedures ensure that your Iceberg storage footprint remains both lean and accurate.

The cadence at which these procedures run depends on workload characteristics. For transactional or continuously updated tables, snapshot expiration should be frequent, sometimes hourly or even more often, to prevent runaway growth of data and metadata files. Tables that serve analytical workloads, where version history supports reproducible reporting, may expire snapshots on a daily or weekly basis instead. In both cases, aligning expiration schedules with operational rhythms such as ingestion windows or compaction jobs helps maintain stability. Orphan cleanup can occur less often, perhaps daily or weekly, since it targets infrequent anomalies rather than normal table evolution.

A key principle in orchestrating retention is ordering. Snapshot expiration should always precede orphan file cleanup. Running orphan cleanup too early risks deleting files still referenced by unexpired snapshots, which could compromise table integrity. Iceberg's design mitigates this risk by preventing the deletion of active files, but maintaining correct sequencing in orchestration ensures predictable behavior and efficient resource use.

Another important consideration is granularity. Not every table should follow the same retention rules. Transient datasets, such as ingestion staging tables, may safely expire snapshots within hours and aggressively remove orphans to minimize cost. Regulatory datasets, however, often require much longer retention windows to preserve auditable change histories. These distinctions can be captured using table properties, catalog annotations, or external policy registries that orchestration systems query before executing retention tasks. Embedding policies into metadata ensures that each table is treated according to its operational and compliance requirements, without requiring manual intervention.

A well-orchestrated retention process ensures that Iceberg tables remain performant, compliant, and cost-efficient. The goal is not simply to delete old data, but to continuously maintain a coherent and optimized metadata state across the lakehouse. By integrating snapshot expiration and orphan cleanup into a unified, automated lifecycle, data teams can preserve the reliability and performance guarantees that make Apache Iceberg an enterprise-ready table format.

### 10.2.5 Secure data deletion

Deleting data in a lakehouse context is not just about marking records as deleted; it is about guaranteeing that they are ultimately removed in a way that satisfies compliance, auditability, and operational safety. In Apache Iceberg, secure data deletion must reconcile the logical world of snapshots and delete files with the physical reality of data files on object storage. This section examines how Copy-on-Write (COW) and Merge-on-Read (MOR) models differ in their deletion semantics, why compaction is often a necessary step for hard deletion, and how to reliably enforce the full deletion lifecycle.

#### DELETION SEMANTICS: LOGICAL VS. PHYSICAL

When you issue a delete or update in Iceberg, the change is first applied at the logical level. In Copy-on-Write (COW) mode, Iceberg rewrites the affected data files immediately, producing new versions that omit the deleted or updated rows. In Merge-on-Read (MOR) mode, the change is recorded in a separate delete file, either a position delete or, in newer versions, a deletion vector, while the original data file remains intact. In both modes, the table's snapshot is updated to reflect the new state. However, this does not immediately remove the deleted data from physical storage. Because Iceberg retains full snapshot history, queries against older snapshots will continue to return logically deleted rows until those snapshots are explicitly expired. Physical deletion occurs only when the system de-references the relevant files during snapshot expiration and orphan cleanup. Until then, deleted rows persist in storage and remain accessible through time-travel queries.

This distinction is critical for compliance use cases. A record that has been logically deleted still resides in data files until those files become unreferenced by all snapshots or delete files. Therefore, enforcing secure deletion requires coordinating snapshots, compaction, and file cleanup in the correct sequence.

## THE ROLE OF COMPACTION IN HARD DELETION

In MOR mode, delete files accumulate over time, applying to data files during read time to filter out deleted rows. But until the data files are actually rewritten to exclude the deleted rows, those rows remain physically present. That means that until compaction merges delete files into fresh base files, a motivated actor might be able to reconstruct deleted data from old snapshots or by bypassing query logic.

Therefore, compaction becomes a necessary step for physical deletion. Once delete files and base files are merged in a compaction job (via `rewrite_data_files` or `OPTIMIZE` in engines like Dremio), the resulting files omit the deleted rows, and old file versions containing the data are eligible for deletion when snapshots expire.

---

**NOTE** In more recent versions (e.g., Iceberg v3), deletion vectors replace or augment position delete files, improving the efficiency and compactness of delete tracking. These newer mechanisms reduce the overhead of delete file accumulation while preserving the ability to clean up deleted rows during compaction. [Amazon Web Services, Inc.](https://aws.amazon.com/iceberg/)

---

## ENFORCING DELETION END-TO-END

A mature deletion strategy must ensure that all stale references are cleared out. In a typical deletion lifecycle for MOR tables, you need to:

1. Issue a delete or update operation (creating new delete files).
2. Run compaction (rewrite data files so that deleted rows are eliminated).
3. Expire snapshots to drop references to older file versions.
4. Clean up orphan files, that is, files that never belonged to a valid snapshot (mostly artifacts from failed writes).

Compaction steps must precede snapshot expiration. If snapshots expire before compaction, you risk permanently losing the ability to remove those rows physically, because the files containing deleted records may be removed while they remain in use by delete files or other snapshots. A robust orchestration ensures compaction, expiration, and cleanup occur in order, with safety checks, such as checking the delete-file volume or snapshot age, to prevent premature execution.

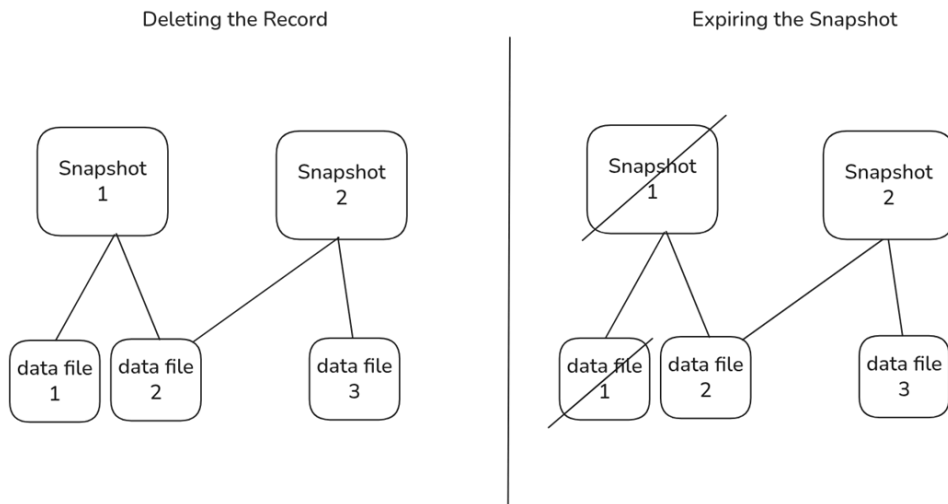
For COW tables, the process is more straightforward but still nontrivial. Since deletes are applied eagerly by rewriting data files, the main task is to expire snapshots and trigger cleanup to free up space. However, you still need to ensure that snapshots referencing old files are appropriately managed so that no active snapshot pins out-of-date data.

## DELETION GUARANTEES AND LIMITATIONS

Secure deletion is strongest when your retention and deletion pipelines are consistent and auditable. But there are a few nuances to keep in mind:

- Branches and tags prevent snapshot expiration for any snapshot they reference. If you create long-lived tags to preserve historic states, those snapshots will block the deletion of underlying files until the tag is removed or expires.
- In MOR mode, delete files reused across snapshots can delay cleanup. If a delete file is referenced by a snapshot you wish to keep, the data file it points to cannot be deleted until that snapshot expires or compaction eliminates those delete references.
- Deletion vectors (in newer Iceberg versions) simplify the delete file model, but compaction is still required to eliminate deleted rows physically. The difference is that deletion vectors reduce fragmentation, metadata overhead, and the number of files involved. [Amazon Web Services, Inc.](#)
- Final deletion only occurs when a file becomes unreferenced. Merely calling deletion procedures does not guarantee immediate purge; expiration must remove all snapshot-level references before cleanup can safely delete the files.

Secure deletion in an Apache Iceberg lakehouse is a multi-step process: it begins with logical deletion (via delete files or direct rewrites) and culminates in physical data removal through compaction and snapshot expiration, as illustrated in figure 10.3. Consistently enforcing this cycle, in the correct order and with visibility, is what separates superficial deletes from those that truly remove sensitive data. In the next section, we will look at practical examples of how deletion strategies can be implemented across orchestration frameworks, tying together code and policies with Iceberg's procedural APIs.



**Figure 10.3** When a record is deleted a new snapshot is created that doesn't include the deleted data, but the data files still exist. Once all snapshots associated with the files with the deleted data are expired are those data files actually removed from storage.

## 10.2.6 Access auditing and governance

In modern data lakehouses, access auditing and governance are no longer optional. As data becomes increasingly valuable and regulations grow more stringent, lakehouse platforms must offer fine-grained visibility into who accessed what, when, and how, along with precise controls to restrict access based on business roles, sensitivity, and data residency requirements. Apache Iceberg provides a strong foundation for this with its metadata-rich, immutable snapshot model, but the real enforcement happens at the catalog and engine layers.

Catalogs are central to this architecture. They act as the control plane for Iceberg tables, mediating access between users and the underlying storage. Modern catalog implementations, such as AWS Glue, Nessie, Apache Polaris, Apache Gravitino, Google BigLake, and Microsoft OneLake, go beyond simple table registration. They increasingly support built-in access control mechanisms, audit trails, and integration with enterprise identity providers.

AWS Glue, for example, offers integration with AWS Lake Formation, enabling column-level access controls enforced across services like Athena, EMR, and Redshift Spectrum. This is particularly useful when sensitive fields like customer PII need to be restricted by role. Google BigLake, on the other hand, integrates with Google Cloud IAM to provide unified access policies across BigQuery and other services that read from Iceberg-compatible tables. Similarly, Microsoft OneLake uses Microsoft OneLake Security for governance, allowing policy-based access and classification across structured and unstructured data.

Apache Polaris, built natively for Iceberg, aims to provide first-class governance tailored to Iceberg's metadata model. It incorporates native role-based access controls (RBAC) and can enforce data security policies down to the row and column level. Apache Gravitino, a more experimental entrant, extends Iceberg's governance reach across multi-modal and federated environments, enabling a unified policy layer that works across diverse catalogs and compute engines.

Nessie, while focused on version control and data branching, indirectly supports governance by offering isolated development environments through catalog-level branching. This allows data engineers and analysts to experiment without impacting production data, enforcing governance through environment separation rather than policy enforcement alone.

But catalog policies alone are not enough. To achieve full auditability, organizations must implement end-to-end lineage and logging. Tools like OpenLineage and Amundsen can be integrated with Iceberg-compatible engines such as Spark, Trino, or Dremio to track query history and data flow across pipelines. These tools rely on Iceberg's deterministic metadata and manifest tracking to reconstruct the full provenance of each dataset. This becomes especially important in regulated industries where data access must be clearly explained and justified.

The following example shows how to enable lineage tracking in a Spark environment using OpenLineage. The configuration registers a listener that captures metadata about the Spark job, including input and output tables, query plans, and timestamps, and forwards it to a centralized OpenLineage collector:

```

from openlineage.spark import OpenLineageSparkListener
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("iceberg-lineage-tracking") \
    .config("spark.sql.catalog.my_catalog",
"org.apache.iceberg.spark.SparkCatalog") \
    .config("spark.sql.catalog.my_catalog.type", "hadoop") \
    .config("spark.sql.catalog.my_catalog.warehouse", "s3://my-bucket/warehouse") \
    .config("spark.extraListeners",
"openlineage.spark.agent.OpenLineageSparkListener") \  #A
    .config("openlineage.transport.type", "http") \
    .config("openlineage.transport.url", "http://openlineage-collector:5000") \
    #B
    .getOrCreate()

df = spark.read.table("my_catalog.db.critical_fact_table")
df.filter("region =
'US').writeTo("my_catalog.db.reporting_fact_table").overwrite()  #C

```

**#A** Registers the OpenLineage listener, which captures metadata from Spark jobs and forwards it to a lineage collector.

**#B** Configures the transport mechanism and endpoint for sending lineage events, making audit records available centrally

**#C** Represents a real data transformation that will be logged by OpenLineage, including input and output table metadata, query plan, and timestamp.

This setup ensures that each job reading from or writing to Iceberg tables is logged with full provenance. When combined with snapshot metadata, OpenLineage allows teams to answer critical questions like “What pipeline generated this table version?” or “Who last updated this data and when?”, all without relying on brittle manual logging.

Multi-engine consistency is another challenge in Iceberg lakehouses. Because Iceberg allows concurrent access from different engines, governance policies must be uniformly respected across them. This is non-trivial: for example, a policy enforced in Spark must be reflected in Trino or Dremio. Catalogs with centralized policy definition, such as Unity Catalog (for Databricks Delta), Polaris, or Lake Formation, help address this by providing a single source of truth for access control and auditing.

Ultimately, access governance in Iceberg lakehouses depends on the strength of the catalog and its ecosystem integrations. While Iceberg enables the underlying data model and metadata tracking, it is the catalog’s responsibility to enforce security boundaries and expose visibility. Choosing a catalog that aligns with your security, compliance, and interoperability goals is critical to delivering a secure, auditable lakehouse.



## 10.2.7 Practical auditing with Iceberg: Example workflows

Auditing in Apache Iceberg revolves around making data access and mutation transparent, traceable, and integrable with governance systems. While Iceberg itself is not a full-fledged audit system, its rich metadata model, append-only snapshot architecture, and REST-compatible catalog ecosystem make it well-suited for audit augmentation. This section walks through example workflows that show how Iceberg metadata can be used to track read/write behaviors and connect with broader auditing systems.

### SNAPSHOT METADATA EXTRACTION FOR CHANGE VISIBILITY

Every mutation in Iceberg creates a new snapshot. By querying the snapshots and history metadata tables, auditors can inspect who made changes, when, and what files or operations were involved.

```
SELECT
  snapshot_id,
  operation,      --#A
  committed_at,  --#B
  user_id        --#C
FROM my_catalog.db.table.snapshots
ORDER BY committed_at DESC;
```

**#A** This captures the type of mutation, such as append, overwrite, or delete.

**#B** The timestamp is crucial for aligning table events with access logs or security incidents.

**#C** When using a REST catalog like Polaris or Gravitino, the `user_id` field records the identity responsible for the commit.

### ACCESS LINEAGE WITH OPENLINEAGE

To understand how data flows through an Iceberg table and into downstream consumers, tools like OpenLineage can be wired into the Spark or Flink engine. These tools hook into job execution to log when and how tables were read or written.

This code constructs a `RunEvent` using the OpenLineage client library. It is executed by the ETL process itself—not by the OpenLineage server, and is responsible for logging metadata about the job execution. The event includes a `runId`, job name, and optional facets like the source code location, which are passed as parameters when the job runs. Although the lineage event is emitted independently of Iceberg, it can be correlated with Iceberg snapshots using timestamp alignment or external job metadata. Catalogs like Nessie or Polaris can then associate the lineage metadata with specific table versions, making it possible to trace which jobs produced or modified particular snapshots. Iceberg's immutable snapshot model and deterministic metadata make this correlation reliable, even though the auditing logic operates outside the table itself.

```

from openlineage.client.run import RunEvent, RunState
from openlineage.client.facet import SourceCodeLocationRunFacet

RunEvent(
    eventType=RunState.START,
    job={
        "namespace": "datalake-engine",
        "name": "daily_etl_job"
    },
    run={
        "runId": "abcd-1234"
    },
    producer="https://my.lineage.collector",
    facets={
        "sourceCodeLocation": SourceCodeLocationRunFacet(
            type="git",
            url="https://github.com/my-org/etl-jobs",
            branch="main"
        )
    }
)

```

## DETECTING UNUSUAL ACCESS PATTERNS VIA METADATA

An often-overlooked audit use case is detecting anomalies in usage. By regularly querying the snapshots and manifests metadata tables, teams can build baselines for how frequently tables change and the types of files modified.

```

SELECT
    COUNT(*) AS commits,
    MAX(committed_at) AS last_commit,
    MIN(committed_at) AS first_commit
FROM my_catalog.db.sales.snapshots
WHERE committed_at > current_date - INTERVAL '30' DAY

```

If a table suddenly experiences an uptick in overwrites or deletions outside normal hours or beyond known job schedules, that may indicate a misconfiguration or even a malicious actor. Combining this with engine-level access logs gives a more complete picture.

## CATALOG-SPECIFIC ENHANCEMENTS

Modern catalogs bring richer auditing capabilities. Apache Polaris logs mutation details to cloud-native audit streams, while Nessie enables commit history inspection across branches. AWS Glue integrates with CloudTrail to log metadata API calls. When Iceberg is used through these catalogs, auditing improves through additional layers of metadata and authorization context.

### 10.3 Disaster recovery in the lakehouse

Data lakehouses must be built not just for scale and speed but for resilience. As the volume and velocity of data increase, so does the risk of failure, from hardware faults and accidental deletions to software bugs and misconfigured automation. Apache Iceberg brings a transactional model to the lakehouse, offering powerful capabilities like snapshot isolation, atomic commits, and metadata rollback. But these features are only one part of a comprehensive disaster recovery strategy.

Unlike traditional data warehouses, Iceberg-based lakehouses rely on loosely coupled components: object storage, compute engines, and metadata catalogs. This decoupling provides flexibility and scalability, but it also introduces new risks. If a metadata catalog becomes corrupted, or if snapshots are mismanaged, data access may be disrupted even if the underlying files remain intact. Similarly, deletion of physical files from cloud storage, whether intentional or accidental, can result in unrecoverable data loss if no safeguards are in place.

This section focuses on the practical aspects of preparing for, detecting, and recovering from such failures. While chapter 9 discussed how to maintain an Iceberg table through compaction, snapshot expiration, and delete file rewriting, those techniques assume a healthy environment. Disaster recovery planning begins where regular maintenance ends. It considers what happens when the metadata is lost, when an object store region fails, or when a human error propagates damaging changes to critical data.

The strength of Apache Iceberg in this context lies in its explicit metadata structure. Tables are composed of immutable manifests and snapshots that can be versioned, backed up, and restored independently of the data files. Tools and techniques such as catalog backups, object storage versioning, and metadata reconstruction from metadata.json files enable organizations to recover quickly and accurately from disruptions. Furthermore, features like branching and time travel provide additional guardrails for logical recovery when physical files remain intact.

Let's explore how to plan for catalog failures, safeguard against file loss, implement cross-region strategies, and use Iceberg's native versioning for rollback. Together, these practices enable durable, self-healing lakehouse architectures that meet both business continuity requirements and compliance obligations.

#### 10.3.1 The role of the metadata catalog in disaster recovery

In Apache Iceberg, the metadata catalog is the gateway to every operation on a table. Whether a user runs a query, inserts new data, or initiates a maintenance task, the action begins with a lookup in the catalog. This component stores the logical name of a table and resolves it to the most recent metadata location, typically a path to a metadata.json file. Without a functioning catalog, a table may still exist on object storage, but it becomes invisible and unusable to downstream engines.

This makes the catalog both a single point of coordination and a potential single point of failure. A corrupted or deleted catalog entry severs access to the Iceberg table even though all metadata and data files might still be intact. Recovery, in such cases, requires not just restoring the catalog's functionality but reestablishing the link between the catalog and the correct metadata location on disk or cloud storage.

Apache Iceberg was designed with this separation in mind. It intentionally decouples table metadata from the catalog to enable flexible deployments and recovery strategies. This design lets users adopt diverse catalog backends such as AWS Glue, Nessie, Apache Polaris, Gravitino, Google BigLake, and Microsoft OneLake, each of which supports Iceberg's catalog interface. However, these systems vary widely in their resilience and backup capabilities.

For example, AWS Glue is a managed Hive-compatible catalog, but it does not natively support versioning of table metadata. This means users must manually export and back up Glue entries if they want point-in-time recovery. Apache Nessie, by contrast, implements a Git-like model for catalogs. It natively supports branches and tags, which can help recover catalog state after accidental deletions or schema changes. Apache Polaris goes further by offering transactional guarantees across the catalog, providing greater consistency and rollback options during failure recovery.

Regardless of the backend, the ability to re-register a table using a known `metadata.json` file is central to Iceberg's recovery model. The `register_table` procedure allows a user to manually create a new catalog entry that points to the appropriate metadata. This means even if the catalog is lost or corrupted, as long as the metadata is still available in storage, the table can be restored.

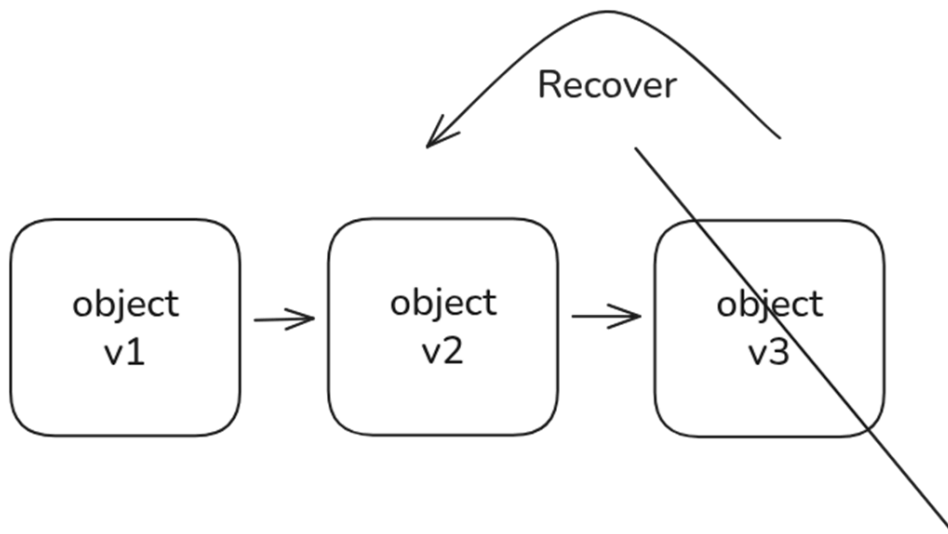
However, relying solely on this mechanism assumes that the correct metadata file is known and accessible. In practice, this requires consistent metadata versioning, external change tracking, and an automated snapshot of catalog contents or metadata file locations. Periodically exporting catalog mappings or enabling audit logging can make a difference between a full recovery and permanent data loss.

Thus, the catalog's role in disaster recovery is both foundational and vulnerable. It requires deliberate planning: selecting a resilient backend, enabling backups where possible, and ensuring metadata can be traced and reconnected. In subsequent sections, we will explore techniques to preserve metadata files, validate catalog consistency, and even perform full reconstruction of tables when catalog records are irretrievable.

### 10.3.2 Protecting against data loss and corruption

While Apache Iceberg's metadata model provides a solid foundation for versioned table operations, no disaster recovery plan is complete without safeguarding the actual data. Data loss and corruption can stem from many sources: accidental deletions from object storage, misconfigured automation jobs, hardware failures, or even malicious actions. Unlike traditional databases, Iceberg stores its data and metadata in external object storage systems such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage. This offers scalability and cost-efficiency, but also shifts the burden of durability and protection to the underlying storage and operational practices.

Object storage systems are designed for high durability but not necessarily for high recoverability. If a file is overwritten or deleted without versioning enabled, it is permanently lost. Therefore, enabling object versioning is a foundational step in protecting Iceberg data as illustrated in figure 10.4. Services like S3 Versioning or Azure Blob Snapshots allow you to retain historical versions of files, which can be critical during recovery. When used with immutable storage policies or object lock configurations, these features can also help defend against ransomware and unintended overwrites.



**Figure 10.4 With object versioning if an object is corrupted or lost you can revert to the previous version of object, the object being a data or metadata file on storage.**

Another critical line of defense is separation of permissions. Many Iceberg deployments give different tools access to the same data lake. For example, one engine may write data while another manages compaction or snapshot expiration. Ensuring that only specific services have delete privileges over critical data files can limit the blast radius of a failure or misconfiguration. Catalogs like Polaris and Nessie offer fine-grained permission models to enforce these policies at the metadata level, and cloud IAM roles can restrict actions at the storage level.

Still, not all risks come from external deletions. Incomplete or failed operations during write-heavy workflows can lead to orphaned files, those written to storage but never committed to an Iceberg snapshot. While these files do not pose a correctness issue, they do increase storage costs and can introduce confusion during audits. Regular use of Iceberg's `remove_orphan_files` procedure can eliminate these files safely, but it requires careful coordination with the write pipelines to avoid deleting files from in-flight operations.

Corruption risks also apply to metadata itself. Although Iceberg's manifests and snapshots are immutable, they can be affected by failures in the storage layer or by bugs in write clients. For this reason, it's essential to configure redundant storage across availability zones or regions. Storage classes with automatic replication, like Amazon S3's cross-region replication or Azure's GRS (Geo-Redundant Storage), provide additional safety against zonal outages or regional disasters.

Guardrails should also be implemented for Iceberg table procedures. Operations such as `rewrite_data_files`, `expire_snapshots`, and `delete_orphan_files` can be destructive if misused. Automating these procedures through orchestrators like Apache Airflow or dbt should always include parameter validation, state checks, and dry-run options where supported. Setting upper and lower bounds on job inputs, such as maximum number of files or snapshot age thresholds, helps catch anomalies early and reduce accidental over-deletion.

In short, protecting Iceberg tables from data loss requires a layered approach: enabling storage-level durability, locking down permissions, removing untracked files, and validating procedure execution. These steps do not eliminate failure, but they ensure that when failure occurs, it does not compromise the recoverability of your data. The next section builds on these protections by exploring replication and cross-environment strategies for disaster recovery across regions and environments.

### 10.3.3 Cross-region and multi-environment recovery

Many organizations operate Iceberg lakehouses that span cloud regions, availability zones, or even multiple environments such as development, staging, and production. Each of these configurations introduces new recovery challenges. A regional cloud outage may prevent access to both data and metadata. A misconfigured job in a staging environment might corrupt shared catalogs or overwrite critical tables. Effective disaster recovery for Iceberg lakehouses must account for these complexities and plan for isolation, replication, and reproducibility.

At the heart of cross-region recovery lies replication. Unlike traditional databases, Iceberg does not offer built-in table replication. However, its separation of data, metadata, and catalog allows for independent strategies at each layer. Object storage replication can handle the data files. Catalog replication or version control can preserve table definitions. Together, these enable the reconstruction of full tables in another region or environment.

Most cloud object stores support cross-region replication (CRR). For example, Amazon S3 allows automatic replication of both data and Iceberg metadata files, including manifests and `metadata.json` snapshots, from one bucket to another in a separate region. Azure and Google Cloud offer similar capabilities. The key requirement is to ensure that *all* metadata files are replicated alongside the data files. If even one manifest or snapshot is missing, the replicated copy cannot serve as a complete backup.

However, CRR alone is not sufficient. Iceberg catalogs are usually not part of the object storage layer. Whether you use AWS Glue, Nessie, Polaris, or another solution, the catalog state must be exported or replicated separately. One approach is to periodically serialize catalog entries and store them alongside the table metadata. For example, Nessie's branching model supports push-pull synchronization between catalog repositories, similar to Git remotes. This enables a central production catalog to be mirrored to staging or disaster recovery regions while retaining lineage and change history.

In more complex environments, particularly those with development and test workloads, the need for reproducibility becomes as important as availability. Teams must be able to recreate a table's exact schema and state in an isolated environment, either for validation or rollback testing. Iceberg supports this through its immutable metadata snapshots. By exporting a known `metadata.json` file, along with the necessary manifest and data files, users can reproduce a table in a separate catalog by calling `register_table` and pointing to the exported metadata.

Maintaining consistent environments also requires careful control of Iceberg table procedures. For example, snapshot expiration or file rewrites in one region should not outpace replication in another. Otherwise, a replica may receive only partial metadata, breaking consistency. This is especially critical when using object storage replication with a time lag. In such cases, orchestrators should delay destructive actions, like expiration or cleanup, until replication has caught up.

For teams managing dev, test, and prod in separate catalogs or even separate cloud accounts, reproducibility tools like Terraform or custom deployment scripts can help ensure schema and configuration consistency. Table properties, partition specs, and catalog settings should be version-controlled to prevent drift. And in production environments, it is often useful to enable stricter controls on catalog operations, while allowing more freedom in non-critical environments.

Finally, regular simulation of disaster scenarios is essential. It is not enough to replicate data and metadata; teams must practice restoring tables in an alternate region or environment. Simulations uncover overlooked dependencies, such as missing IAM permissions or forgotten bucket policies. These exercises also help teams gain confidence in using Iceberg's recovery tools under pressure.

Cross-region and multi-environment recovery are not optional in enterprise-scale lakehouses. They are a core requirement for business continuity. Apache Iceberg's modular architecture allows for robust replication and recovery strategies, but only when supported by deliberate planning and automation. The next section will explore how to leverage Iceberg's built-in snapshot and time travel features to support rapid incident response and logical rollbacks.

### 10.3.4 Rollback and time travel in incident response

Apache Iceberg's snapshot architecture provides a powerful mechanism for recovering from logical errors, accidental deletions, or data corruption introduced by upstream systems. How do you recover from *human* or *software* mistakes within the table itself? These may include bad writes, dropped partitions, or unintended schema changes. Time travel and rollback are two key capabilities that make Iceberg particularly resilient in such situations.

Every change to an Iceberg table, whether an append, update, delete, or schema evolution, results in a new metadata snapshot. These snapshots are immutable and stored in the metadata directory of the table's location. Each snapshot includes a reference to the files that constitute the current state of the table, along with a log of changes made. This design enables users to reference previous states of the table by either snapshot ID or timestamp.

Time travel allows you to query the table *as it existed* at a given point in time or at a specific snapshot. This is supported across all engines that integrate natively with Iceberg, including Apache Spark, Dremio, Flink, and Trino. For example, in Spark, you can specify a snapshot ID in the read options to query historical data. This enables validation, auditing, or even ad-hoc comparisons between current and prior states. Crucially, it also allows engineering teams to diagnose the root cause of a data issue without needing to revert the production state.

However, for recovery purposes, querying old snapshots is not enough. In most incidents, the goal is to *revert* the current state of the table to a previous good state. Iceberg supports this through the `rollback_to_snapshot` procedure. This operation updates the table's metadata pointer to a prior snapshot, making it the new current version. Unlike a read-only time travel query, rollback affects all future queries and writes. It effectively "undoes" all changes made since the chosen snapshot.

There are some caveats. First, rollback is a metadata-only operation. It does not clean up any files introduced in newer snapshots unless those snapshots are later expired. This means storage may temporarily include unused files from later revisions, which can be removed by running `expire_snapshots`. Second, rollback is limited by retention settings. If older snapshots have already expired, they are no longer available for rollback. For this reason, operational policies should include retaining several hours or days of snapshots, especially in high-change environments.

The snapshot model also supports use cases where full rollback is not needed. For example, if only one partition was corrupted, users can isolate that partition's state from a previous snapshot and selectively copy its data forward. This approach is more surgical and avoids reverting unrelated changes in other partitions.

Time travel also comes with limitations in Merge-on-Read (MOR) tables. Since deleted files are applied at query time rather than materialized into the base data, older snapshots may reference data files with accumulated deletes that no longer reflect the actual historical state. In practice, this makes MOR less suitable for accurate forensic time travel, especially if compaction and cleanup have not been run consistently. In COW tables, time travel is far more predictable because each snapshot fully materializes its state.

Operationalizing rollback involves integrating snapshot tracking into your data observability stack. Some teams tag important snapshots with metadata (for example, post-deployment markers or quality check approvals) to make them easier to identify in recovery scenarios. Others log snapshot history to external observability systems for alerting and investigation.

Finally, rollback can be orchestrated just like any other Iceberg procedure. Platforms such as Airflow or Dagster can be used to automate rollback with snapshot lookups, rollback execution, and follow-up cleanup. In regulated industries, rollback actions may also require audit logging and approval workflows to meet compliance standards.

Iceberg's snapshot model is more than a versioning tool; it is a first-class incident response mechanism. When properly understood and integrated into operations, it allows teams to move quickly, recover cleanly, and avoid permanent damage from temporary errors.

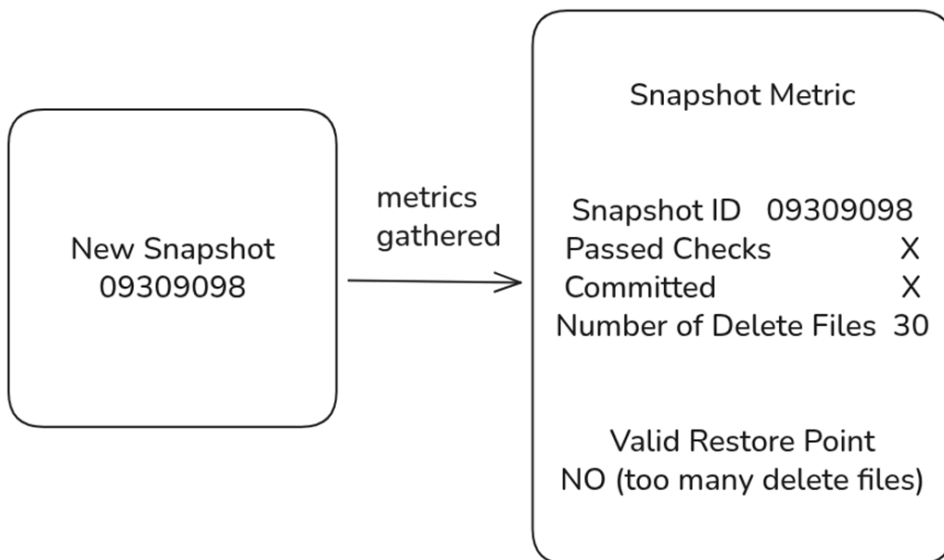
### 10.3.5 Automating disaster recovery procedures

Disaster recovery strategies are only as reliable as their execution. A well-designed plan that is never tested or automated often fails in practice. Apache Iceberg's declarative procedures, metadata-driven architecture, and decoupled storage model provide a strong foundation for recovery. But to ensure real resilience, teams must automate recovery workflows, monitor their effectiveness, and regularly test their execution.



The first step in automating disaster recovery is defining recovery objectives. Two key metrics guide this process: Recovery Point Objective (RPO) and Recovery Time Objective (RTO). RPO defines how much data loss is tolerable in the event of failure. RTO defines how quickly systems must be restored. These metrics vary across workloads. A critical financial reporting table may have an RPO of minutes and an RTO of under an hour. A derived marketing analytics table may tolerate much longer windows. These values shape how frequently backups are taken, where replicas are stored, and how quickly failover can be triggered.

Automation begins with snapshot management. Since every Iceberg table snapshot captures a complete, immutable view of the table at a point in time, it serves as a natural recovery checkpoint, as illustrated in figure 10.5. By automating the tagging and cataloging of snapshots, teams can create labeled restore points. For example, an orchestrator can add metadata tags to each snapshot that passes a data quality check or follows a successful ingestion cycle. These tags make it easier to identify valid rollback targets during a recovery event.



**Figure 10.5** You can automate gathering of snapshot metrics when a new snapshot arrives to evaluate it based on your own criteria whether it is a sound restore point, so automated processes can revert to an ideal point if something goes wrong.

Next is catalog state preservation. Catalogs are often the weakest link in Iceberg recoverability. They are not versioned by default and may not support built-in backup and restore. To mitigate this, teams can periodically export catalog state into reproducible formats such as JSON or Avro. In the case of Nessie, automated fetch-and-push scripts can synchronize branches across environments, preserving both table definitions and lineage. Glue and REST-based catalogs may require custom export tools to track schema, partition specs, and metadata location URIs.

Storage replication also needs automation. While most cloud providers support replication at the bucket level, these policies must be validated regularly. Automation scripts can check that recent metadata and data files exist in the target location and raise alerts if gaps are found. For buckets that include multiple tables or non-Iceberg objects, tagging and directory conventions help identify and verify Iceberg-specific content.

Beyond data and metadata, automation must cover procedures. Common Iceberg recovery actions like `rollback_to_snapshot`, `expire_snapshots`, and `register_table` can all be scripted and parameterized. This enables orchestrators like Airflow, Dagster, or managed cloud workflows to treat disaster recovery as a repeatable process, not a manual intervention. For example, a pipeline could detect data corruption, identify the last good snapshot from a metadata table query, perform a rollback, and then notify stakeholders through a messaging system.

Monitoring is the other half of the automation equation. A successful recovery process depends on timely detection. Monitoring should not only track availability and storage usage but also metadata integrity. Signs of trouble may include failed commits, missing manifest files, unusually large snapshot histories, or a drop in file compaction activity. These indicators can be captured using Iceberg's metadata tables or external tools like Apache Griffin, Monte Carlo, or custom rules in Spark and Flink.

Regular disaster recovery drills are essential. Automation provides consistency, but only human validation can confirm readiness. Simulations should test failover to a different region, rehydration of tables from metadata and object storage, and rollback scenarios under production-like constraints. Each drill should log results, duration, and any issues encountered. Over time, this builds confidence and reveals gaps in automation coverage.

Finally, disaster recovery automation must align with governance. Recovery actions may have compliance implications, especially in regulated sectors. Automations that perform rollbacks, deletions, or catalog rewrites should be versioned, auditable, and, where needed, require approval. This adds complexity but is necessary to avoid introducing new risks while addressing the original failure.

Iceberg enables disaster recovery by design. But like any tool, its effectiveness depends on how it is used. Automation transforms a theoretical capability into a reliable safety net. It allows organizations to recover not just data, but trust in their platform's ability to withstand the unexpected.

### 10.3.6 Validating recovery readiness

It is easy to assume that a recovery plan will work just because procedures exist on paper or scripts are checked into version control. In practice, recoverability is not guaranteed by design alone. It must be validated through regular testing, audits, and scenario-based exercises. Apache Iceberg offers several structural advantages for recovery, including immutable snapshots and externalized metadata, but these features only help if the organization knows how to use them under pressure.

Validation starts with simulating standard failure modes. These include catalog corruption, accidental data deletion, object storage unavailability, and logical data errors. Each type of failure affects a different part of the system, and each demands a tailored recovery approach. Catalog loss requires restoring table definitions and metadata URLs. Data loss requires rehydrating files from replicated storage or archived snapshots. Logical errors require rolling back to a known good snapshot and, in some cases, reprocessing downstream tables. Each of these procedures can be tested in isolation to ensure that both automation and manual steps are correctly implemented.

A common oversight in recovery validation is assuming that access to metadata guarantees access to data. For example, a backup of the `metadata.json` file may allow you to re-register an Iceberg table, but if the associated Parquet or Avro files have been deleted, corrupted, or relocated, the snapshot is unusable. Validation should confirm not only the presence of metadata but also the integrity and accessibility of data files across all manifests. This includes verifying that file paths resolve correctly, especially in multi-region deployments where manifests may contain absolute paths pointing to a primary location. If restoring from a backup to a different region or bucket, manifests may need to be rewritten to reflect the new storage layout. These checks can be performed through read-only scan jobs that validate schema consistency, partition pruning, and full snapshot loadability.

Recovery validation also includes verifying version compatibility. Apache Iceberg's format evolves, and changes to metadata layouts, manifest structures, or supported features can introduce incompatibilities. A backup written with version 0.13 may not be readable by a version 1.5 engine, depending on how features like deleting files or row-level operations were implemented. Recovery drills should therefore include engine version alignment checks and, where possible, test restores using the exact client versions that would be deployed during an incident.

Operational readiness is another dimension. Recovery is rarely performed by a single individual. It typically involves collaboration between data engineers, platform administrators, cloud operations, and governance teams. Regular tabletop exercises help clarify roles, surface gaps in documentation, and ensure that everyone knows where the recovery playbooks are stored and how to execute them. These drills can be scheduled quarterly and should include both partial failures, like a missing manifest file, and full scenarios, like region-wide unavailability.

Monitoring and alerting are essential to validating readiness. Dashboards should track the age of the last known good snapshot, the size and frequency of catalog exports, and the health of replication pipelines. Alerts should be configured for anomalies such as a drop in compaction activity, a sudden increase in orphan files, or the absence of new snapshots over time. These indicators not only help detect real-time issues but also validate that recovery instrumentation is in place and functioning.

Documentation plays a critical supporting role. All automated procedures should be versioned and documented with input parameters, dependencies, expected outcomes, and rollback steps. Manual procedures must be clearly written, ideally with screenshots or CLI examples. For example, instructions for restoring a Nessie catalog should include the exact API calls or commands used to create a new branch, register a table from existing metadata, and validate its lineage. Documentation must also cover cross-region and cross-environment recovery, highlighting differences between dev, staging, and production.

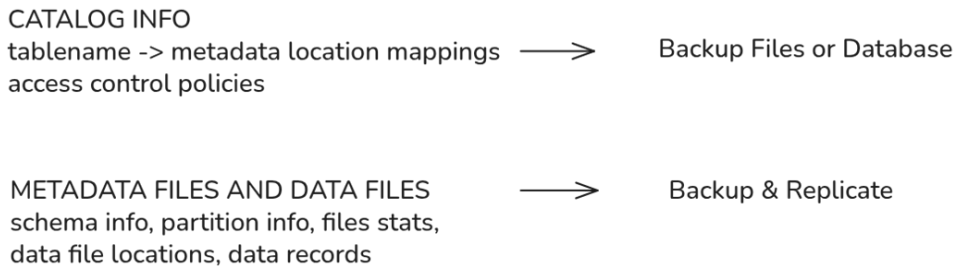
Finally, all validation results should be logged and reviewed. A successful recovery simulation is only valuable if it leads to learning and refinement. Logs should capture duration, errors, recovery steps taken, and post-recovery health checks. Over time, this log becomes a historical record of the platform's resilience maturity.

Apache Iceberg makes recovery possible, but validation makes it reliable. In the end, it is not the presence of a plan that defines readiness; it is the confidence, built through repeated and structured practice, that the plan will work when needed.

### 10.3.7 Disaster recovery through automation

Manual recovery procedures are prone to error, delay, and inconsistency. In complex systems where data changes rapidly and dependencies span multiple tools, waiting for a human to troubleshoot and respond to failure often means prolonged downtime and potential data loss. Disaster recovery in Apache Iceberg environments must be automated not just for speed, but for predictability and repeatability. The goal is to turn recovery into an engineered workflow rather than a set of tribal instructions.

The first layer of automation begins with backups, as illustrated in figure 10.6. For Iceberg tables, this means automating the export of metadata files and catalog state. The frequency of these backups depends on how often tables change, but for transactional workloads, hourly backups are often a safe baseline.



**Figure 10.6 Regular backups not just of data files, but of metadata files and catalog state can make disaster recovery and ensure that storage is in sync with the catalog after a recovery scenario.**

Object storage replication is the second layer. Tools like AWS S3 Replication or Azure Storage replication can be configured to mirror all Iceberg data and metadata files to a secondary region or account. This process should include full paths, including the metadata, data, and snapshots directories for each table. Automating this replication ensures that data is not only stored redundantly but that recovery does not depend on a single cloud region's availability.

At the orchestration level, recovery plans can be encoded as DAGs (directed acyclic graphs) using tools like Apache Airflow or Dagster. A recovery DAG may include steps to register a new table using a copied metadata.json, restore the associated data files from a replicated bucket, validate snapshot lineage, and optionally update a catalog with the restored table's metadata. These steps can be conditioned on the nature of the failure, such as a single table loss versus a full catalog outage. Because these DAGs can be version-controlled and tested like application code, they allow platform teams to iterate on recovery strategies safely and transparently.

Monitoring systems like Prometheus or Datadog can provide triggers for disaster recovery workflows. For example, if a critical Iceberg table has not received a new snapshot within a predefined time window, or if the number of orphan files spikes beyond a known threshold, the system can invoke a diagnostic pipeline or initiate failover procedures. This form of event-driven recovery allows detection and action to occur with minimal human intervention.

Some vendors and managed platforms are starting to integrate disaster recovery automation directly into their offerings. For example, Dremio supports failover-aware metadata access, while Apache Polaris introduces concepts like metadata mirroring and cross-region catalog replication. In environments using these platforms, recovery may involve enabling features rather than building pipelines from scratch. Still, it is critical to validate how these services behave during actual failure scenarios and to identify any limitations regarding consistency, version compatibility, or recovery lag.

Another key automation target is snapshot validation. Even with backups in place, not all snapshots are equally helpful for recovery. Validating snapshots involves confirming that all data and manifest files referenced in the snapshot are still present and uncorrupted. Automating this check can help ensure that your backups are not silently broken. Tools that scan and validate snapshot chains, either built in-house or as part of metadata management frameworks, can proactively raise alerts when snapshots become unusable.

Finally, recovery automation should include a communication layer. When a recovery procedure is initiated, it is essential to notify relevant teams and log each action taken. This is especially important in regulated environments where audit trails are required. Integrations with tools like Slack, Jira, or Microsoft Teams can ensure that operators are aware of the incident, can review the automated actions, and are prepared to take over if needed.

In an Apache Iceberg deployment, disaster recovery is not a single task or script. It is a set of coordinated and tested workflows that align with the platform's modular architecture. By automating these workflows, teams can ensure that recovery becomes a repeatable, observable, and verifiable process, ready to perform when it matters most.

### 10.3.8 Practical examples: Automating recovery workflows

To bring the disaster recovery strategies into real-world practice, this section presents sample workflows and code snippets that illustrate how to automate recovery in an Apache Iceberg environment. The examples focus on catalog re-registration, snapshot rollback, and consistency validation following a recovery event.

## EXAMPLE 1: RE-REGISTERING A TABLE AFTER CATALOG LOSS

Suppose your catalog is corrupted or lost, but all metadata and data files still reside in object storage. You can recover by pointing the catalog back at a known metadata.json. This snippet uses Spark SQL to re-register an Iceberg table:

```
CALL my_catalog.system.register_table(
  table => 'recovered_db.recovered_table',
  metadata_file => 's3://my-bucket/iceberg/metadata/metadata-000123.json'
); --#A

CALL my_catalog.system.set_current_snapshot(
  table => 'recovered_db.recovered_table',
  snapshot_id => 123
); --#B
```

**#A** Registers a new table entry in the catalog, referencing the metadata file that describes the desired table state.

**#B** Sets the current snapshot of the table to match the one stored in the metadata, making that snapshot the active state.

After this operation, downstream engines can query `recovered_db.recovered_table` as though it had never been lost, provided all referenced data files are intact.

## EXAMPLE 2: AUTOMATED ROLLBACK AND CLEANUP ORCHESTRATED IN AIRFLOW

If a bad write corrupted data, you may want to roll back and then purge later snapshots. This Airflow DAG example automates that process:

```

from airflow import DAG
from airflow.providers.apache.spark.operators.spark_sql import SparkSqlOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'platform',
    'retries': 1,
    'retry_delay': timedelta(minutes=5)
}

with DAG(
    dag_id='iceberg_recovery_dag',
    default_args=default_args,
    start_date=datetime(2025, 10, 1),
    schedule_interval=None,
    catchup=False
) as dag:

    rollback = SparkSqlOperator(
        task_id='rollback_snapshot',
        sql="""
            CALL my_catalog.system.rollback_to_snapshot(
                table => 'prod.sales',
                snapshot_id => 205
            )
            """, #A
        spark_conn_id='spark_default'
    )

    expire = SparkSqlOperator(
        task_id='expire_newer_snapshots',
        sql="""
            CALL my_catalog.system.expire_snapshots(
                table => 'prod.sales',
                older_than => TIMESTAMP '2025-09-10 00:00:00'
            )
            """, #B
        spark_conn_id='spark_default'
    )

    rollback >> expire #C

```

**#A** Executes the rollback to snapshot 205, reverting the table's current state to that snapshot.  
**#B** Expires any snapshots older than a cutoff, effectively removing newer, invalid versions after rollback.  
**#C** Ensures expiration only runs after rollback, maintaining consistency and minimizing the risk of removing required files.

Once the DAG completes, the table's current metadata points to snapshot 205, and newer snapshots are eliminated. At this point, orchestrated cleanup or compaction may follow to reclaim storage.

### EXAMPLE 3: VERIFYING SNAPSHOT INTEGRITY VIA VALIDATION SCAN

After recovery, it is critical to confirm that all referenced files in the snapshot are accessible and correct. This snippet shows how to run a validation query to detect missing or corrupted files:

```
SELECT file_path
FROM my_catalog.prod.sales.manifests
WHERE NOT EXISTS (
    SELECT 1
    FROM my_catalog.prod.sales.files
    WHERE files.file_path = manifests.file_path
); --#A
```

**#A** Identifies manifest entries referencing file paths that cannot be found in the file's metadata table, indicating missing or corrupted data files.

If this query returns any rows, it means some files listed in manifests are not present or not registered. You can alert operators, attempt to rehydrate missing files from backups, or abort further recovery steps.

### EXAMPLE 4: CROSS-REGION FAILOVER WITH CATALOG BRANCH RECREATION

In a multi-region scenario with a versioned catalog (e.g., Nessie), you might replicate catalog branches and switch failover contexts programmatically:



```
CALL nessie.catalog.branch(
  name => 'recovery_zone',
  from_ref => 'main'
); --#A

CALL nessie.catalog.fast_forward(
  table => 'prod.sales',
  branch => 'recovery_zone',
  to => 'main'
); --#B
```

**#A** Creates a new catalog branch named `recovery_zone`, starting from the current main branch, enabling isolation for recovery operations.

**#B** Fast-forwards the `recovery_zone` branch's `prod.sales` table to match main, bringing the branch in sync with production.

This branching pattern enables you to recover, validate, and test on a replica branch before promoting it to production.

With this, you've reached the final stage of building a production-grade Apache Iceberg lakehouse. From foundational concepts and architectural decisions to ingestion, governance, and operationalization, this book has walked you through the full lifecycle of an Iceberg deployment. The tools, patterns, and tradeoffs covered across these chapters are not just technical guidance. They form a framework for making decisions in real-world data environments. Whether you're just starting or scaling out a mature platform, you now have the knowledge to design, implement, and run an Iceberg lakehouse with confidence, clarity, and control.

## 10.4 Summary

- Iceberg lakehouses require more than just design and ingestion planning; operational maturity is key to long-term reliability.
- Orchestration frameworks like Airflow and Dagster help automate compaction, snapshot expiration, and governance across workloads.
- Auditing practices must span both data and metadata, using metadata tables, lineage tools, and catalog logs to track access and change.
- File retention in Iceberg is governed by snapshot expiration and orphan file cleanup, both of which must be configured to match workload patterns.
- Secure deletion in COW and MOR tables depends on snapshot lifecycle management and, in the case of MOR, regular delete file compaction.
- Catalog access controls vary by provider (e.g., AWS Glue, Nessie, Polaris), so governance models must align with platform capabilities.
- Disaster recovery for Iceberg tables involves backing up catalog metadata, leveraging snapshot rollback, and designing for cross-region resilience.
- Iceberg's snapshot model provides powerful rollback options but also requires procedural safeguards to protect from unintended data loss.

- Simulating and automating recovery workflows helps validate readiness for real-world incidents and reduces recovery time.

## *Appendix A The metadata tables*

Apache Iceberg tracks rich metadata to efficiently manage its tables. This metadata isn't just for internal bookkeeping; it's also exposed to users through special metadata tables that can be queried like regular tables. These metadata tables provide visibility into the physical and logical layout of an Iceberg table, offering insights into data file sizes, partitions, snapshots, and more.

Understanding how to use these metadata tables is essential for monitoring table health, diagnosing performance issues, and automating maintenance tasks like compaction and snapshot expiration. This section provides an overview of these tables, demonstrates how to query them using common engines like Spark and Dremio, and explains how to interpret their outputs.

We'll walk through each of Iceberg's primary metadata tables and describe how they can be used in practical maintenance scenarios. We'll also examine how these tables enable proactive optimization workflows by serving as a foundation for dynamic maintenance triggers.

### **A.1 Querying Iceberg metadata tables**

Iceberg exposes its metadata through a rich set of system tables that enable users to analyze and manage the physical and logical aspects of their datasets. These metadata tables are integral for understanding table evolution, data layout, and snapshot management. They can be accessed using both Spark and Dremio, offering flexibility across different analytics engines.

#### **ACCESSING METADATA TABLES**

In Spark, metadata tables are referenced by appending the metadata table name to the original table name:

```
SELECT * FROM db.table.history;
SELECT * FROM db.table.snapshots;
SELECT * FROM db.table.entries;
```

In Dremio, metadata tables are accessed using the `TABLE(<iceberg_metadata>(<table_name>))` syntax:

```
SELECT * FROM TABLE(table_history('my_table'));
SELECT * FROM TABLE(table_snapshot('my_table'));
SELECT * FROM TABLE(table_files('my_table'));
```

## KEY METADATA TABLES

- **history:** Shows the table's commit timeline, including each snapshot's timestamp, ID, parent, and whether it is part of the current history.
- **snapshots:** Lists all valid snapshots along with operation types (append, overwrite, delete), and manifest lists.
- **entries:** Shows current manifest entries for both data and delete files.
- **files:** Displays metadata for the current data and delete files in the latest snapshot.
- **manifests:** Lists manifest files used in the current table state.
- **partitions:** Provides detailed statistics on partition-level metrics.
- **refs:** Shows named references like branches and tags associated with snapshots.

## PRACTICAL USES

These tables enable a variety of diagnostic and operational workflows:

- **Auditing Changes:** Use history and snapshots to track modifications and restore prior states.
- **Diagnosing Storage Patterns:** Query files, entries, or manifests to identify skew, excessively small files, or large partitions.
- **Time Travel:** Combine metadata tables with time travel queries to investigate the table state at a previous snapshot or timestamp.
- **Maintenance Triggers:** Automate compaction or cleanup based on insights from partitions and manifests.

## A.2 The history metadata table

The history metadata table captures the complete lineage of an Iceberg table by recording each commit made to the table. This includes the creation of snapshots, rollbacks, and the parent-child relationships between snapshots. It is one of the most essential tools for understanding how a table has evolved.

## WHAT IT CONTAINS

Each row in the history table includes the following fields:

- `made_current_at`: The timestamp when the snapshot was made current.
- `snapshot_id`: The unique ID for the snapshot.
- `parent_id`: The snapshot ID this one is derived from.
- `is_current_ancestor`: A boolean indicating whether the snapshot is in the direct history of the current table state.

This structure enables you to track not just the current state, but also any branches, rollbacks, or abandoned snapshot paths.

## EXAMPLE QUERIES

Using Spark:

```
SELECT * FROM db.table.history;
```

Using Dremio:

```
SELECT * FROM TABLE(table_history('my_table'));
```

## USE CASES

- **Auditing Table Evolution:** Track when specific changes were made to the table and by which engine or job.
- **Debugging Rollbacks:** Identify when and why a rollback occurred by observing non-ancestor snapshots.

**Snapshot Management:** Combine with the snapshots table to see what operations were performed for each snapshot (e.g., append, overwrite, delete).

```
SELECT
  h.made_current_at,
  s.operation,
  h.snapshot_id,
  h.is_current_ancestor,
  s.summary['spark.app.id']
FROM db.table.history h
JOIN db.table.snapshots s ON h.snapshot_id = s.snapshot_id
ORDER BY h.made_current_at;
```

This table is foundational for time travel, snapshot expiration strategies, and understanding how your data changed over time.

## A.3 The snapshots metadata table

The snapshots metadata table provides detailed information about each snapshot in an Apache Iceberg table. While the history table outlines the lineage of changes, the snapshots table offers the operational and structural specifics of each change event.

### WHAT IT CONTAINS

Each row in the snapshots table includes:

- `committed_at`: Timestamp when the snapshot was committed.
- `snapshot_id`: Unique ID for the snapshot.
- `parent_id`: The ID of the snapshot this one is based on.
- `operation`: The operation type (e.g., append, overwrite, delete, etc.).
- `manifest_list`: The path to the Avro file listing the manifests used in this snapshot.
- `summary`: A map of metadata, such as the number of records added or deleted, data files involved, and engine-specific metadata like Spark application IDs.

### EXAMPLE QUERIES

Using Spark:

```
SELECT * FROM db.table.snapshots;
```

Using Dremio:

```
SELECT * FROM TABLE(table_snapshot('my_table'));
```

### USE CASES

**Understanding Table Modifications:** By filtering snapshots by operation type, users can understand the nature of changes applied over time.

```
SELECT snapshot_id, operation, committed_at
FROM db.table.snapshots
WHERE operation = 'delete';
```

**Debugging Writes:** Identify unexpected behavior by correlating snapshot timing with job/application IDs.

```
SELECT committed_at, operation, snapshot_id, summary['spark.app.id']
FROM db.table.snapshots
ORDER BY committed_at DESC;
```

**Supporting Snapshot Expiration:** This table is essential for identifying which snapshots are candidates for expiration, especially when managing storage footprint or meeting retention policies.

The snapshots metadata table is a vital lens into the lifecycle of a table's state and is frequently joined with the history table to form a complete picture of change provenance and lineage.

## A.4 The metadata\_log\_entries metadata table

The metadata\_log\_entries metadata table in Apache Iceberg exposes the evolution of a table's metadata files over time. Each entry corresponds to a new metadata file committed to the table, providing a forensic view of how the table's structure and state have progressed.

### WHAT IT CONTAINS

Each row includes:

- **timestamp:** When the metadata file was committed.
- **file:** The URI to the metadata JSON file.
- **latest\_snapshot\_id:** The snapshot ID associated with the metadata file at the time of the commit.
- **latest\_schema\_id:** The schema ID in use when the file was committed.
- **latest\_sequence\_number:** A monotonically increasing number representing the latest sequence of changes.

### EXAMPLE QUERIES

Using Spark:

```
SELECT * FROM db.table.metadata_log_entries;
```

Using Dremio:

```
SELECT * FROM TABLE(table_history('my_table'));
```

### USE CASES

- **Tracking Metadata Changes:** Allows users to inspect how frequently metadata files are committed and whether changes are associated with schema updates, snapshot creations, or catalog-level operations.
- **Debugging Table Evolution:** If a table suddenly becomes unreadable, this table can help identify the last known good metadata file and assist in recovery (especially in disaster recovery scenarios).

- **Cross-validating with Snapshots:** Join this table with snapshots or history to analyze whether any orphan metadata files exist or to validate snapshot and schema usage patterns.

```
SELECT l.timestamp, l.file, s.operation
FROM db.table.metadata_log_entries l
LEFT JOIN db.table.snapshots s
  ON l.latest_snapshot_id = s.snapshot_id;
```

## OPERATIONAL RELEVANCE

Maintaining a clean and rational history of metadata files is essential for efficient table performance and troubleshooting. While snapshot expiration handles data file cleanup, the `metadata_log_entries` table offers a structural audit trail that is especially important in regulated environments or when maintaining long-lived tables.

## A.5 The manifests metadata table

The manifests metadata table provides detailed information about the manifest files used by the current snapshot of an Apache Iceberg table. Manifest files are intermediate metadata files that store references to groups of data or delete files, making them critical for understanding the layout and performance characteristics of the table.

### WHAT IT CONTAINS

Each row in this table represents a manifest file and includes:

- `path`: URI to the manifest file (typically an Avro file).
- `length`: Size of the manifest file in bytes.
- `partition_spec_id`: Identifier for the partition spec used.
- `added_snapshot_id`: Snapshot that introduced the manifest.
- `added_data_files_count`, `existing_data_files_count`, `deleted_data_files_count`: Counts of file changes.
- `partition_summaries`: Summary statistics such as lower and upper bounds for partition values.

### EXAMPLE QUERIES

Using Spark:

```
SELECT * FROM db.table.manifests;
```

Using Dremio:



```
SELECT * FROM TABLE(table_manifests('db.table'));
```

## USE CASES

- **Compaction Health:** An excessive number of small manifest files or many manifests referencing only a few data files can signal the need for metadata compaction. This fragmentation adds overhead to query planning and scanning.
- **Partition Distribution Insights:** By analyzing partition\_summaries, you can assess the skew or distribution across partitions, which is useful for tuning partitioning strategies.
- **Debugging Snapshot Behavior:** Use added\_snapshot\_id to trace which snapshot added specific manifests, and correlate this with operations like append, overwrite, or delete.

```
SELECT path, length, added_snapshot_id
FROM db.table.manifests
ORDER BY length DESC
LIMIT 10;
```

## BEST PRACTICES

- **Monitor Manifest Counts:** A growing number of manifests can slow down planning. Use metrics from this table to trigger metadata compaction dynamically.
- **Review Partition Summaries:** Identify underutilized or skewed partitions that may need repartitioning or better clustering strategies.
- **Track Manifest Bloat:** Over time, manifests may grow in number or size, even if data hasn't increased proportionally. This may indicate inefficient write patterns or a lack of periodic optimization.

## A.6 The partitions metadata table

The partitions metadata table in Apache Iceberg provides visibility into how data is physically laid out across partitions in the current snapshot. This metadata is critical for understanding storage distribution, identifying skew, and optimizing query performance.

### WHAT IT CONTAINS

Each row in the partitions table represents a partition and includes:

- **partition:** The partition key (e.g., a date, region, or composite key).
- **spec\_id:** The ID of the partition specification used.
- **record\_count:** Number of records in the partition.
- **file\_count:** Total number of data files.
- **total\_data\_file\_size\_in\_bytes:** Total size of data files in the partition.

- Delete file metrics:
  - position\_delete\_record\_count, position\_delete\_file\_count
  - equality\_delete\_record\_count, equality\_delete\_file\_count
- last\_updated\_at: Timestamp of the last change.
- last\_updated\_snapshot\_id: Snapshot ID associated with the last update.

## EXAMPLE QUERIES

Using Spark:

```
SELECT * FROM db.table.partitions;
```

Using Dremio:

```
SELECT * FROM TABLE(table_partitions('db.table'));
```

## USE CASES

- **Skew Detection:** Large imbalances in record\_count or file\_count across partitions can degrade performance. Identifying these outliers helps target compaction or repartitioning efforts.

```
SELECT partition, file_count
FROM db.table.partitions
ORDER BY file_count DESC
LIMIT 5;
```

- **Delete File Accumulation:** High counts of deleted files in specific partitions may indicate frequent updates or deletes, which could require targeted compaction or cleanup.
- **Storage Monitoring:** The total\_data\_file\_size\_in\_bytes metric allows teams to monitor data growth and storage utilization per partition.
- **Targeted Optimization:** By filtering partitions with high file counts or large sizes, you can perform selective compaction or rewriting to minimize query planning overhead.

## BEST PRACTICES

- **Monitor Partition Health Regularly:** Use this table to detect hotspots and ensure that partitioning strategies remain aligned with evolving query patterns.
- **Combine With Clustering Metrics:** In Dremio, supplement partition insights with clustering health metrics to decide when to run OPTIMIZE TABLE.

- **Filter Out Stale Partitions:** Identify partitions that haven't been updated recently and evaluate them for cold storage tiering or deletion if no longer needed.

## A.7 The files metadata table

The files metadata table provides a comprehensive view of all active data and delete files in the current snapshot of an Apache Iceberg table. This metadata table is essential for inspecting file-level statistics, evaluating table health, and diagnosing performance issues caused by file distribution.

### WHAT IT CONTAINS

Each row in the files metadata table corresponds to a file and includes:

- `content`: Type of file (0 for data, 1 for position delete, 2 for equality delete)
- `file_path`: Full path to the file in storage
- `file_format`: File format (e.g., PARQUET)
- `spec_id`: Partition specification ID
- `record_count`: Number of records in the file
- `file_size_in_bytes`: Size of the file
- Column-level metrics: `column_sizes`, `value_counts`, `null_value_counts`, `nan_value_counts`
- `lower_bounds`, `upper_bounds`: Column value ranges
- `split_offsets`, `equality_ids`, `sort_order_id`
- `readable_metrics`: A human-readable summary of file-level statistics

### EXAMPLE QUERIES

Using Spark:

```
SELECT * FROM db.table.files;
```

Using Dremio:

```
SELECT * FROM TABLE(table_files('db.table'));
```

### USE CASES

- **Small File Detection:** Identify files smaller than your compaction threshold to target them for rewrite operations.

```
SELECT file_path, file_size_in_bytes
FROM db.table.files
WHERE file_size_in_bytes < 134217728;
```

- **Content Type Filtering:** Differentiate between data and delete files to diagnose Merge-on-Read performance degradation.

```
SELECT file_path, content
FROM db.table.files
WHERE content != 0;
```

- **Column-Level Analysis:** Review null counts or value distribution across columns to understand data quality and clustering patterns.

```
SELECT readable_metrics
FROM db.table.files
WHERE file_path LIKE '%parquet';
```

- **Sort Order & Partitioning Validation:** Use lower\_bounds and upper\_bounds to verify clustering and data colocation efficiency.

## BEST PRACTICES

- **Regular Review:** Periodically query the files table to monitor file sizes and balance, especially for streaming or high-throughput write workloads.
- **Filter by File Type:** Target delete files separately for cleanup or investigation, particularly in MOR use cases.
- **Column-Level Optimization:** Use metrics to detect columns with excessive nulls or skew, potentially adjusting partitioning or sort order for better performance.

## A.8 The manifests metadata table

The manifests metadata table in Apache Iceberg provides visibility into the manifest files that make up the current snapshot of a table. Manifests are metadata files that list the data and delete files used in a snapshot. Analyzing them can help diagnose metadata sprawl, identify optimization opportunities, and understand partition-level data distribution.

### WHAT IT CONTAINS

Each row in the manifests table represents one manifest file and includes:

- **content:** Indicates the type of files listed (0 for data, 1 for delete)
- **path:** Full path to the manifest file
- **length:** Size of the manifest file in bytes
- **partition\_spec\_id:** The partition spec used for the files listed
- **added\_snapshot\_id:** The snapshot that added this manifest
- **added\_data\_files\_count, existing\_data\_files\_count, deleted\_data\_files\_count**

- `added_delete_files_count`, `existing_delete_files_count`, `deleted_delete_files_count`
- `partition_summaries`: Summary statistics for partitions listed in the manifest

## EXAMPLE QUERIES

Using Spark:

```
SELECT * FROM db.table.manifests;
```

Using Dremio:

```
SELECT * FROM TABLE(table_manifests('db.table'));
```

## USE CASES

- **Detecting Metadata Sprawl:** Tables with excessive manifests can slow down planning and scanning. Look for many small manifest files.

```
SELECT COUNT(*), SUM(length) AS total_manifest_bytes
FROM db.table.manifests;
```

- **Partition Coverage Insight:** Use `partition_summaries` to evaluate how partitions are distributed across manifests, which can hint at poor clustering.

```
SELECT partition_summaries
FROM db.table.manifests
WHERE content = 0;
```

- **Monitoring Changes Over Time:** Join with the `snapshots` table to trace how many manifests are added or removed across snapshots.

```
SELECT s.committed_at, m.path, m.length
FROM db.table.snapshots s
JOIN db.table.manifests m ON s.snapshot_id = m.added_snapshot_id;
```

## BEST PRACTICES

- **Trigger Compaction:** If many small manifests are detected, consider running metadata compaction to consolidate them.
- **Optimize Partitioning:** Use `partition_summaries` to reassess how well your partition strategy aligns with query patterns.
- **Track Manifest Growth:** Keep tabs on the number and size of manifests to prevent unexpected performance regressions.

## A.9 The partitions metadata table

The partitions metadata table offers a window into how data is physically distributed across a table's partitions in its current snapshot. This is particularly useful in evaluating partition effectiveness, monitoring skew, and deciding whether repartitioning or compaction is necessary.

### WHAT IT CONTAINS

Each row in the partitions table represents a partition in the current snapshot and includes:

- **partition:** The actual partition key values
- **spec\_id:** The ID of the partition spec used
- **record\_count:** Number of records in this partition
- **file\_count:** Number of files (data and delete) in this partition
- **total\_data\_file\_size\_in\_bytes:** Total size of all data files
- **position\_delete\_record\_count / equality\_delete\_record\_count:** Volume of deletes
- **last\_updated\_snapshot\_id:** The snapshot where this partition was last modified

### EXAMPLE QUERIES

Using Spark:

```
SELECT * FROM db.table.partitions;
```

Using Dremio:

```
SELECT * FROM TABLE(table_partitions('db.table'));
```

### USE CASES

- **Detecting Skew:** Find partitions with disproportionate record counts or file sizes.

```
SELECT partition, record_count, total_data_file_size_in_bytes
FROM db.table.partitions
ORDER BY record_count DESC
LIMIT 10;
```

- **Compaction Targets:** Identify partitions with many small files to target for partial compaction.

```
SELECT partition, file_count
FROM db.table.partitions
WHERE file_count > 50;
```

- **Delete File Pressure:** Understand where delete file accumulation is impacting performance.

```
SELECT partition, position_delete_record_count
FROM db.table.partitions
WHERE position_delete_record_count > 0;
```

## BEST PRACTICES

- **Partition Monitoring:** Regularly inspect this table to ensure the partition strategy is aligned with evolving data and query patterns.
- **Incremental Optimization:** Use partition filters with compaction or snapshot expiration to focus on problematic partitions.
- **Repartitioning Decisions:** If skew or inefficiency persists despite compaction, consider adjusting the partition spec altogether.

## A.10 The position\_deletes metadata table

The position\_deletes metadata table tracks positional delete files, files that instruct Iceberg to skip specific row positions within data files. These are critical for supporting Merge-on-Read (MOR) behavior, particularly in upsert and delete operations.

### WHAT IT CONTAINS

Each row represents a single delete action and includes:

- file\_path: Path of the original data file
- pos: Row position to be deleted
- row: The deleted row data (optional)
- partition: The partition key values
- spec\_id: Partition spec ID
- delete\_file\_path: The path to the delete file where the instruction resides

### EXAMPLE QUERIES

In Spark:

```
SELECT * FROM db.table.position_deletes;
```

In Dremio:

```
SELECT * FROM TABLE(table('db.table').position_deletes);
```

## USE CASES

- **Understanding Upsert Costs:** As more upserts are performed under MOR, the number of delete records increases, which can degrade read performance. Use this table to quantify and localize that impact.

```
SELECT COUNT(*) FROM db.table.position_deletes;
```

- **Targeting Compaction:** Identify which partitions or files have accumulated many positional deletes.

```
SELECT partition, COUNT(*) AS delete_count
FROM db.table.position_deletes
GROUP BY partition
ORDER BY delete_count DESC;
```

- **Audit & Debugging:** Inspecting which rows were deleted and when, to verify transactional integrity or troubleshoot unexpected results.

## BEST PRACTICES

- **Monitor Accumulation:** Positional delete files are read during every query in MOR tables. Monitoring this table helps signal when compaction is necessary to merge deletes into new base files.
- **Strategic Snapshot Expiration:** Since deletes are only purged after relevant snapshots expire, aligning the delete file volume with snapshot retention policies is key to managing performance.
- **Avoiding Read Amplification:** Excessive delete files can cause Iceberg readers to perform multiple disk reads per row. Use this metadata to keep delete file growth in check through scheduled rewrites.

## A.11 The all\_data\_files metadata table

The `all_data_files` metadata table provides a historical view of every data file ever added to an Iceberg table across all snapshots, active or expired. Unlike the `files` metadata table, which only reflects the current snapshot, `all_data_files` gives you complete lineage and visibility into the table's evolution over time.

## WHAT IT CONTAINS

Each row represents a single data file and includes fields such as:



- `file_path`: Full path to the data file.
- `file_format`: e.g., PARQUET, AVRO, ORC.
- `partition`: Partition values (if partitioned).
- `record_count`: Number of rows in the file.
- `file_size_in_bytes`: File size on disk.
- `column_sizes`, `value_counts`, `null_value_counts`, `lower_bounds`, `upper_bounds`: Statistics useful for tuning, analysis, and optimization.
- `spec_id`, `sort_order_id`: Structural metadata for interpreting the file within the table context.
- `split_offsets`: Used for splitting files during parallel reads.

## EXAMPLE QUERIES

In Spark:

```
SELECT * FROM db.table.all_data_files;
```

In Dremio:

```
SELECT * FROM TABLE(table_all_data_files('db.table'));
```

## USE CASES

- **Forensic Debugging**: When a file disappears from the files table but was present before, `all_data_files` helps trace when it was added and potentially removed.
- **Historical Optimization Analysis**: Measure how file sizes and partition distributions evolved.

```
SELECT
  partition,
  MAX(file_size_in_bytes) AS max_size,
  MIN(file_size_in_bytes) AS min_size
FROM db.table.all_data_files
GROUP BY partition;
```

- **Regulatory Auditing**: In compliance workflows (e.g., GDPR), understanding the lifecycle of specific files is critical. This table provides the necessary traceability.
- **Performance Diagnosis**: Identify old files that may have been missed by compaction routines or retained due to snapshot expiration lag.

## BEST PRACTICES

- **Join with Snapshots:** Pair this table with snapshot metadata to build a full timeline of file activity.
- **Leverage in Automation:** Use this table in compaction planning pipelines to detect stale or oversized files that are still consuming space.
- **Balance with Retention:** While valuable, the volume of data in `all_data_files` can grow significantly. Ensure snapshot expiration policies are tuned to avoid runaway metadata costs.

## A.12 The `all_delete_files` metadata table

The `all_delete_files` metadata table offers a historical view of every delete file ever added to an Apache Iceberg table across all snapshots. This includes both *position deletes* and *equality deletes*, whether they are still active or have been superseded by newer snapshots. This table is particularly useful for diagnosing data retention, auditing delete operations, and evaluating the impact of Merge-On-Read (MOR) workloads.

### WHAT IT CONTAINS

Each row corresponds to a delete file and includes metadata such as:

- `file_path`: The full path to the delete file.
- `file_format`: Typically PARQUET.
- `partition`: Partition information for the rows targeted by the delete file.
- `record_count`: Number of rows marked for deletion in the file.
- `file_size_in_bytes`: Size of the delete file on disk.
- `column_sizes`, `value_counts`, `null_value_counts`: Statistics describing the delete file contents.
- `lower_bounds`, `upper_bounds`: Value ranges affected by the delete file.
- `split_offsets`: Used for parallel reads during scan planning.
- `content`: Distinguishes between:
  - `PositionDelete`: Position delete
  - `EqualityDelete`: Equality delete

### EXAMPLE QUERIES

In Spark:

```
SELECT * FROM db.table.all_delete_files;
```

In Dremio:

```
SELECT * FROM TABLE(table_all_delete_files('db.table'));
```

### USE CASES

- **Monitoring MOR Accumulation:** MOR workloads tend to accumulate delete files over time. This table helps quantify the buildup and its impact on query performance.

```
SELECT COUNT(*) AS num_delete_files,
       SUM(record_count) AS total_deleted_rows
FROM db.table.all_delete_files;
```

- **Compliance Audits:** Validate whether deletions requested for regulatory compliance (e.g., GDPR or CCPA) were registered and persisted appropriately.
- **Compaction Readiness:** Use the presence and volume of delete files to determine when it's time to compact a table to reclaim performance.
- **File Lineage and Debugging:** Track when and how deletes were applied across time, and identify deletes that may have been missed or incorrectly applied.

## BEST PRACTICES

- **Snapshot Expiration:** Ensure regular expiration of snapshots to allow Iceberg to clean up obsolete delete files and prevent unnecessary data scans.
- **Size Monitoring:** Keep delete file size and count within reasonable thresholds to avoid degrading query performance. Use metrics here to trigger background maintenance jobs.
- **MOR Awareness:** In MOR tables, monitor how long delete files remain active, and plan periodic rewrites to reduce read-time overhead.

## A.13 The all\_entries metadata table

The all\_entries metadata table provides the most granular and comprehensive view of an Iceberg table's file-level evolution. It surfaces the full list of manifest entries across all snapshots, including both data and delete files, along with their historical status changes, whether a file was added, removed, or is currently active.

### WHAT IT CONTAINS

Each row in all\_entries represents a manifest entry and includes:

- **status:** Indicates the file's lifecycle state:
  - **EXISTING**: Existing file from a previous snapshot.
  - **ADDED**: Added in the current snapshot.
  - **DELETED**: Deleted in the current snapshot.
- **snapshot\_id:** The snapshot in which the entry was added or removed.
- **sequence\_number** and **file\_sequence\_number:** Help determine the order in which operations occurred.
- **data\_file:** A struct with metadata like file path, format, partition, and row count.

- **readable\_metrics:** A human-readable breakdown of column-level stats for quick diagnostics.

## EXAMPLE QUERIES

In Spark:

```
SELECT * FROM db.table.all_entries;
```

In Dremio:

```
SELECT * FROM TABLE(table_all_entries('db.table'));
```

## USE CASES

- **Lineage Tracking:** This table is ideal for building a detailed lineage of when each file was added and removed. It helps answer questions like: *Which snapshot introduced a particular file? When was it replaced?*
- **Audit and Debugging:** By joining all\_entries with snapshots, you can recreate the lifecycle of a file and investigate how and when a data or delete file changed state.

```
SELECT e.status, e.snapshot_id, s.committed_at, e.data_file.file_path
FROM db.table.all_entries e
JOIN db.table.snapshots s ON e.snapshot_id = s.snapshot_id
ORDER BY s.committed_at;
```

- **Change Data Capture (CDC):** For use cases where fine-grained change tracking is needed, this table offers a basis for analyzing the file-level diffs across snapshots.
- **Impact Analysis:** When planning snapshot expiration or compaction, this table helps determine the reach and redundancy of file usage across versions.

## BEST PRACTICES

- **Snapshot Expiration Awareness:** As this table includes entries from all snapshots, it can become large. Regularly expire old snapshots to keep the metadata volume manageable.
- **Manifest Efficiency Checks:** Look for redundant patterns, such as the same files appearing across many snapshots, and assess if optimization is needed.
- **Diagnostic Scripting:** Build alerting scripts that scan for patterns like many deleted files persisting across snapshots, indicating a need for maintenance.

## A.14 The all\_manifests metadata table

The all\_manifests metadata table in Apache Iceberg presents a complete historical view of all manifest files that have ever been associated with the table, across every snapshot, not just the current one. This makes it a critical resource for understanding the long-term evolution and health of table metadata.

### WHAT IT CONTAINS

Each row in all\_manifests provides metadata about a single manifest file, including:

- **content:** Type of content stored, 0 for data files, 1 for position delete files, 2 for equality delete files.
- **path:** Full URI to the manifest file.
- **length:** Size of the manifest in bytes.
- **partition\_spec\_id:** Identifier of the partition specification associated with the manifest.
- **added\_snapshot\_id:** Snapshot that introduced this manifest.
- **added\_data\_files\_count, existing\_data\_files\_count, deleted\_data\_files\_count:** Show how the manifest evolved.
- **partition\_summaries:** Summary statistics on the partitions represented in this manifest.
- **reference\_snapshot\_id:** Optional field referencing the snapshot that created the manifest.

### EXAMPLE QUERIES

In Spark:

```
SELECT * FROM db.table.all_manifests;
```

In Dremio:

```
SELECT * FROM TABLE(table_all_manifests('db.table'));
```

### USE CASES

- **Metadata Volume Analysis:** By inspecting how many manifests exist and how large they are, you can monitor for metadata sprawl, a common source of degraded query performance.
- **Change Tracking:** Understand how the number of data and delete files has changed over time by comparing manifest entries across snapshots.
- **Partition Insights:** partition\_summaries can help reveal evolving data distribution and guide partition spec changes.

- **Historical Debugging:** If a table's performance regressed, querying `all_manifests` can help trace whether a large number of delete files or data files were introduced suddenly.

## BEST PRACTICES

- **Compaction Triggers:** Use `added_data_files_count` and `deleted_data_files_count` to flag manifests that repeatedly accumulate many small files or deletions, potential signals to run compaction.
- **Snapshot Expiration:** Like other "all" metadata tables, `all_manifests` reflects the complete history. Expiring old snapshots reduces their size and improves performance when queried.
- **Partition Summary Monitoring:** Automate checks on `partition_summaries` to detect skew, data duplication across partitions, or overly fragmented partitioning.

## A.15 The refs metadata table

The refs metadata table provides visibility into all named references associated with an Iceberg table, including branches and tags. This capability is a result of Iceberg's native support for branching and tagging, which introduces versioned data workflows that mirror some of the familiar constructs from source control systems.

### WHAT ARE BRANCHES AND TAGS?

In Apache Iceberg, every table maintains a snapshot log to support features like time travel and snapshot isolation. Traditionally, users could only reference these snapshots indirectly, by timestamp or snapshot ID. However, branching and tagging extend this model by allowing users to create named references to specific snapshots, each with an independent lifecycle.

- **Tags** are immutable references to a specific snapshot. They are useful for marking historical states for audit, compliance, or data lineage tracking. For example, a year-end snapshot can be tagged as `EOY-2024` and retained indefinitely.
- **Branches** are mutable references that evolve with new snapshots. They allow independent snapshot histories to develop in parallel. This is particularly useful for experimental pipelines, write-audit-publish workflows, or regulatory sandboxing.

Each reference can have its own retention policies, defining how long the reference is retained and how many snapshots it must keep.

### WHAT DOES THE REFS TABLE CONTAIN?

This metadata table includes one row per reference and provides the following fields:

- **name:** The name of the branch or tag.
- **type:** Either BRANCH or TAG.
- **snapshot\_id:** The snapshot ID the reference currently points to.
- **max\_reference\_age\_in\_ms:** The maximum age (in milliseconds) that this reference can be retained.
- **min\_snapshots\_to\_keep:** For branches, the minimum number of snapshots that must be preserved.
- **max\_snapshot\_age\_in\_ms:** The maximum age for individual snapshots to be retained under this reference.

These retention properties are evaluated during snapshot expiration to determine which metadata and data files can be safely deleted.

## EXAMPLE QUERY

In Spark:

```
SELECT * FROM db.table.refs;
```

## WHY IT MATTERS

The refs table is essential for managing advanced lifecycle scenarios:

- **Auditability and Compliance:** Retain snapshots relevant to legal or regulatory investigations using long-lived tags.
- **Experimental Workflows:** Isolate writes on separate branches without impacting production data.
- **Data Recovery:** Safely time-travel or restore table state using tagged or branched snapshots.

Using the refs table, you can dynamically assess which snapshots are protected from expiration and ensure that critical references are preserved according to business policies.

## A.16 Monitoring table health with metadata tables

Maintaining an Iceberg lakehouse at scale requires proactive monitoring of table health. Fortunately, Iceberg's metadata tables offer a powerful set of tools for evaluating data layout, snapshot churn, and file proliferation. By leveraging these tables, you can design automated processes to trigger compaction, snapshot expiration, or other maintenance activities based on observed metrics rather than fixed schedules.

## KEY METRICS TO TRACK

The following metadata tables are especially useful for assessing table health:

- files: Analyze distribution of file sizes and clustering of data. A high number of small files may indicate the need for compaction.
- manifests: Observe the number of manifests and the number of files per manifest. A growing number of manifests may signal excessive snapshot churn.
- snapshots: Review the frequency and volume of snapshot creation. Frequent snapshots with minimal change may suggest inefficient write patterns.
- refs: Ensure critical branches and tags are not inadvertently expired during snapshot cleanup. By querying these tables routinely, you can gather insights such as:
  - How many files fall below the target size threshold?
  - Are files well-aligned with partitioning and sort orders?
  - Are there orphaned or dangling delete files accumulating?
  - How many snapshots were created in the last 24 hours?

These questions help you move beyond manual operations to intelligent automation.

### EXAMPLE: TRIGGERING COMPACTION BASED ON FILE METRICS

Suppose you want to initiate a compaction when more than 10% of a table's files are smaller than 128 MB. You could run a query like the following:

```
SELECT COUNT(*) AS total_files,
       SUM(CASE WHEN file_size_in_bytes < 134217728 THEN 1 ELSE 0 END) AS
small_files
FROM db.table.files;
```

You could then use these metrics in conjunction with orchestration tools like Apache Airflow, dbt, or custom Spark jobs to decide when to run `rewrite_data_files`.

### EXAMPLE: MONITORING SNAPSHOT FREQUENCY

To assess whether snapshots are being generated too frequently:

```
SELECT COUNT(*) AS snapshots_last_24h
FROM db.table.snapshots
WHERE committed_at > CURRENT_TIMESTAMP() - INTERVAL 1 DAY;
```

This can help diagnose excessive update operations or misconfigured ingestion processes.

### AUTOMATING MAINTENANCE WITH INSIGHTS

By setting thresholds and tracking trends in these metrics, you can automate:

- **Compaction:** When small files exceed a threshold.
- **Snapshot Expiration:** When the snapshot count or age exceeds limits.



- **Orphan File Cleanup:** When storage usage is inconsistent with tracked files.

Combined with branching, tagging, and retention-aware expiration, metadata-driven automation ensures your Iceberg lakehouse stays performant and lean, even under heavy and evolving workloads.

## *Appendix B Python for Apache Iceberg*

Apache Iceberg has become a central standard for modern data lakehouses, and Python provides one of the most adaptable ecosystems for working with it. This appendix introduces practical ways to use Iceberg directly and indirectly through leading Python libraries and frameworks. Each section focuses on a single library, explains its connection to Iceberg, and includes step-by-step examples for both ETL and analytical workloads.

The goal is to show how to build, manage, and analyze Iceberg data entirely in Python, without depending on JVM-based systems such as Spark. You'll learn how to define schemas, create tables, append and overwrite data, and perform queries using tools like Pylceberg, Polars, DuckDB, Daft, PyDremio, Bauplan, and SpiceAI.

Each tool plays a different role in the Python-Iceberg ecosystem:

- **Pylceberg** provides direct, low-level access to Iceberg tables and catalogs.
- **Polars** and **DuckDB** deliver high-performance, in-memory analytics on Iceberg data.
- **Daft** adds distributed computation built on Apache Arrow.
- **Dremio** provides comprehensive SQL support for Iceberg, featuring scalable query execution.
- **Bauplan** extends Iceberg's data model with Git-style branching and version control.
- **SpiceAI** enables federated queries and intelligent analytics over Iceberg datasets.

Together, these libraries demonstrate how Python can serve as both the control plane and execution environment for an Iceberg-based lakehouse. The following sections assume basic familiarity with Python and Iceberg concepts such as schemas, partitions, and catalogs. Each example is written to be runnable, concrete, and easy to adapt to your own workflows.

## B.1 Pylceberg

Pylceberg is the official Python client for Apache Iceberg. It gives Python users full control over Iceberg tables without requiring a JVM or Spark environment. With Pylceberg, you can connect to an Iceberg catalog, create and manage tables, and perform operations such as appending, overwriting, and scanning data, all in native Python.

To begin, install the library:

```
pip install pyiceberg[sql] pyarrow
```

This setup enables both catalog management and Arrow integration for reading and writing data.

Once installed, you can connect to an Iceberg catalog. The following example uses a local SQL-based catalog backed by SQLite, which is ideal for testing or standalone setups.

```

from pyiceberg.catalog import load_catalog
from pyarrow import parquet as pq

catalog = load_catalog(
    "default",
    type="sql",
    uri="sqlite:///demo.db",
    warehouse="file:///tmp/warehouse"
) #A

catalog.create_namespace("default") #B

table = catalog.create_table(
    "default.taxi_data",
    schema=pq.read_schema("trips.parquet")
) #C

arrow_table = pq.read_table("trips.parquet")
table.append(arrow_table) #D

result = table.scan().to_arrow() #E

print(len(result)) #F

```

**#A** Loads an Iceberg catalog using SQLite as the backend. The warehouse argument defines where metadata and data files are stored.

**#B** Creates a logical namespace to group related tables, similar to a schema in a database.

**#C** Creates a new Iceberg table in the catalog, inferring its schema from an existing Parquet file.

**#D** Appends a PyArrow table to the Iceberg table, which automatically creates new data files and updates metadata.

**#E** Reads the Iceberg table back into memory as a PyArrow table using Iceberg's scan API.

**#F** Displays the number of rows retrieved, confirming the data load.

Once your table exists, PyIceberg lets you modify its structure and content. Schema evolution, for example, can be done without rewriting old data:

```

update = table.update_schema()
update.add_column("pickup_borough", "string").commit() #A

new_data = pq.read_table("updated_trips.parquet")
table.append(new_data) #B

```

**#A** Opens a schema update transaction, adds a new column, and commits it atomically. Iceberg handles the metadata evolution seamlessly.

**#B** Writes new data that includes the added column; old rows will remain compatible.

You can also perform overwrites and filtered appends. The following replaces data for a specific partition while keeping the rest intact:

```
from pyiceberg.expressions import EqualTo

overwrite = table.overwrite(where=EqualTo("pickup_date", "2023-01-01"))
overwrite.add_file("new_data.parquet").commit() #A
```

**#A Creates a transactional overwrite scoped to a predicate. Only files matching the condition are replaced, ensuring atomicity and isolation.**

Finally, PyIceberg supports scanning subsets of data for analytical queries using filters:

```
from pyiceberg.expressions import GreaterThan

expensive_trips = table.scan(
    row_filter=GreaterThan("total_amount", 100)
).to_arrow() #A

print(expensive_trips.to_pandas().head()) #B
```

**#A Pushes down a filter to the scan operation, allowing Iceberg to skip files that don't match.**

**#B Converts the result into a Pandas DataFrame for quick inspection or analysis.**

Through these examples, PyIceberg demonstrates how to manage Iceberg tables directly in Python, from table creation to schema evolution and analytics. It serves as the foundation for higher-level integrations like Polars, DuckDB, and Daft, which rely on its APIs for catalog and table operations.

## B.2 Polars

Polars has emerged as one of the fastest dataframe libraries in Python, offering a query engine that rivals even DuckDB in performance. With recent releases, Polars introduced native Iceberg integration, allowing direct reading from and (experimentally) writing to Iceberg tables. This makes Polars a natural choice for analytics workloads over Iceberg data, combining efficient vectorized computation with lazy query execution.

Before working with Iceberg, ensure you have Polars and PyIceberg installed since Polars delegates Iceberg catalog handling to PyIceberg underneath.

```
pip install polars pyiceberg pyarrow
```

Once installed, you can connect to an existing Iceberg table and perform queries using Polars' lazy API. The following example reads an Iceberg table directly from its metadata file.

```
import polars as pl

lazy_df = pl.scan_iceberg(
    "s3://data-lake/warehouse/trips/metadata.json",
    storage_options={"AWS_ACCESS_KEY_ID": "key", "AWS_SECRET_ACCESS_KEY":
"secret"}
) #A

df = lazy_df.collect() #B

print(df.describe()) #C
```

**#A** Initializes a lazy scan on an Iceberg table by pointing to its metadata.json. Polars uses the Iceberg metadata to discover all Parquet data files.

**#B** Executes the lazy query, loading only the required columns and partitions into memory.

**#C** Prints a summary of the table, demonstrating how seamlessly Iceberg integrates into Polars' dataframe model.

Once data is loaded, Polars provides efficient transformations and analytical operations. The following example filters and aggregates Iceberg data to compute the average trip distance by passenger count.

```
summary = (
    lazy_df
    .filter(pl.col("passenger_count") > 1) #A
    .group_by("passenger_count")           #B
    .agg(pl.col("trip_distance").mean().alias("avg_distance")) #C
    .sort("passenger_count")              #D
    .collect()                            #E
)

print(summary)
```

**#A** Pushes a predicate down into the Iceberg scan, ensuring only necessary files are read.

**#B** Groups rows by the passenger count column for aggregation.

**#C** Computes the mean trip distance per passenger count.

**#D** Orders the results to produce a clean output.

**#E** Executes the lazy plan, triggering all Iceberg reads and computations efficiently.

Although Polars' Iceberg write functionality is still experimental, it provides a simple API for appending or overwriting data. The following example demonstrates writing a Polars DataFrame into an Iceberg table.

```
# Create a small Polars DataFrame
new_data = pl.DataFrame({
    "trip_id": [101, 102, 103],
    "passenger_count": [1, 2, 3],
    "trip_distance": [3.5, 5.1, 2.9]
}) #A

# Append this data to an existing Iceberg table
new_data.write_iceberg(
    "default.trips_table",
    mode="append"
) #B
```

**#A** Defines a new Polars DataFrame to be written into the Iceberg table.

**#B** Appends the DataFrame to the specified table. Polars delegates the write operation to Pylceberg, ensuring Iceberg's ACID guarantees.

Polars' combination of lazy evaluation, columnar optimization, and native Iceberg support makes it a powerful choice for analytical workloads. You can use it to explore data, run aggregations, and prototype transformations before writing results back to Iceberg for persistent storage. While write operations are still evolving, read and compute performance are already production-ready, providing a fast, Pythonic experience for working with Apache Iceberg tables.

## B.3 DuckDB

DuckDB is an in-process analytical database engine that provides lightning-fast SQL queries directly in Python. Its extensible design allows it to read and write Iceberg tables through the Iceberg extension, enabling seamless access to Iceberg data for both ad-hoc analytics and ETL tasks. Using DuckDB, you can query Iceberg tables stored locally or in the cloud, insert new data, and even connect to Iceberg catalogs for managed metadata operations.

Before working with Iceberg, you must install DuckDB and enable the Iceberg extension.

```
pip install duckdb
```

Once installed, you can initialize a connection and load the Iceberg extension to make Iceberg functions available inside DuckDB.

```
import duckdb

con = duckdb.connect("iceberg_demo.db") #A

con.execute("INSTALL iceberg;") #B

con.execute("LOAD iceberg;") #C
```

**#A** Establishes a persistent DuckDB connection that can query Iceberg tables and store local metadata.

**#B** Installs the Iceberg extension, which adds Iceberg-specific functions such as `iceberg_scan`.

**#C** Activates the extension for the session, making Iceberg operations immediately available.

After loading the extension, you can query an existing Iceberg table by referencing its metadata file. The following example reads data directly from a table's `metadata.json` file.

```
result = con.execute("""
    SELECT vendor_id, trip_distance, total_amount
    FROM iceberg_scan('/data/warehouse/trips/metadata.json')
    WHERE trip_distance > 5
    ORDER BY total_amount DESC
    LIMIT 10
""").df() #A

print(result.head()) #B
```

**#A** Executes a SQL query over the Iceberg table using `iceberg_scan`, which reads the table metadata and retrieves Parquet files that match the filter.

**#B** Displays the top rows of the query, now loaded as a Pandas DataFrame.

DuckDB also supports integrating with Iceberg catalogs, including REST-based catalogs such as Apache Polaris or AWS Glue. Once attached, you can query and manage Iceberg tables with familiar SQL DDL and DML commands.



```

con.execute("""
    ATTACH 'my_catalog' AS iceberg_catalog (
        TYPE iceberg,
        ENDPOINT 'https://catalog-host.com/v1',
        SECRET 'my_secret_key'
    );
""") #A

tables = con.execute("SHOW TABLES FROM iceberg_catalog.default;").fetchall() #B

con.execute("""
    CREATE TABLE iceberg_catalog.default.daily_trips AS
    SELECT * FROM iceberg_catalog.default.trips WHERE trip_date = '2023-01-01';
""") #C

```

**#A** Attaches a catalog connection so DuckDB can resolve table names without direct paths.

**#B** Lists tables registered under the default namespace in the attached catalog.

**#C** Creates a new Iceberg table directly through the catalog by running a **CREATE TABLE AS SELECT** statement.

You can also insert new data into existing Iceberg tables. DuckDB handles writing Parquet data and updating Iceberg metadata transparently through the extension.

```

con.execute("""
    INSERT INTO iceberg_catalog.default.daily_trips
    VALUES (1001, 3.2, 15.75), (1002, 7.1, 28.10);
""") #A

```

**#A** Appends new rows to the Iceberg table using SQL syntax; DuckDB commits data atomically by updating the table's manifest and metadata files.

For analytical tasks, you can perform joins, aggregations, and time-travel queries across Iceberg tables as if they were standard SQL tables. DuckDB's Iceberg extension supports reading snapshots by ID or timestamp, allowing reproducible queries on historical table states.

```
# Query a specific snapshot using a snapshot ID
historical = con.execute("""
    SELECT COUNT(*) AS trips_count
    FROM iceberg_scan(
        '/data/warehouse/trips/metadata.json',
        snapshot_id := '9182736455'
    );
""").df() #A

print(historical)
```

**#A** Reads data from a previous snapshot of the Iceberg table, providing time-travel capability for reproducible analytics.

DuckDB's Iceberg integration makes it one of the most convenient engines for both interactive queries and scripted ETL workflows. You can attach catalogs, execute transformations, and export analytical results with SQL simplicity, all while maintaining Iceberg's transactional consistency and schema evolution guarantees.

## B.4 Daft

Daft is a distributed dataframe library built on Apache Arrow and optimized for large-scale data processing. It provides a familiar, pandas-like API while offering high-performance I/O and query execution. Daft's integration with Apache Iceberg is powered by **Pylceberg**, allowing users to read Iceberg tables directly into Daft DataFrames for analysis. While Daft currently focuses on reading Iceberg data rather than writing it, it excels at analytical workloads that require scanning and transforming large Iceberg datasets.

To get started, install Daft along with Pylceberg and any necessary storage dependencies.

```
pip install daft pyiceberg pyarrow s3fs
```

Once installed, you can use Daft to read Iceberg tables through a Pylceberg catalog. The catalog provides table metadata and object storage paths, while Daft handles data retrieval and computation.

```

import daft
from pyiceberg.catalog import load_catalog

catalog = load_catalog(
    "default",
    type="sql",
    uri="sqlite:///demo.db",
    warehouse="file:///tmp/warehouse"
) #A

table = catalog.load_table("default.taxi_data") #B

df = daft.read_iceberg(table) #C

df.show(5) #D

```

**#A** Connects to a SQL-based Iceberg catalog that stores table definitions and metadata.

**#B** Loads an existing Iceberg table from the catalog.

**#C** Reads the table into a Daft DataFrame using the PyIceberg integration. Daft handles parallel reads from the table's Parquet files.

**#D** Displays the first few rows for quick inspection, similar to `head()` in pandas.

Once data is loaded, you can perform distributed transformations and filtering operations using Daft's expression API. Daft pushes filters down to the Iceberg scan, minimizing unnecessary I/O by reading only the relevant files.

```

filtered = (
    df.filter(df["total_amount"] > 100)      #A
    .groupby("passenger_count")             #B
    .agg({"trip_distance": ["mean"]})        #C
)

filtered.show()                             #D

```

**#A** Applies a filter expression that Iceberg uses for partition pruning and file skipping.

**#B** Groups rows by passenger count for aggregated analytics.

**#C** Computes the mean of the `trip_distance` column within each group.

**#D** Displays the aggregated results, evaluated in a distributed fashion.

Daft's tight coupling with Arrow allows zero-copy data interchange with other tools. For example, you can convert Daft DataFrames into Arrow Tables or Polars DataFrames for downstream processing.

```
# Convert Daft DataFrame to Arrow Table
arrow_table = df.to_arrow_table() #A

# Convert Arrow Table to Polars DataFrame
import polars as pl
polars_df = pl.from_arrow(arrow_table) #B
```

**#A Exports the Daft DataFrame into a PyArrow Table for interoperability.**  
**#B Converts the Arrow Table into a Polars DataFrame, enabling mixed-library pipelines for analytics and visualization.**

While Daft does not yet support writing Iceberg tables directly, you can export processed data to Parquet and register it through Pylceberg. This two-step workflow enables Daft to act as a high-performance compute layer in Iceberg-based data pipelines.

```
# Write Daft DataFrame to Parquet files
df.write_parquet("s3://data-lake/cleaned/trips/") #A

# Register exported data as a new Iceberg table
catalog.create_table(
    "default.cleaned_trips",
    schema=df.schema().to_arrow(),
    location="s3://data-lake/cleaned/trips/"
) #B
```

**#A Writes the Daft DataFrame to Parquet files in object storage.**  
**#B Creates a new Iceberg table in the catalog that references those files, allowing future queries through Iceberg-compatible engines.**

In practice, Daft's integration with Apache Iceberg offers a modern, Pythonic approach to reading large datasets efficiently. It is particularly useful for workloads where data is stored in Iceberg tables, but analytical logic is executed in Python. By combining Pylceberg's metadata capabilities with Daft's distributed execution engine, developers can process massive Iceberg datasets interactively and feed results into downstream systems or visualizations, all within a single Python environment.

## B.5 Dremio

Dremio provides a full SQL engine for Apache Iceberg, and Python users can access its capabilities through Dremio's APIs and connectors. Using these connectors, you can query, create, and modify Iceberg tables while taking advantage of Dremio's query planner, reflections, and data federation capabilities. The two most common ways to interact with Dremio from Python are through the ODBC driver and the Apache Arrow Flight interface. Both allow execution of SQL commands on Iceberg tables as if they were standard database tables.

To begin, install the necessary dependencies.

```
pip install pyodbc pyarrow
```

Once installed, you can connect to a Dremio cluster and execute SQL commands directly from Python. The following example demonstrates how to query an Iceberg table using the ODBC interface.

```
import pyodbc

conn = pyodbc.connect(
    "Driver={Dremio Connector 64-bit};"
    "ConnectionType=Direct;"
    "HOST=demo.dremio.cloud;"
    "PORT=443;"
    "AuthenticationType=Plain;"
    "UID=my_username;"
    "PWD=my_token;"
    "SSL=1"
) #A

cursor = conn.cursor() #B

cursor.execute('SELECT * FROM "Samples"."NYC_Trips" LIMIT 5;') #C

rows = cursor.fetchall() #D
for row in rows:
    print(row)
```

**#A** Creates a direct connection to Dremio using the ODBC driver and a personal access token.

**#B** Instantiates a cursor for sending SQL queries to the Dremio engine.

**#C** Runs a SQL query against an Iceberg table stored in Dremio's catalog.

**#D** Fetches and prints the query results, showing how Iceberg tables can be queried like any relational source.

The ODBC method is ideal for lightweight applications, but Arrow Flight offers faster, more efficient transport of large datasets using the Arrow columnar format. The following example demonstrates how to query an Iceberg table using Arrow Flight.

```

from pyarrow import flight

client = flight.FlightClient("grpc+tls://data.dremio.cloud:443") #A

headers = [(b"authorization", f"bearer {token}".encode())] #B

sql = 'SELECT trip_distance, total_amount FROM "Samples"."NYC_Trips" WHERE
total_amount > 100' #C

flight_info = client.get_flight_info(
    flight.FlightDescriptor.for_command(sql),
    flight.FlightCallOptions(headers=headers)
) #D

reader = client.do_get(flight_info.endpoints[0].ticket,
flight.FlightCallOptions(headers=headers)) #E
arrow_table = reader.read_all() #F

print(arrow_table.to_pandas().head()) #G

```

**#A** Connects to Dremio's Arrow Flight endpoint for high-performance data transport.

**#B** Sets authentication headers using a bearer token for secure access.

**#C** Defines a SQL query that selects and filters data from an Iceberg table.

**#D** Requests query execution information from Dremio, which returns metadata about result endpoints.

**#E** Streams the data using Flight's efficient gRPC transport layer.

**#F** Reads the entire dataset into an in-memory Arrow table for further processing in Python.

**#G** Converts the Arrow Table into a Pandas DataFrame for analysis or visualization.

Beyond querying, you can also create and modify Iceberg tables directly through Dremio. The following example shows how to create a new Iceberg table and insert data into it.

```

cursor.execute("""
    CREATE TABLE "Samples"."daily_summary" AS
    SELECT passenger_count, AVG(total_amount) AS avg_total
    FROM "Samples"."NYC_Trips"
    GROUP BY passenger_count;
""") #A

cursor.execute("""
    INSERT INTO "Samples"."daily_summary"
    VALUES (4, 32.75), (5, 45.12);
""") #B

conn.commit() #C

```

**#A** Creates a new Iceberg table in Dremio by selecting data from another Iceberg table.

**#B** Inserts new rows into the table using standard SQL syntax.

**#C** Commits the transaction so changes are reflected in the Iceberg metadata and manifest files.

Through either interface, Dremio allows full DDL and DML support for Iceberg tables, including CREATE, INSERT, DELETE, MERGE, and UPDATE operations. This makes Dremio an ideal control plane for managing Iceberg-based data lakes while maintaining interoperability with Python's data science stack. By combining PyArrow's performance with Dremio's Iceberg integration, developers can run scalable, SQL-driven analytics on open data without moving or duplicating datasets.

## B.6 Bauplan

Bauplan is a modern data orchestration framework designed to manage Iceberg-based data lakes with version control. It introduces Git-like workflows for your data, allowing you to create branches, experiment with transformations, validate results, and merge changes, all backed by Apache Iceberg's metadata and storage structure. Every operation in Bauplan, branch creation, table import, schema evolution, and merge, ultimately manipulates Iceberg tables under the hood, making it a powerful abstraction layer for Iceberg data engineering.

To get started, install the Bauplan SDK.

```
pip install bauplan
```

Once installed, you can authenticate and initialize a client connection. The client provides access to your Iceberg-backed repositories and grants full control over data operations.

```
import bauplan

client = bauplan.Client(api_key="my_api_key") #A

client.create_branch("dev", from_ref="main") #B

client.create_namespace("default", branch="dev") #C
```

**#A** Initializes a Bauplan client to communicate with the Bauplan service or local environment.

**#B** Creates a new branch named “dev” derived from “main,” enabling isolated experimentation.

**#C** Defines a logical namespace for grouping related Iceberg tables, similar to a schema or database.

Once a branch and namespace are ready, you can create and load Iceberg tables. Bauplan uses PyIceberg internally to register tables and manage their schema and metadata.

```
client.create_table(
    table="trips",
    search_uri="s3://data-lake/trips/*.parquet",
    namespace="default",
    branch="dev",
    replace=True
) #A

client.import_data(
    table="trips",
    search_uri="s3://data-lake/trips/*.parquet",
    namespace="default",
    branch="dev"
) #B

arrow_table = client.scan(
    table="trips",
    ref="dev",
    namespace="default",
    columns=["passenger_count", "total_amount"]
) #C

print(arrow_table.to_pandas().head()) #D
```

**#A** Creates a new Iceberg table in the dev branch by scanning and registering Parquet data as Iceberg files.

**#B** Imports or replaces data in the table; each operation generates a new Iceberg snapshot.

**#C** Reads the Iceberg table as a PyArrow Table, allowing downstream analytics in Pandas or Polars.

**#D** Converts the Arrow table into a Pandas DataFrame for quick inspection.



Bauplan also supports validation and publishing workflows that mirror Git's branching model. After data has been validated or transformed on a branch, it can be merged back into main for production use.

```
assert arrow_table["passenger_count"].null_count == 0  #A

client.merge_branch(source_ref="dev", into_branch="main")  #B

client.delete_branch("dev")  #C
```

**#A** Verifies that the data meets quality requirements before publishing.

**#B** Merges the dev branch into main, committing all Iceberg table changes atomically.

**#C** Cleans up by deleting the temporary branch after a successful merge.

Because Bauplan's operations are transactional and Iceberg-native, every change, whether an import, overwrite, or schema update, creates a new snapshot in the underlying Iceberg table. This ensures full traceability, auditability, and rollback capability.

You can also evolve schemas on a branch before merging. For example, you might add a new column for data enrichment.

```
client.update_schema(
    table="trips",
    namespace="default",
    branch="dev",
    add_columns=[{"name": "pickup_zone", "type": "string"}]
)  #A
```

**#A** Modifies the table's schema by adding a new column within the branch context, preserving historical compatibility across snapshots.

Bauplan provides a powerful model for Iceberg lakehouse management: it treats data as code. You can branch, commit, validate, and merge with the same discipline as software development, while maintaining all of Iceberg's transactional guarantees. This approach allows data engineers to safely experiment with transformations, run quality checks, and publish trusted data into production, all without breaking or duplicating datasets.

## B.7 SpiceAI

SpiceAI is a modern query engine that allows federated access to multiple data sources, including Apache Iceberg. It treats Iceberg tables as first-class datasets that can be queried alongside other sources such as PostgreSQL, BigQuery, or object storage. SpiceAI combines open standards, Arrow, Flight SQL, and REST, with Iceberg's table format to enable high-performance analytical queries through a simple Python interface.

To use SpiceAI, install its Python SDK:

```
pip install spicepy
```

After installation, you can configure a dataset to point to your Iceberg catalog. SpiceAI uses the REST catalog endpoint to discover Iceberg tables and expose them through its unified SQL layer. Once configured, you can interact with those tables just as you would with any other SQL data source.

The following example demonstrates how to query an Iceberg table with the SpiceAI Python client.

```
from spicepy import Client

client = Client() #A

result = client.query("""
    SELECT pickup_borough, AVG(total_amount) AS avg_fare
    FROM iceberg.default.trips
    WHERE total_amount > 50
    GROUP BY pickup_borough
    ORDER BY avg_fare DESC
""") #B

df = result.read_pandas() #C

print(df.head()) #D
```

**#A** Creates a new SpiceAI client instance that communicates with the Spice runtime or cloud service.

**#B** Executes a SQL query over an Iceberg dataset registered in Spice's catalog. The query runs on the federated engine using the Iceberg REST API under the hood.

**#C** Reads the results of the query into a Pandas DataFrame for analysis or visualization.

**#D** Displays a preview of the aggregated results for quick inspection.

Beyond simple querying, SpiceAI supports complex analytical operations across multiple sources. You can join Iceberg tables with external datasets to perform multi-domain analysis without data duplication.

```
joined = client.query("""
    SELECT i.trip_id, i.total_amount, p.driver_rating
    FROM iceberg.default.trips AS i
    JOIN postgres.analytics.driver_ratings AS p
    ON i.driver_id = p.driver_id
    WHERE i.total_amount > 100
""") #A

joined_df = joined.read_pandas() #B

print(joined_df.describe()) #C
```

**#A** Executes a federated query that joins an Iceberg table with a PostgreSQL table, showing how SpiceAI handles multi-source analytics.

**#B** Reads the result into a Pandas DataFrame, preserving column types and metadata.

**#C** Prints summary statistics, demonstrating how seamlessly Iceberg data integrates into broader analytical contexts.

You can also register Iceberg tables dynamically through configuration or API calls, making it easy to connect new datasets as your lakehouse evolves. The following example registers an Iceberg table through the dataset configuration interface.

```
datasets:
  - name: trips
    from:
      iceberg:https://catalog.mycompany.com/v1/namespaces/default/tables/trips #A
```

**#A** Declares an Iceberg dataset by referencing its REST catalog endpoint, allowing SpiceAI to manage it as part of its unified query layer.

SpiceAI does not modify Iceberg data directly but provides a powerful querying and integration layer. All write or maintenance operations, such as inserts, deletes, or schema updates, are performed using your Iceberg catalog tools or engines like Dremio or Pylceberg. SpiceAI's focus is on fast, federated analytics and AI-driven insights across your Iceberg tables and other structured sources.

By integrating Iceberg into its federated architecture, SpiceAI allows Python developers to analyze open-format data with minimal configuration, leveraging both Iceberg's data reliability and Spice's unified query model. This approach makes it ideal for organizations building AI-ready lakehouses that need flexible access to governed data stored in Iceberg.

## B.8 Summary and best practices

Throughout this appendix, you've explored how Python interacts with Apache Iceberg through a range of libraries, each with different strengths, abstractions, and levels of integration. While **Pylceberg** provides the direct and foundational interface for Iceberg table operations, higher-level frameworks like **Polars**, **DuckDB**, **Daft**, **Dremio**, **Bauplan**, and **SpiceAI** build on top of these principles to offer richer analytical and orchestration capabilities. Understanding how to combine these tools effectively is the key to maintaining a clean, scalable, and efficient Iceberg-based data platform.

At the heart of every Iceberg workflow in Python lies the same set of operations: creating, reading, writing, evolving schemas, and scanning data efficiently. Let's summarize the best practices and patterns that emerge across all these integrations.

```

from pyiceberg.catalog import load_catalog
from pyarrow import parquet as pq

# Initialize a local Iceberg catalog for testing
catalog = load_catalog(
    "default",
    type="sql",
    uri="sqlite:///demo.db",
    warehouse="file:///tmp/warehouse"
) #A

# Create a new Iceberg table from a Parquet dataset
table = catalog.create_table(
    "default.sales",
    schema=pq.read_schema("sales_data.parquet")
) #B

# Read Parquet file as Arrow Table and append to Iceberg
data = pq.read_table("sales_data.parquet")
table.append(data) #C

# Perform incremental overwrite for a given partition
from pyiceberg.expressions import EqualTo
overwrite = table.overwrite(where=EqualTo("region", "US")) #D
overwrite.add_file("new_sales_data.parquet").commit() #E

# Query the table with a filter for analytics
from pyiceberg.expressions import GreaterThan
results = table.scan(row_filter=GreaterThan("amount", 1000)).to_arrow() #F
print(results.to_pandas().head()) #G

```

**#A** Connects to a local Iceberg catalog that manages metadata and table registration.

**#B** Creates a new Iceberg table using a schema derived from a Parquet file.

**#C** Appends data as new files, letting Iceberg manage atomic commits.

**#D** Defines a conditional overwrite targeting only the "US" partition.

**#E** Commits the overwrite transaction, producing a new snapshot.

**#F** Executes a filtered scan to retrieve high-value transactions efficiently.

**#G** Converts the Arrow table into a Pandas DataFrame for inspection.

This example encapsulates the lifecycle that all Python-based Iceberg tools follow, whether the operations are executed through PyIceberg directly or abstracted through other engines.

Across all libraries discussed, several best practices consistently lead to more reliable and efficient Iceberg workflows:

1. **Use catalogs consistently.**

Always register and query Iceberg tables through a catalog. This ensures that metadata remains synchronized and transactional guarantees hold, even across multiple compute engines.

2. **Leverage schema evolution wisely.**  
Adding columns is easy in Iceberg, but plan for backward compatibility. Always evolve schemas on a test branch (in Bauplan) or a development catalog before merging into production.
3. **Optimize writes upfront.**  
Write data in appropriately sized files and use partition strategies that reflect common query filters. Doing this early reduces the need for heavy optimization jobs later.
4. **Prefer lazy and pushdown-enabled scans.**  
Libraries like Polars, Daft, and DuckDB push filters into Iceberg's metadata layer, which minimizes I/O and accelerates queries. Always design queries to take advantage of these optimizations.
5. **Adopt branch-based experimentation.**  
Bauplan's branching model is ideal for isolating experimental changes. Treat branches as staging environments for data transformations before merging clean data into production tables.
6. **Unify analytics and AI access.**  
Use federated engines like Dremio or SpiceAI when multiple teams need shared access to Iceberg tables across cloud and on-prem systems. This approach centralizes governance while preserving open access.

Apache Iceberg's design allows these diverse tools to interoperate seamlessly. Whether you're managing data ingestion, optimizing queries, or powering AI applications, the combination of Python and Iceberg provides both flexibility and control. Together, they form a powerful foundation for an open, versioned, and fully governed lakehouse ecosystem.

## *Appendix C The Apache Iceberg specification*

Apache Iceberg is more than an open table format; it is a specification with clearly defined rules for how table metadata, snapshots, partitioning, and schema evolution behave. These specifications ensure consistent behavior across tools and engines, enabling interoperability, reliability, and long-term stability. As Iceberg evolves, each new version of the spec introduces new capabilities while balancing backward compatibility.

This appendix provides a concise but thorough reference to the Iceberg specification. It begins by explaining the purpose and structure of the specification, then walks through format versions 1, 2, and 3, highlighting the major features introduced in each. You'll also find guidance on how metadata, snapshots, and concurrency work under the hood, along with coverage of the REST Catalog API and Puffin file format specification.

Whether you're implementing your own engine integration or evaluating compatibility between tools, this appendix will help you understand what guarantees Iceberg makes, and how they continue to evolve.

### **C.1 Understanding the Iceberg specification**

The Iceberg specification is a living standard that defines how data tables behave within the Apache Iceberg ecosystem. It outlines how metadata is structured, how snapshots are created and updated, how delete files are handled, and how tables evolve. This spec provides the foundation that allows different engines to interoperate reliably with the same data.

Rather than leaving table behavior up to implementation details, Iceberg formalizes key operations so that they are predictable, durable, and versioned. This specification allows tools like Spark, Flink, Trino, and Dremio to read and write Iceberg tables without conflicts or surprises.

Understanding how the spec is structured and how it evolves across versions helps you make informed decisions about compatibility, upgrades, and tool support. It also clarifies what guarantees Iceberg provides at each layer of the table format.

### C.1.1 What is a table format specification?

A table format specification defines the rules for how tabular data is stored, managed, and accessed. In Iceberg, this includes:

- The structure of metadata files
- How snapshots and manifests track data files
- How deletes and updates are represented
- How schema and partitioning changes are recorded
- How concurrency and commits are managed

Unlike legacy table formats that rely on directory conventions or implicit behaviors, Iceberg makes all of these rules explicit and versioned. This reduces ambiguity and makes the format safer to adopt in multi-engine environments.

### C.1.2 Why Iceberg formalizes table behavior

Without a specification, each tool that interacts with a table would need to reverse-engineer the format or apply assumptions. This leads to inconsistencies, data corruption, or lock-in. Iceberg avoids these pitfalls by enforcing a shared contract that all engines must follow.

This contract provides:

- Predictable read and write behavior
- Isolation between readers and writers
- Backward-compatible schema and partition evolution
- Consistent snapshot and version tracking

The spec enables safe collaboration across teams and tools. It also decouples table storage from compute engines, making Iceberg well-suited for cloud-native and multi-tenant environments.

### C.1.3 Evolution of the spec: versioning principles and compatibility

The Iceberg specification evolves through discrete format versions. Each version introduces new capabilities that may not be readable by older clients. To protect compatibility, tables declare the format version they use, and readers check this version before processing metadata.

So far, three versions have been formally adopted:

- **Version 1** introduced the core snapshot model and immutable data tracking.
- **Version 2** added support for row-level deletes and stricter validation rules.
- **Version 3** expanded type support and introduced deletion vectors, row lineage, and encryption features.

Future versions are developed with community input and reviewed for compatibility. Spec changes that don't affect file format structure or metadata parsing may be introduced in minor library releases, but any feature that breaks forward-compatibility triggers a version bump.



By keeping the spec versioned and documented, Iceberg allows users to choose the right balance between stability and new features, depending on their environment and engine support.

## C.2 Iceberg table format versions

Iceberg tables evolve, and so does the format that governs them. Each new version of the Iceberg table format adds features that improve performance, flexibility, or correctness. But to ensure stability, these changes are gated behind explicit format version numbers.

When you create or upgrade a table, you choose a format version. This version determines what features are available and which rules apply. For example, row-level deletes and deletion vectors are only supported in newer versions. Older versions remain valid and readable, but newer engines may write files or metadata that older readers cannot understand.

This section walks through each officially adopted format version, v1, v2, and v3, explaining the features introduced, the problems they solve, and how they affect table behavior. If you're planning to integrate a new engine, upgrade your tables, or understand how versioning affects interoperability, this overview will give you the technical context you need.

### C.2.1 Version 1: Foundation for analytical tables

Version 1 is the original Iceberg table format. It was designed to address the limitations of directory-based table layouts, such as those used in Hive. Instead of treating folders and file names as the source of truth, Iceberg v1 introduced a metadata-driven approach. This shift laid the groundwork for features like snapshot isolation, schema evolution, and efficient query planning.

Key characteristics of version 1 include:

- **Immutable file tracking:** Tables track data files explicitly through manifest files, rather than relying on directory listings.
- **Snapshot-based access:** Readers use a snapshot to get a consistent view of the table, even as new data is written.
- **Schema evolution:** Tables support safe column adds, drops, renames, and reordering.
- **Partition independence:** Partition values are stored in metadata, not inferred from file paths.

Version 1 supports Parquet, Avro, and ORC as data file formats. It also introduces the manifest list and manifest file structures that form the basis for tracking data and metadata changes. These structures are compact, readable, and designed to scale with the number of files and partitions.

Tables created with version 1 are compatible with version 2 readers. No metadata changes are required to read a v1 table in a v2 engine, which makes it safe to upgrade processing tools before upgrading tables.

## C.2.2 Version 2: Row-level deletes and stricter writes

Version 2 extends the Iceberg format to support row-level deletes without rewriting full data files. This is a critical feature for compliance, data correction, and slowly changing dimensions. The main addition is the concept of *delete files*, which track deleted rows separately from the original data files.

There are two types of delete files:

- **Position deletes:** Identify rows by file path and row position.
- **Equality deletes:** Identify rows by matching one or more column values, such as `user_id = 42`.

These delete files are written and tracked using the same metadata structures as data files. When a scan is planned, the engine applies delete files during query execution to produce a correct result.

In addition to delete support, version 2 introduces stronger requirements for writers:

- Writers must record the data and delete files they add or remove with each commit.
- Engines must validate snapshot isolation before applying updates.
- Table updates are more strictly defined to avoid inconsistent states in multi-writer environments.

All v1 metadata and data files remain valid in version 2. When reading v1 tables with a v2 engine, default values are applied to fill in missing fields added in the v2 spec. This ensures that backward compatibility is preserved even as capabilities expand.

## C.2.3 Version 3: Extended types and advanced capabilities

Version 3 introduces the most expansive set of changes to date. It deepens Iceberg's support for complex use cases while preparing the format for tighter integration with data governance and fine-grained change tracking.

New features introduced in version 3 include:

- **Extended data types:** Adds support for timestamp with timezone, variant, geometry, geography, and unknown types, enabling broader interoperability with analytics engines and location-aware applications.
- **Default column values:** Allows schemas to specify default values for columns, simplifying ingestion pipelines and reducing data transformation overhead.
- **Multi-argument transforms:** Partition specs and sort orders can now include expressions that span multiple columns, enabling more expressive partitioning strategies.
- **Row lineage tracking:** Introduces optional metadata fields that capture where data came from and how it was transformed, improving auditability.
- **Deletion vectors:** Adds binary structures that efficiently track which rows have been logically deleted, without needing to apply separate delete files during reads.

- **Table encryption keys:** Enables table-level encryption metadata to be tracked natively within the format, laying the groundwork for more secure, governed environments.

Together, these enhancements allow Iceberg to serve a wider range of operational, analytical, and regulatory needs. They also create the foundation for real-time pipelines that require both performance and traceability.

#### C.2.4 Version 4: Performance, portability, and real-time readiness

Version 4 is still under development, but it represents a strategic shift in how Iceberg handles metadata, commits, and operations at scale. While earlier versions focused on correctness and capability, v4 emphasizes speed, flexibility, and operational simplicity.

Proposed improvements include:

- **Single-file commits:** Consolidates metadata changes into a single file per commit. This reduces file system operations, lowers write amplification, and improves throughput in high-ingestion environments.
- **Parquet-based metadata:** Replaces Avro with Parquet for storing manifest and manifest list files. As a columnar format, Parquet enables engines to scan only the metadata fields they need, reducing I/O and improving query planning.
- **Redesigned column statistics:** Adds typed and projectable statistics, including null counts and support for nested structures. This gives query engines more precision when pruning files and estimating costs.
- **Relative paths in metadata:** Supports storing file references relative to a base path, rather than using absolute URIs. This makes tables easier to move, clone, or back up across storage environments.

These changes are designed for large-scale, mixed workload environments where tables may receive thousands of updates per hour. They reflect real-world pain points shared by users managing streaming ingestion, massive tables, or complex deployment pipelines.

While not yet finalized, the v4 proposals highlight Iceberg's continued focus on performance and usability. They aim to remove operational friction, reduce metadata overhead, and bring Iceberg closer to supporting truly interactive and real-time data workloads.

### C.3 Snapshot management and table metadata

Iceberg's strength lies not just in how it stores data, but in how it organizes and evolves that data over time. Unlike traditional file-based systems that treat table state as a collection of folders and files, Iceberg treats the table as a stream of snapshots, each representing a complete, immutable view of the data at a point in time.

Snapshots allow Iceberg to deliver ACID guarantees without requiring locks or coordination across compute engines. When a table is updated, a new snapshot is created with metadata that points to the current set of data files. This snapshot becomes the new table state, while the old snapshots remain available for time travel, rollback, or audit purposes.

Managing snapshots and metadata is central to Iceberg's design. This section explains how metadata is structured, how snapshots are created and tracked, and how Iceberg maintains isolation and concurrency even in write-heavy environments.

### C.3.1 Table metadata files

Every Iceberg table has a root metadata file. This file contains the table's current schema, partition spec, properties, and a reference to the most recent snapshot. Each time a change is committed, whether it's a new data file, a schema update, or a snapshot expiration, a new metadata file is written, and the catalog is updated to point to it.

These metadata files are immutable. They are never edited in place. Instead, each one is a standalone description of the full table state. This design supports atomicity: the catalog either points to the old version or the new one, but never to a partial or corrupted state.

To support time travel, the metadata file also includes a history log that records every snapshot and the timestamp when it became current. Engines can use this log to reconstruct the table's evolution or to query the state as it existed at a given moment.

### C.3.2 Snapshots and the manifest list

A snapshot in Iceberg is more than just a pointer; it is a complete description of the table's data at a moment in time. It contains references to one or more *manifest lists*, each of which is a metadata file that enumerates the manifests for that snapshot.

Manifests are metadata files themselves. Each one lists the data files that make up a subset of the table. It also includes partition-level stats and file-level metrics. Because manifests can be reused across snapshots, Iceberg avoids rewriting metadata that hasn't changed. This reuse is a key reason why snapshot creation is fast, even for very large tables.

The manifest list serves as an index of these manifests. It records the location of each manifest file, along with metadata such as its associated partition range, snapshot ID, and sequence number. When a query engine plans a scan, it starts with the manifest list, evaluates whether each manifest is needed, and then drills into the relevant ones to find matching data files.

This layered metadata structure makes snapshot operations efficient, even when a table has millions of files. Each layer provides just enough information to prune unnecessary work.

### C.3.3 Sequence numbers and optimistic concurrency

Iceberg ensures snapshot isolation using optimistic concurrency. Writers do not block other writers or readers. Instead, each writer reads the current table state, prepares a new snapshot, and attempts to commit it. The commit succeeds only if the table hasn't changed since the writer read it.

To track and resolve conflicts, Iceberg uses *sequence numbers*. Every committed snapshot receives a unique, incrementing sequence number. This number is inherited by all data and delete files associated with the snapshot. If a commit fails due to a conflict, the writer retries the update against the new table state, possibly with a new sequence number.

These sequence numbers also play a role in delete application and scan planning. For example, when applying a delete file, engines can use the sequence number to determine whether the delete is newer than the data it might affect.

Because of this concurrency model, readers always see a consistent snapshot of the table, and writers never interfere with one another unless they attempt conflicting changes. There are no locks, no coordination services, and no hidden race conditions. Just atomic commits and a clear record of what changed, when, and why.

## C.4 The REST Catalog specification

Iceberg tables rely on catalogs to store and resolve metadata pointers. Traditionally, these catalogs have been tightly coupled to specific technologies, Hive Metastore, JDBC databases, or versioned systems like Nessie. While functional, these implementations introduce dependencies that vary across environments, making it harder to share Iceberg tables across engines and teams.

To solve this, the Iceberg community introduced a new standard: the REST Catalog specification. First released in Iceberg 0.14.0, this specification defines a consistent, engine-agnostic API for interacting with catalogs over HTTP. It decouples the catalog client from any particular storage backend, enabling interoperability and simplification across the lakehouse stack.

The goal is straightforward: make it possible for any Iceberg client to talk to any compliant REST Catalog server, without needing to load custom connectors or embed vendor-specific logic.

### C.4.1 Overview and purpose

The REST Catalog specification defines a set of routes and behaviors that catalog servers must support. These include endpoints for managing namespaces, creating and loading tables, committing transactions, and listing configuration options.

A typical Iceberg engine, like Spark or Flink, uses these endpoints to discover tables, load metadata, and write updates. Because the API is based on HTTP and uses standard JSON payloads, it can be implemented in any language and hosted on any platform.

This architecture gives organizations flexibility. They can deploy a REST Catalog as a standalone service, integrate it with internal tooling, or proxy it to another system like Hive or a database. The client doesn't need to know what's behind the catalog; it just follows the API.

### C.4.2 Catalog configuration and default endpoints

Every REST Catalog starts with a configuration step. Clients first call the `/v1/config` endpoint to retrieve default and override settings from the server. These settings may include the warehouse location, the supported endpoints, or limits on client behavior.

The spec defines a base set of endpoints for working with namespaces and tables. These include:

- Creating, listing, and deleting namespaces
- Creating, registering, and loading tables
- Updating table metadata
- Committing transactions atomically
- Gathering metrics about tables or views
- Renaming tables and managing properties

These endpoints follow REST conventions and support standard authentication methods such as OAuth2 and bearer tokens. Security and multi-tenant isolation can be layered on top without breaking compatibility with clients.

### **C.4.3 Namespaces, tables, and views**

In a REST Catalog, all tables are organized under namespaces. These are logical groupings, similar to schemas or folders, that help structure large collections of tables. Each table is referenced using a fully qualified name, such as `analytics.sales.orders`.

Clients can use REST calls to list namespaces, query the tables within them, or create new entries. Views, where supported, can be handled similarly. While Iceberg itself doesn't prescribe view semantics, the catalog can expose view definitions to compatible engines.

Table creation is a two-step process. First, the client sends a metadata file and a request to register it under a namespace and name. Then, the server commits the new entry and returns a response pointing to the current snapshot. This allows custom logic, such as validation, enforcement, or replication, to be applied before the table becomes available.

### **C.4.4 Table registration, metrics, and transactions**

One of the most powerful aspects of the REST spec is its support for transactions. The `/transactions/commit` endpoint allows multiple changes, new data files, delete files, and metadata updates, to be bundled and committed as a single atomic operation.

This mirrors the optimistic concurrency model used throughout Iceberg. The client prepares a snapshot update, submits it to the catalog, and the catalog applies it only if the base version is still current. If a conflict occurs, the client retries.

Metrics endpoints allow clients to collect statistics about table usage or performance. These are optional, but they provide a standard path for exposing custom observability data, such as file counts, snapshot age, or storage footprint.

### **C.4.5 OAuth2 support and security considerations**

Security is an important part of any distributed system. The REST spec includes built-in support for token-based authentication and OAuth2 flows. Clients can use the Authorization header to pass credentials, and servers can validate sessions, roles, and scopes as needed.

Earlier versions of the spec included a dedicated `/oauth/tokens` endpoint, but this has been deprecated in favor of external token exchanges. This shift reflects a broader emphasis on separating catalog logic from identity management, making it easier to plug into existing security infrastructure.

Whether you're deploying a REST Catalog in a public cloud, an internal network, or across multiple environments, the goal is the same: allow clients to authenticate, discover, and safely modify Iceberg tables through a consistent, versioned API.

### C.4.6 The scan planning endpoint

As Iceberg tables grow in size and complexity, query engines need efficient ways to discover which data files are relevant to a specific scan. Traditionally, engines load a table's metadata, read the manifest list, then scan multiple manifest files to identify candidate files. While this works well for many cases, it introduces overhead, especially when queries are small or frequent.

To address this, the REST Catalog specification introduces a dedicated endpoint for scan planning. This endpoint allows engines to delegate part of the query planning process to the catalog itself. Instead of downloading and interpreting metadata locally, the engine can submit a planning request and receive a ready-to-scan list of files and deletes.

This endpoint simplifies client logic and reduces latency. It also creates opportunities for catalogs to optimize the planning step internally, such as by caching, filtering, or applying custom logic based on configuration or access policies.

The typical flow works as follows:

1. The client issues a request to `/v1/{prefix}/namespaces/{namespace}/tables/{table}/plan`.
2. The request includes optional filters, projection columns, and scan options.
3. The catalog processes the request and returns a list of data and delete files, along with partition and column-level statistics.

The result is a fully resolved scan plan, ready for execution by the engine. This structure closely mirrors what the engine would have constructed manually, but avoids multiple metadata downloads and parsing steps.

Scan planning via the REST Catalog is particularly valuable in serverless or multi-tenant environments. It reduces network traffic, improves cold-start performance, and allows the central catalog to enforce policies or optimizations that may not be visible to individual engines.

While not every catalog implementation supports this endpoint today, it represents a strategic direction for Iceberg: making metadata work more scalable by giving the catalog a more active role in query orchestration. As catalogs become smarter and more stateful, endpoints like this will continue to close the gap between metadata and execution.

## C.5 Puffin file format specification

Not all metadata in an Iceberg table is stored in manifest files or table-level schemas. Some use cases require rich, extensible metadata that lives outside the standard table structures. That's where the Puffin file format comes in.

Puffin is a binary format for storing auxiliary metadata in a compact, structured, and versioned way. It was designed to be flexible, allowing engines, catalogs, or custom tooling to embed and consume metadata that is not strictly required for core table operations but still valuable for performance, observability, or advanced analytics.

The most common use case today is column-level metrics. While manifest files already include basic statistics like min, max, and null count at the file level, Puffin files can store precomputed metrics with greater precision or density. This can accelerate scan planning by providing detailed stats without forcing the engine to read every file or manifest.

### C.5.1 What is a Puffin file?

A Puffin file consists of a header, a list of metadata entries, and a footer that maps entry keys to byte ranges in the file. Each entry is encoded as a compressed binary blob, along with its format version and length. This structure allows readers to scan the footer, locate the entries they care about, and read them directly, without loading the full file into memory.

This makes Puffin files both efficient and extensible. New entry types can be added over time without breaking existing consumers, and different engines can choose to support different subsets of entries. Common entry types include column statistics, bloom filters, and custom indexes.

Because Puffin files are optional, they don't change the behavior of the Iceberg table itself. A table without Puffin files still functions normally, reads, writes, snapshots, and deletes work as expected. But when Puffin files are present, they provide an additional layer of metadata that advanced tools can use to optimize performance.

### C.5.2 Storing column-level metrics and custom indexes

The primary motivation behind Puffin was to store fine-grained column statistics in a way that is both portable and performant. These statistics are often too large to include directly in manifest files, especially for wide tables or streaming ingestion scenarios. Puffin solves this by decoupling the stats from the manifest and storing them in a separate, optimized structure.

Tools that generate Puffin files might run in batch, asynchronously from ingestion. They scan the table's data files, compute detailed metrics, such as histograms, quantiles, or cardinality estimates, and write them into a Puffin file. Later, during query planning, an engine can read those metrics to decide which files to scan, which filters to apply, and how to order execution.

Beyond statistics, Puffin can also be used to store custom indexes. For example, a search-heavy workload might benefit from inverted indexes or sketches that speed up point lookups. Rather than building those structures at query time, they can be written once into a Puffin file and reused across queries.



### C.5.3 Integration with Iceberg table metadata

Puffin files are not referenced directly by manifest lists or snapshot metadata. Instead, they are typically registered as table-level properties or discovered through catalog integrations. This keeps the core table metadata lightweight while still allowing advanced tooling to benefit from the additional data.

Because Puffin is designed to evolve independently of the main Iceberg spec, it allows for experimentation and specialization. Different engines can adopt Puffin in different ways, whether to improve scan planning, support new query types, or build observability layers on top of Iceberg.

As Iceberg matures, Puffin is expected to play a larger role in performance tuning and adaptive query execution. It reflects a broader pattern in the Iceberg ecosystem: separating core behavior from optional enhancements, so that complexity remains manageable, but customization is always possible.

## C.6 Compatibility and migration

One of Iceberg's core strengths is its focus on stability. From the beginning, the project has prioritized backward compatibility, allowing users to upgrade their tables and engines without rewriting data or interrupting pipelines. But as the specification evolves, so do the expectations around how features work, what metadata is required, and how tools interpret table state.

Understanding how version compatibility works and how to manage migrations between versions is essential for operating Iceberg tables in production. Whether you're upgrading from v1 to v2, adopting v3 features, or evaluating the upcoming changes in v4, the transition points are clear, documented, and designed to be safe.

### C.6.1 Reading and writing across format versions

Each Iceberg table includes a format-version field in its metadata file. This field indicates which version of the table format is in use. Engines use this value to determine what features are allowed, what metadata fields to expect, and how to interpret certain behaviors like deletes or transforms.

Reading across versions is typically straightforward. Newer engines are designed to understand older formats, so a v3 engine can safely read v1 or v2 tables. This ensures that clients don't break just because a table hasn't been upgraded yet.

Writing across versions is more sensitive. An engine that supports v3 features, like deletion vectors or new data types, cannot write to a v1 table. In these cases, the engine must either downgrade its behavior or trigger a table upgrade. Most engines handle this negotiation automatically and provide clear error messages if an operation isn't supported.

### C.6.2 Upgrading tables to newer spec versions

Iceberg makes table upgrades an explicit action. Changing the format version is not automatic; it requires the user (or engine) to write a new metadata file with a higher format-version and to validate that the environment supports it.

Upgrades are typically triggered in one of two ways:

- Manually, through catalog tools or table property updates

- Automatically, by a write operation that requires a newer feature

For example, if a user writes an equality delete to a v1 table, the engine will either block the operation or upgrade the table to v2 before applying the commit.

Upgrading does not rewrite existing data or manifests. All prior snapshots remain valid, and the new version applies only to changes going forward. This makes upgrades lightweight and safe, even on large tables.

### **C.6.3 Handling backward compatibility in practice**

Backward compatibility is central to Iceberg's design, but it comes with expectations. While older readers can continue working with tables that haven't been upgraded, they will not be able to understand newer features once the format version increases.

For this reason, upgrades should be planned in tandem with engine and catalog updates. If some components in your stack only support v1 or v2, upgrading to v3 or v4 may cause compatibility issues. Most production deployments lock table versions until all critical tools are verified against the newer format.

In practice, migrations are straightforward because:

- Data files remain unchanged between versions
- Older metadata files are retained for time travel
- The upgrade path is always forward-only and incremental

There is no expectation that all users must adopt the latest format. Each version builds on the previous one but preserves compatibility and stability. That means Iceberg users can adopt new features at their own pace, ensuring that upgrades align with operational readiness, engine support, and organizational requirements.